

Two Currencies 解説

ynymxiaolongbao

クエリの言い換え

- X_j 枚の金貨と Y_j 枚の銀貨がある
 - 「金貨1払う or 銀貨 C_i 払う」という要求がいくつかある
 - 金貨、銀貨の枚数を0未満にせずにすべての要求を満たせるか？満たせる場合、残せる金貨の枚数の最大値はいくつか？
-
- C_i が小さいものから順に、和が Y_j を超えない限り銀貨を払う
 - 残りを金貨で払う（足りなかったら-1）

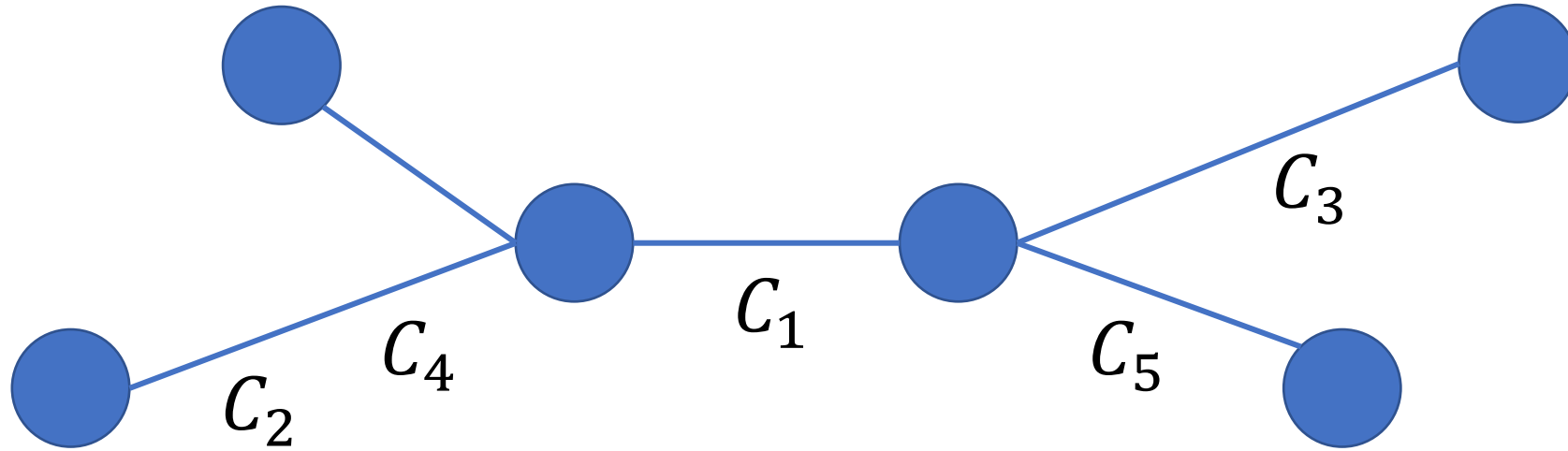
クエリの言い換え

- (答え) = $\max(X_j - (\text{払いたい金貨の枚数}), -1)$
- (払いたい金貨の枚数)
= (要求の個数) - (C_i が小さいものから最大何個の和が Y_j 以下か)

以下の二つの値が分かれば良い

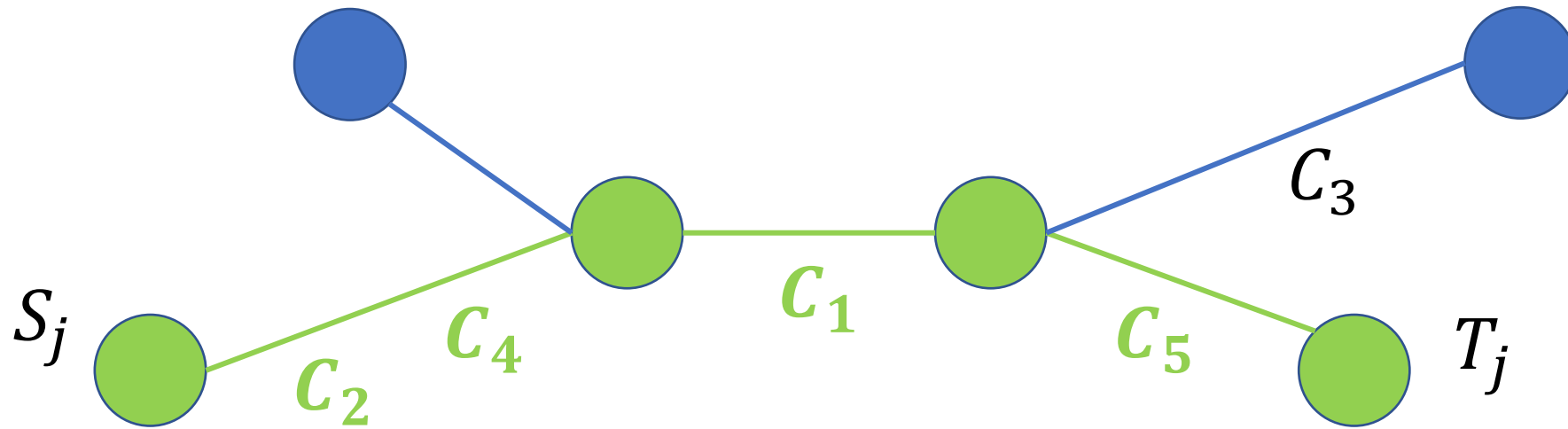
- (要求の個数)
- (C_i が小さいものから最大何個の和が Y_j 以下か)

問題概要



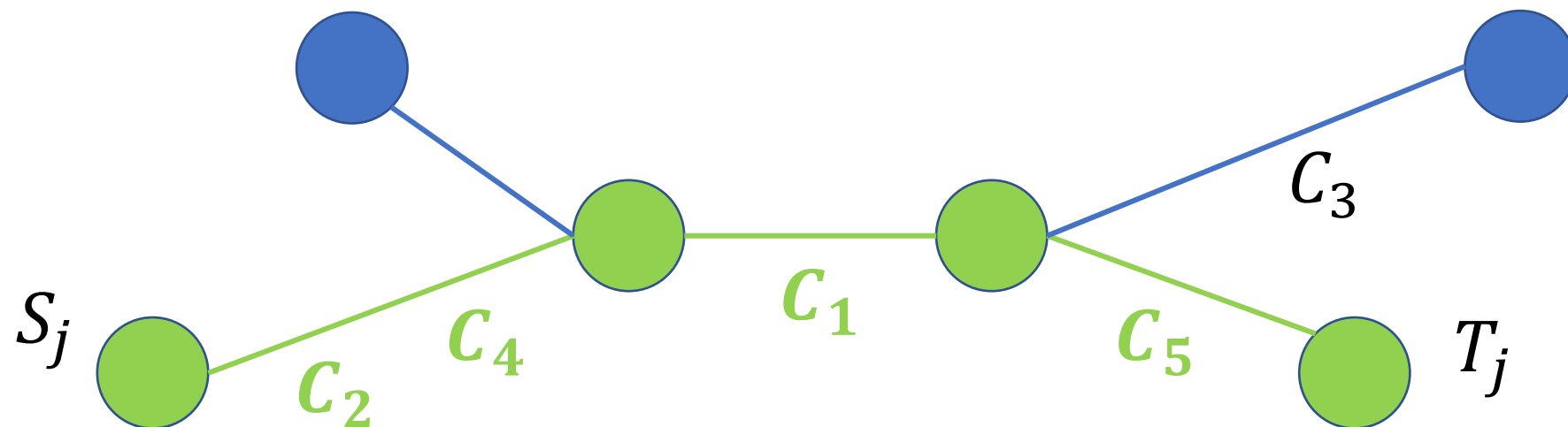
- 木がある
- 辺上に、値 c_i がいくつかある
- 頂点 s_j, t_j を結ぶ最短パス上の値 c_i の集合について、(c_i の個数)、(c_i が小さいものから最大何個の和が Y_j 以下か)を求める

問題概要



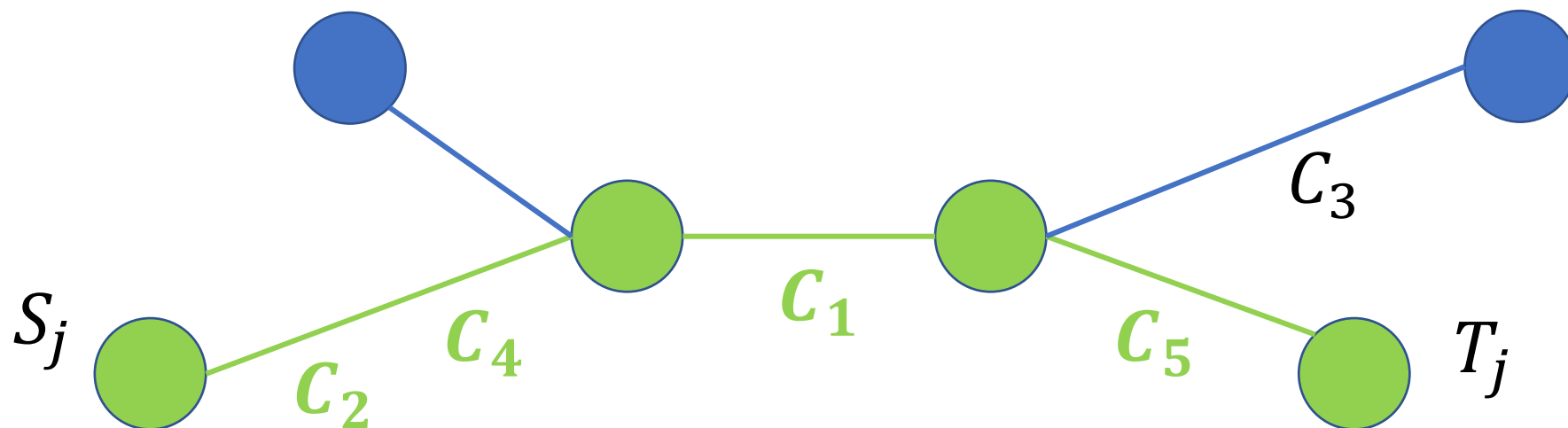
- 木がある
- 辺上に、値 c_i がいくつかある
- **頂点 S_j, T_j を結ぶ最短パス上の値 c_i の集合** について、(c_i の個数)、(c_i が小さいものから最大何個の和が Y_j 以下か) を求める

小課題 1 ($N, M, Q \leq 2000$)



- (c_i の個数)、(c_i が小さいものから最大何個の和が Y_j 以下か) 求める
- 二つの値を愚直に求める
- $O((N + M \log M)Q)$

小課題 2 ($C_1 = C_2 = \dots = C_M$)

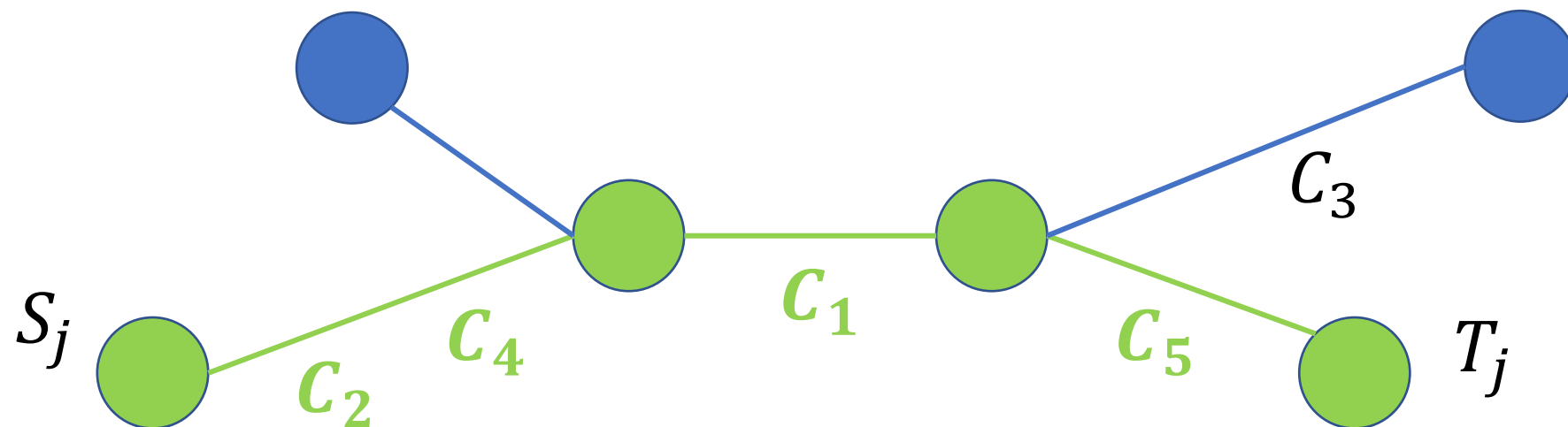


- (C_i の個数)、(C_i が小さいものから最大何個の和が Y_j 以下か) 求める
- (C_i が小さいものから最大何個の和が Y_i 以下か)

$$=\min\left((C_i\text{の個数}), \left\lfloor \frac{Y_j}{C_1} \right\rfloor\right)$$

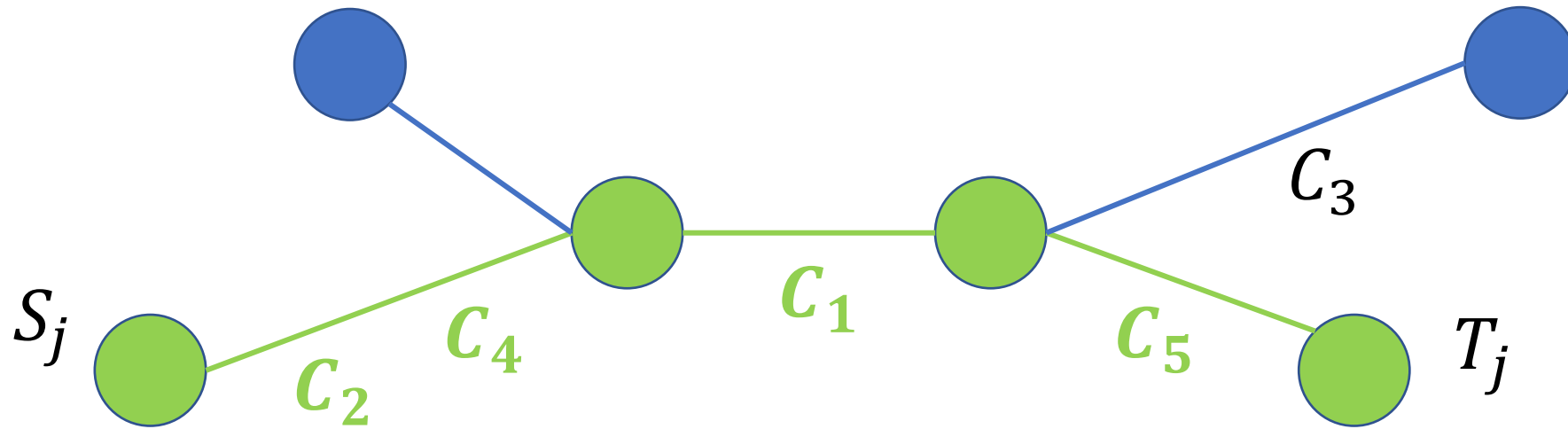
→ (C_i の個数) だけ求めれば良い

小課題 2 ($C_1 = C_2 = \dots = C_M$)



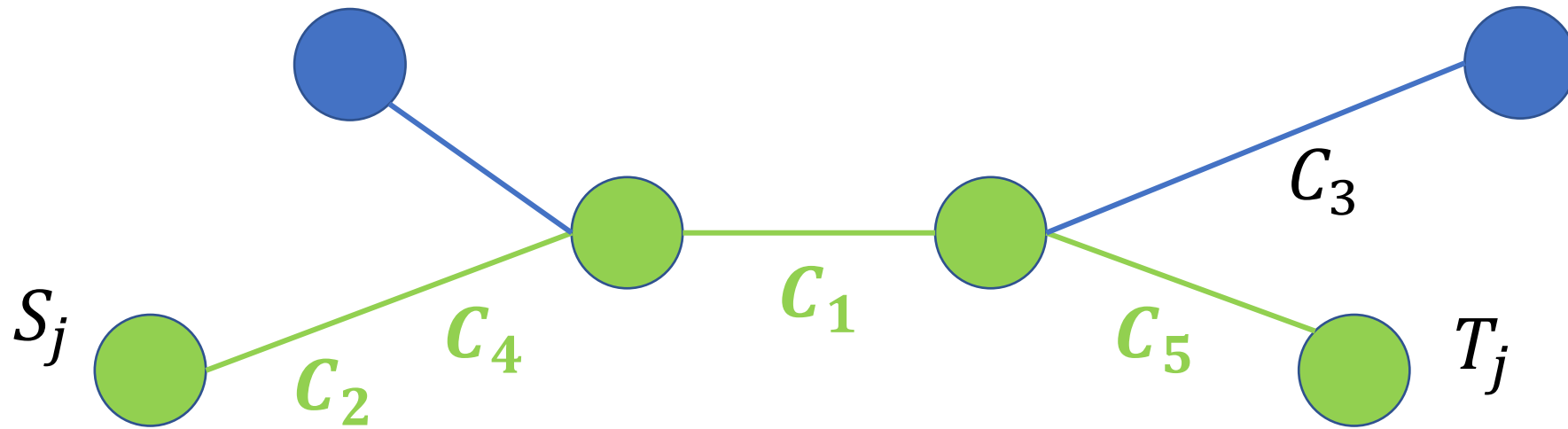
- (C_i の個数) を求める
- 木のパス上の辺の重みの和 (更新なし)
→ LCA + 累積和
- $O(Q \log N + N \log N + M)$

小課題3 (パス), 満点解法



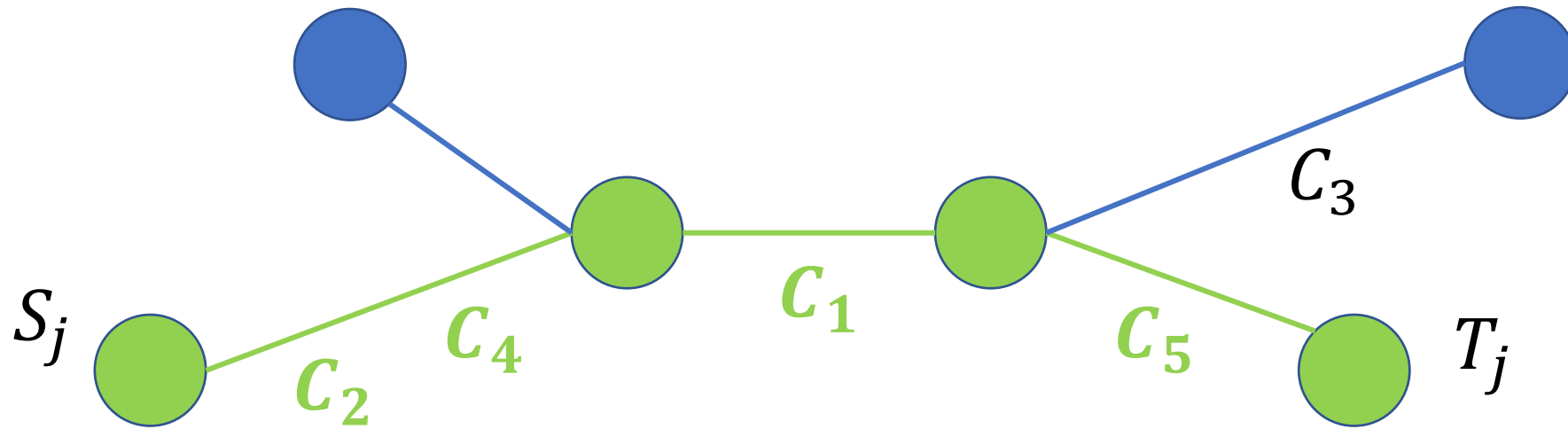
- (C_i の個数)、(C_i が小さいものから最大何個の和が Y_j 以下か) 求める
 - (C_i の個数)は小課題2で求まった
- ※もし木のケースが分からなくても、パスの場合は累積和で求まる
→ (C_i が小さいものから最大何個の和が Y_j 以下か) を求めればよい

小課題3 (パス), 満点解法



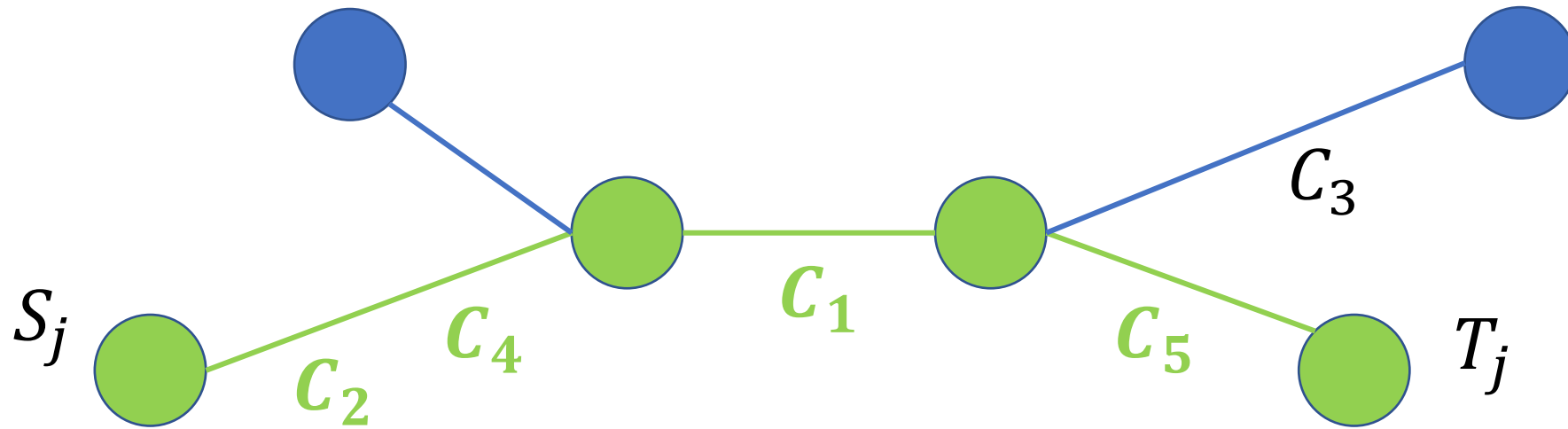
- (C_i が小さいものから最大何個の和が Y_j 以下か) を求める
- パス $S_j - T_j$ 上にある C_i であって、全ての C_i を昇順にソートしたとき k 番目までに入るものの和が Y_j 以下となる最大の k は何か?
- このままでは難しい

小課題3 (パス), 満点解法



- (C_i が小さいものから最大何個の和が Y_j 以下か) を求める
- **パス $S_j - T_j$ にある C_i であって、全ての C_i を昇順にソートしたとき k 番目までに入るものの和が Y_j 以下となる最大の k は何か?**
- このままでは難しい
→ 二分探索したときの**判定問題**を考える

小課題3 (パス), 満点解法



- パス $S_j - T_j$ にある C_i であって、全ての C_i を昇順にソートしたとき k 番目までに入るものの和が Y_j 以下か？
- この Q 個の判定問題を解く
- 何もない状態から始めて、 C_i を昇順に辺上に追加していく
- k 番目まで追加したら、パス $S_j - T_j$ 上の和を求めて Y_j 以下か判定

小課題3 (パス), 満点解法

パスの場合

- 一点更新
- 区間の値の和の取得

→ **BIT**

判定問題 Q 個が $O((M + Q) \log N)$ で解ける

小課題 3 (パス), 満点解法

木の場合

- 一辺の重みを更新
- $S_j - T_j$ パスの重みの和の取得

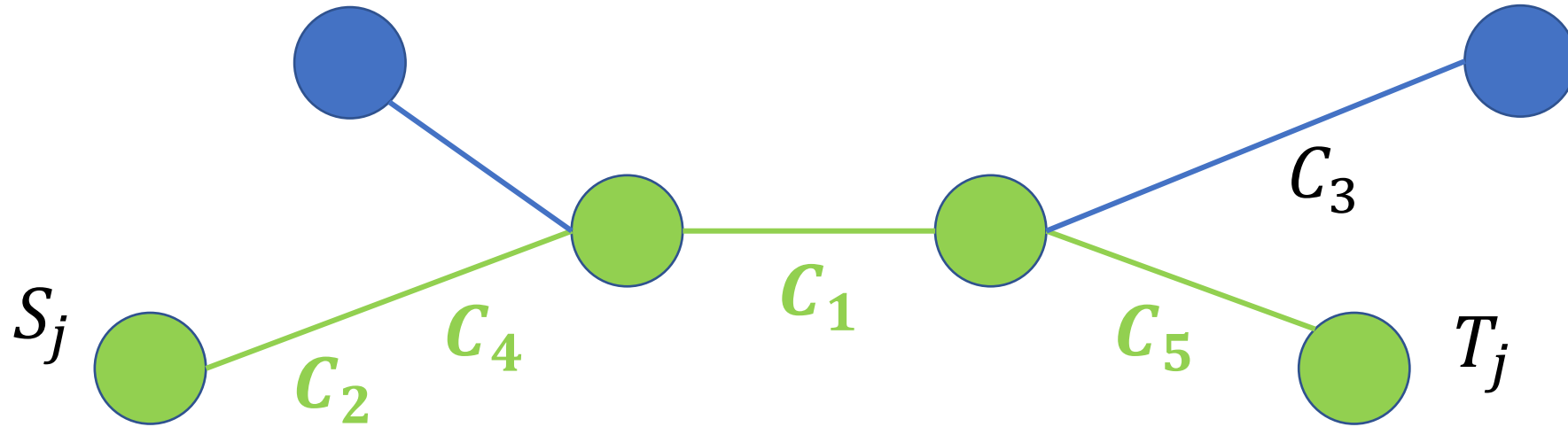
→ **LCA + EulerTour + BIT**

判定問題 Q 個が $O((M + Q) \log N)$ で解ける

小課題 3 (パス), 満点解法

- 二分探索したときの判定問題 Q 個が $O((M + Q) \log N)$ で解けた
- これを $O(\log M)$ 回繰り返して二分探索を行えば、元の問題 Q 個が $O((M + Q) \log N \log M)$ で解ける
- **並列二分探索 (Parallel Binary Search)**と呼ばれる

小課題3 (パス), 満点解法



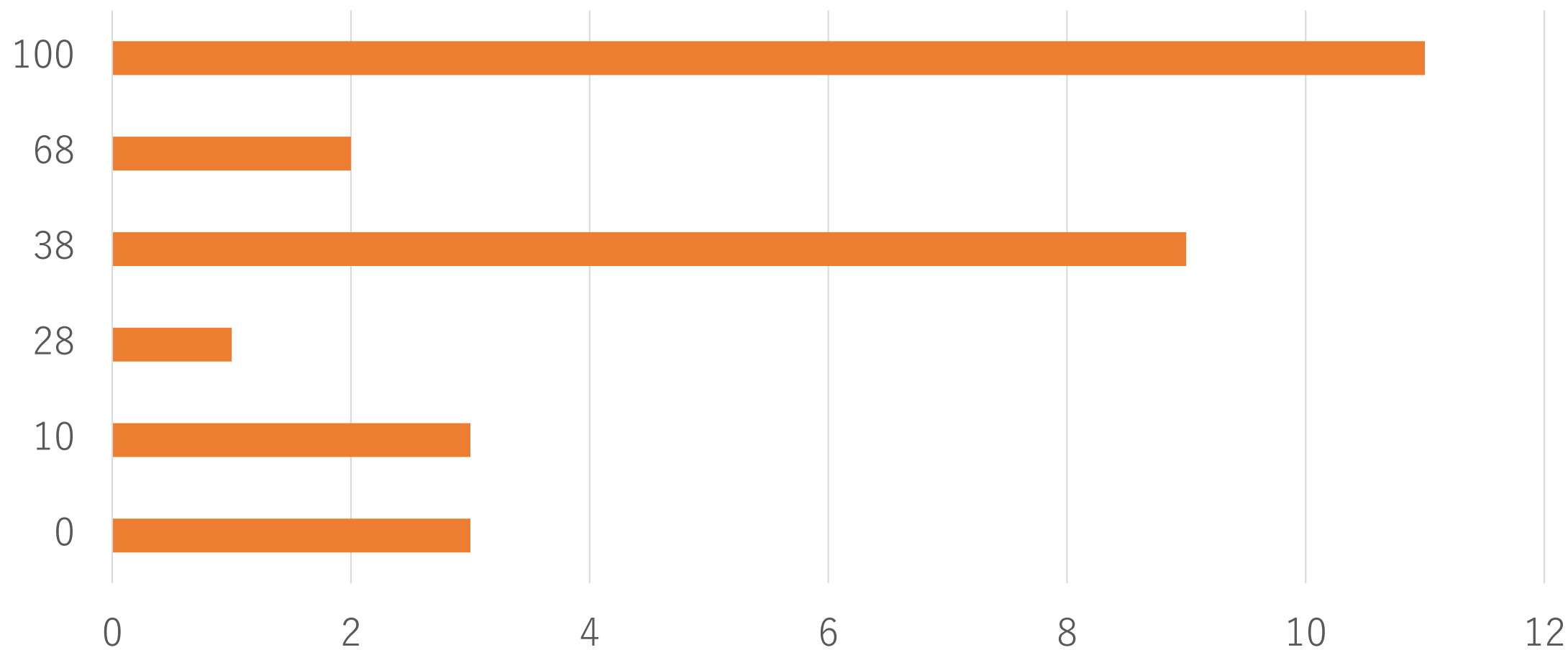
- パス $S_j - T_j$ 上にある C_i であって、全ての C_i を昇順にソートしたとき k 番目までに入るものの和が Y_j 以下となる最大の k が分かった
- (C_i が小さいものから最大何個の和が Y_j 以下か) を求める
- k 番目まで追加した後、パス $S_j - T_j$ 上にある C_i の個数を求める
- 同様の手法でよい $\rightarrow O((M + Q) \log N \log M + N \log N + M \log M)$

余談

- 並列二分探索をせず、Mo's algorithmを使う方法もある
- (簡単のため $N = M = Q$ とする)
- 更新 $O(N^{1.5})$ 回
- 計算 $O(N)$ 回
- BITだと両方に $\log N$ が付いて $O(N^{1.5} \log N)$
- 平方分割で更新 $O(1)$, 計算 $O(\sqrt{N})$ にすると $O(N^{1.5})$

得点分布

得点分布



Festival 2 解説

解説：yuto1115

区間スケジューリング問題（前提知識）

- N 個の区間があり、 i 番目の区間は $[A_i, B_i]$ で表される ($A_i < A_{i+1}$, $A_i < B_i$)
- 区間同士が共通部分を持たないようにいくつかの区間を選ぶとき、最大で何個選べるか？
- この問題は貪欲法で解ける
 - B_i の昇順に区間を見ていって、「いままで選んだどの区間とも共通部分を持たないならばその区間を選ぶ」ことを繰り返す

区間スケジューリング問題（前提知識）

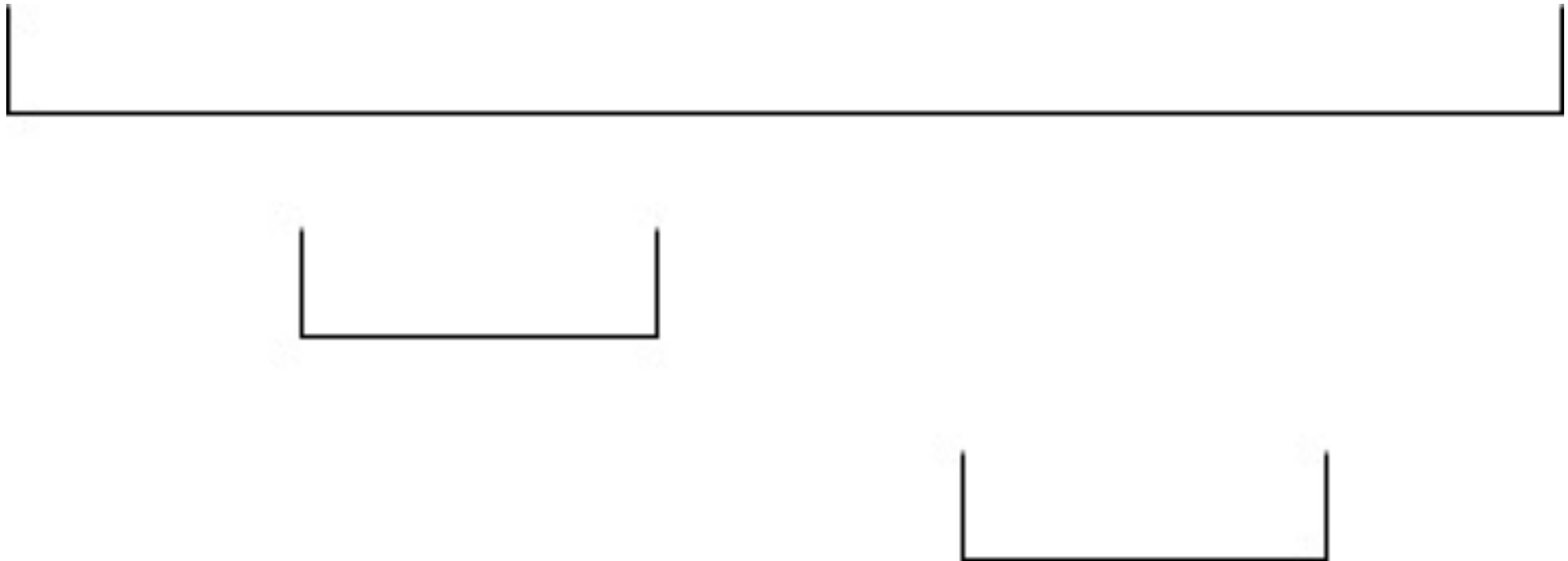
- 一方で、誤った貪欲法の例として以下のアルゴリズムがよく紹介される
 - A_i の昇順に区間を見ていって、「いままで選んだどの区間とも共通部分を持たないならばその区間を選ぶ」ことを繰り返す
- 以降、 $\uparrow$ の貪欲法のことを嘘貪欲と呼ぶ

問題概要

- N が与えられる。 N 区間からなる区間スケジューリング問題の入力としてありうるもののうち、正しい貪欲で得られる解が嘘貪欲で得られる解より改善するものの数を数えよ。
- 区間の端点は 1 以上 $2N$ 以下の整数であり、 $2N$ 個の整数がすべてちょうど一度ずつ現れるものとする
- 大きい素数 P が与えられるので、答えを P で割った余りを出力する
(埋め込み防止)

入力例 ($N = 3$)

$A = (1, 2, 4), B = (6, 3, 5)$

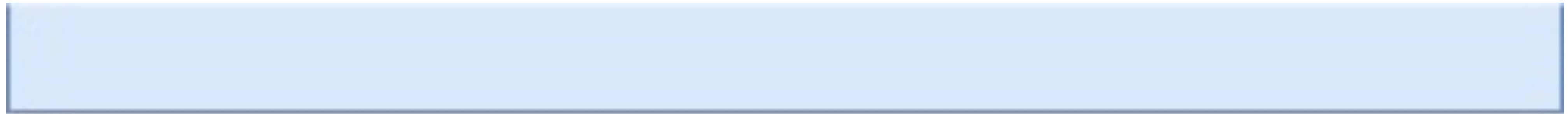


入力例 ($N = 3$)

$$A = (1, 2, 4), B = (6, 3, 5)$$

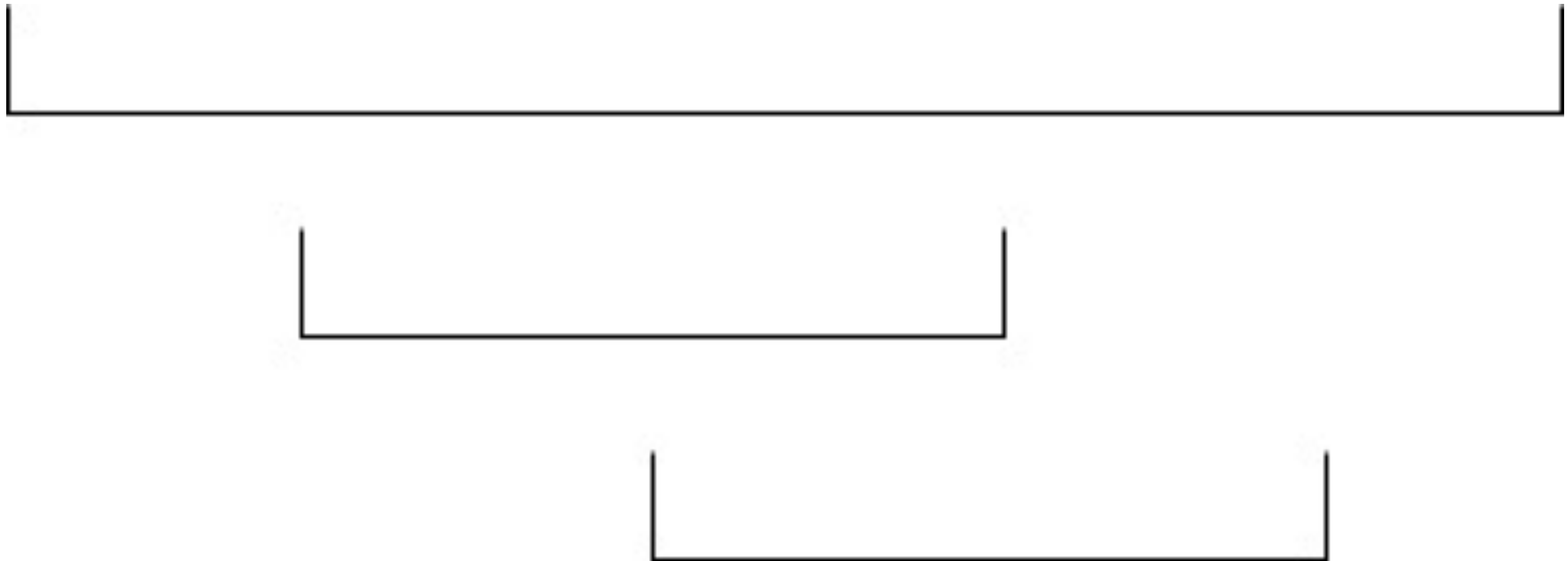
正しい貪欲

嘘貪欲



入力例 ($N = 3$)

$A = (1, 2, 3), B = (6, 4, 5)$

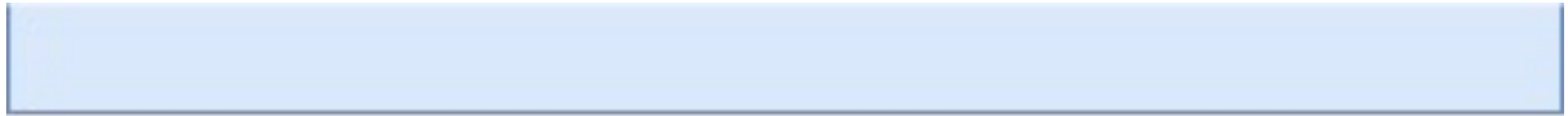


入力例 ($N = 3$)

$$A = (1, 2, 3), B = (6, 4, 5)$$

正しい貪欲

嘘貪欲



小課題 1 ($N \leq 5$)

- A_i, B_i に 1 から $2N$ までの数を割り当てる方法として $(2N)!$ 通りを全探索する
- それぞれの割り当てについて、
 - valid な入力 ($A_i < A_{i+1}, A_i < B_i$) かどうかチェックする
 - 正しい貪欲 > 嘘貪欲かどうかチェックする

ここまでで 5 点

小課題 2 ($N \leq 8$)

- $(2N)!$ 通りを全探索するのは無駄なので、最初から valid な入力だけを探索したい
- DFS を用いて、 i の昇順に A_i, B_i を定めていく
 - A_i を選ぶとき：まだ使っていない数のうち最小のものを選ぶしかない
 - B_i を選ぶとき：まだ使っていない数のうちどれを選んでもいい
- $N = 8$ のとき、valid な入力は $15 \times 13 \times 11 \times 9 \times 7 \times 5 \times 3 \times 1 = 2027025$ 通り
- それぞれについて、正しい貪欲 > 嘘貪欲かどうかチェックすればよい

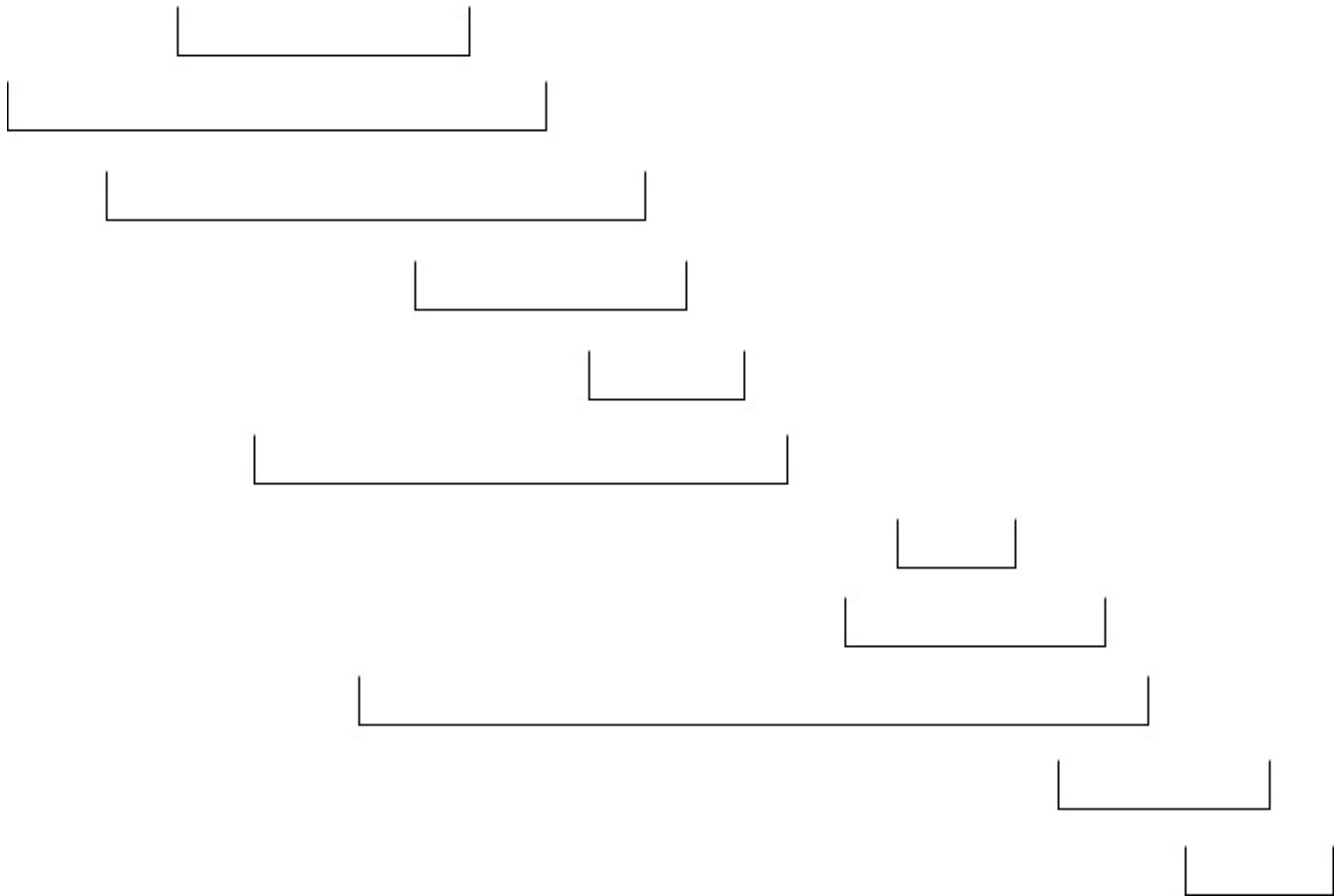
ここまでで 10 点

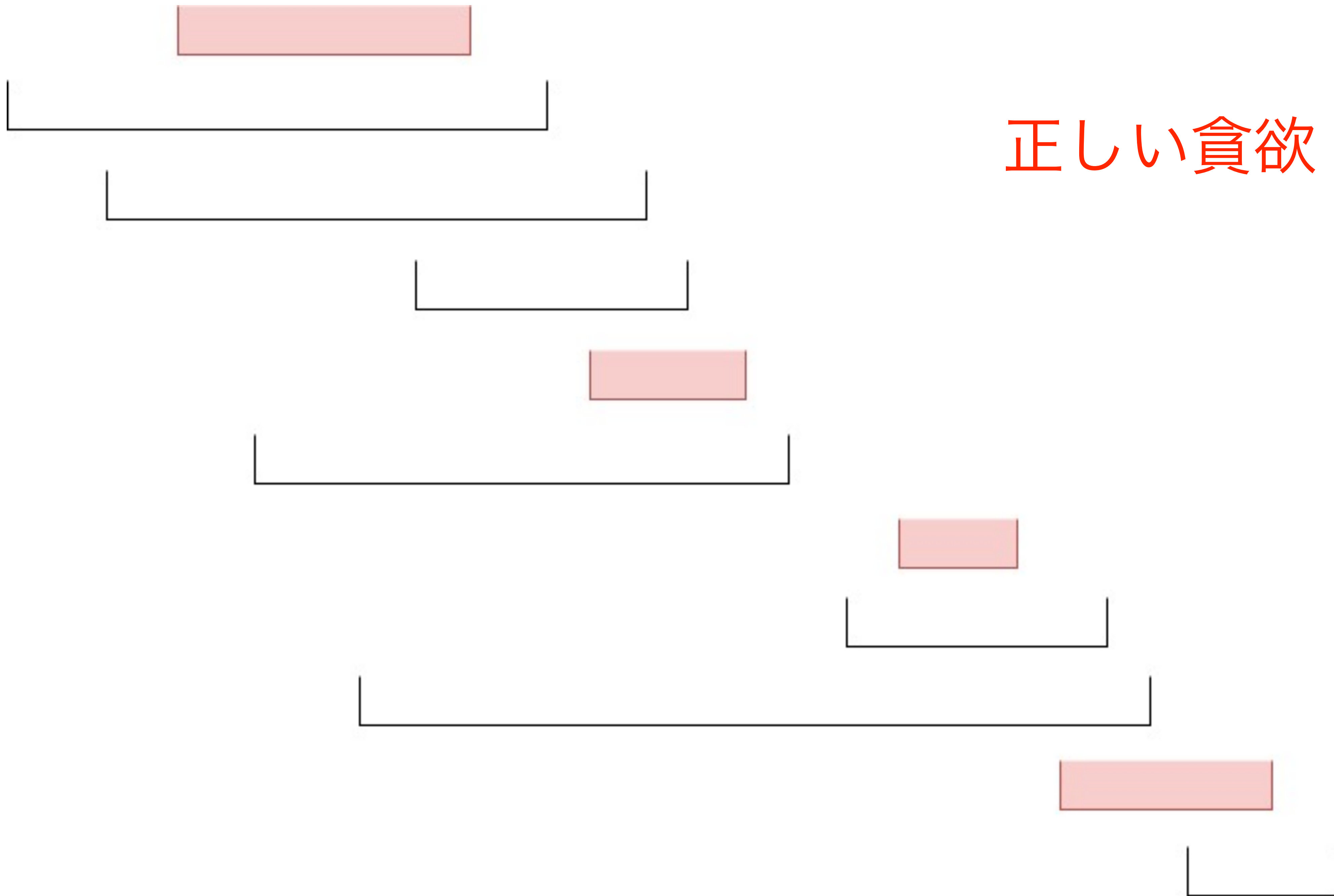
小課題 3 ($N \leq 30$)

- ここからが本番
- うまく動的計画法に落とし込んで、多項式時間で解こう
- いろいろな解法が考えられるが、一例を紹介
- 簡単のため、嘘貪欲を正しい貪欲にしても改善**しない**ものの数を数えることにする

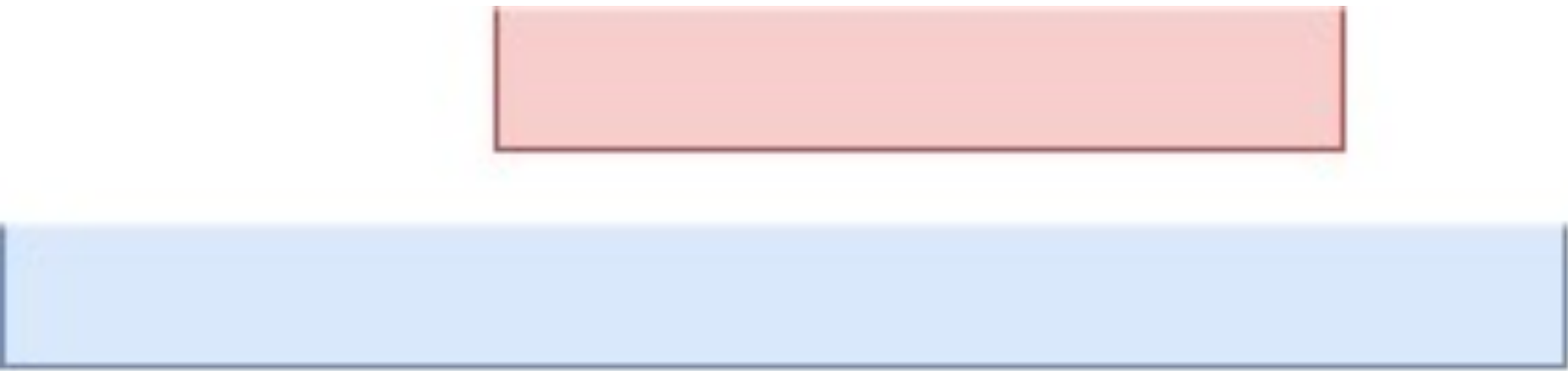
小課題 3 ($N \leq 30$)

- 区間を順に定めていきながら、「正しい貪欲で選ばれる区間」と「嘘貪欲で選ばれる区間」をそれぞれ管理していく
- それぞれの貪欲で選ばれる区間を本当にすべて保持したら破綻する
- 図を書いて、正しい貪欲 = 嘘貪欲となるときの構造を観察しよう



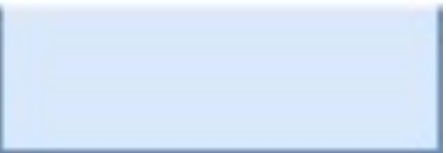


正しい貪欲



正しい貪欲

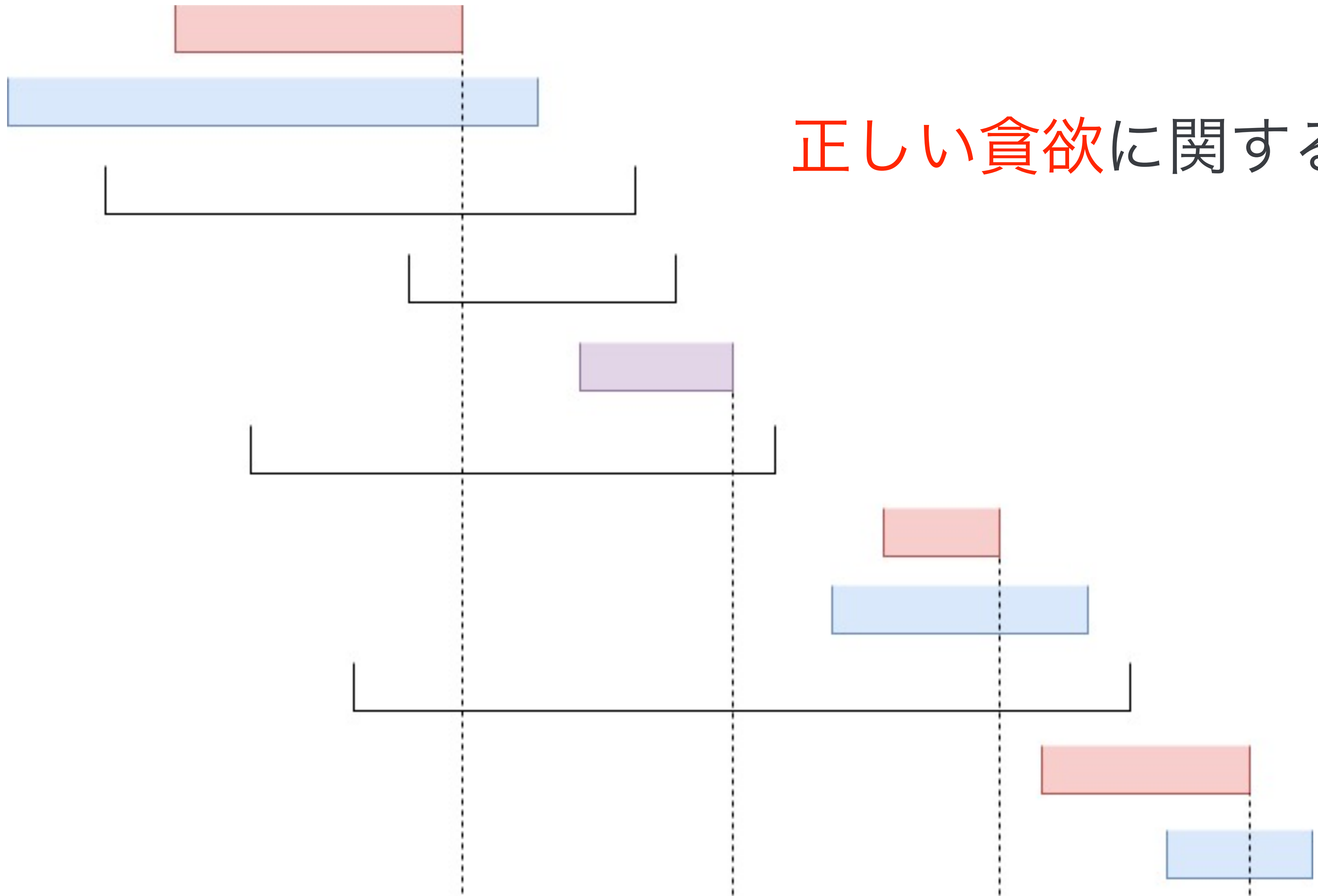
嘘貪欲



考察

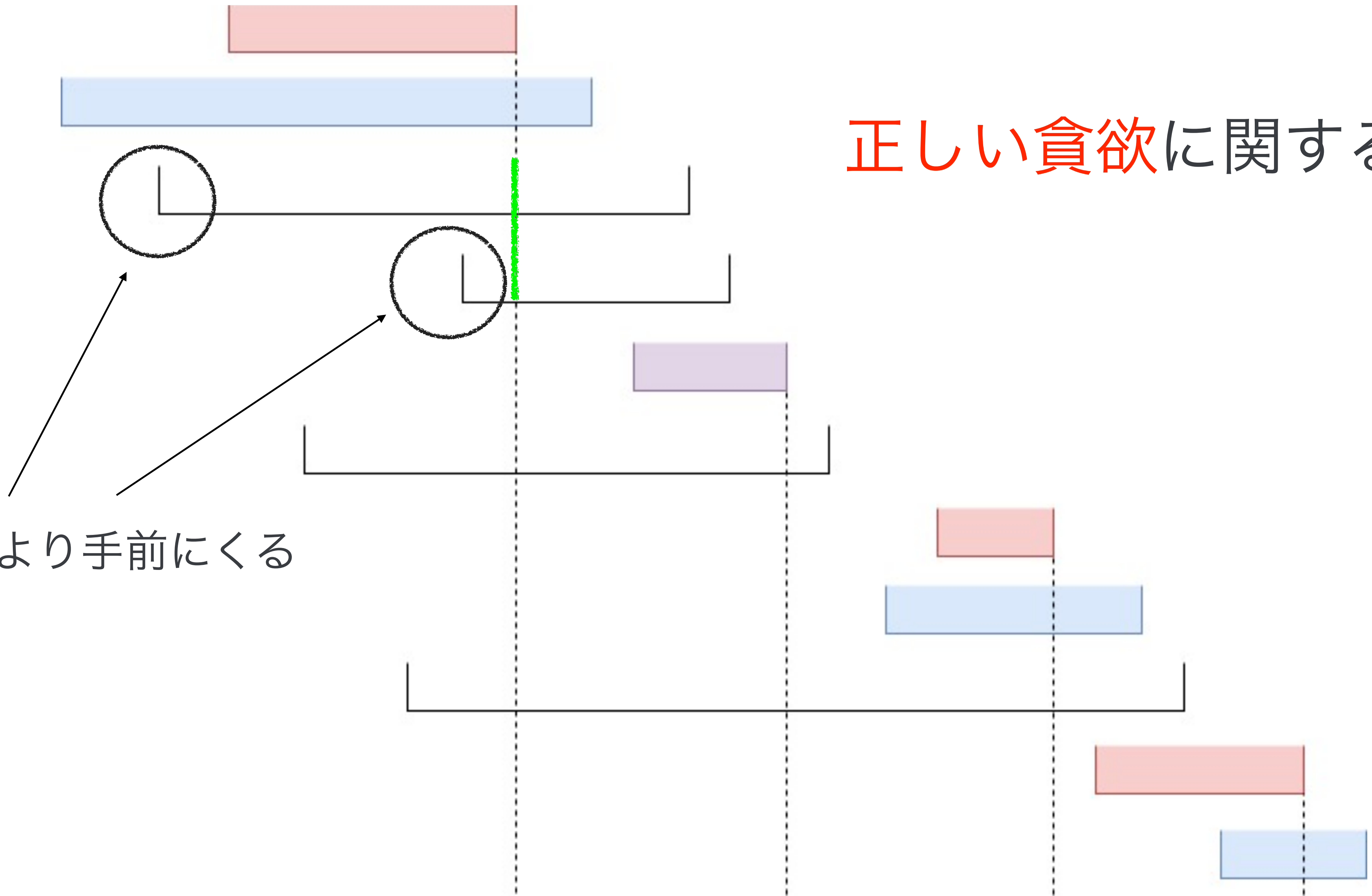
- B_i の昇順に区間を並べると、「正しい貪欲で選ばれる区間」と「嘘貪欲で選ばれる区間」が交互にやってくる（被るときもある）
- B_i の昇順に区間を定めながら、「正しい貪欲で選ばれる区間」と「嘘貪欲で選ばれる区間」を交互に指定する感じで DP をすればよさそう
- それぞれの貪欲で選ばれる区間が、指定した区間に本当に一致するための条件は？

正しい貪欲に関する条件

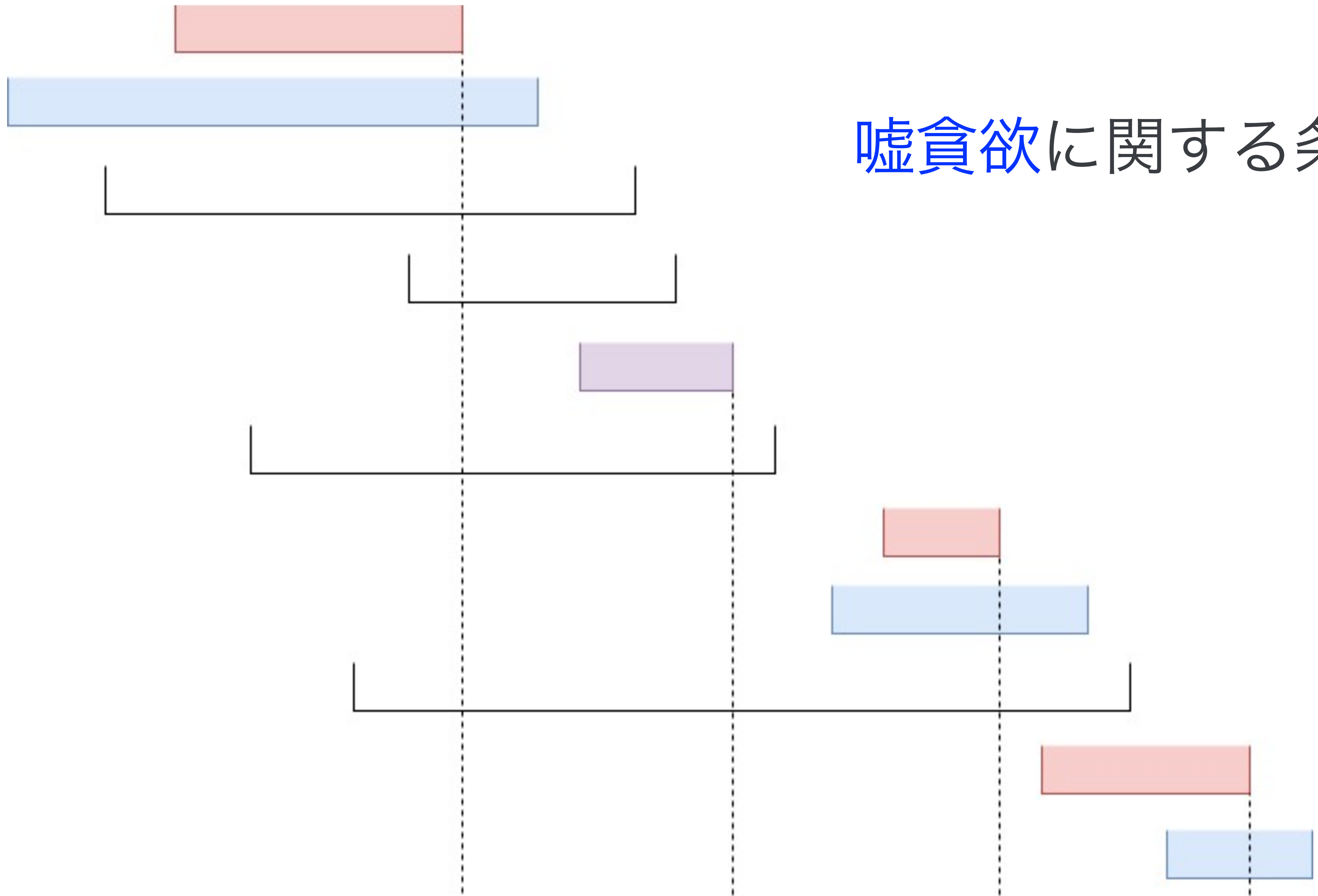


正しい貪欲に関する条件

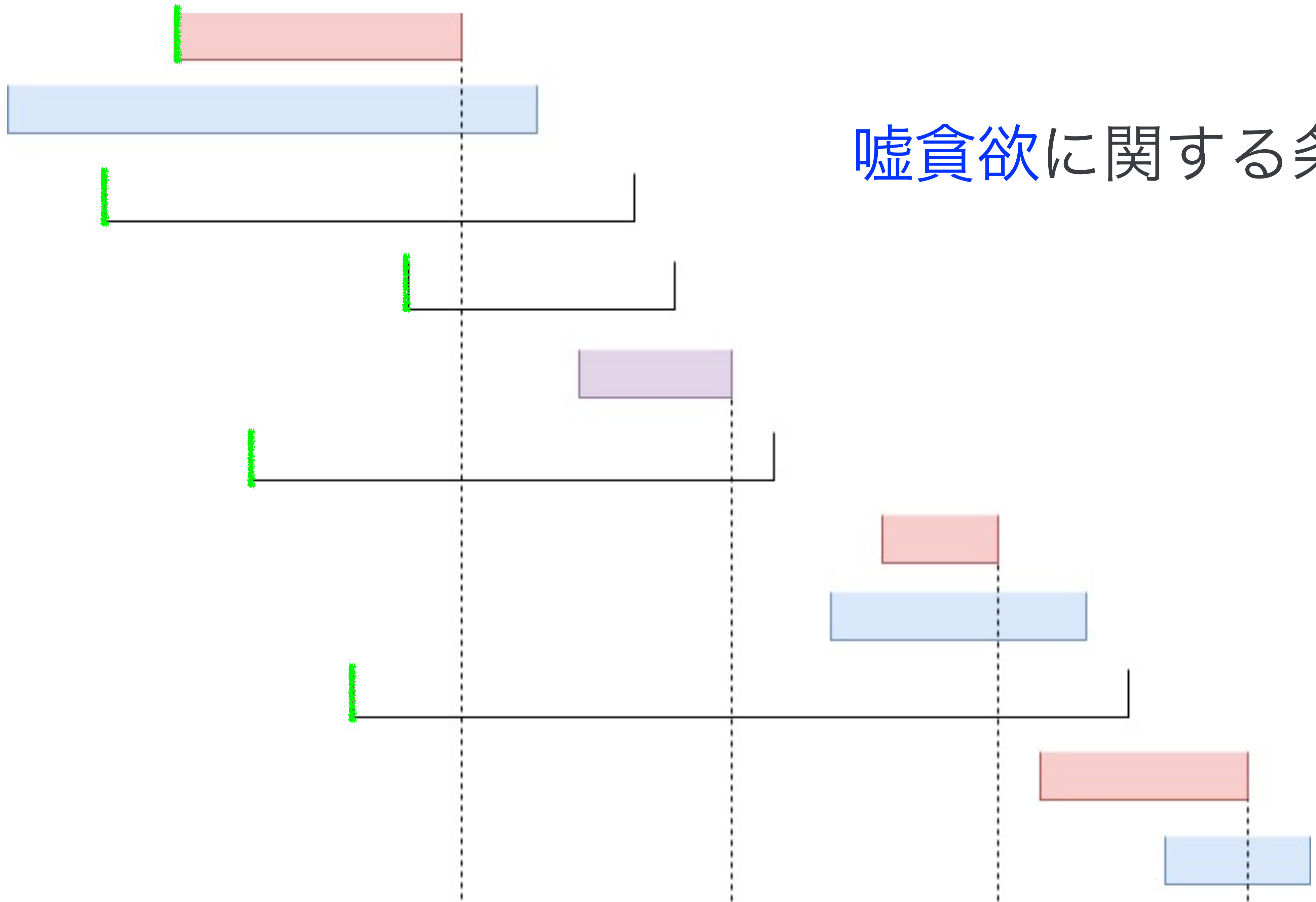
緑の線より手前にくる



嘘貪欲に関する条件

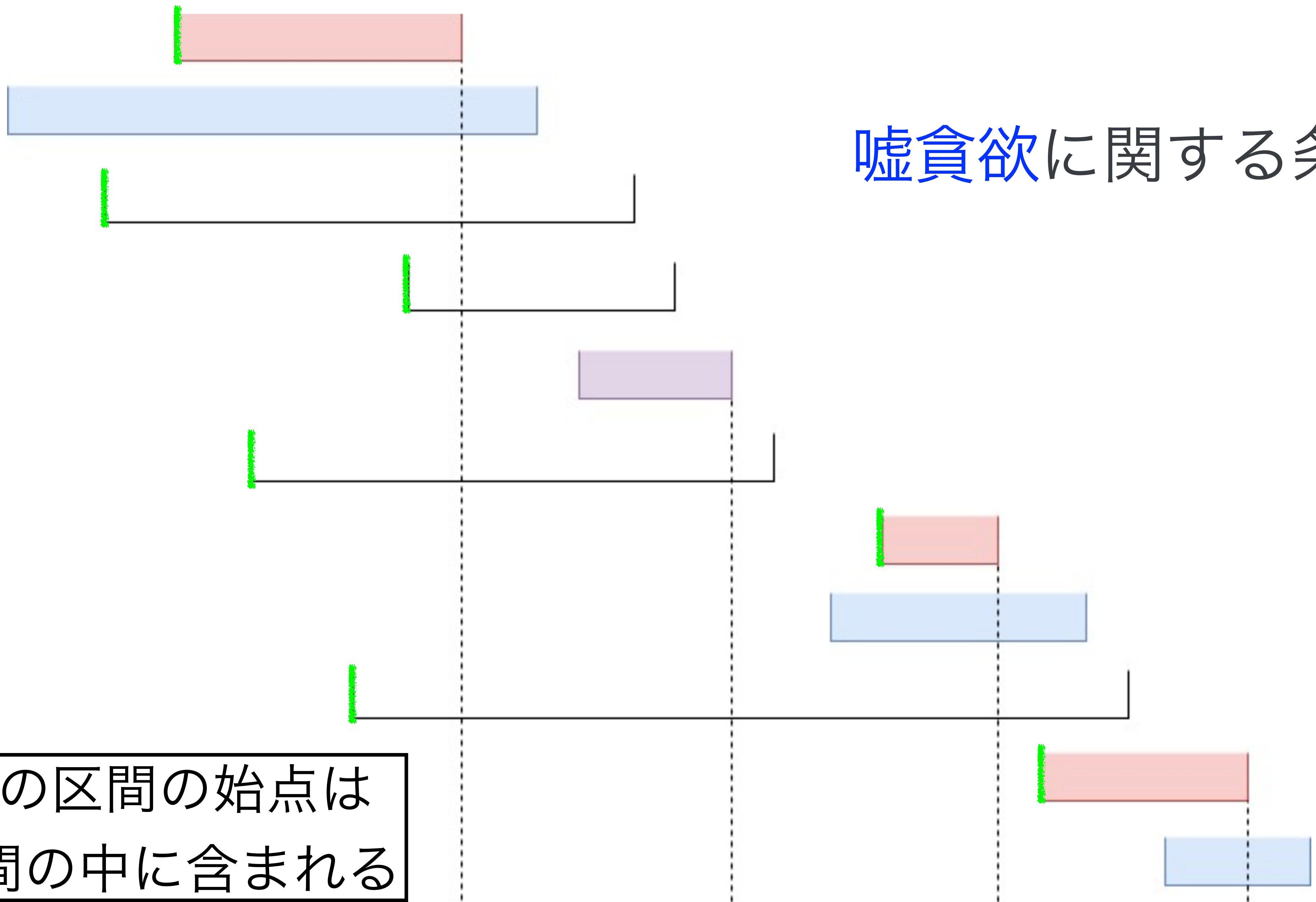


嘘貪欲に関する条件



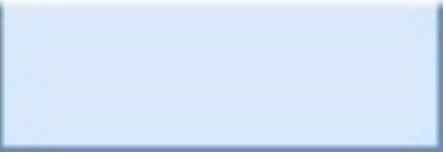
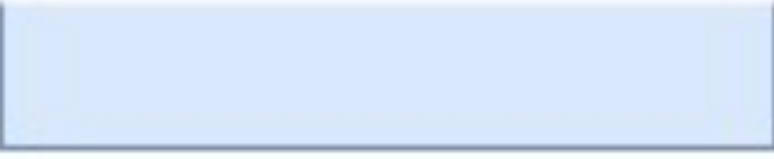
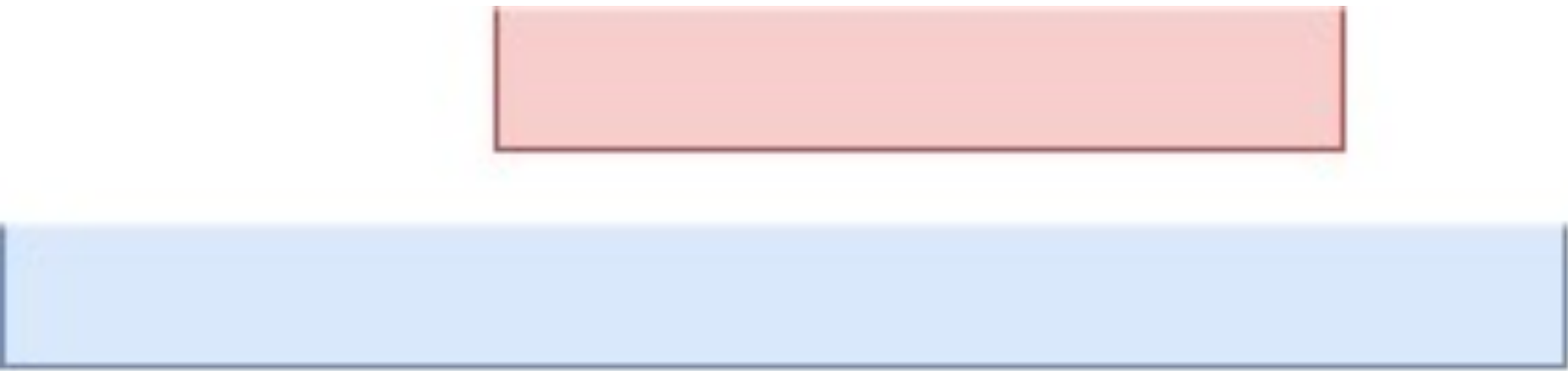
嘘貪欲に関する条件

青以外の区間の始点は
青い区間の中に含まれる



小課題 3 ($N \leq 30$)

- B_i の昇順に区間を定める挿入 DP
- 以下の情報を持つ
 - 定めた区間の個数
 - 選んだ赤い区間の数 - 青い区間の数 (0 or 1)
 - 最後に選んだ赤い区間は何番目か
 - 最後に選んだ青い区間は何番目か
 - 挿入できる箇所のうち、青い区間の中に含まれるのは何箇所か



正しい貪欲

嘘貪欲

小課題 3 ($N \leq 30$)

- 丁寧に遷移を書くと、 $O(N^5)$, $O(N^6)$ などになる
- この方針を高速化すると $O(N^3)$ まで減らせるが、実装が煩雑なので省略

ここまでで 37 点

小課題 4 ($N \leq 300$)

- 挿入 DP ではなく、1 から $2N$ までの数を順番に区間の端点に割り当てていく
- 「始点は決まったが終点がまだ決まっていない区間」がいくつか存在することになる
(これを単に「開いた区間」と呼びます)
- まず図でざっくり説明します

正しい貪欲

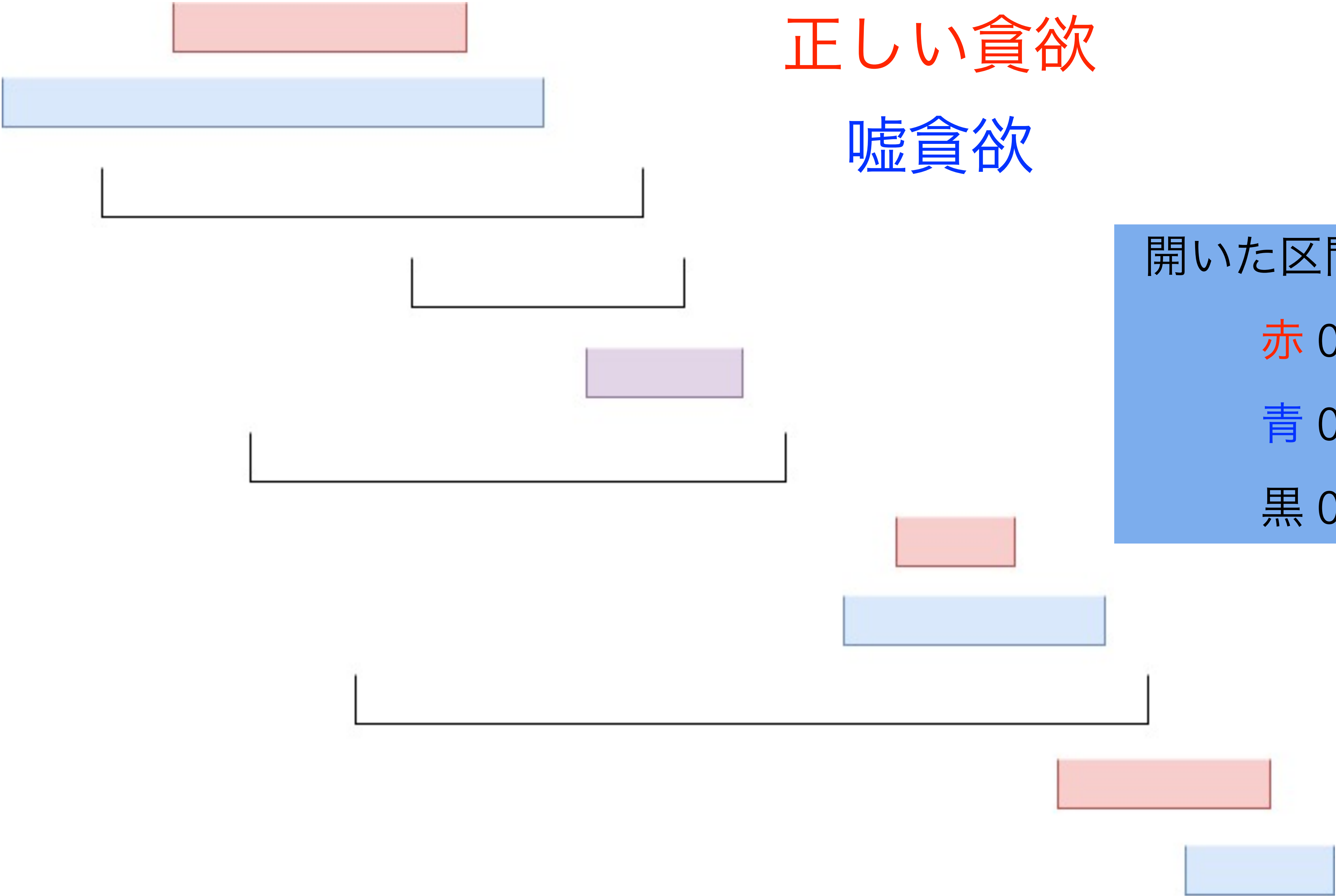
嘘貪欲

開いた区間の数

赤 0

青 0

黒 0



正しい貪欲

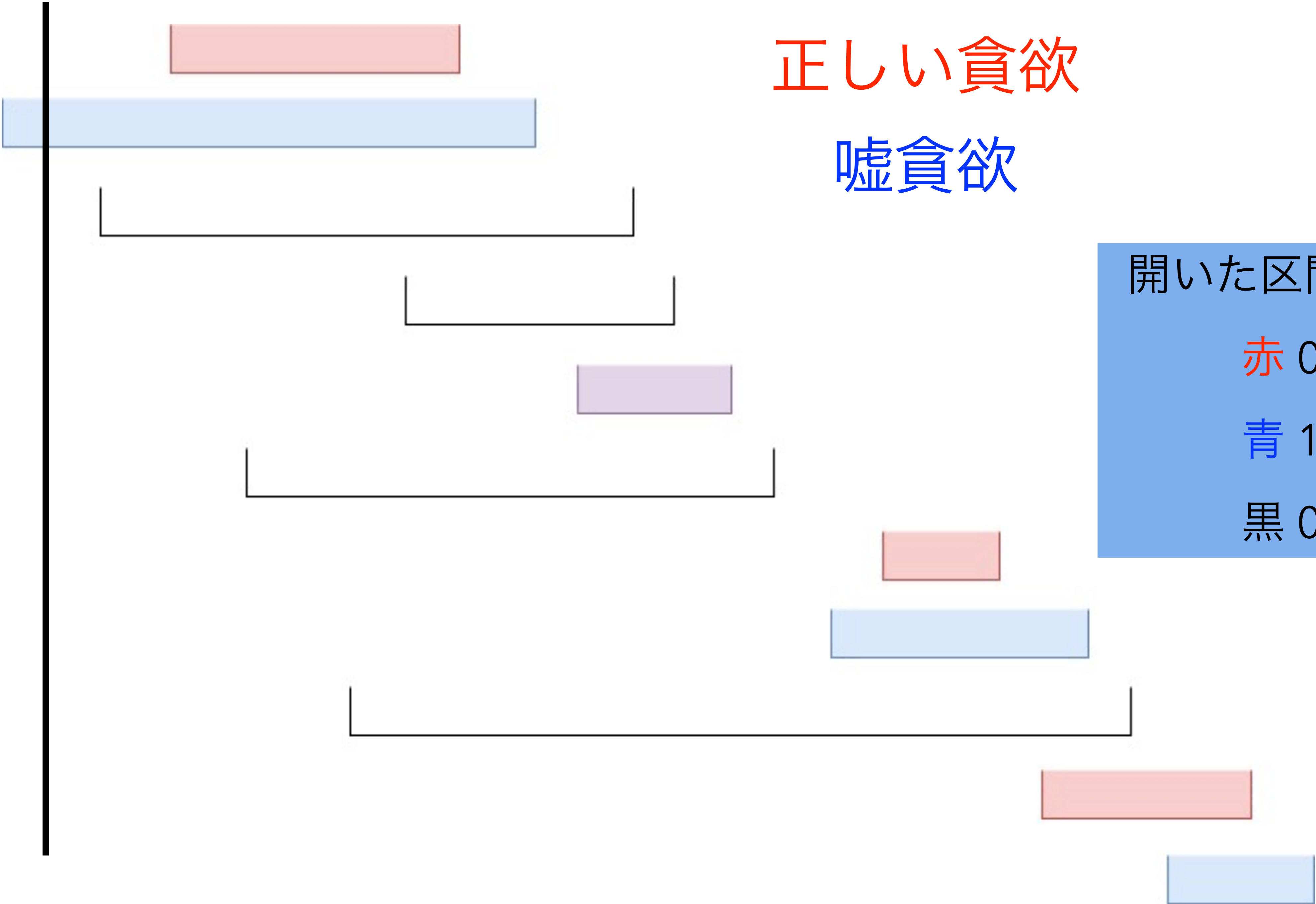
嘘貪欲

開いた区間の数

赤 0

青 1

黒 0



正しい貪欲

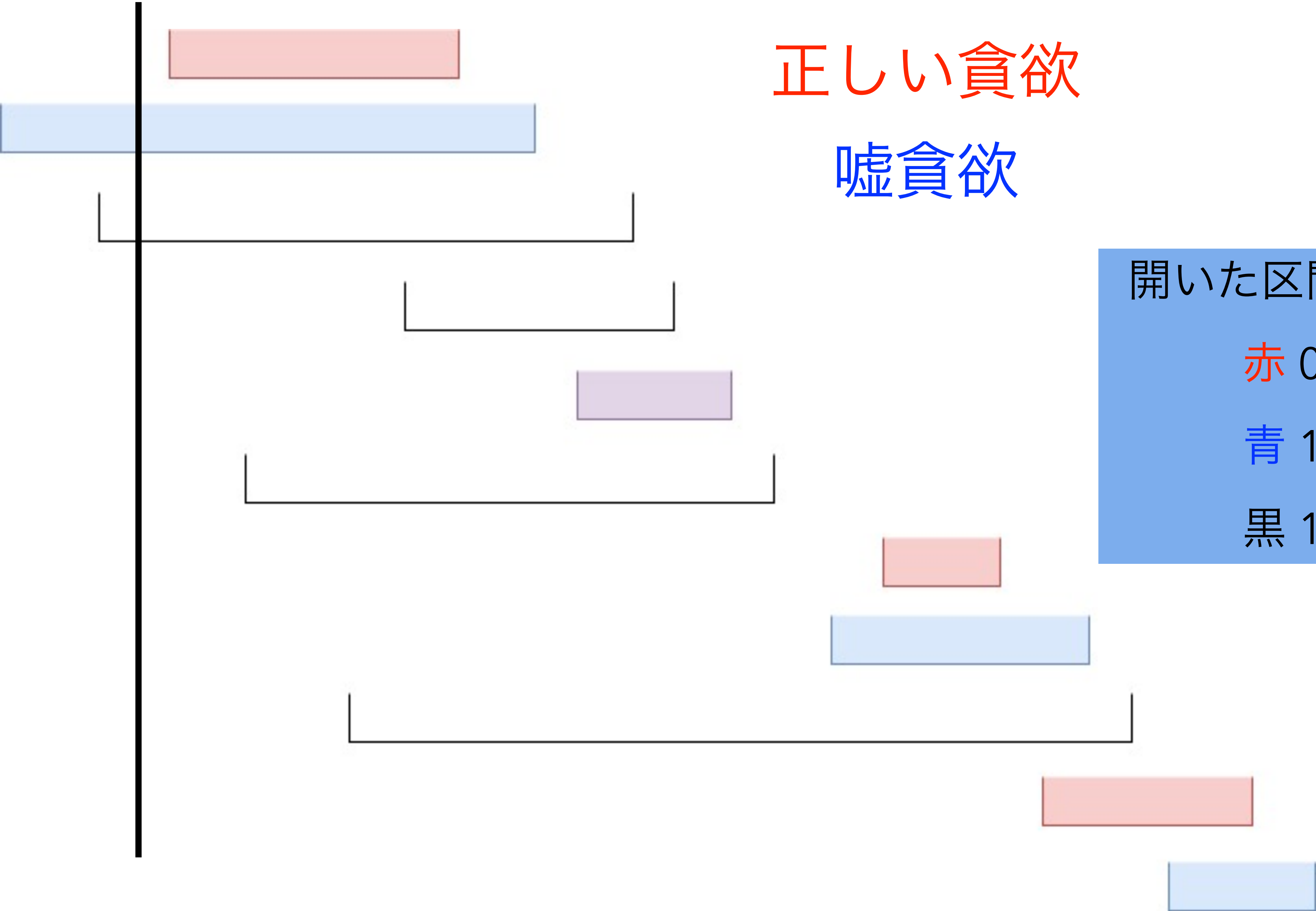
嘘貪欲

開いた区間の数

赤 0

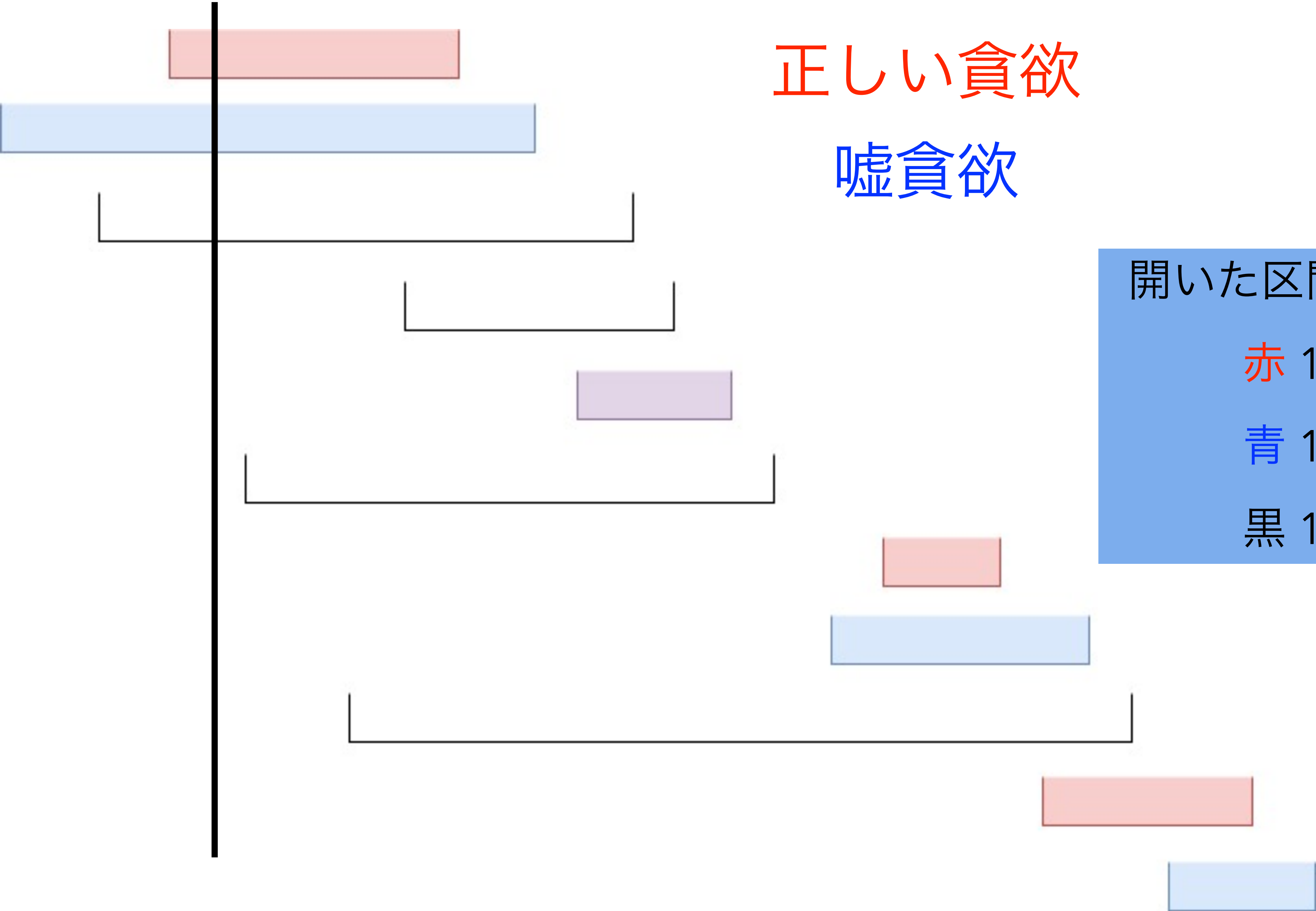
青 1

黒 1



正しい貪欲

嘘貪欲



開いた区間の数

赤 1

青 1

黒 1

正しい貪欲

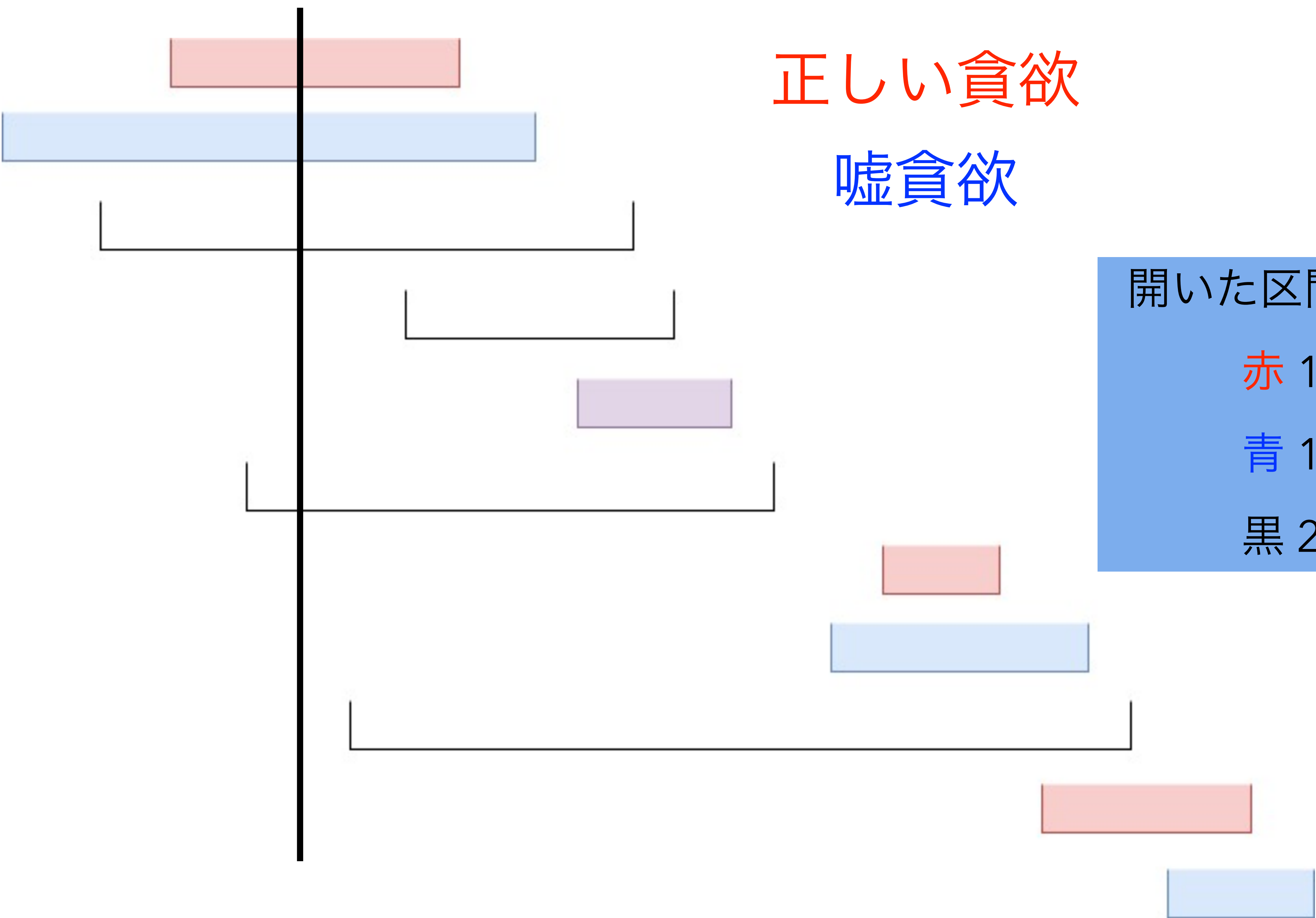
嘘貪欲

開いた区間の数

赤 1

青 1

黒 2



正しい貪欲

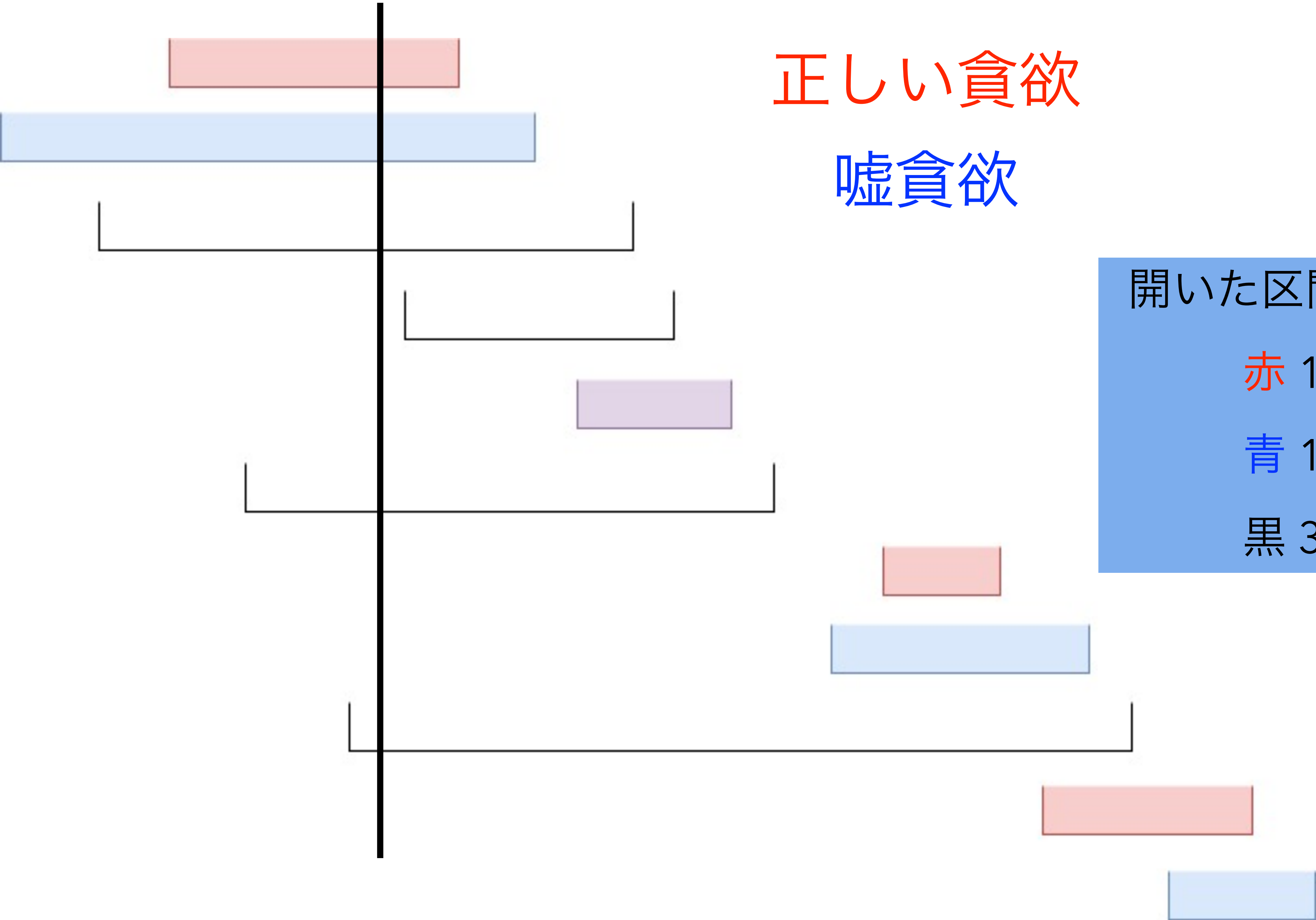
嘘貪欲

開いた区間の数

赤 1

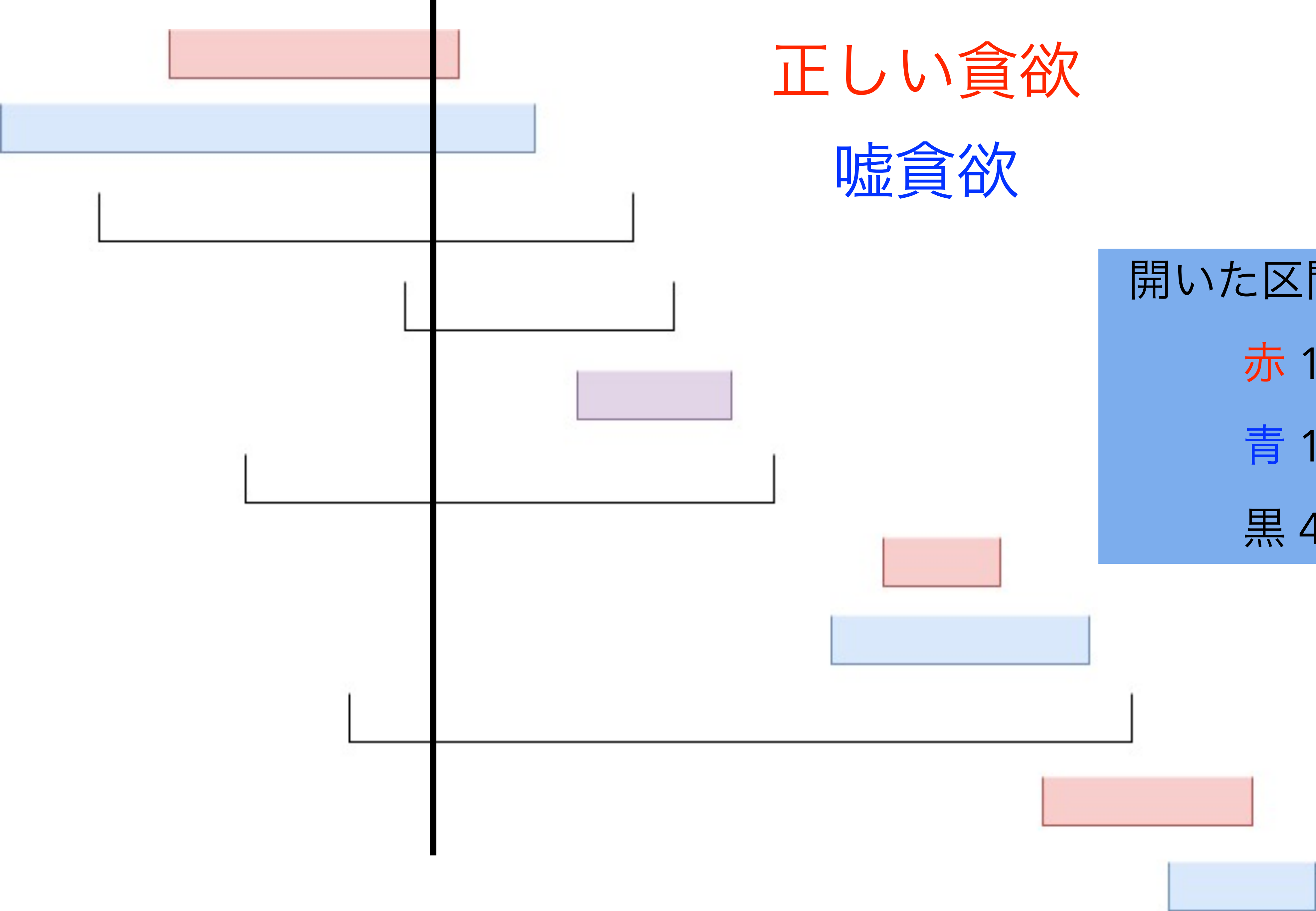
青 1

黒 3



正しい貪欲

嘘貪欲



開いた区間の数

赤 1

青 1

黒 4

正しい貪欲

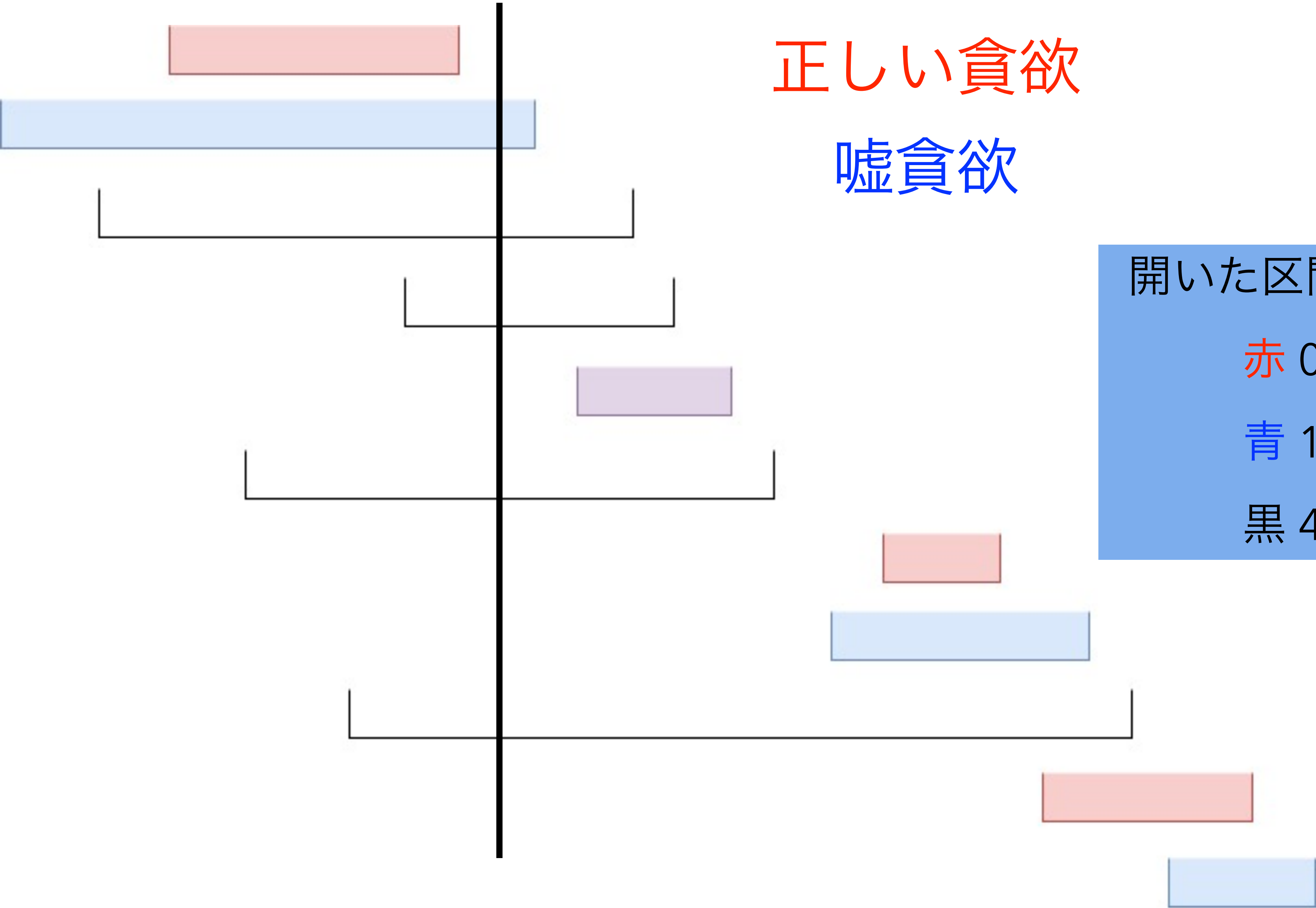
嘘貪欲

開いた区間の数

赤 0

青 1

黒 4



正しい貪欲

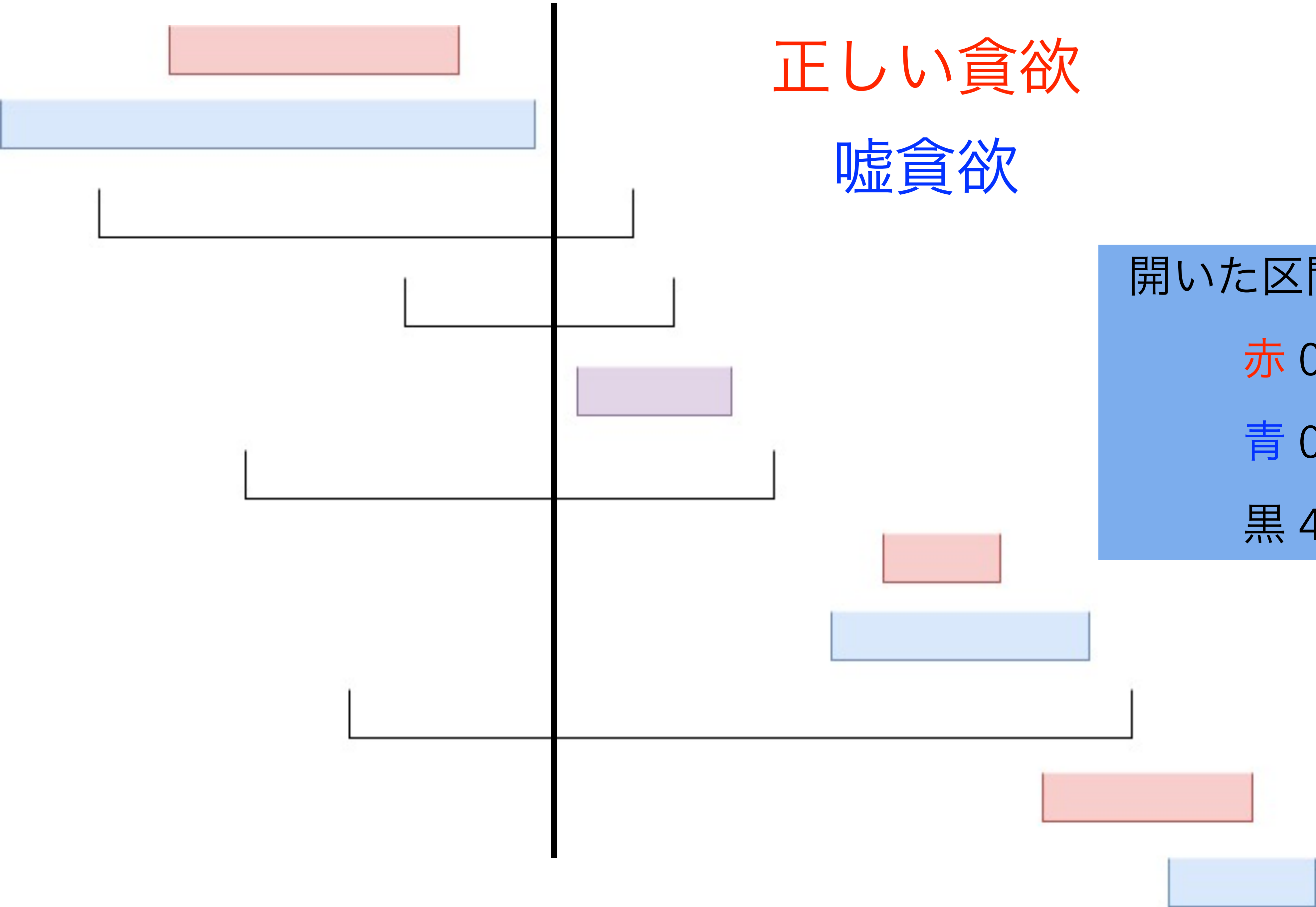
嘘貪欲

開いた区間の数

赤 0

青 0

黒 4



正しい貪欲

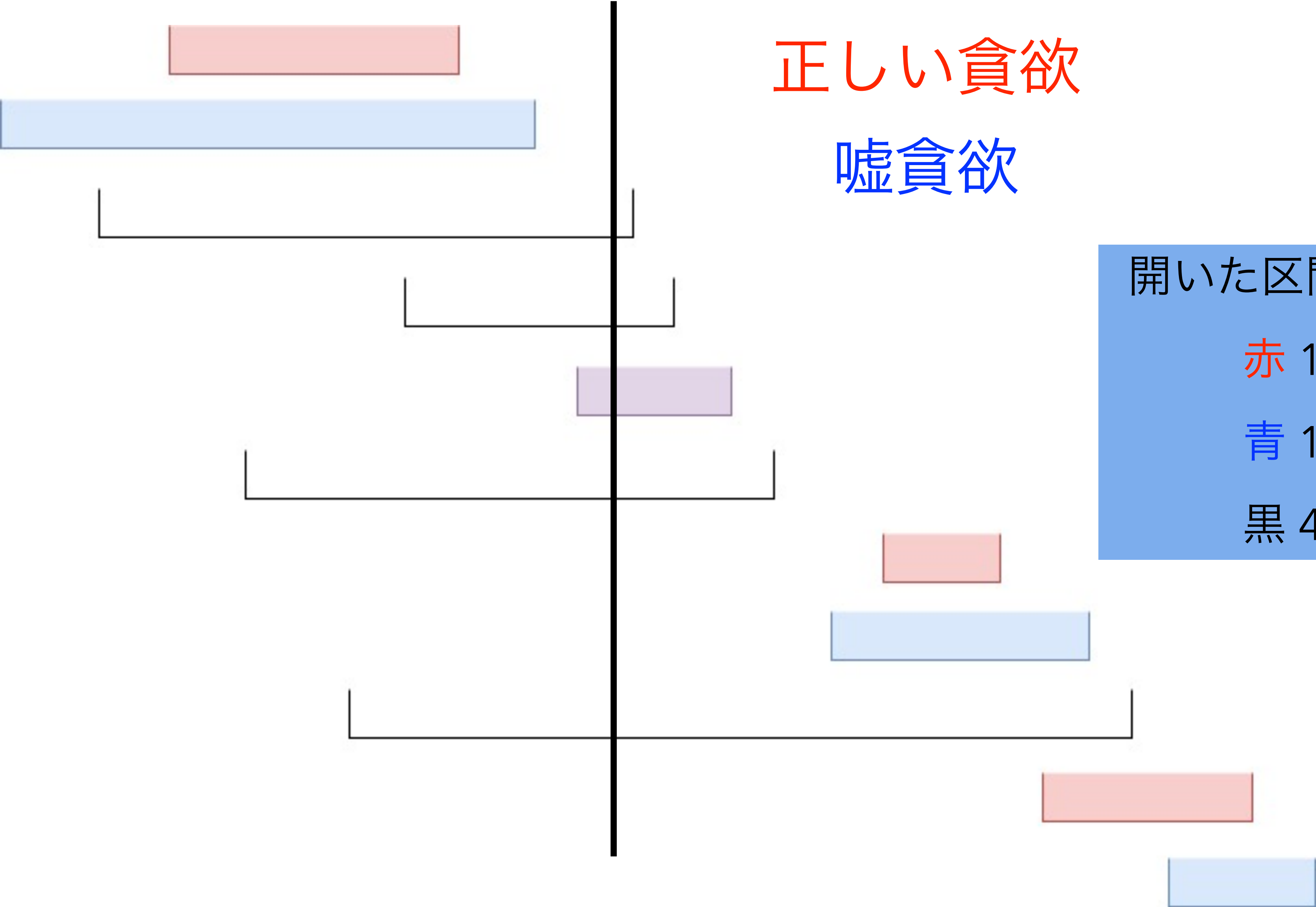
嘘貪欲

開いた区間の数

赤 1

青 1

黒 4



正しい貪欲

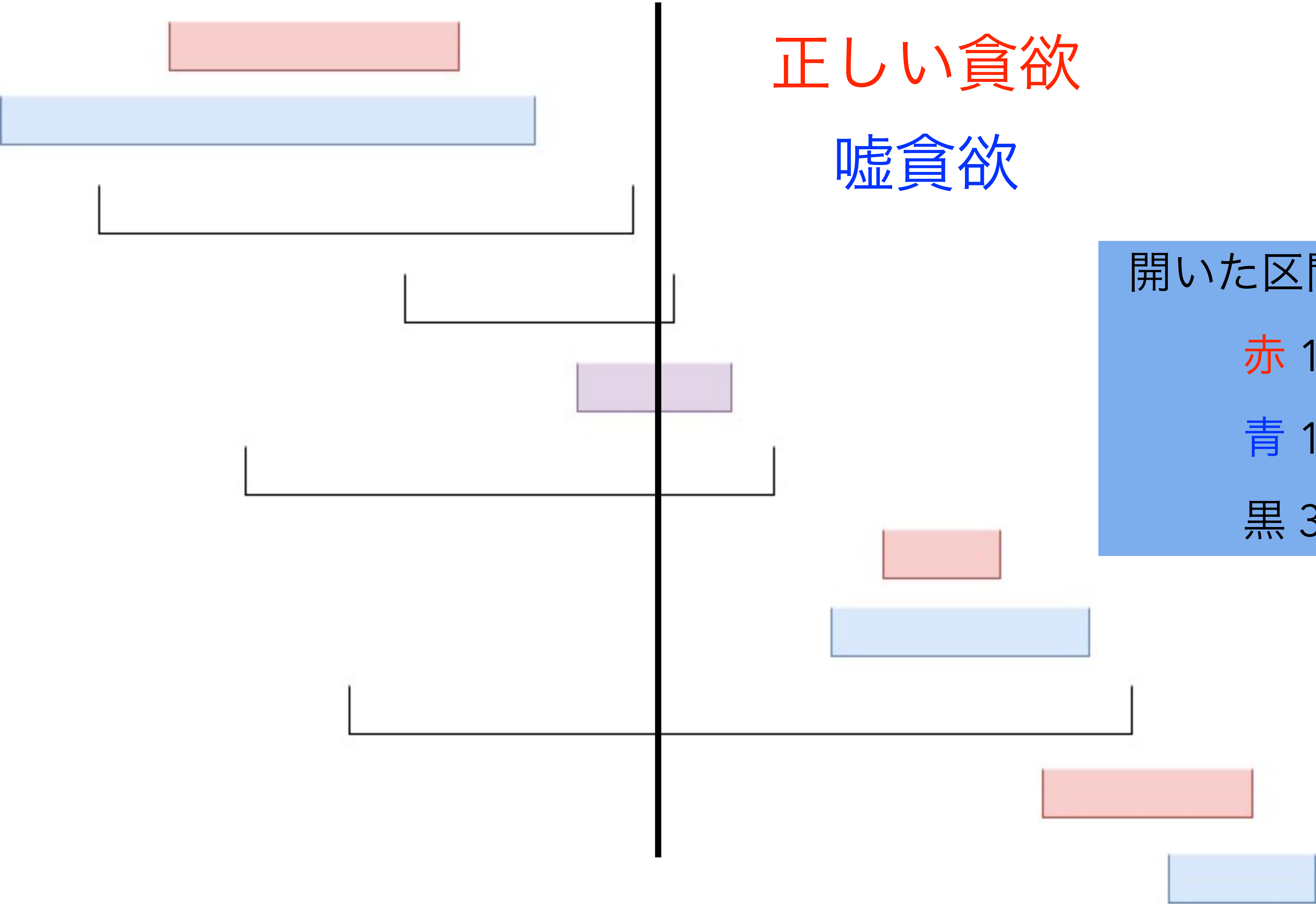
嘘貪欲

開いた区間の数

赤 1

青 1

黒 3



正しい貪欲

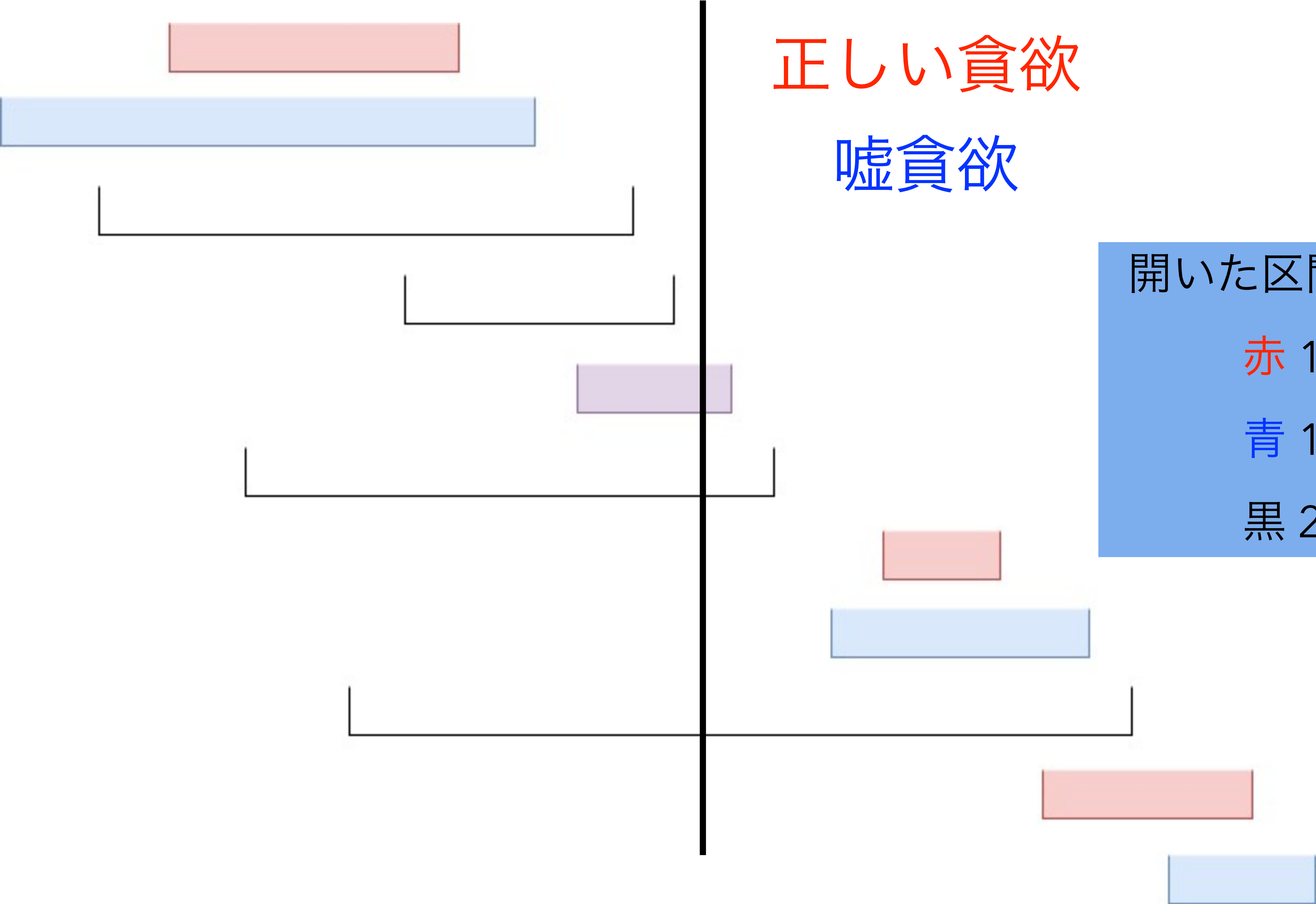
嘘貪欲

開いた区間の数

赤 1

青 1

黒 2



正しい貪欲

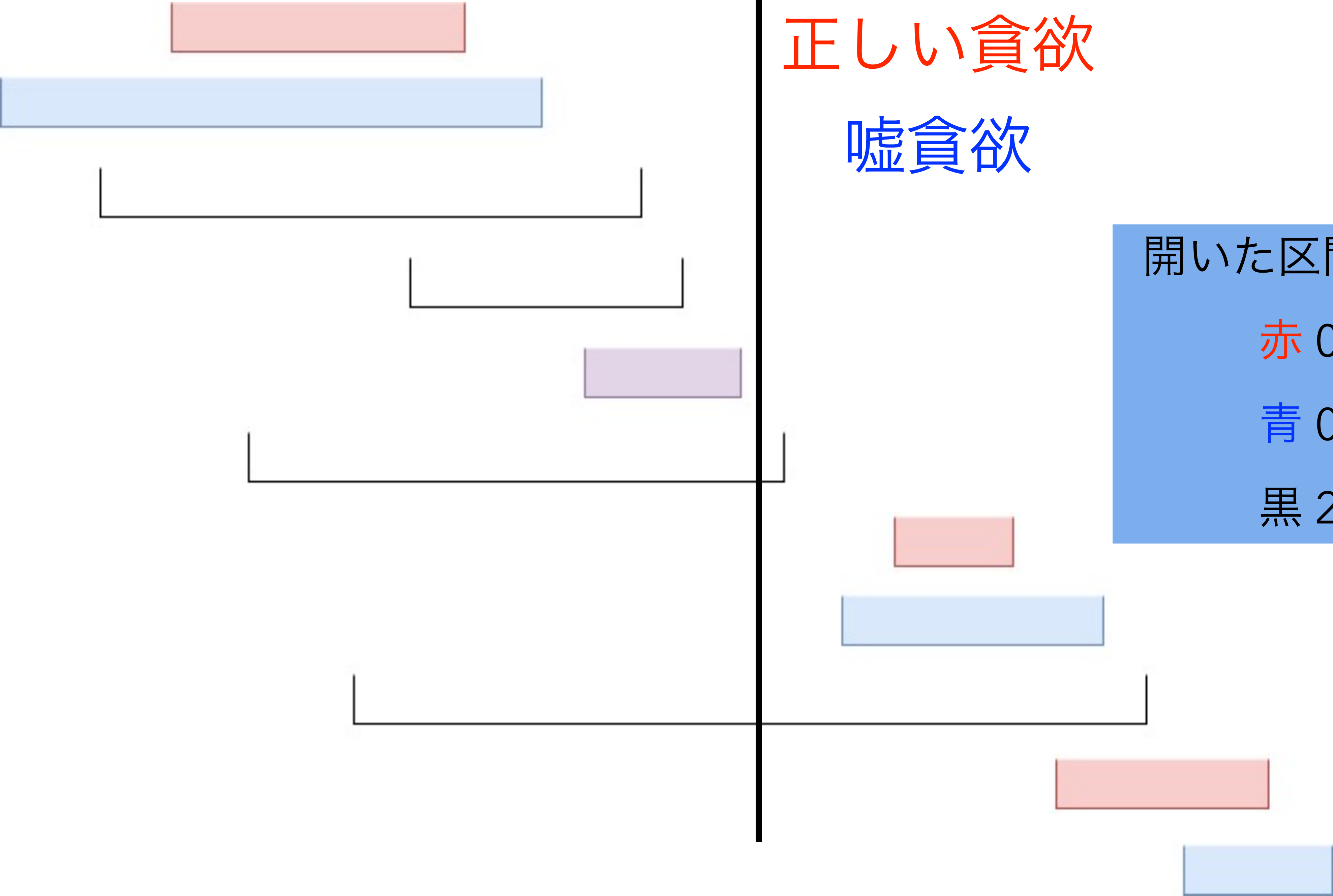
嘘貪欲

開いた区間の数

赤 0

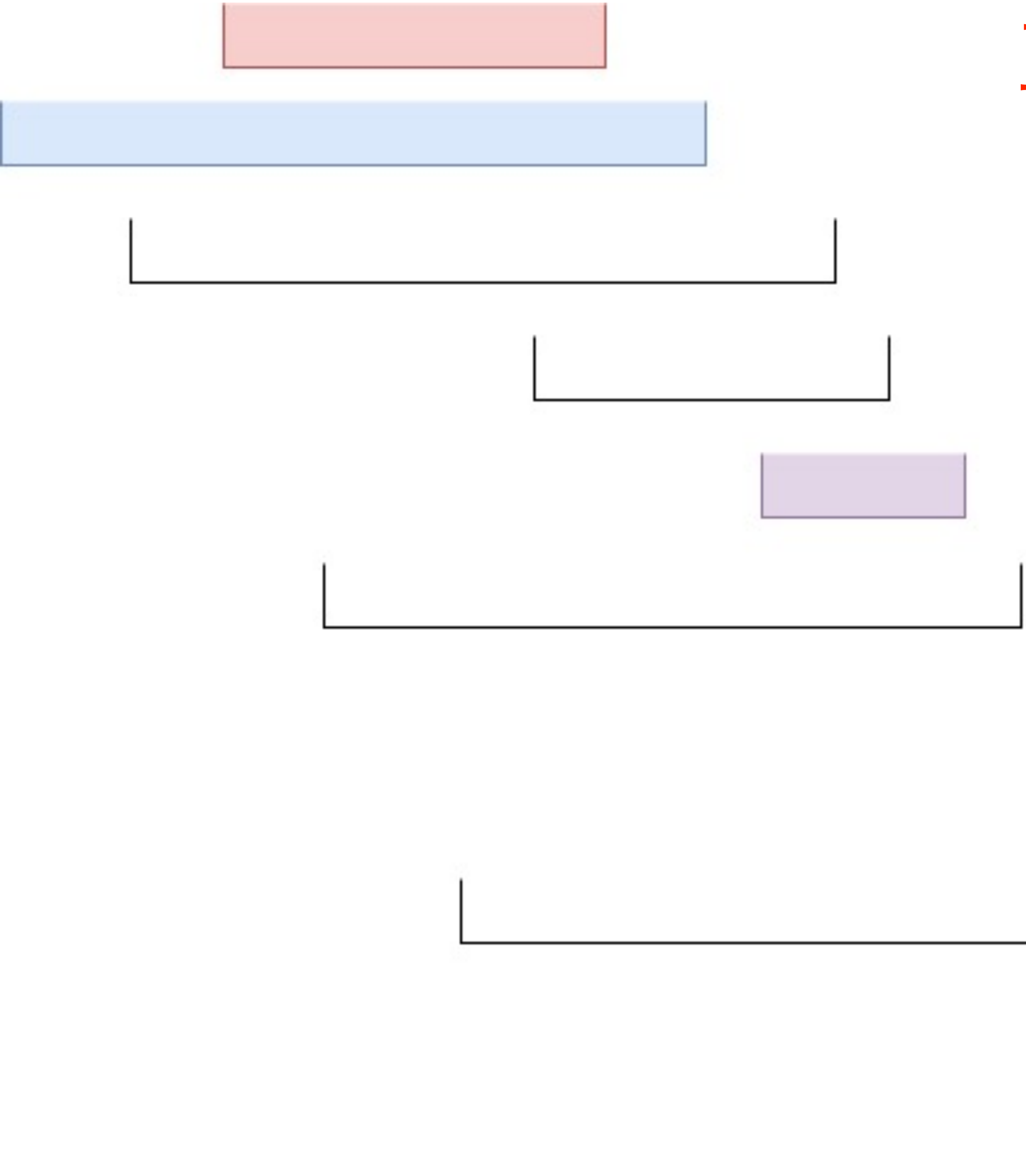
青 0

黒 2



正しい貪欲

嘘貪欲



開いた区間の数

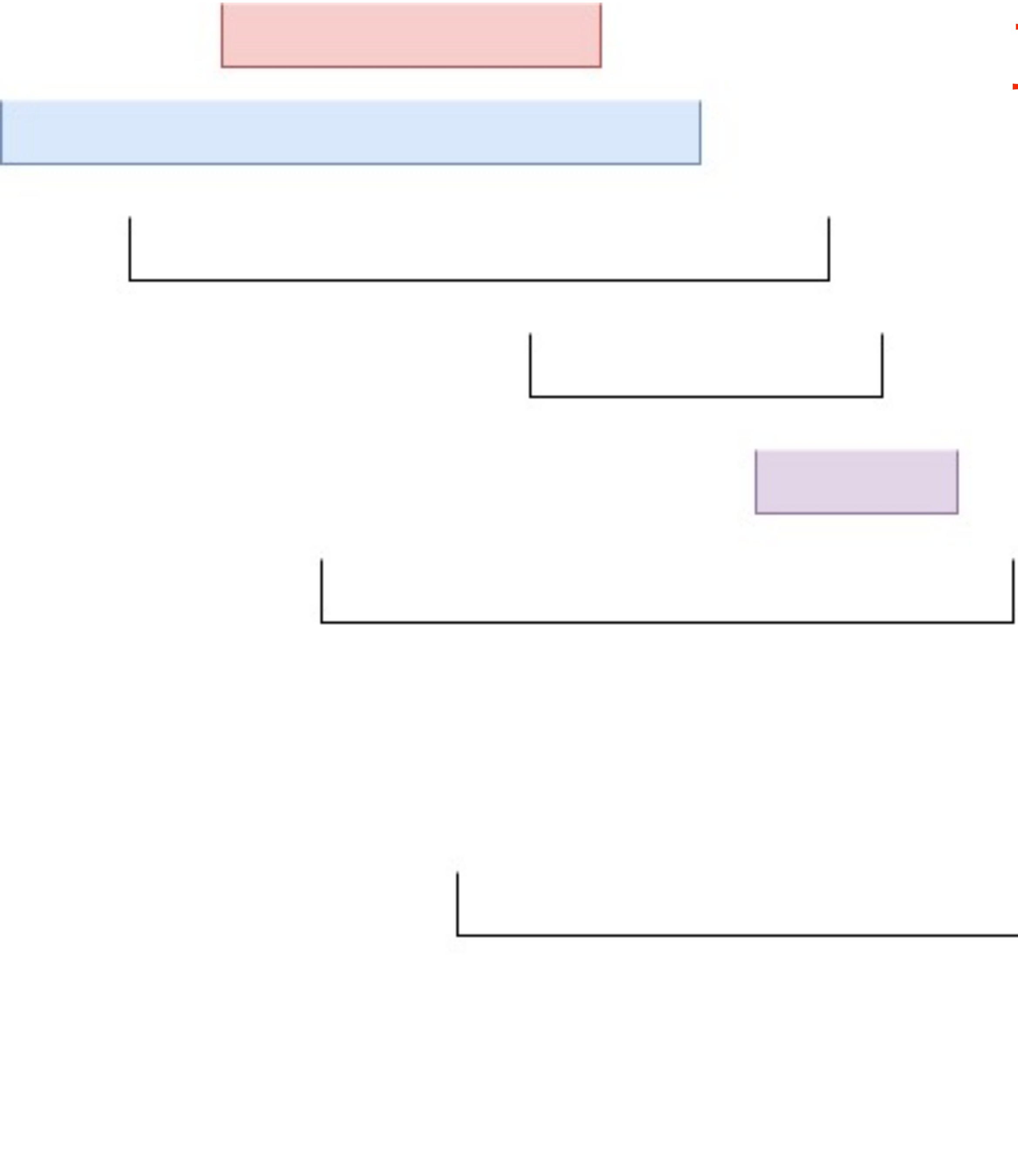
赤 0

青 0

黒 1

正しい貪欲

嘘貪欲



開いた区間の数

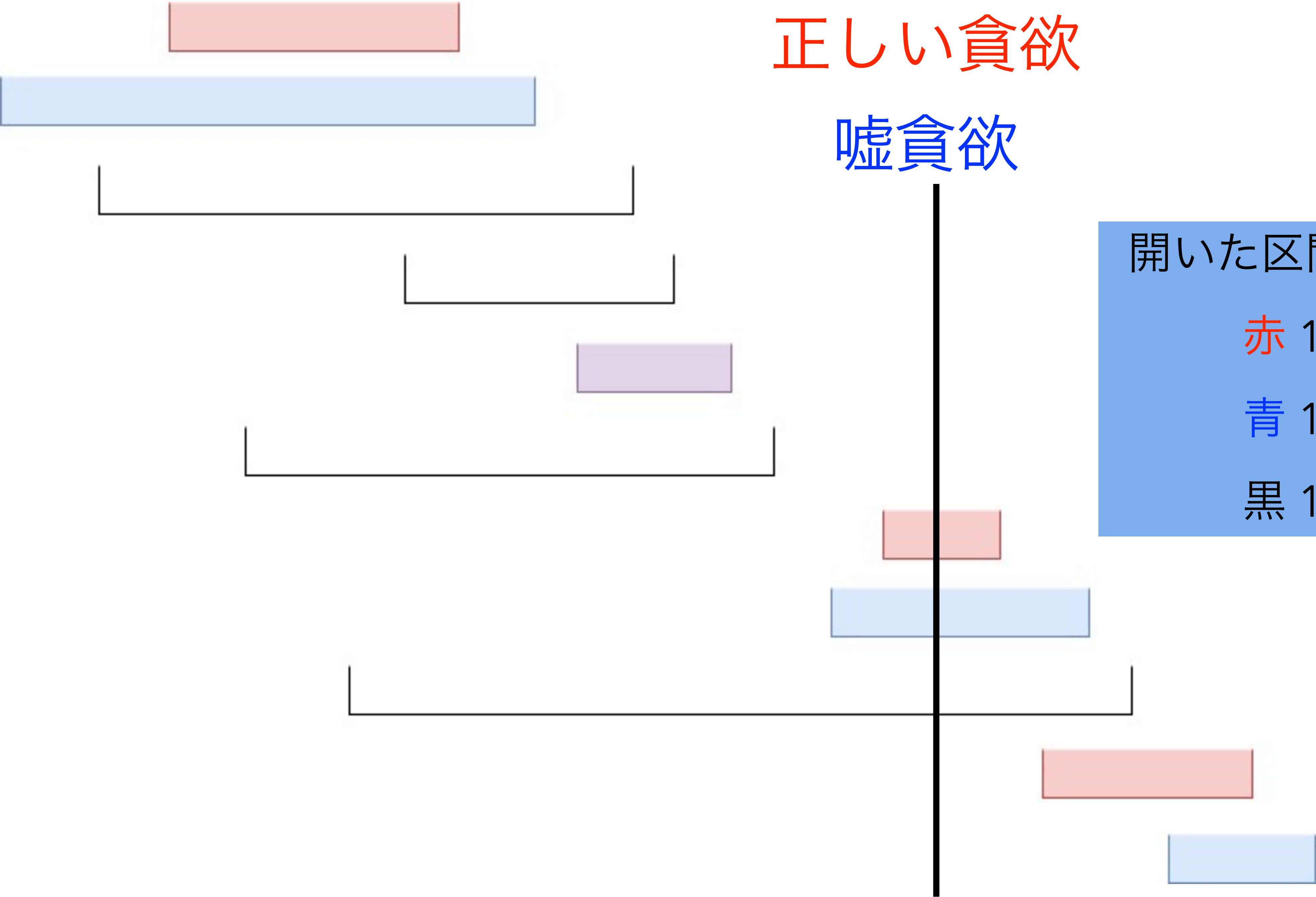
赤 0

青 1

黒 1

正しい貪欲

嘘貪欲



開いた区間の数

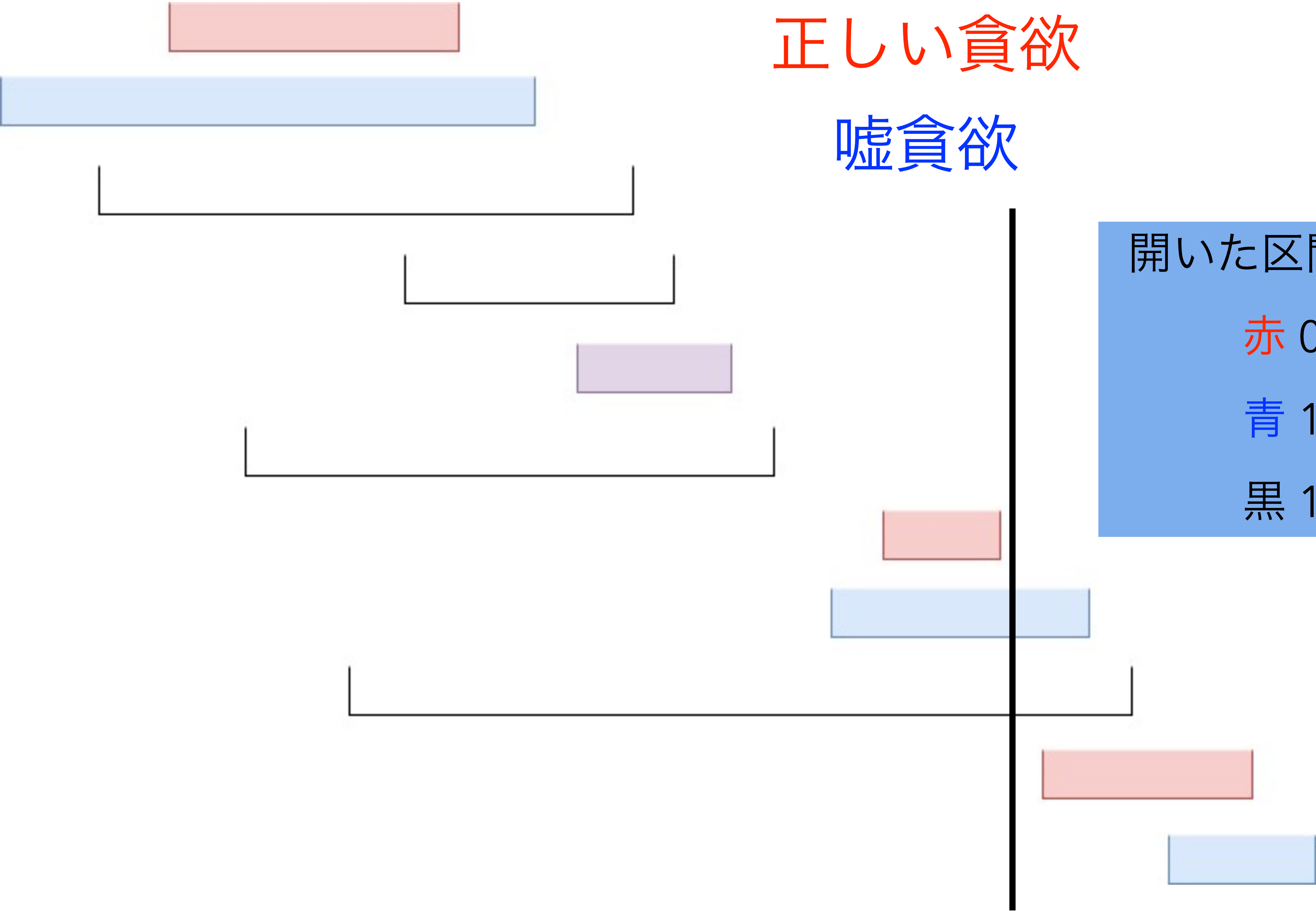
赤 1

青 1

黒 1

正しい貪欲

嘘貪欲



開いた区間の数

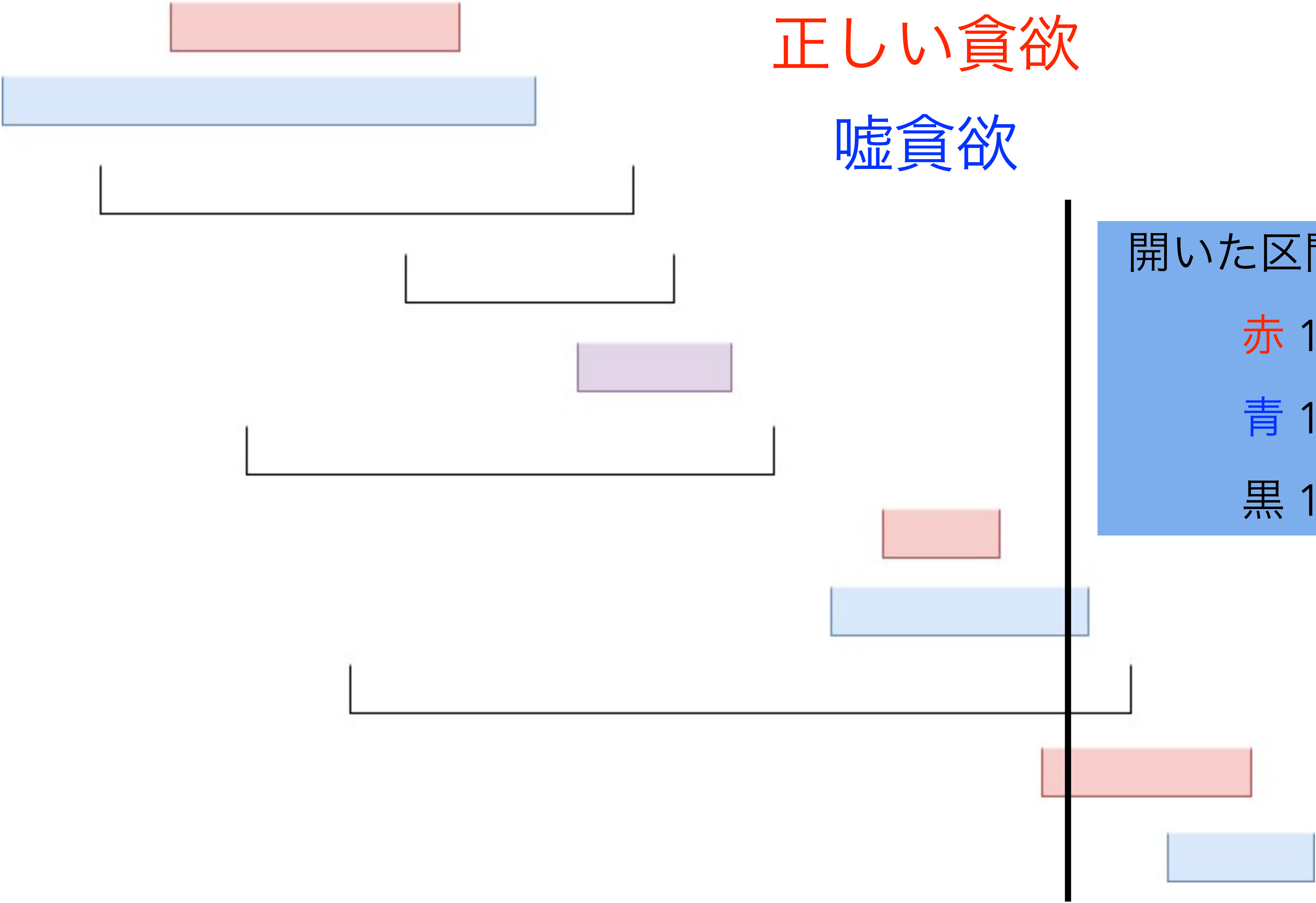
赤 0

青 1

黒 1

正しい貪欲

嘘貪欲



開いた区間の数

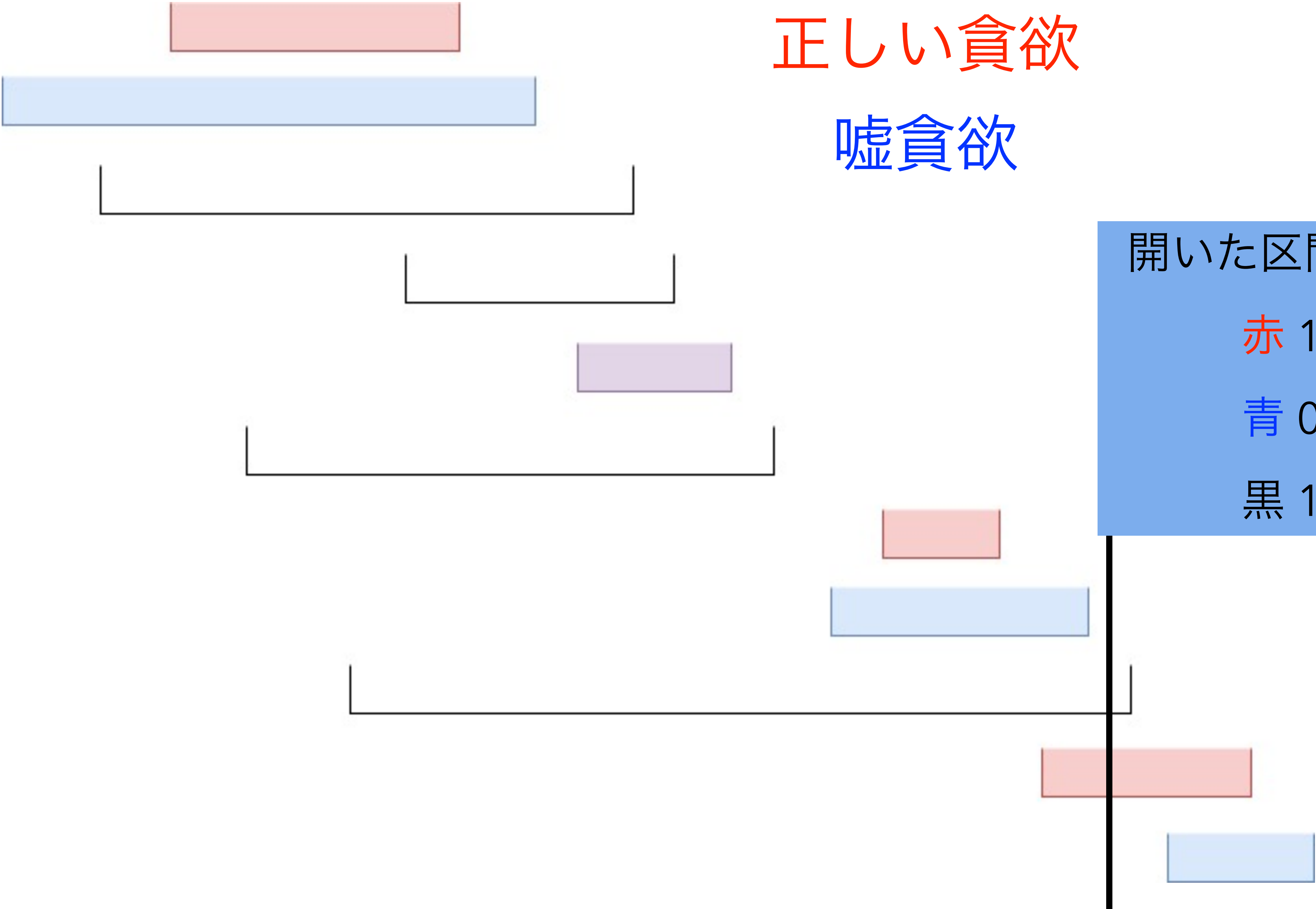
赤 1

青 1

黒 1

正しい貪欲

嘘貪欲



開いた区間の数

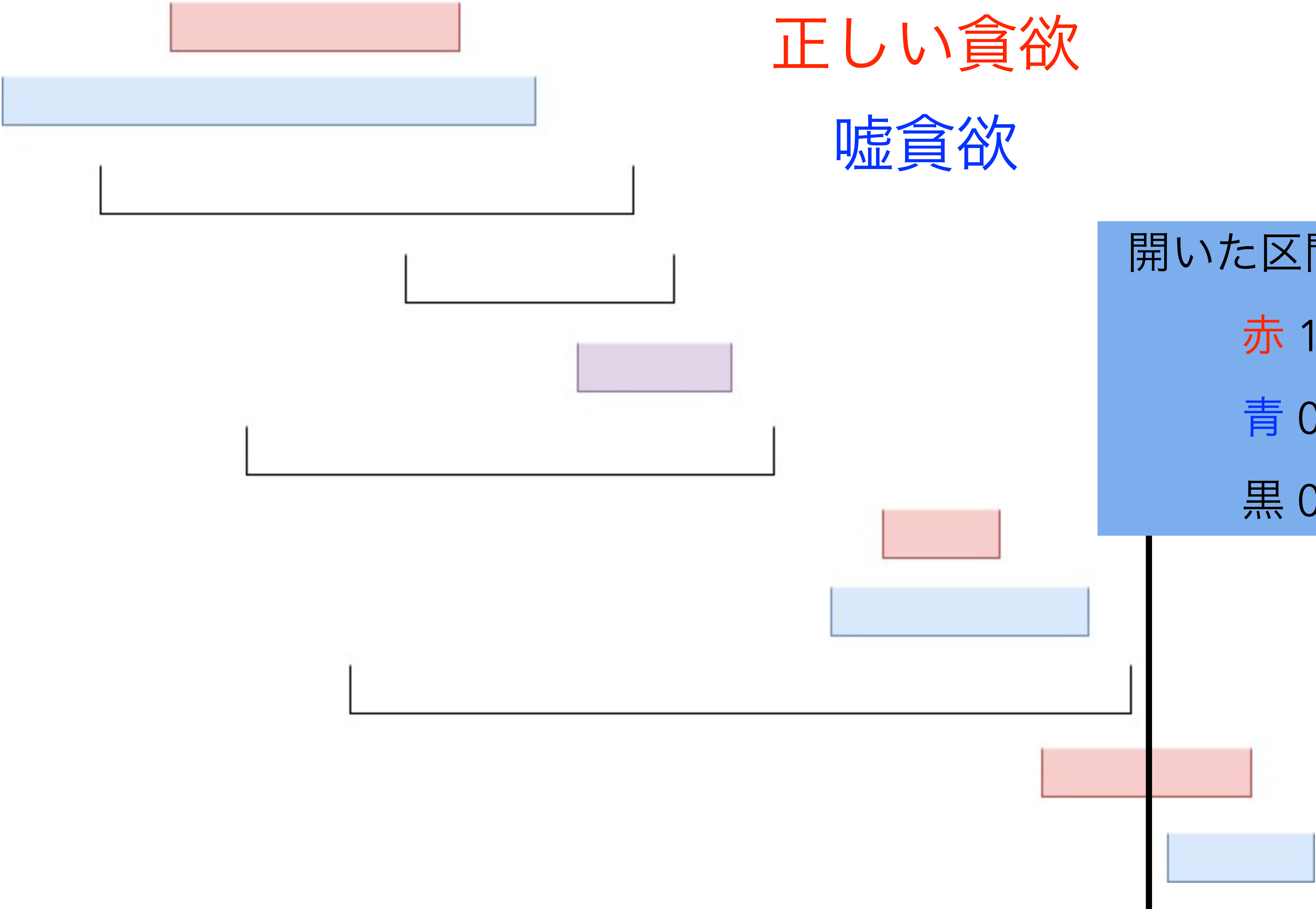
赤 1

青 0

黒 1

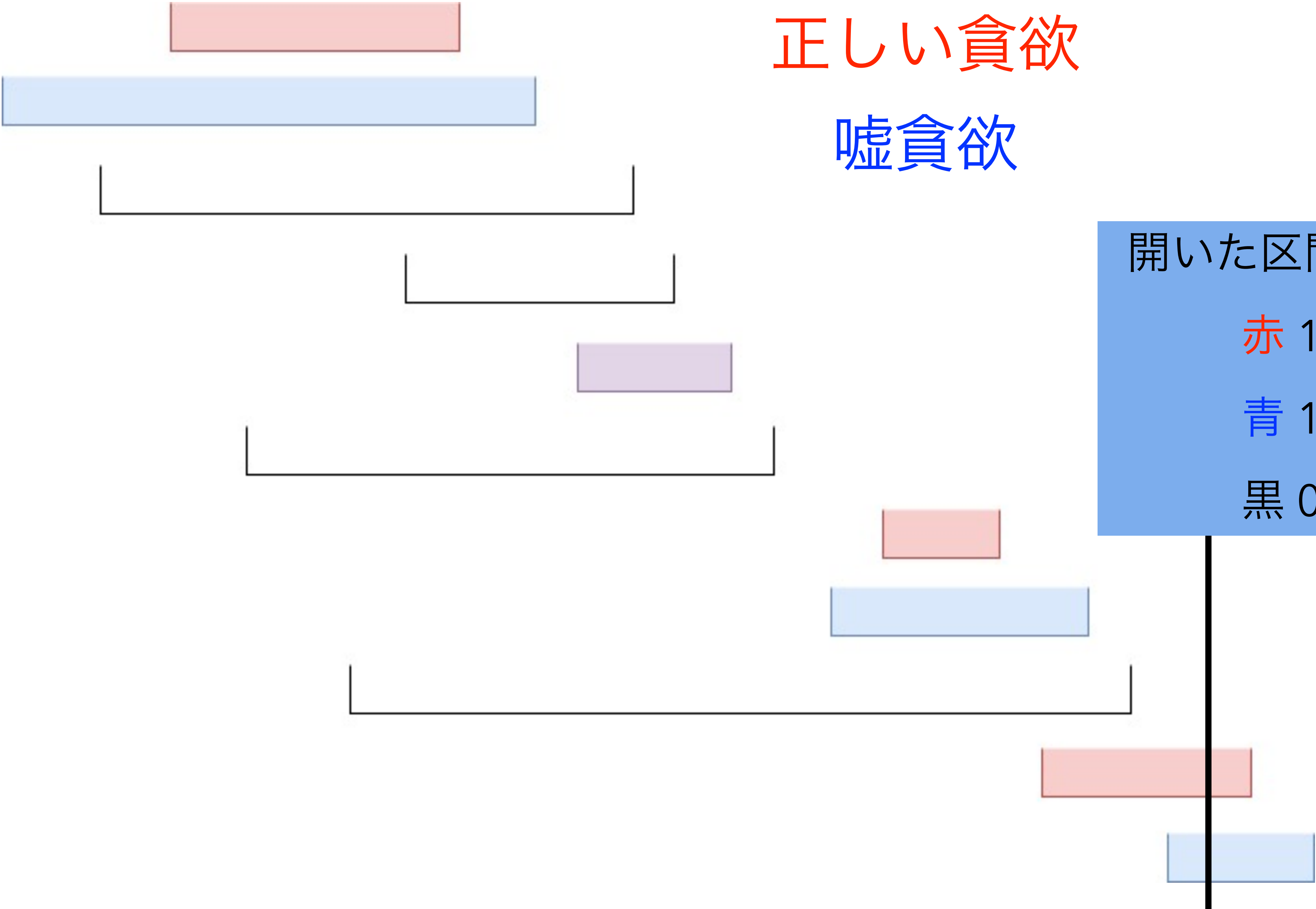
正しい貪欲

嘘貪欲



正しい貪欲

嘘貪欲



開いた区間の数

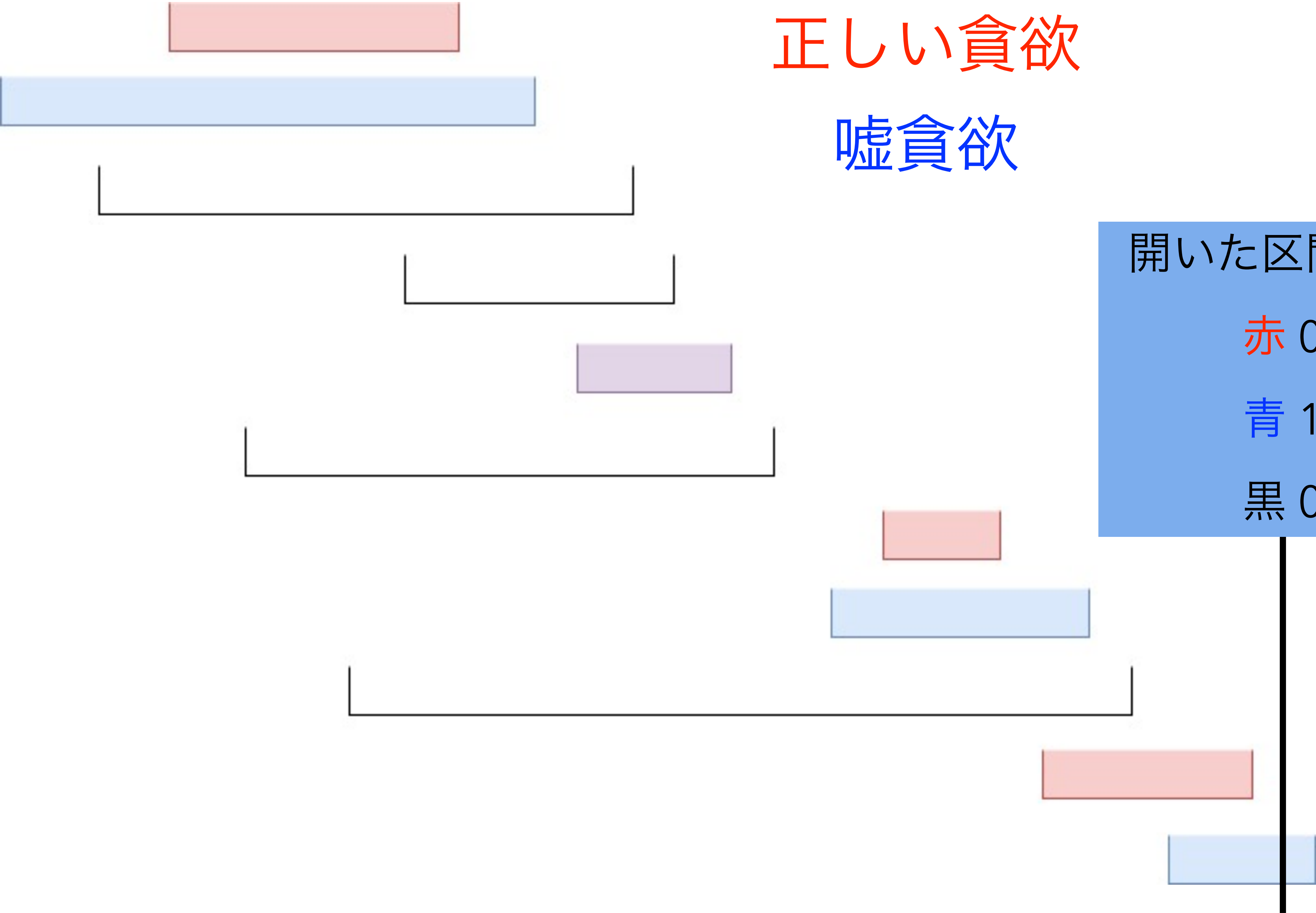
赤 1

青 1

黒 0

正しい貪欲

嘘貪欲



開いた区間の数

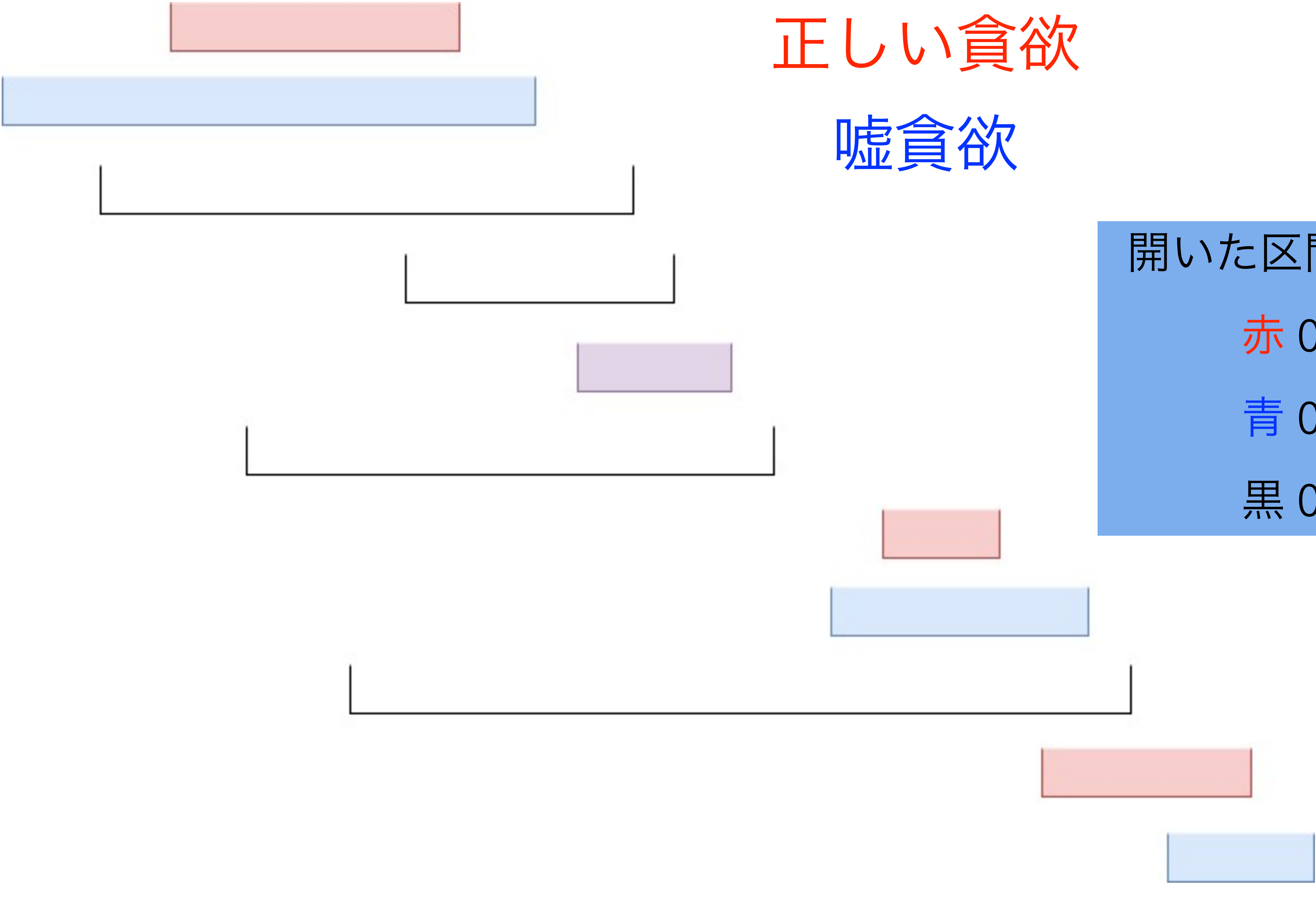
赤 0

青 1

黒 0

正しい貪欲

嘘貪欲



小課題 4 ($N \leq 300$)

- 持つべき情報は、
 - 1 から $2N$ までの数のうち、どこまで割り当てたか
 - 開いた黒い区間の数
 - ↑のうち、最後に赤い区間を閉じたときより後に開けたものの数
 - 赤、青の区間に関する“状態”
- 状態って？
 - ざっくりいうと「赤い区間が開いているかどうか」「青い区間が開いているかどうか」だが、それだけでは足りない

正しい貪欲

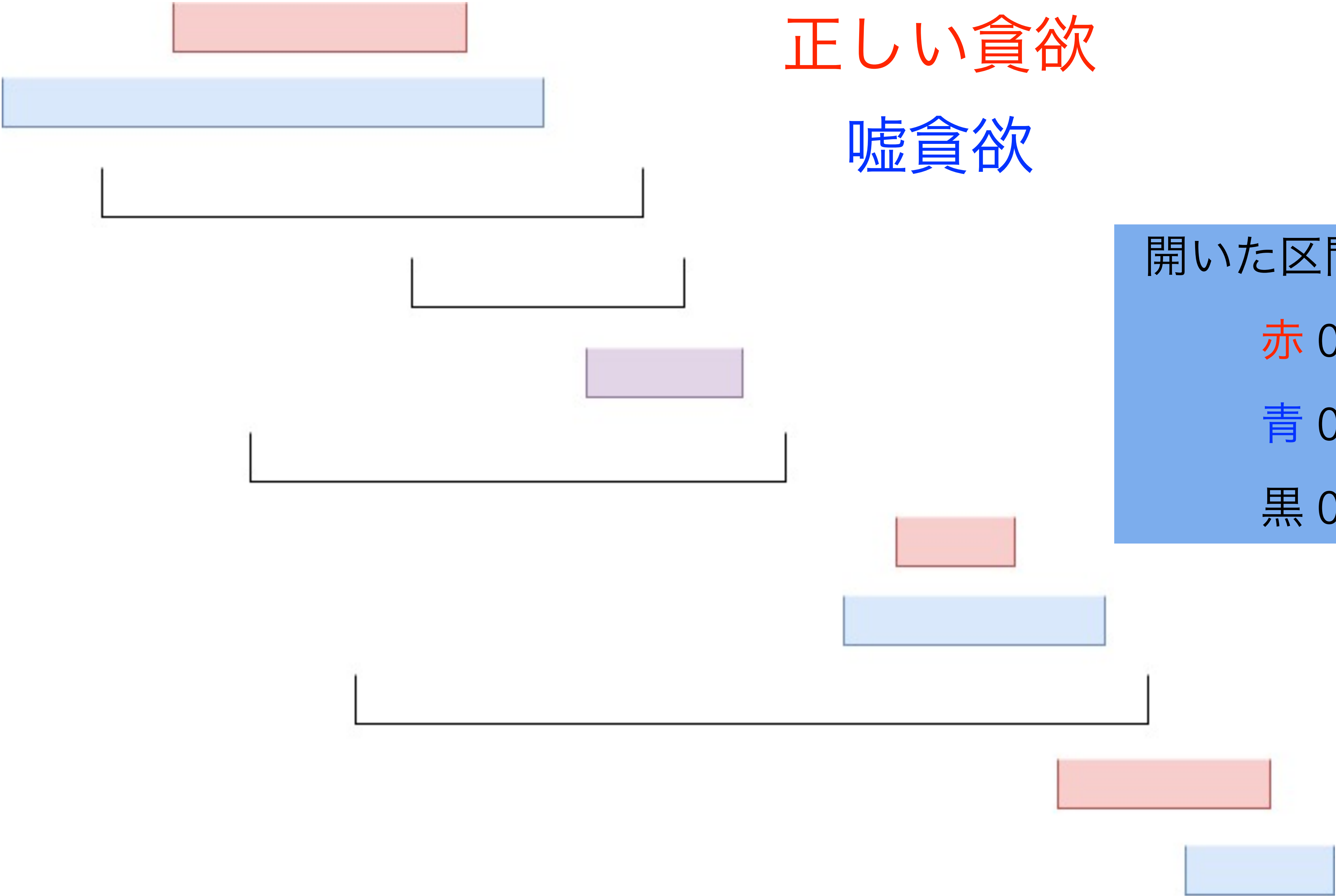
嘘貪欲

開いた区間の数

赤 0

青 0

黒 0



正しい貪欲

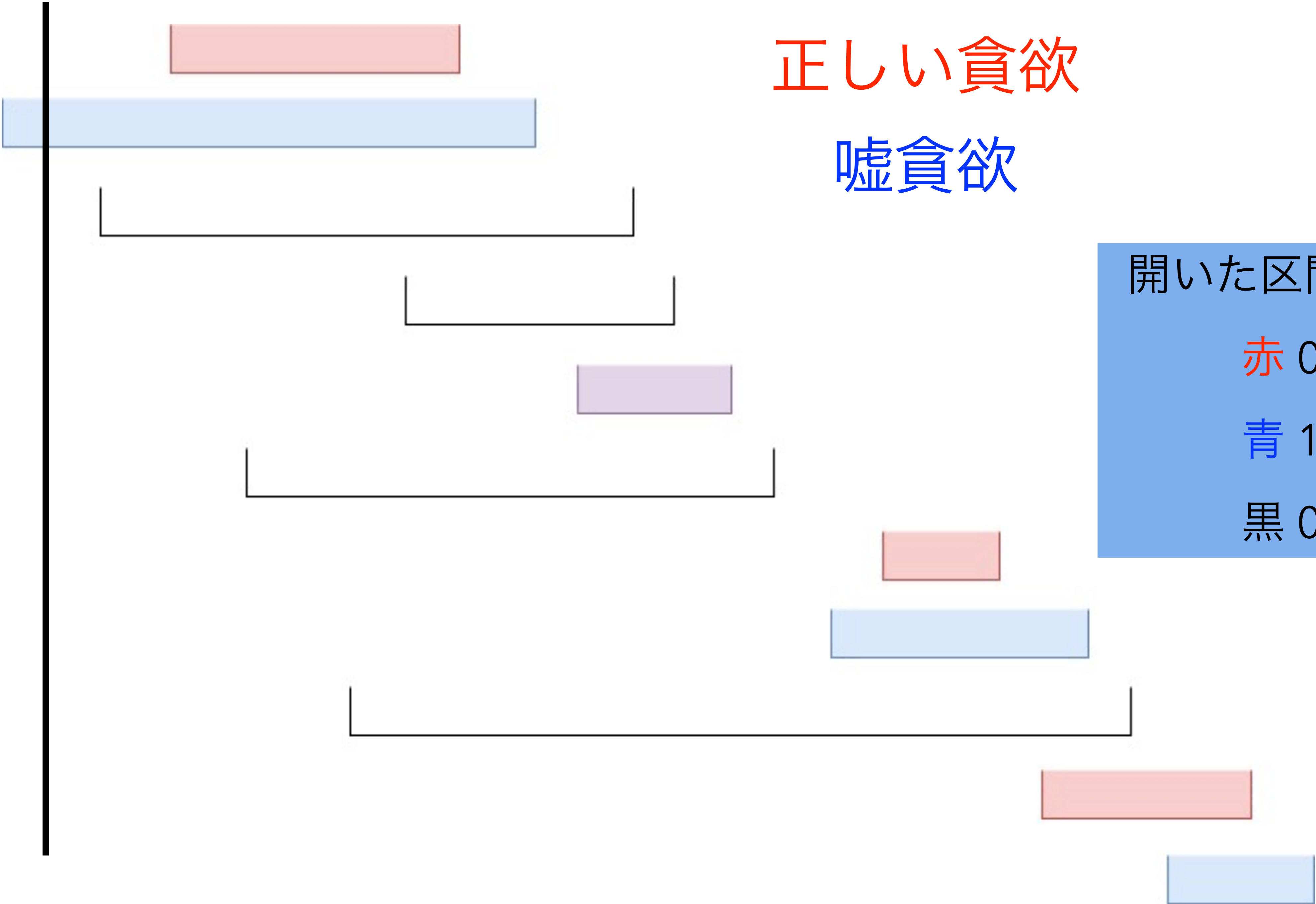
嘘貪欲

開いた区間の数

赤 0

青 1

黒 0



正しい貪欲

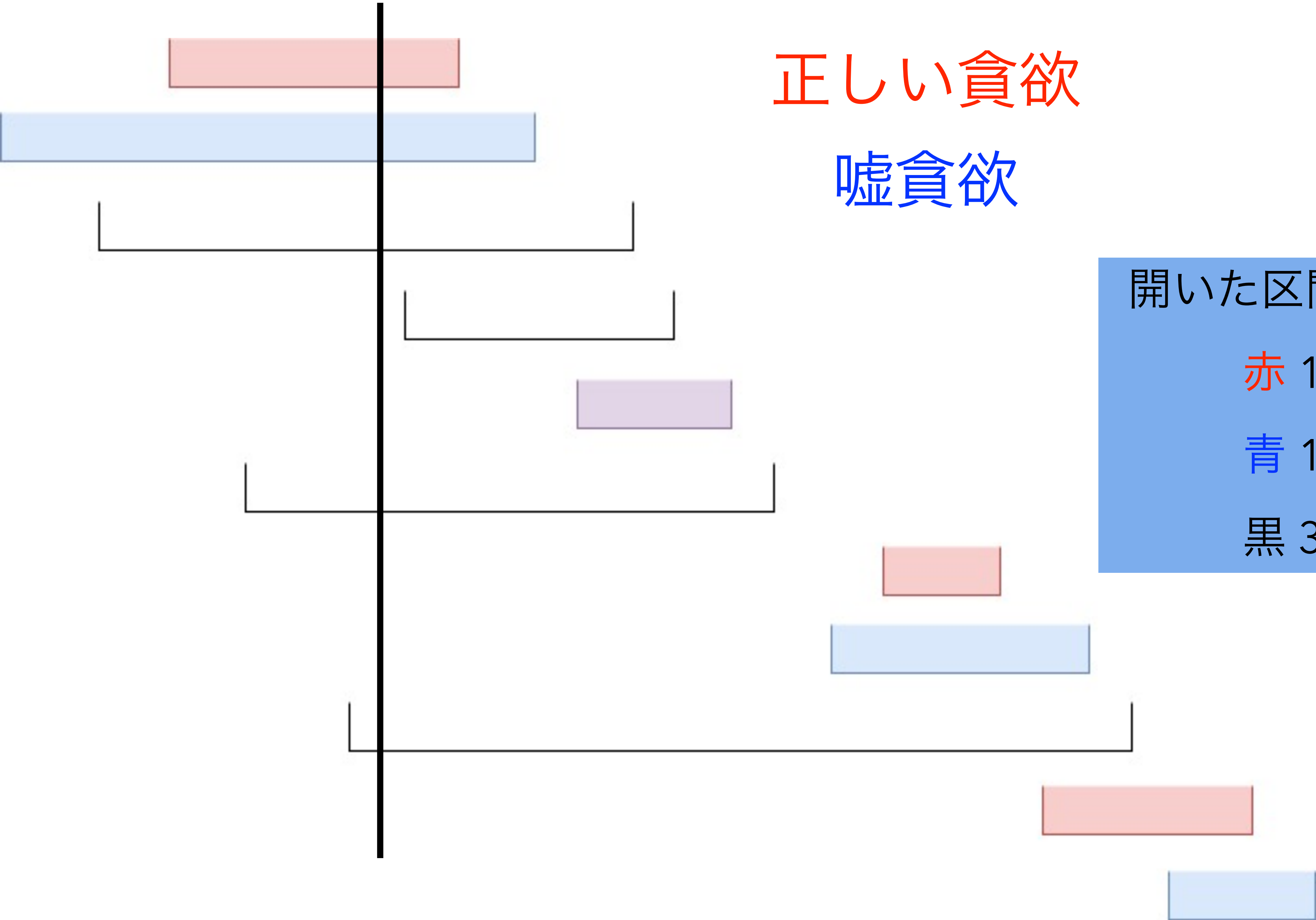
嘘貪欲

開いた区間の数

赤 1

青 1

黒 3



正しい貪欲

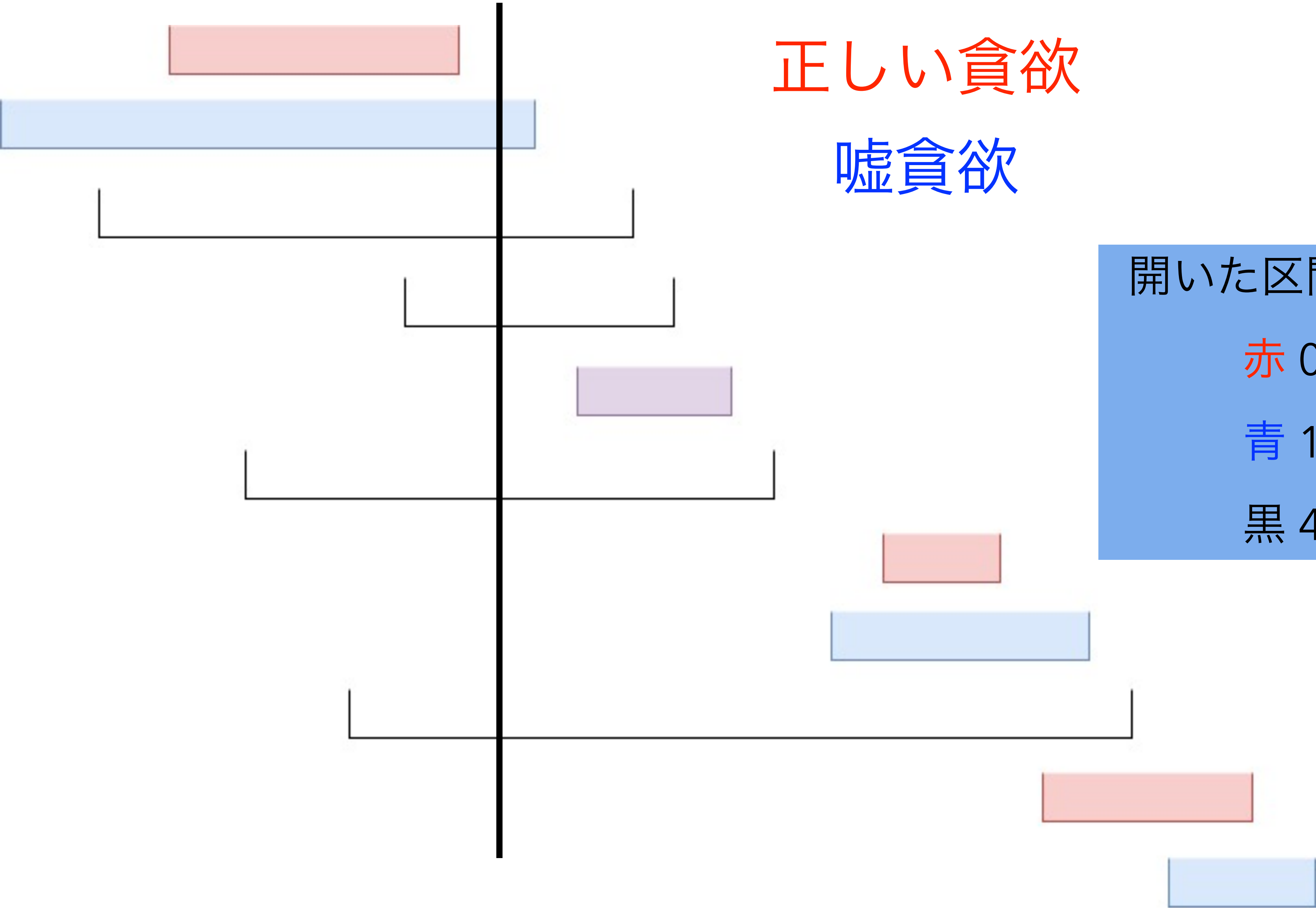
嘘貪欲

開いた区間の数

赤 0

青 1

黒 4



正しい貪欲

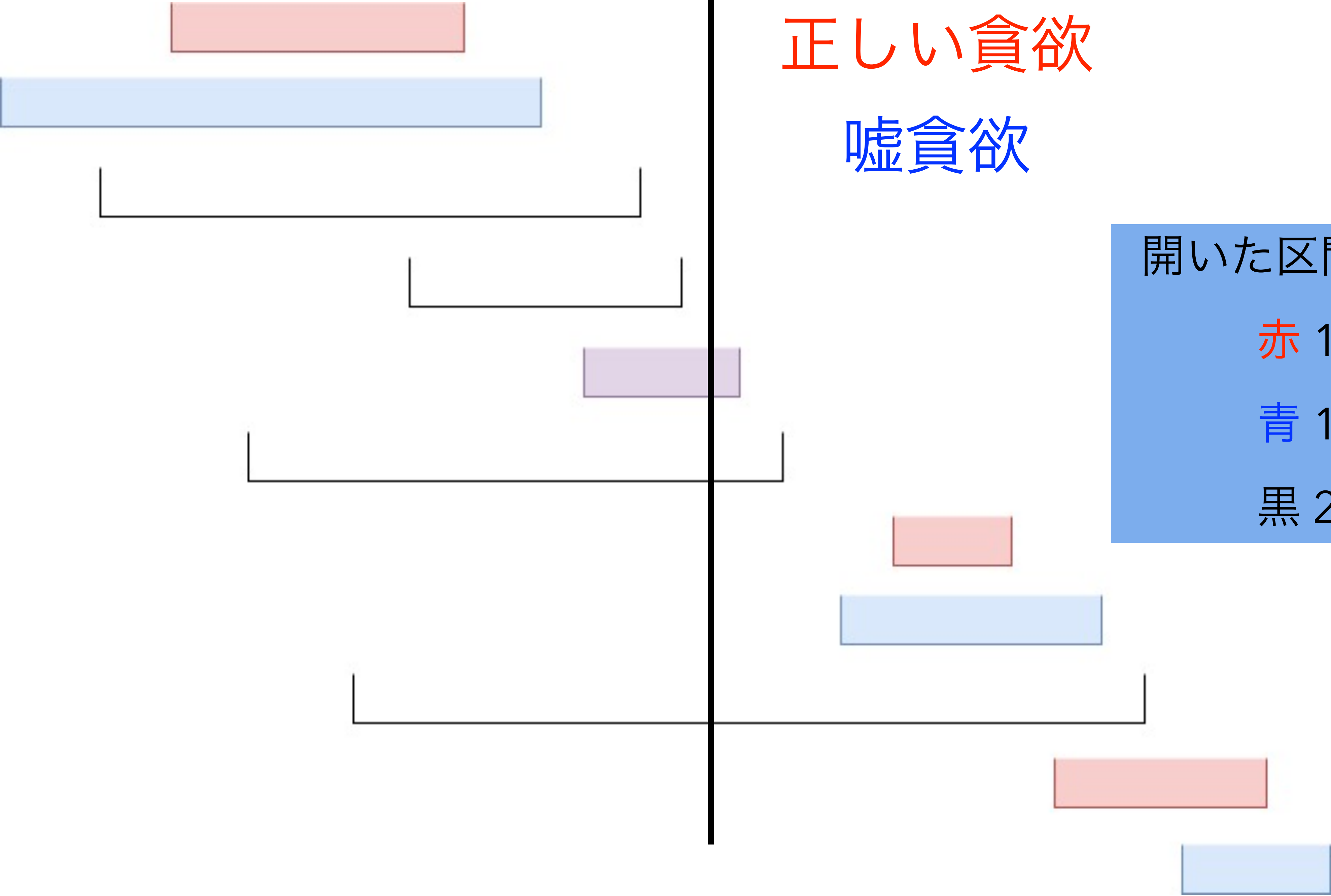
嘘貪欲

開いた区間の数

赤 1

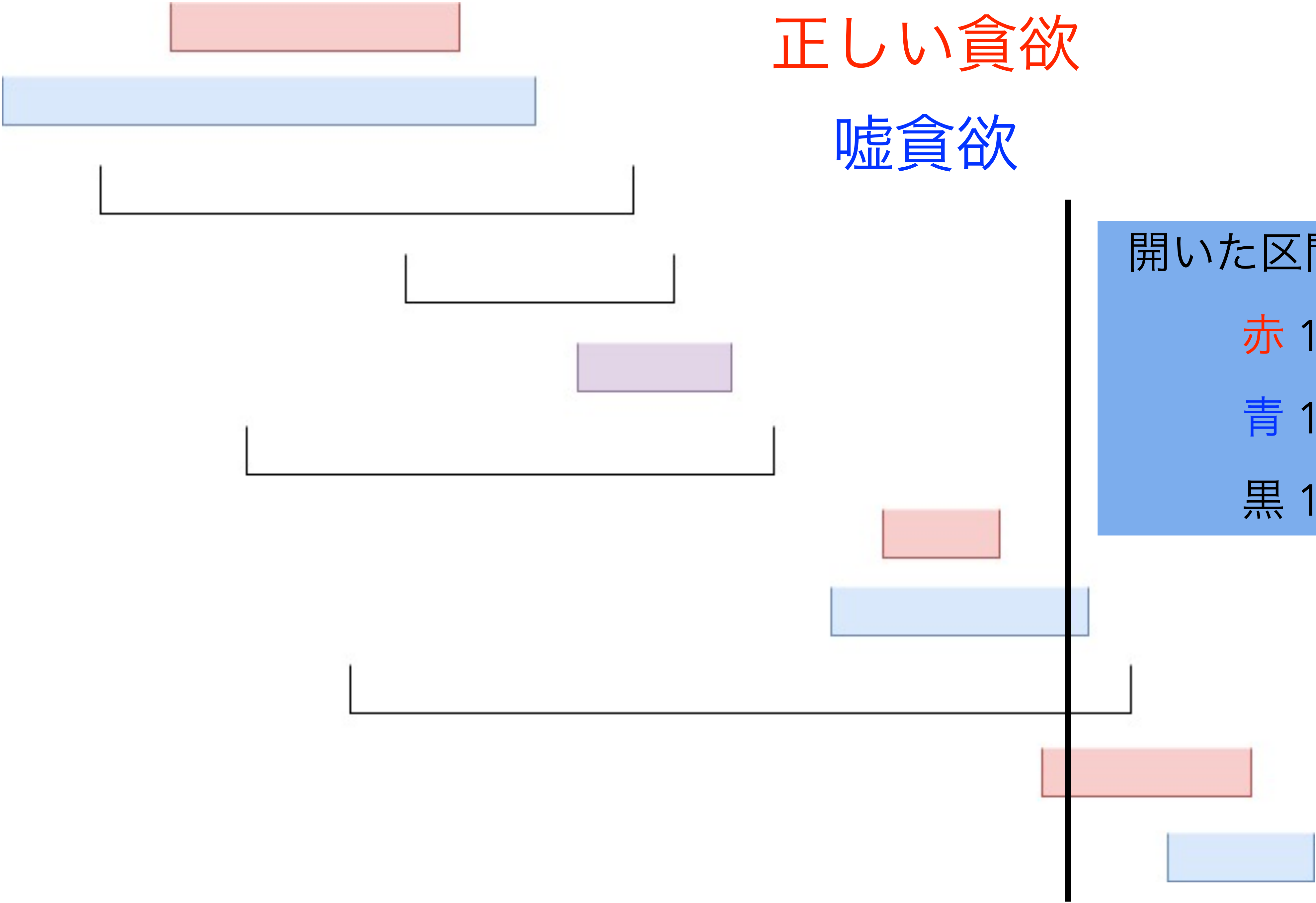
青 1

黒 2



正しい貪欲

嘘貪欲



開いた区間の数

赤 1

青 1

黒 1

小課題 4 ($N \leq 300$)

- 全部で 6 通りの状態がある
- 工夫すればもっと状態をまとめられるが、いずれにせよ場合分けが必要
- 焦らず丁寧に実装しましょう
- 計算量 $O(N^3)$

ここまでで 51 点

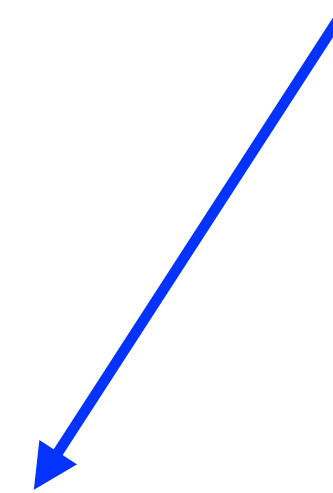
小課題 5 ($N \leq 3000$)

- 再び挿入 DP に戻ります
- さっきは B_i の昇順に区間を見たが、今度は A_i の降順に区間を見る
- 反転してるだけで実質同じじゃないの？
 - 嘘貪欲は区間の始点をベースに動いているので、実は対称性がない
 - 実際、区間の始点に関する制約の方が終点より“厳しい”

小課題 3 ($N \leq 30$)

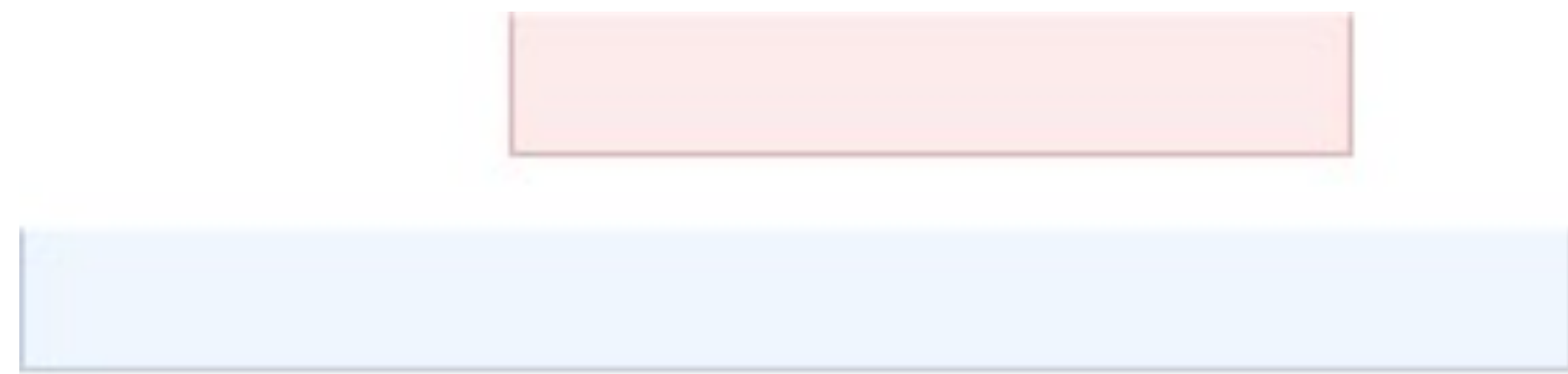
- B_i の昇順に区間を定める挿入 DP
- 以下の情報を持つ
 - 定めた区間の個数
 - 選んだ赤い区間の数 - 青い区間の数 (0 or 1)
 - 最後に選んだ赤い区間は何番目か
 - 最後に選んだ青い区間は何番目か
 - 挿入できる箇所のうち、青い区間の中に含まれるのは何箇所か

これがムダ



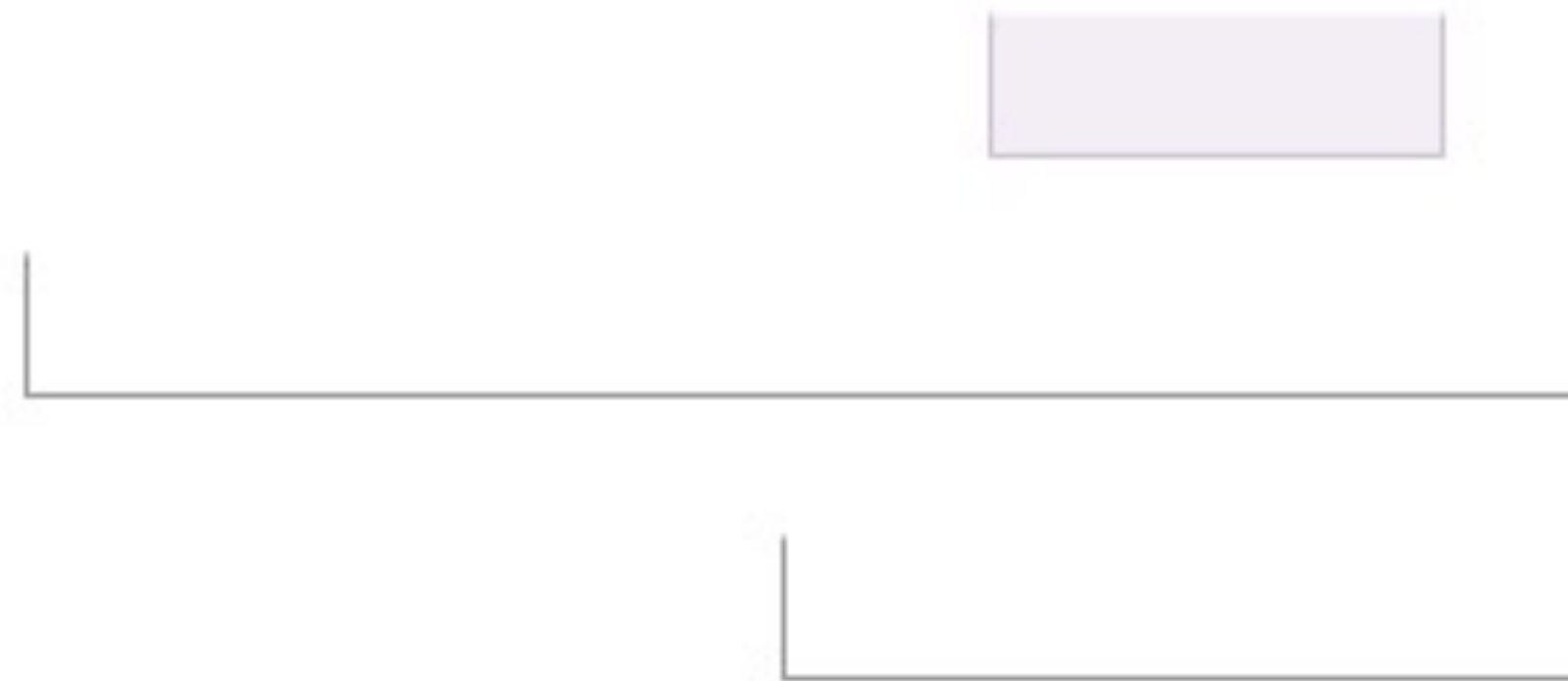
小課題 5 ($N \leq 3000$)

- 本当にすべて A_i の降順に区間を見るわけではなくて、赤い区間・青い区間の端点は特別扱いします
- 今回も図で説明します

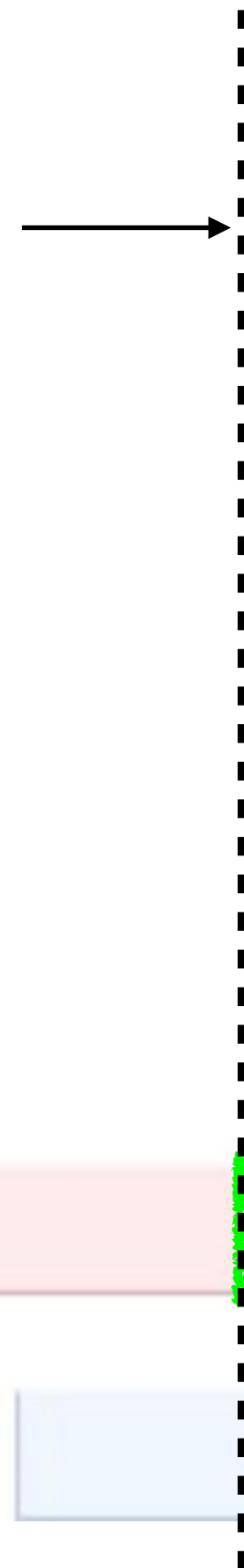


正しい貪欲

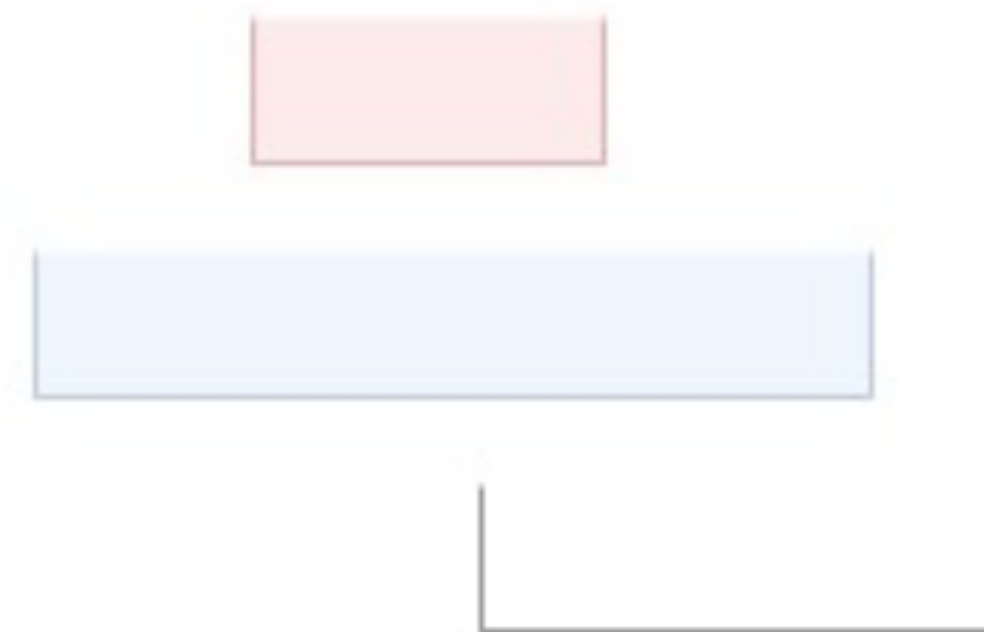
嘘貪欲

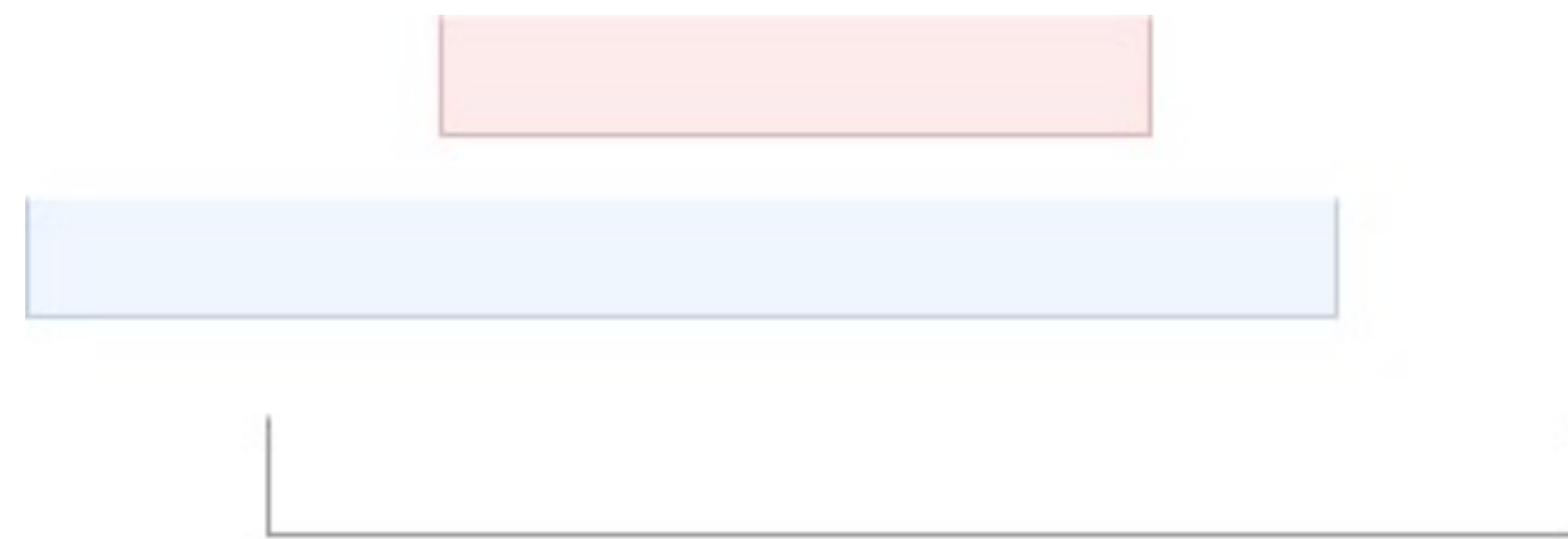


始点がこのラインより
後にある区間を定めた



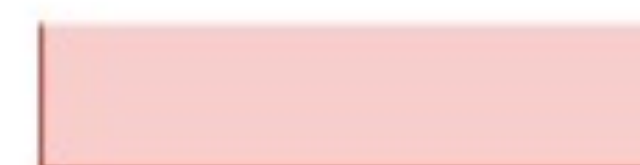
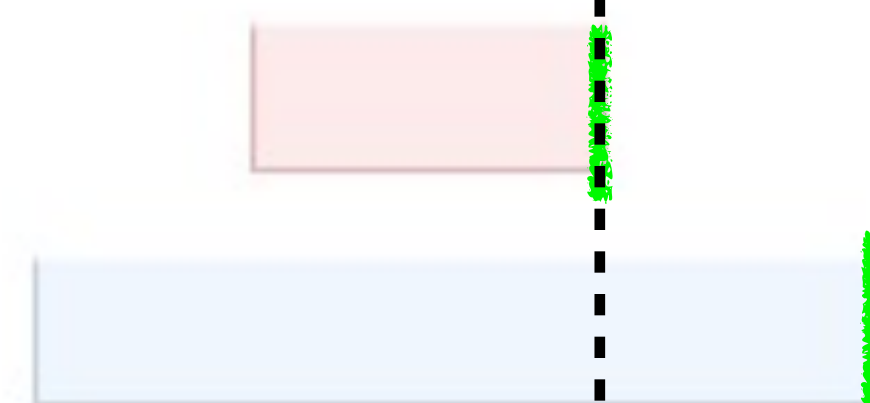
終点だけ定めた →



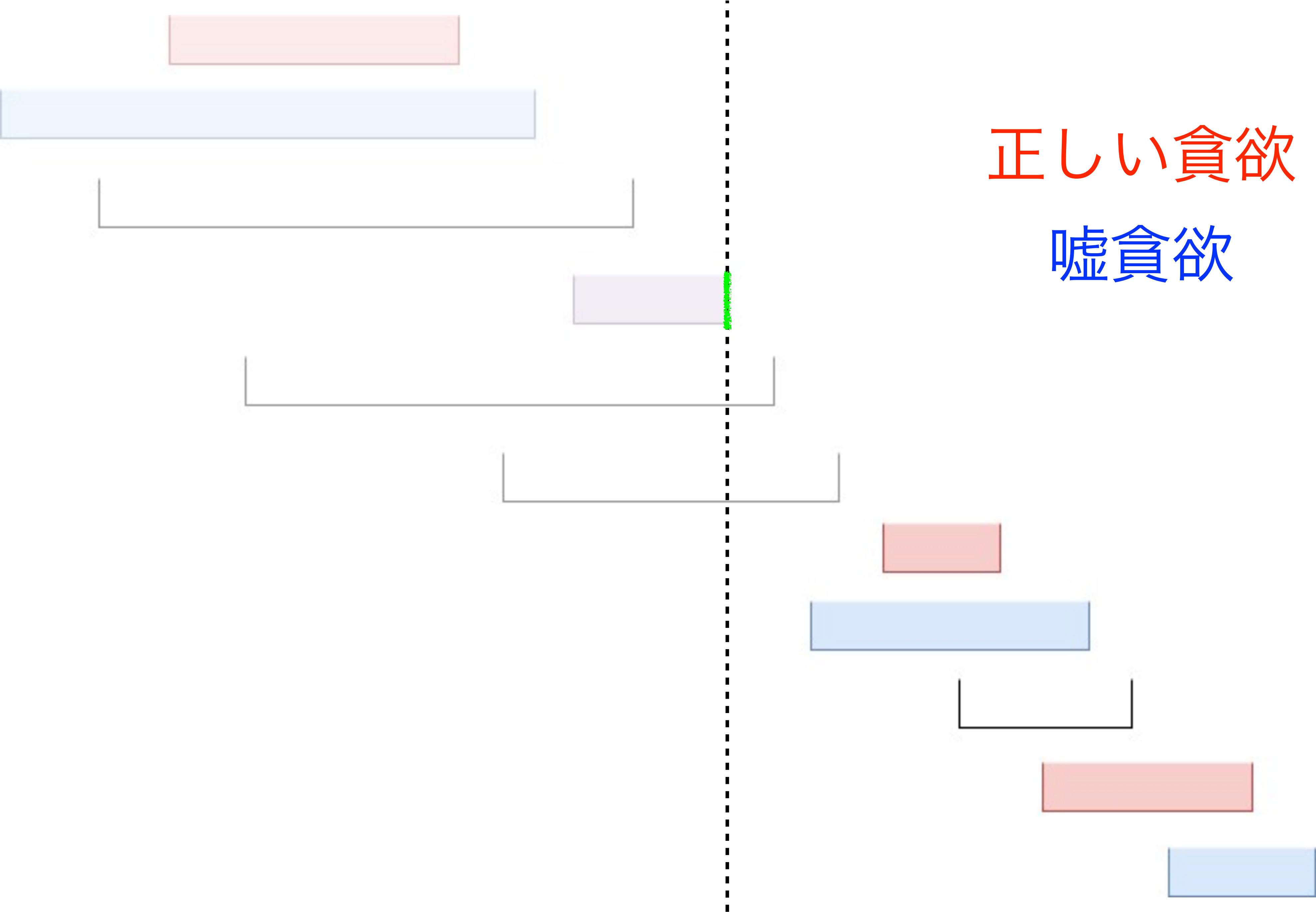


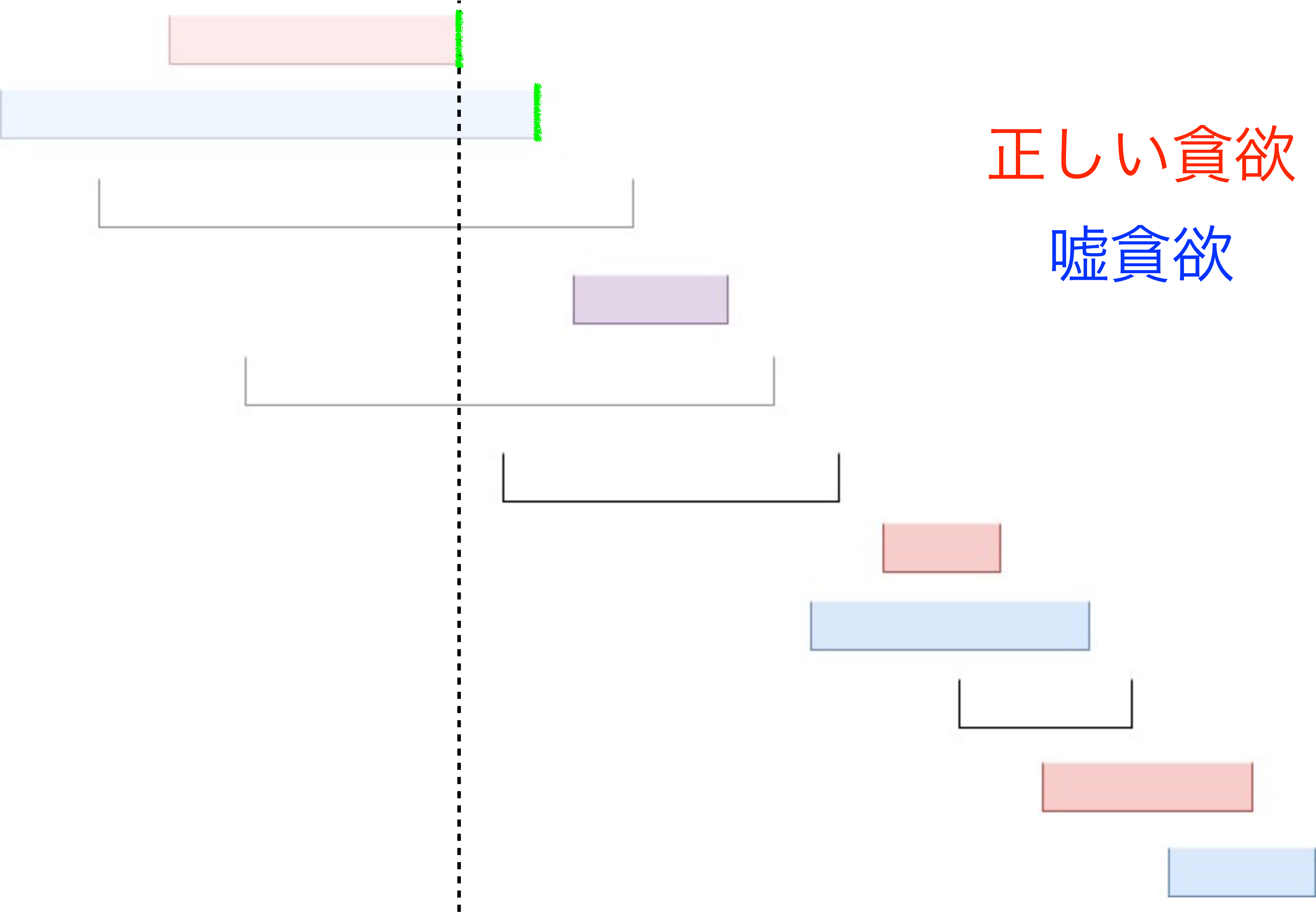
正しい貪欲

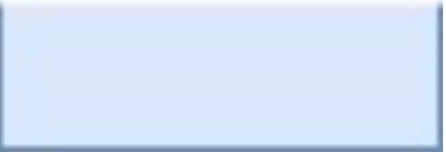
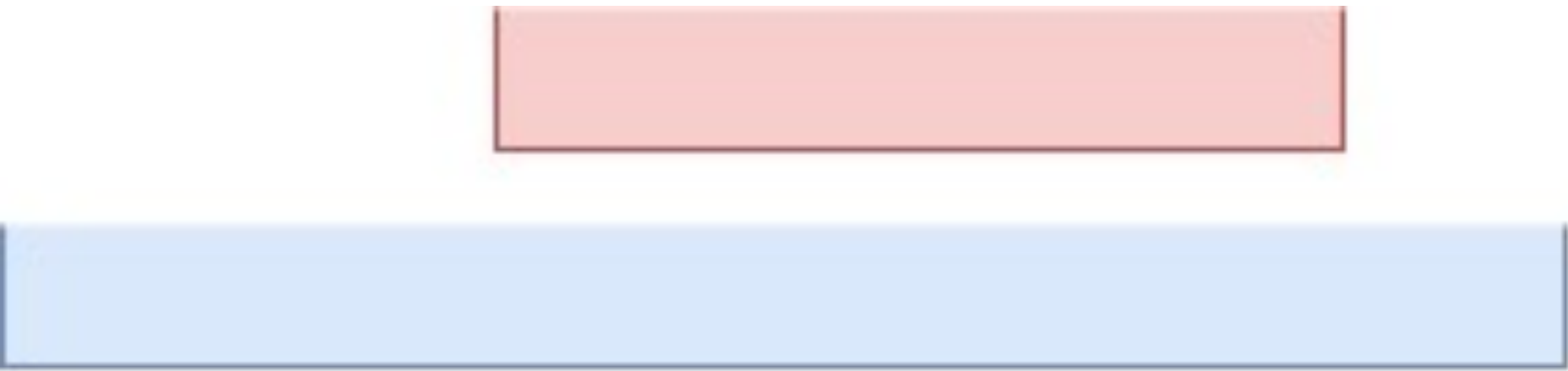
嘘貪欲



- 点線を動かすにあたって、
1. 赤・青の区間の位置関係を定める
 2. 点線が動いた範囲の中に始点を持つ黒い区間を挿入



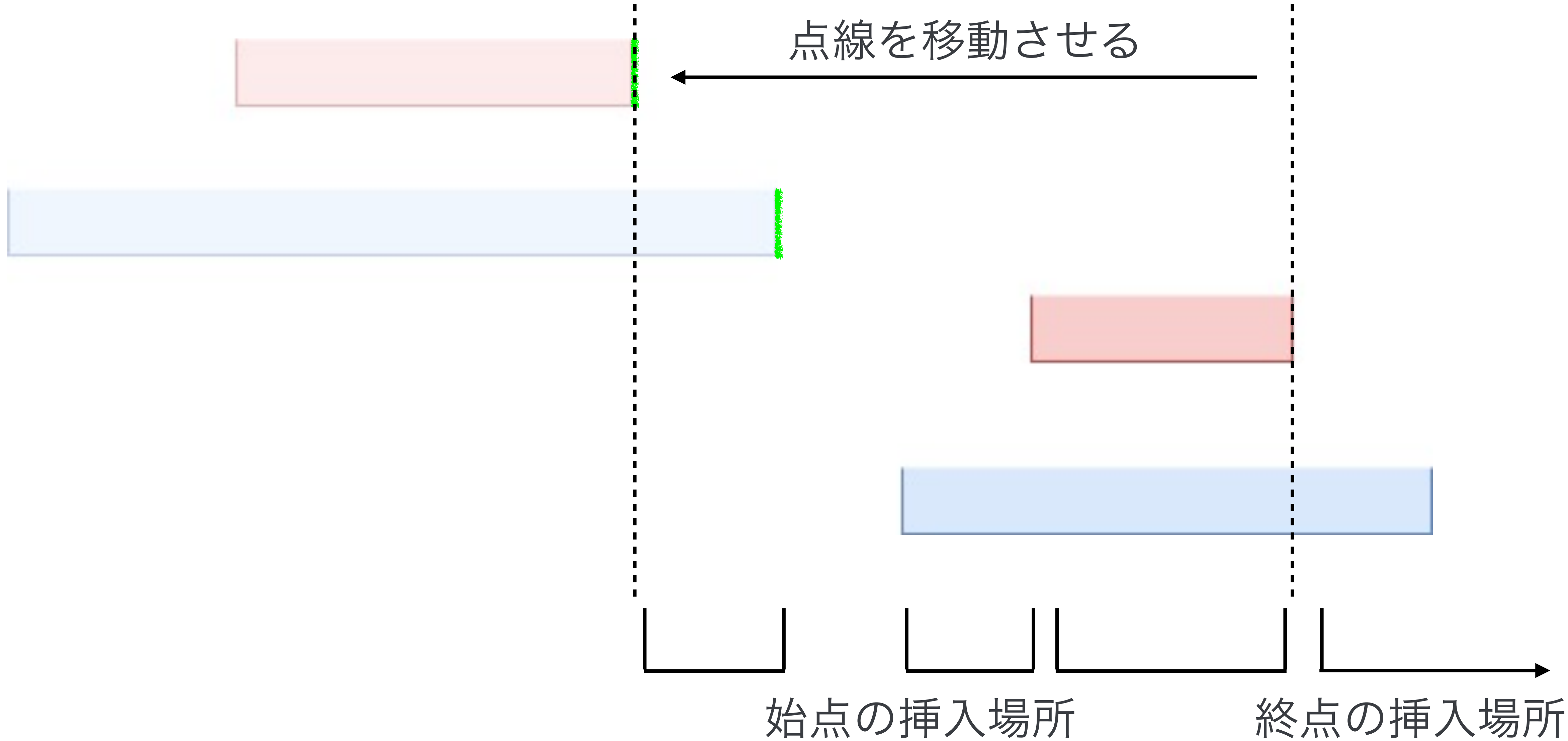


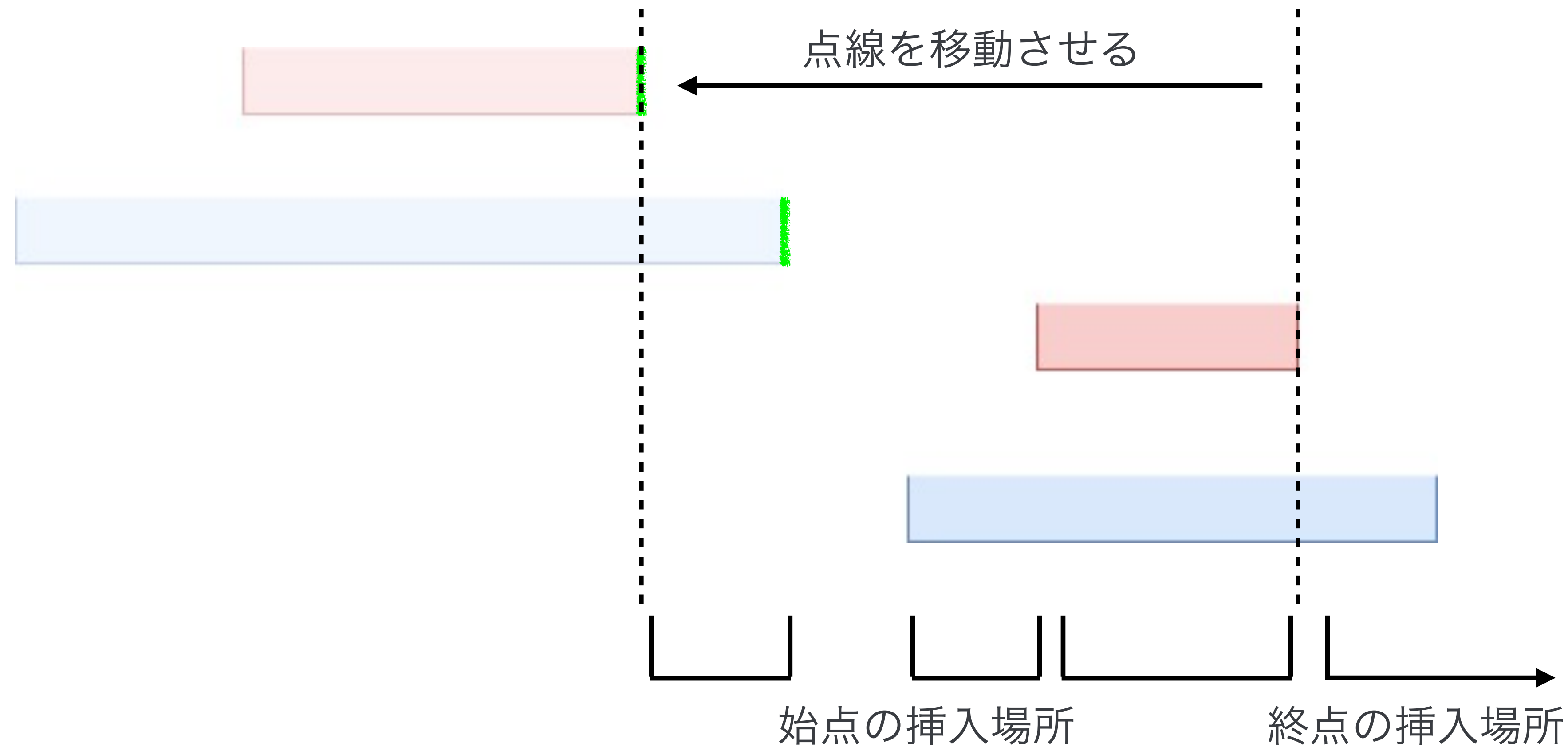


正しい貪欲
嘘貪欲

小課題 5 ($N \leq 3000$)

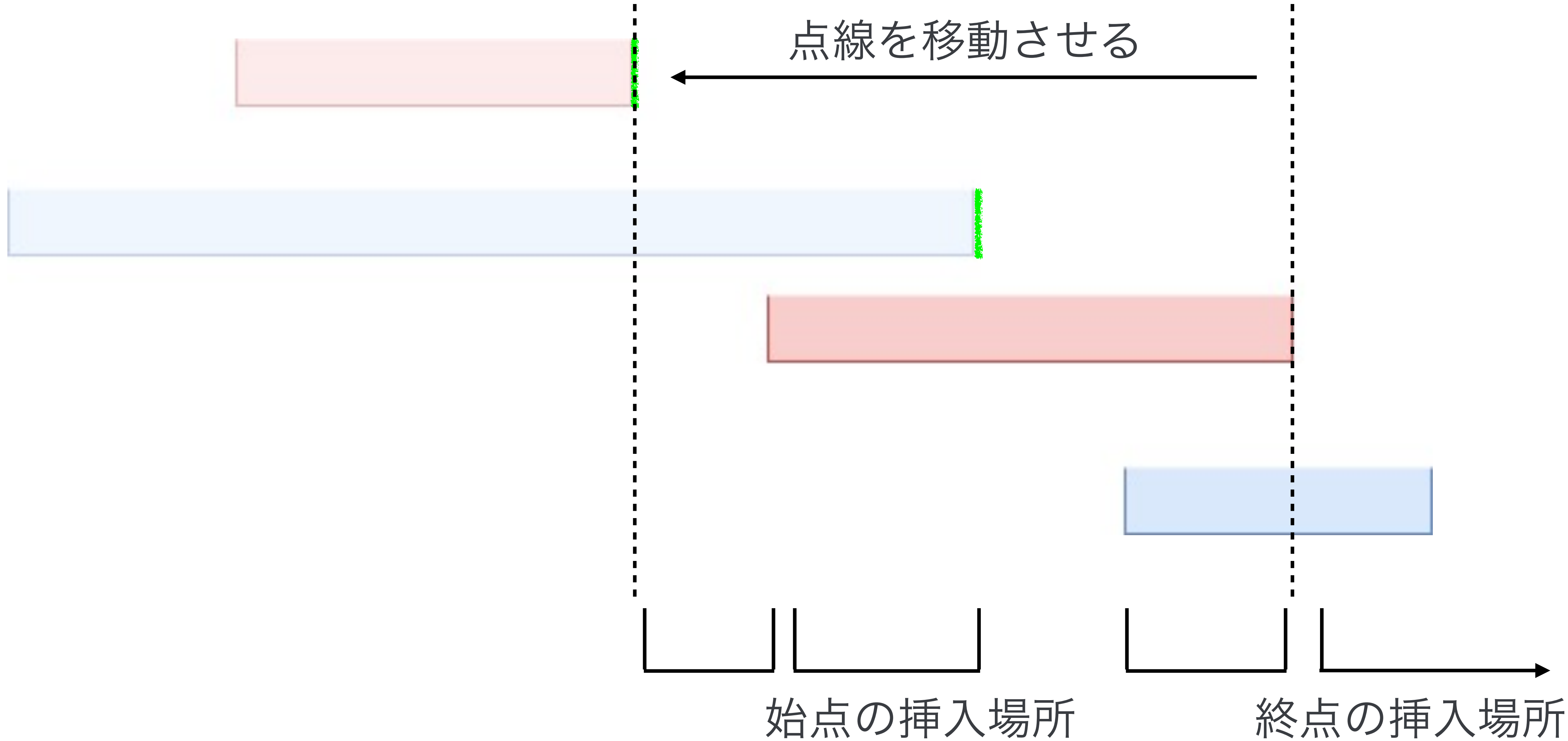
- 持つべき情報は、
 - 区間を何個定めたか
 - 終点だけ定まった赤・青の区間が一致したもの (図中における紫区間) かどうか
- 遷移の式をもうちょっと具体的に説明します





元々区間が i 個あって、新たに挿入する黒区間が j 個であるとき、

$$\text{dp}[i + j + 2][0] \leftarrow \text{dp}[i + j + 2][0] + \text{dp}[i][0] \cdot {}_{j+2}C_2 \cdot (2i - 2)(2i - 1) \cdots (2i - 2 + (j - 1))$$



※ この位置関係の場合もあることに注意

小課題 5 ($N \leq 3000$)

- $dp[*][0] \leftarrow dp[*][0]$ の形の遷移だけを扱ったが、他の 3 通りについても同様
- 状態 $O(N)$ 、遷移 $O(N)$ なので $O(N^2)$ で解けた

ここまでで 87 点

小課題 6 ($N \leq 20000$)

- そういえば使っていない制約があった
 - P は大きい素数
- 小課題 6 で初めて使います (人によってはもっと早く使うかもしれない)

小課題 6 ($N \leq 20000$)

- さっきの遷移の式をもう一度よく眺めてみよう

$$\text{dp}[i+j+2][0] \leftarrow \text{dp}[i+j+2][0] + \text{dp}[i][0] \cdot {}_{j+2}C_2 \cdot (2i-2)(2i-1)\cdots(2i-2+(j-1))$$

- $i+j+2$ を改めて j と置き直す

$$\text{dp}[j][0] \leftarrow \text{dp}[j][0] + \text{dp}[i][0] \cdot {}_{j-i}C_2 \cdot (2i-2)(2i-1)\cdots(i+j-5)$$

$$\text{dp}[j][0] \leftarrow \text{dp}[j][0] + \text{dp}[i][0] \cdot \frac{(j-i)(j-i-1)}{2} \cdot (2i-2)(2i-1)\cdots(i+j-5)$$

小課題 6 ($N \leq 20000$)

$$\text{dp}[j][0] \leftarrow \text{dp}[j][0] + \text{dp}[i][0] \cdot \frac{(j-i)(j-i-1)}{2} \cdot (2i-2)(2i-1)\cdots(i+j-5)$$

$$\text{dp}[j][0] \leftarrow \text{dp}[j][0] + \text{dp}[i][0] \cdot \frac{(j-i)(j-i-1)}{2} \cdot \frac{(i+j-5)!}{(2i-3)!}$$

- ある配列 P, Q 、定数 A, B, C が存在して、

$$\text{dp}[j][0] \leftarrow \text{dp}[j][0] + \text{dp}[i][0] \cdot P[i] \cdot Ai^2 \cdot Q[i+j]$$

$$\text{dp}[j][0] \leftarrow \text{dp}[j][0] + \text{dp}[i][0] \cdot P[i] \cdot Bij \cdot Q[i+j]$$

$$\text{dp}[j][0] \leftarrow \text{dp}[j][0] + \text{dp}[i][0] \cdot P[i] \cdot Cj^2 \cdot Q[i+j]$$

小課題 6 ($N \leq 20000$)

$$\text{dp}[j][0] \leftarrow \text{dp}[j][0] + \text{dp}[i][0] \cdot P[i] \cdot Ai^2 \cdot Q[i + j]$$

$$\text{dp}[j][0] \leftarrow \text{dp}[j][0] + \text{dp}[i][0] \cdot P[i] \cdot Bij \cdot Q[i + j]$$

$$\text{dp}[j][0] \leftarrow \text{dp}[j][0] + \text{dp}[i][0] \cdot P[i] \cdot Cj^2 \cdot Q[i + j]$$

$i, j, i + j$ に関する項しか出てこない

→ 畳み込める形になった🎉

畳み込みとは？

- 長さ N の配列 A, B が与えられる
- 以下の式で定義される配列 C を求めたい

$$C[i] = \sum_{j=0}^i A[j] \cdot B[i-j]$$

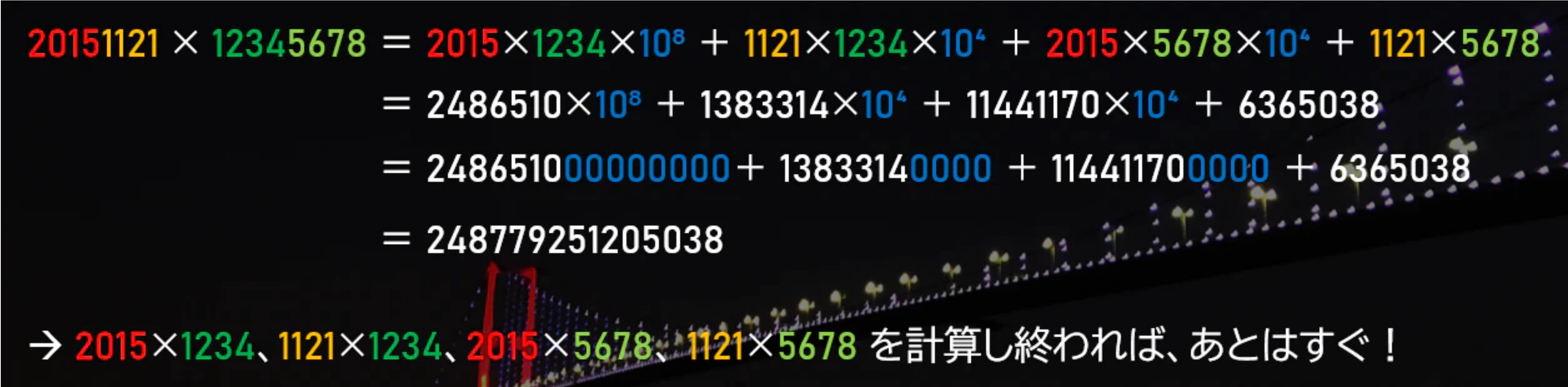
- ただし、出てくる値は全て $\text{mod } P$ で考えるものとする
- 愚直にやると $O(N^2)$ だが、もっと速く求めたい

手法 1 : NTT (数論変換)

- 畳み込みを $O(N \log N)$ で求められる
- $\text{mod } P$ の世界でのフーリエ変換のようなもの
- 知らない人で中身が気になる人は自分で調べてください
- 今回の問題では P が任意なので、NTT に適した素数を 3 つほど持ってきてそれぞれにおいて NTT した上で、中国剰余定理から復元
- 任意 mod NTT を時間内に空書きするのは非現実的

手法 2 : Karatsuba 法

- 畳み込みを $O(N^{\log_2 3}) \approx O(N^{1.59})$ で求められる
- 以下、square1001 さんの記事 『超高速！多倍長整数の計算手法【前編：大きな数の四則計算を圧倒的な速度で！】』 (Qiita) から引用します


$$\begin{aligned} 20151121 \times 12345678 &= 2015 \times 1234 \times 10^8 + 1121 \times 1234 \times 10^4 + 2015 \times 5678 \times 10^4 + 1121 \times 5678 \\ &= 2486510 \times 10^8 + 1383314 \times 10^4 + 11441170 \times 10^4 + 6365038 \\ &= 24865100000000 + 13833140000 + 11441170000 + 6365038 \\ &= 248779251205038 \end{aligned}$$

→ 2015×1234 、 1121×1234 、 2015×5678 、 1121×5678 を計算し終われば、あとはすぐ！

手法 2 : Karatsuba 法

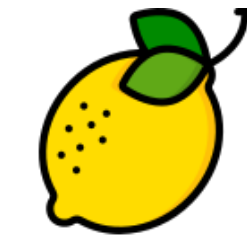
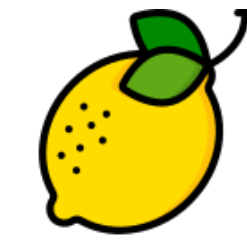
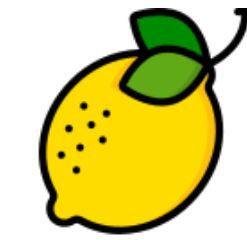
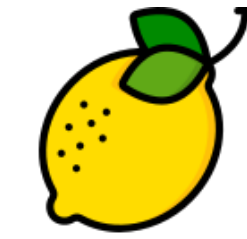
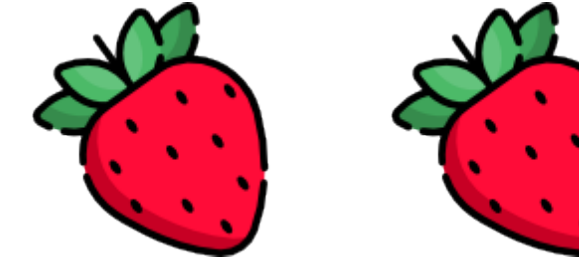
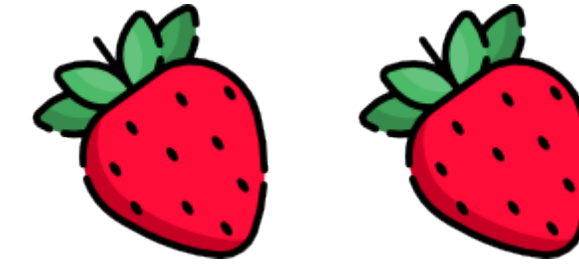
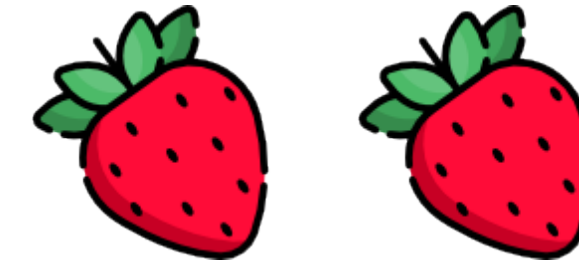
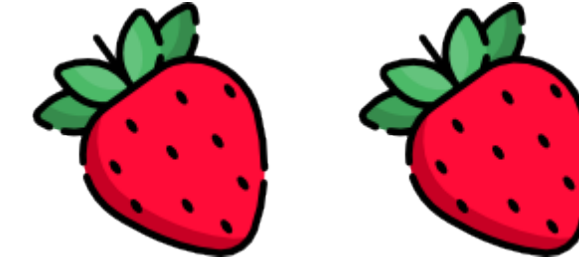
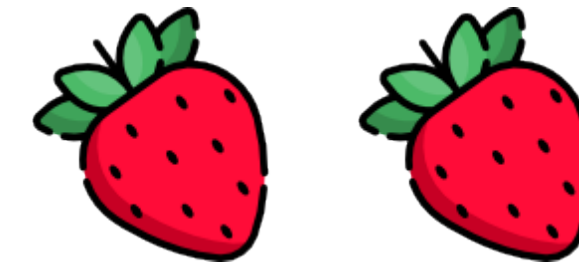
- NTT より遅い代わりに、実装がかなり楽
- 競技時間中に空書きするのも非現実的ではない

小課題 6 ($N \leq 20000$)

- この問題における畳み込みはオンライン畳み込みとよばれるものなので、ただ畳み込みめばいいというものでもない
- 分割統治と組み合わせる必要がある (オンライン・オフライン変換)
- 任意 mod NTT で $O(N(\log N)^2)$
Karatsuba 法で $O(N^{1.59})$

ここまでで 100 点

得点分布



100

87

51

37

10

5

0

Passport

Editorial: Hiroataka Yoneda (square1001)



海外に行ったことはありますか？



そのために必要な「パスポート」

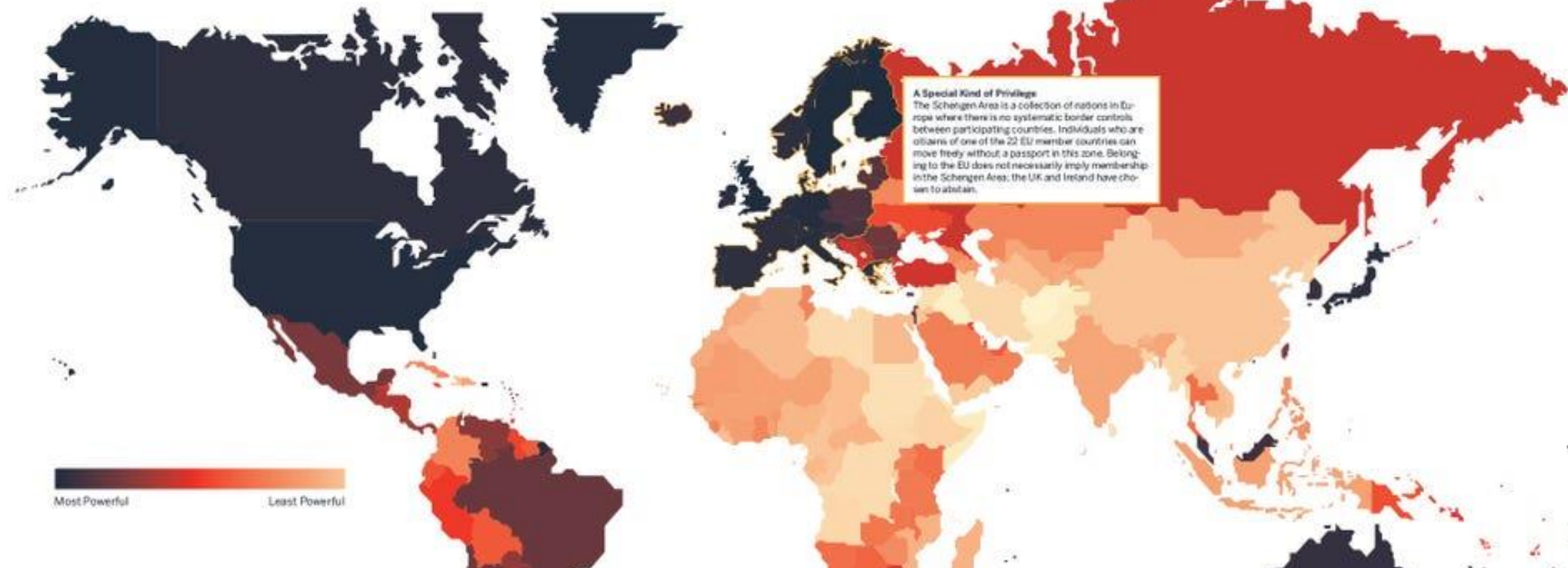


HOW POWERFUL IS YOUR PASSPORT?

We may live in a world defined by invisible meta-data and the Cloud, but the roughly 3x5 inch paper booklet known as the passport still carries a lot of weight. More than a simple grant of access into a country, passports and the visas they contain are a reflection of geopolitics, the relationship between two nations, and a country's stature relative to the rest of the world. They're also of immense value. Just ask any one of the

developing world's citizens: if they'd rather win the lottery or a U.S. passport, chances are they'd say the latter (though they'd be wrong in thinking the American passport is the most powerful of them all).

Here's a look at the strength of the world's passports—ranked by the travel freedom a passport holder enjoys.



パスポートを使って行ける国は限られています

[画像は Business Insider の記事 “The Countries With The Most Powerful Passports” より]

Passport holders of the following countries are granted visa-free access or receive a visa upon arrival to the number of countries noted.

問題の説明

N 個の国があります

国 i のパスポートで国 $L_i, L_i + 1, \dots, R_i$ に行けます

最初国 $X_1, X_2, \dots, X_Q$ に住んでいる場合について

- N カ国全部行くのに最小何個のパスポートを得る必要があるか？

※1 不可能な場合は、そのことを報告するため -1 を出力する

※2 最初パスポートを持っていないとする



各小課題の制約

SUBTASK 1

$X = 1$

SUBTASK 2

$N \leq 300, Q = 1$

SUBTASK 3

$N \leq 2000, Q = 1$

SUBTASK 4

$N \leq 2000$

SUBTASK 5

追加の制約はない

Chapter I

小課題 1 の解法

はじめに：問題の言い換え

この解説全体で、以下のように問題を言い換えて話します

※ この方が問題として考えやすいと思います

性質

N カ国全部行くのに得るパスポートの数を最小化



「パスポートで国 $1, 2, \dots, N$ に入国できる状態」に
できるだけ早くすること

※ できるだけ早く = できるだけ短いステップ数（今いる国でパスポートを取得 → 別の国へ移動のプロセス）で

はじめに：問題の言い換え

この言い換えの意味を、入力例 1 で説明します

	国 1	国 2	国 3	国 4
パスポート 1				
パスポート 2				
パスポート 3				
パスポート 4				

※ 赤+黒の領域は、各パスポートが行ける国を表す

最初、国 1 にのみ行ける状態

はじめに：問題の言い換え

この言い換えの意味を、入力例 1 で説明します

	国 1	国 2	国 3	国 4
パスポート 1				
パスポート 2				
パスポート 3				
パスポート 4				

※ 赤+黒の領域は、各パスポートが行ける国を表す

パスポート 1 を得ると、国 1, 2, 3 に行ける状態に

はじめに：問題の言い換え

この言い換えの意味を、入力例 1 で説明します

	国 1	国 2	国 3	国 4
パスポート 1				
パスポート 2				
パスポート 3				
パスポート 4				

※ 赤+黒の領域は、各パスポートが行ける国を表す

パスポート 2 を得ると、全部行ける状態に
(ここまでに 2 ステップかかった)

はじめに：問題の言い換え

この言い換えの意味を、入力例 1 で説明します

	国 1	国 2	国 3	国 4
パスポート 1				
パスポート 2				
パスポート 3				
パスポート 4				

※ 赤+黒の領域は、各パスポートが行ける国を表す

この状態になれば、全部の国をめぐって目標達成！

小課題 1 ($X_i = 1$) の解法

小課題 1 では「一番左の国からスタート ($X = 1$)」

各ステップにおける最適な行動は

(今行ける国の中で) R_i が最大の国のパスポートを得ること

	国 1	国 2	国 3	国 4
パスポート 1				
パスポート 2				
パスポート 3				
パスポート 4				

パスポート 2 よりもパスポート 3 の方が区間が広げられるので、絶対に得する。

小課題 1 ($X_i = 1$) の解法

この行動を繰り返し、「行ける国が全部」の状態になるまで何回かかるかシミュレーション

- 最大 N ステップかかるので、全体計算量 $O(N^2)$
- 「目標達成不可能な場合」にも注意

R_i の累積 max を前計算すると、各ステップ $O(1)$ でできる

→ 全体計算量が $O(N)$ になる!

6 点

小課題 1 ($X_i = 1$) の解法

注意: この貪欲法は、一般のケースで成り立つとは限らない

※ 行ける国の数が最大になるパスポートを選び続ける貪欲法

	国 1	国 2	国 3	国 4	国 5	国 6	国 7
パスポート 1							
パスポート 2							
パスポート 3							
パスポート 4							
パスポート 5							
パスポート 6							
パスポート 7							

貪欲法の反例: $N = 7, X_1 = 5$ (国 5, 6, 7 のパスポートを順に得るのが最適)

Chapter II

小課題 2, 3, 4 の解法

小課題 2 以降の準備

「最小何手で目的の状態にできるか？」の問題は

最短経路問題 に帰着させるのが定石！

(実用例：スライドパズルの最適解を求める など)

小課題 2 ($N \leq 300, Q = 1$) の解法

実は、この問題も最短経路問題になる

帰着

状態 $(l, r) \cdots$ 国 $l, l+1, \dots, r$ に入国できる状態、とする

状態 (l, r) からパスポート k ($l \leq k \leq r$) を取ると

状態 $(\min(l, L_k), \max(r, R_k))$ に変化する

(X, X) から $(1, N)$ に最小何ステップで行けるか？

小課題 2 ($N \leq 300, Q = 1$) の解法

実は、この問題も最短経路問題になる

$O(N^2)$ 頂点 $O(N^3)$ 辺のグラフなので
BFS で答えが計算量 $O(N^3)$ で求まる!!

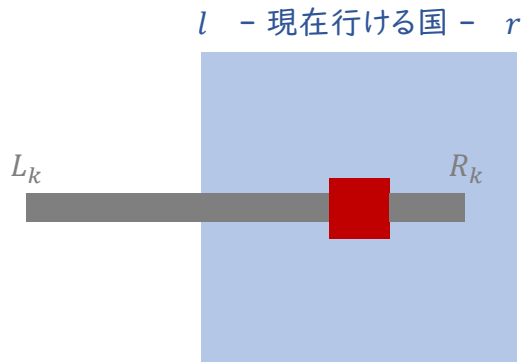
(X, X) から ($1, N$) に最小何ステップで行けるか?

22 点

小課題 3 ($N \leq 2500, Q = 1$) の解法

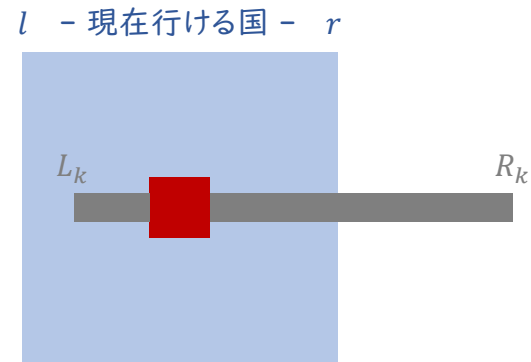
辺の数を減らすため、どんな状態遷移があるか考えてみよう

ケース 1



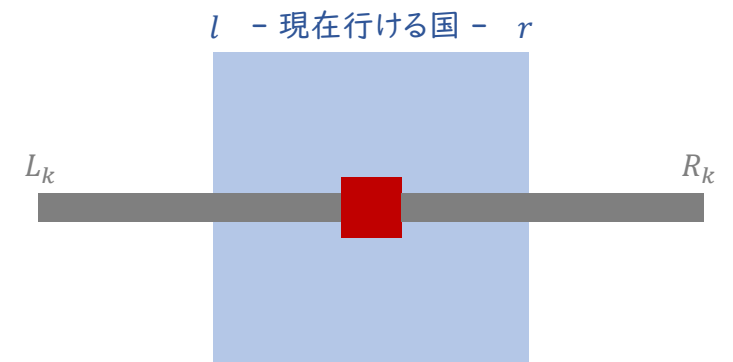
左だけ伸びるパターン

ケース 2



右だけ伸びるパターン

ケース 3



両側伸びて (L_k, R_k) になるパターン

小課題 3 ($N \leq 2500, Q = 1$) の解法

辺の数を減らすため、どんな状態遷移があるか考えてみよう

ケース 1

l - 現在行ける国 - r

L_k ($l \leq k \leq r$) が最小の
パスポートが最適

それ以外は意味ない

左だけ伸びるパターン

ケース 2

l - 現在行ける国 - r

R_k ($l \leq k \leq r$) が最大の
パスポートが最適

それ以外は意味ない

右だけ伸びるパターン

ケース 3

l - 現在行ける国 - r

どれを選べばいいか
分からない!

両側伸びて (L_k, R_k) に
なるパターン

小課題 3 ($N \leq 2500, Q = 1$) の解法

ケース 3 に対処するため、次の考察を考える

考察

状態 (l, r) を「少なくとも 国 $l, l+1, \dots, r$ に入国できる状態」とする

この変更で $(l, r) \rightarrow (l+1, r)$ 、 $(l, r) \rightarrow (l, r-1)$ の辺を張ることになる

すると、各 k ($1 \leq k \leq N$) に対して $(k, k) \rightarrow (L_k, R_k)$ の辺を張れば

ケース 3 も $(l, r) \rightarrow (k, k) \rightarrow (L_k, R_k)$ と行くことで考慮できたことになる!!

※ $(l, r) \rightarrow (k, k)$ (ただし $l \leq k \leq r$) と行くのは、区間を 1 ずつ縮める辺 (第 1 段落参照) を適当な順番でたどることのできる

小課題 3 ($N \leq 2500, Q = 1$) の解法

少し複雑なので、小課題 3 の解法をまとめます

解法

各 (l, r) に対して以下のように辺を張ったグラフを考える

- $l < r$ のとき、 $(l, r) \rightarrow (\min(L_l, \dots, L_r), r)$ のコスト 1 の辺
 $(l, r) \rightarrow (l, \max(R_l, \dots, R_r))$ のコスト 1 の辺
 $(l, r) \rightarrow (l + 1, r), (l, r - 1)$ のコスト 0 の辺
- $l = r (= k)$ のとき、 $(k, k) \rightarrow (L_k, R_k)$ のコスト 1 の辺

状態 (X, X) から $(1, N)$ までの最短距離が答え

小課題 3 ($N \leq 2500, Q = 1$) の解法

少し複雑なので、小課題 3 の解法をまとめます

解法 $O(N^2)$ 頂点 $O(N^2)$ 辺のグラフなので

01-BFS で答えが計算量 $O(N^2)$ で求まる!!

- $l = r (= k)$ のとき、 $(k, k) \rightarrow (L_k, R_k)$ のコスト 0 の辺

状態 (X, X) から $(1, N)$ までの最短距離が答え

46 点

小課題 4 ($N \leq 2500$) の解法

小課題 4 では、 $X = 1, 2, \dots, N$ それぞれに対して
状態 (X, X) から $(1, N)$ までの最短距離を求めたい

先ほどのグラフの辺を全部逆向きにすると...

頂点 $(1, N)$ から各頂点への最短距離を求めることで全部解決!

計算量 $O(N^2)$ で小課題 4 に正解

54 点

Chapter III

小課題 5 (満点) の解法

満点解法の準備

ところで、もし目標が「国 1 に行くこと」だったら...

方法

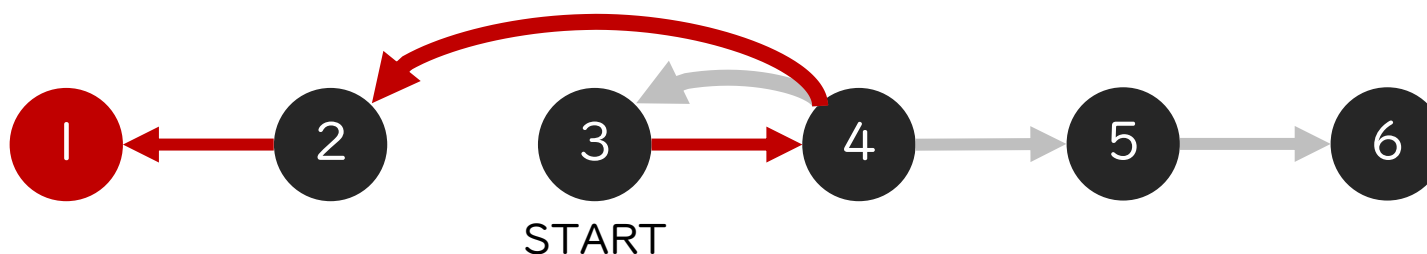
N 頂点あって、頂点 i から頂点 $L_i, \dots, R_i$ に辺を張ったグラフで最短経路問題を解けばよい

なぜなら、直前に得たパスポート以外忘れてもいいから

実際は「国 1, 2, ..., N 全部行ける状態」が目標なので難しい

満点解法の準備

ところで、もし目標が「国 1 に行くこと」だったら…



作るグラフとその最短路の例

満点解法の準備

ここで、次のように問題を言い換えてみよう

性質

国 $1, 2, \dots, N$ 全部行ける状態にする

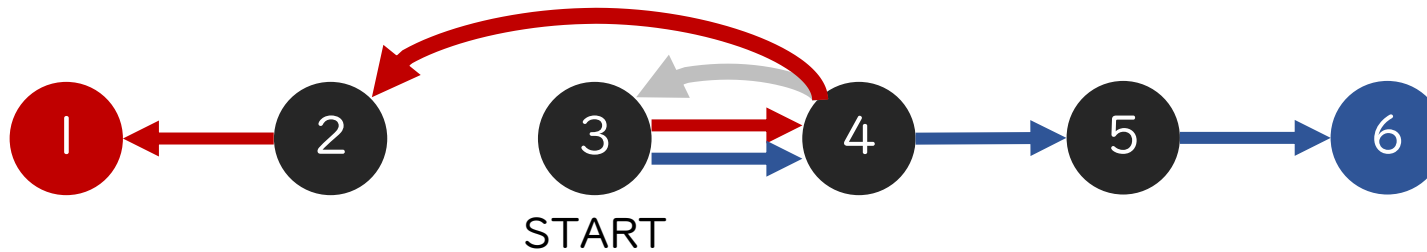


「国 1 」と「国 N 」に行ける状態にする

理由：制約「 $L_i \leq i \leq R_i$ 」のおかげで、行ける国は区間になるから！

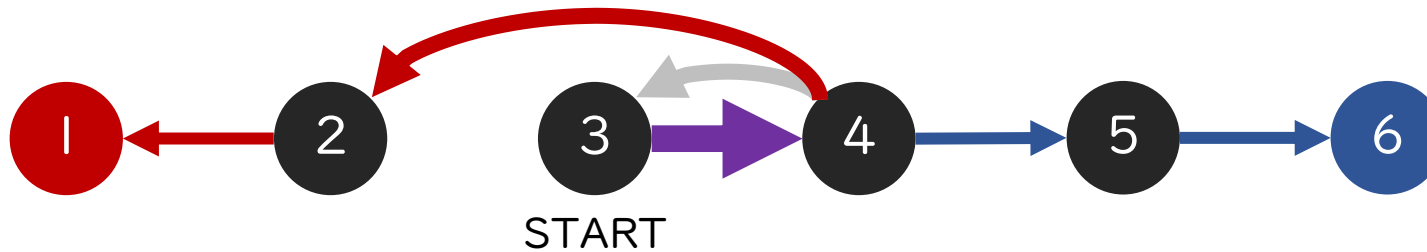
満点解法

先ほどの N 頂点のグラフで
「国 1 に行く経路」と「国 N に行く経路」を別々に考えると...



満点解法

先ほどの N 頂点のグラフで
「国 1 に行く経路」と「国 N に行く経路」を別々に考えると...

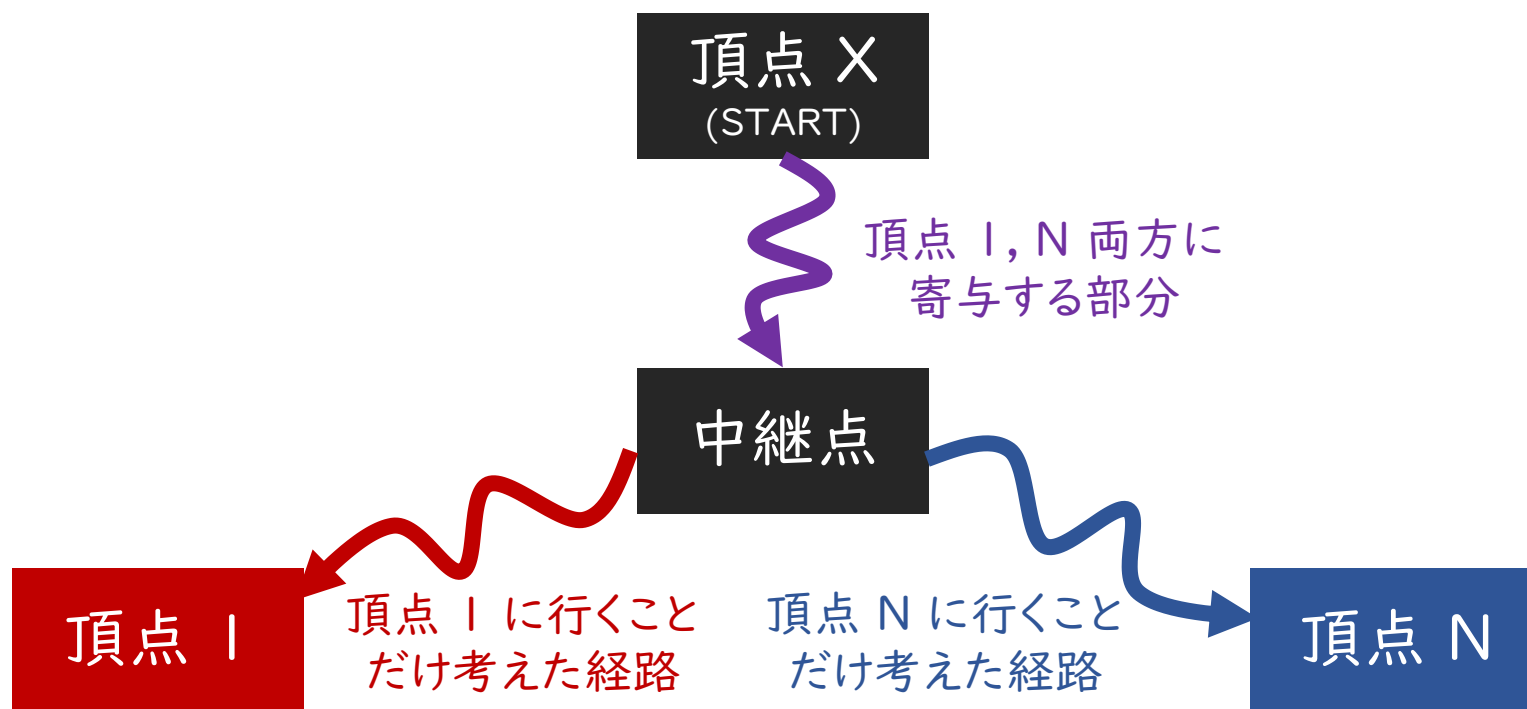


並列する部分は 1 つのパスポートでまとめられるので
これでステップ数が削れるかもしれない

満点解法

つまり一般に、最適解は次のような Y 字型の“経路”になる！

※ 通る辺の本数の合計 (= パスポートの個数) を最小化するものが最適解



満点解法

これはどうやって解くか？

方法 1

頂点 1 から逆辺を通った最短距離 $a_1, a_2, \dots, a_N$

頂点 N から逆辺を通った最短距離 $b_1, b_2, \dots, b_N$

頂点 X (スタート) からの最短距離 $c_1, c_2, \dots, c_N$ を求める

このとき $a_i + b_i + c_i$ の最小値が答え

(これは $Q = 1$ の場合に解ける)

満点解法

これはどうやって解くか？

方法 2

グラフの辺を逆向きにして考える

頂点 1 からの最短距離 $a_1, a_2, \dots, a_N$

頂点 N からの最短距離 $b_1, b_2, \dots, b_N$ を求める

頂点 N+1 を追加し、ここから頂点 i にコスト $a_i + b_i$ の辺を張る
その後、頂点 N+1 からの最短距離 $d_1, d_2, \dots, d_N$ を求める

$d_1, d_2, \dots, d_N$ が各スタート地点に対する答えになる！

満点解法

これはどうやって解くか？

方法 2

グラフの辺を逆向きにして考える

計算量 $O(N^2)$ で答えが求まる！

頂点 N からの最短距離 $b_1, b_2, \dots, b_N$ を求める

でも、さらに改善できないか？

頂点 $N+1$ を追加して、頂点 $N+1$ から頂点 N までの辺を張る
その後、頂点 $N+1$ からの最短距離 $d_1, d_2, \dots, d_N$ を求める

$d_1, d_2, \dots, d_N$ が各スタート地点に対する答えになる！

満点解法

ここで「区間に辺を張るテク」と呼ばれるテクニックを紹介します
具体例として、16 頂点のグラフを考えます

1

2

3

4

5

6

7

8

9

10

11

12

13

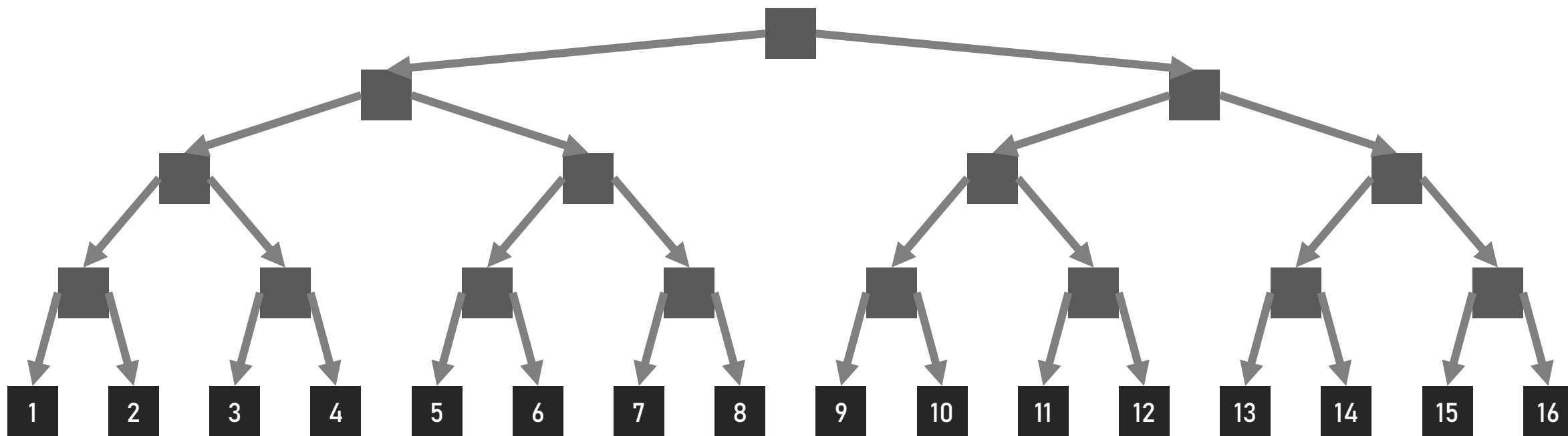
14

15

16

満点解法

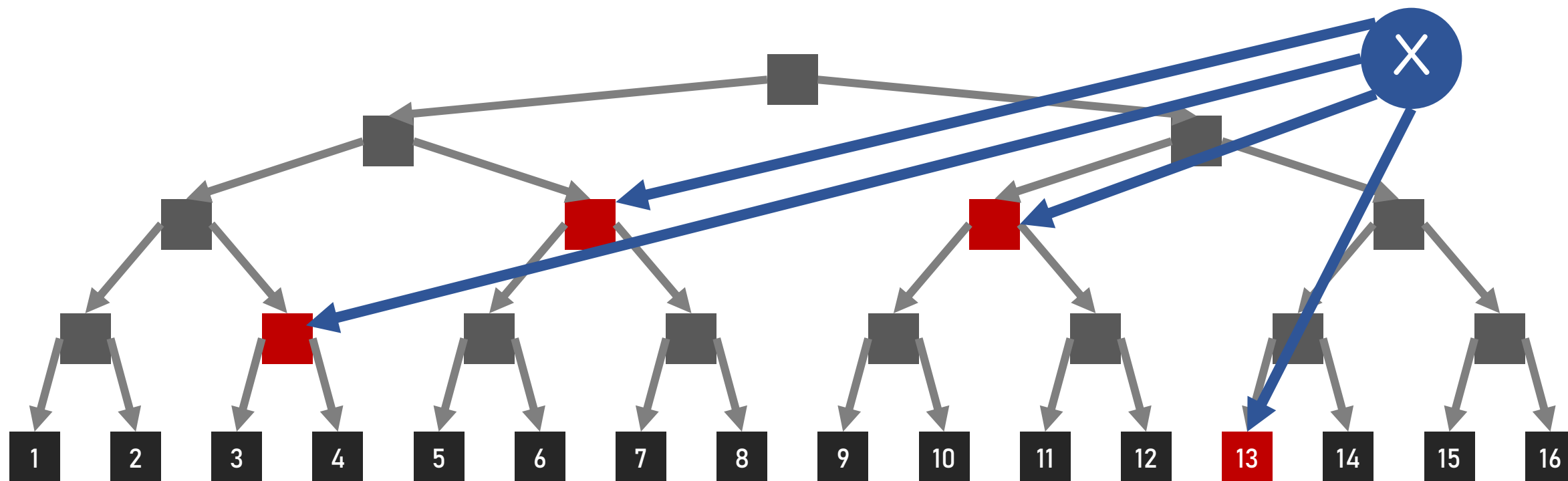
ここで「区間に辺を張るテク」と呼ばれるテクニックを紹介します
まず、セグメント木のような形で、新たな頂点と有向辺を追加します



満点解法

ある頂点 X から頂点 3, 4, ..., 13 に辺を張りたいとします

頂点 X から赤い頂点に辺を張ると、目的達成



満点解法

ある頂点 x から頂点 $3, 4, \dots, 13$ に辺を張りたいとします

頂点 x から赤い頂点に辺を張ると、目的達成

このようにして「区間に辺を張る操作」 x を

$2 \log N$ 辺以下の追加で実現できる!



満点解法

満点解法のグラフは「区間に辺を張るテク」を使うことで
 $2N$ 頂点 $O(N \log N)$ 辺のグラフとして扱える

よって、この問題は工夫した BFS で
計算量 $O(N \log N)$ で解ける!

100 点

Chapter IV

得点分布

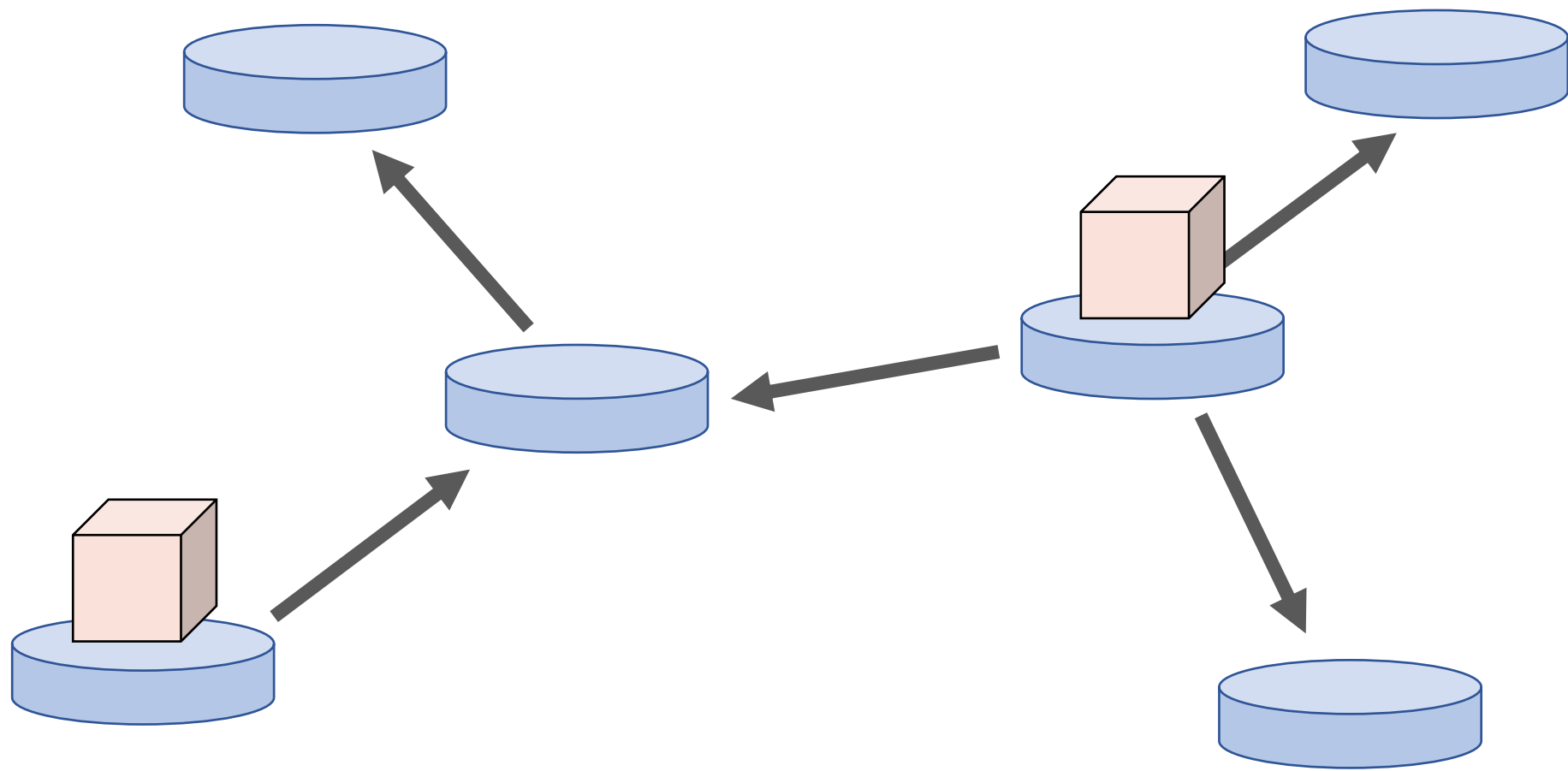
得点分布

得点	小課題					人数
	1	2	3	4	5	
100	✓	✓	✓	✓	✓	✈
54	✓	✓	✓	✓		✈✈✈✈✈✈✈✈
46	✓	✓	✓			✈✈✈
40		✓	✓			✈
22	✓	✓				✈✈✈✈✈✈✈✈
16		✓				✈
6	✓					✈✈✈✈✈
0						✈✈

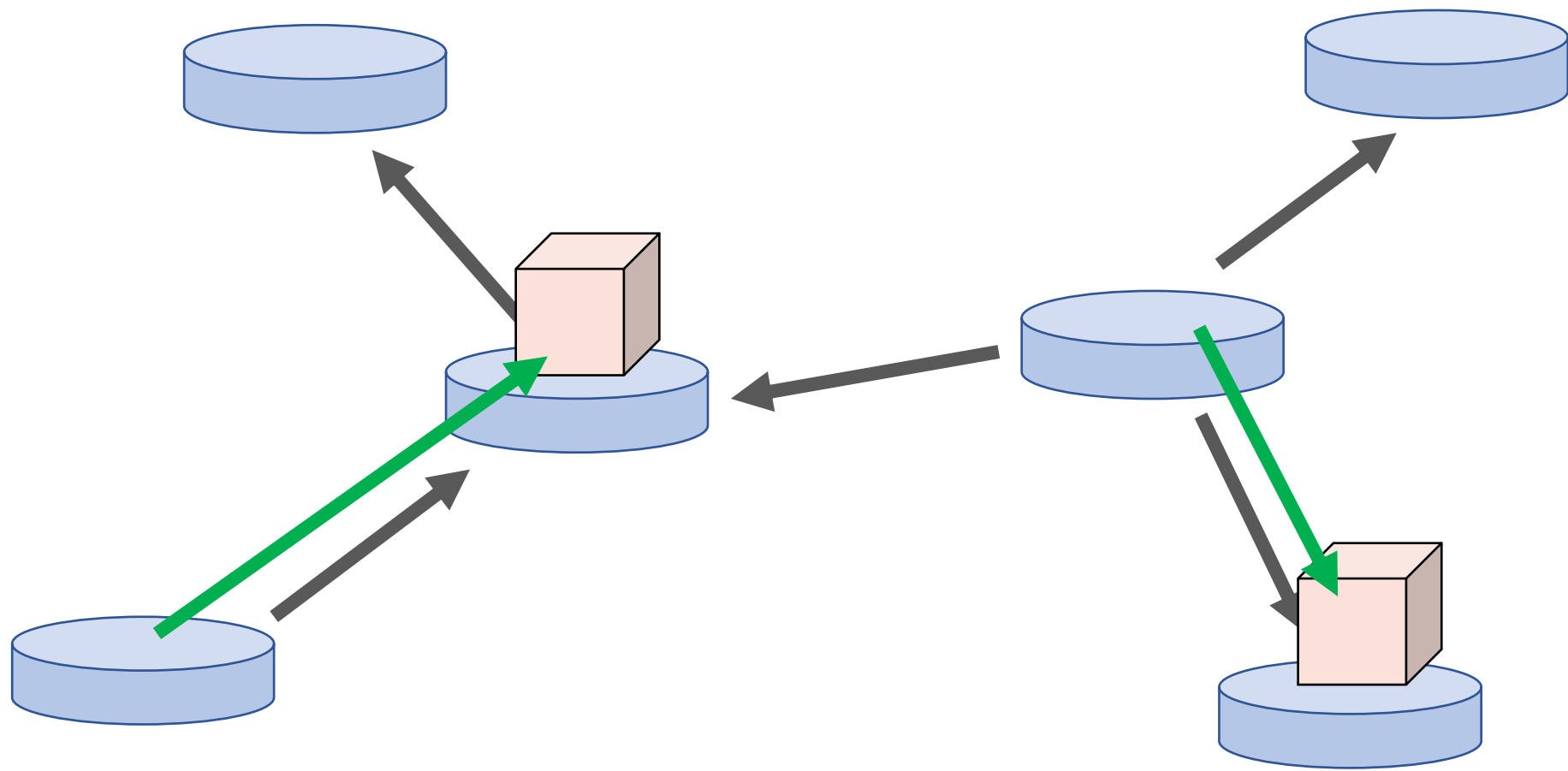
JOI 2022/2023 春 ベルトコンベア

解説: 木ノ下恭範 (noshi91)

問題概要



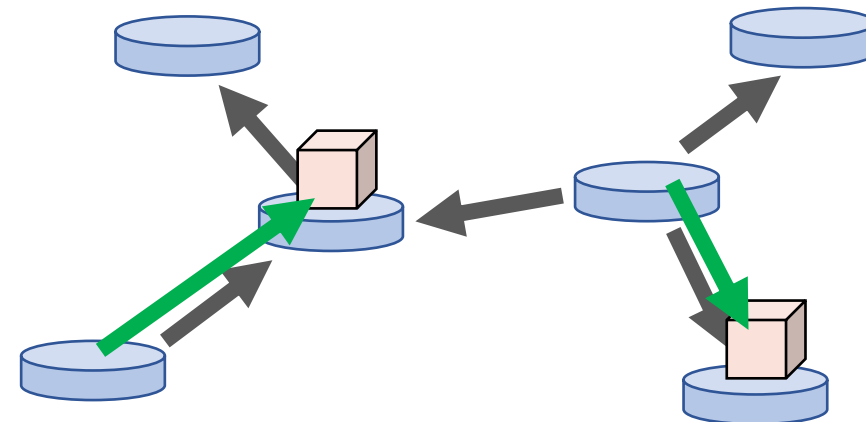
問題概要



問題概要

N 頂点の木がある

各辺は製品を一方向に運ぶが、方向が分からない

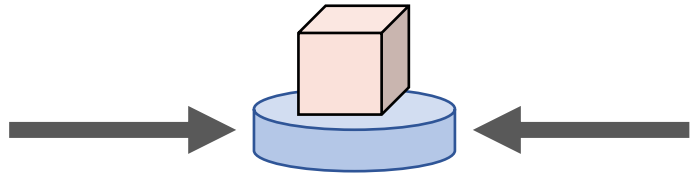


1. いくつかの辺を選んで反転する
2. いくつかの頂点に製品を置き、製品が移動する
3. 各頂点に製品が **1** 個以上あるか確認する
4. 反転した辺を戻す

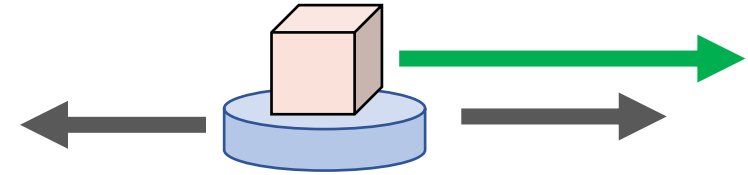
30 クエリ以内に辺の向きを判定せよ

問題概要

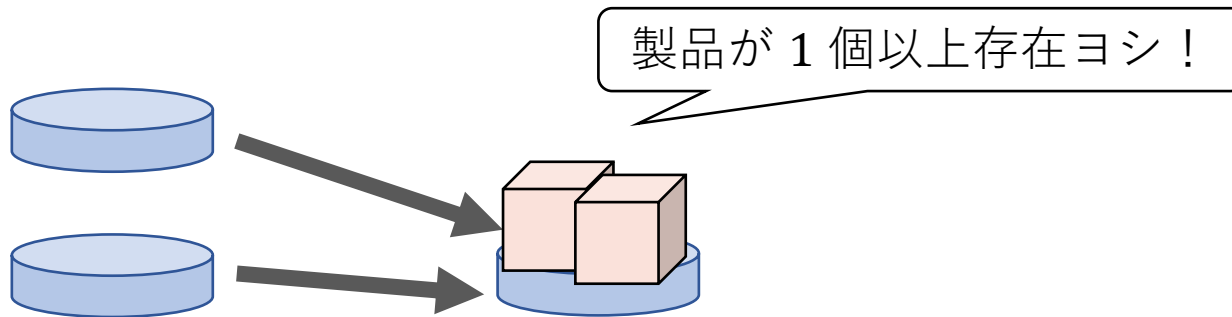
出ていく辺が無ければ動かない



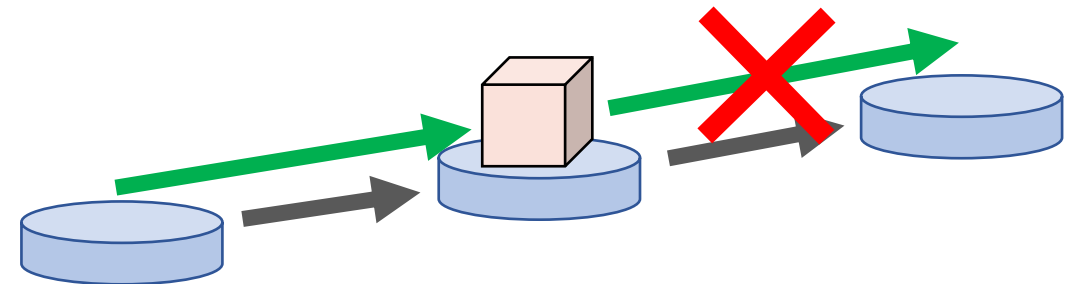
どの辺で移動するかは分からない



製品が合流しても分からない



移動は連鎖せず、1 回だけ

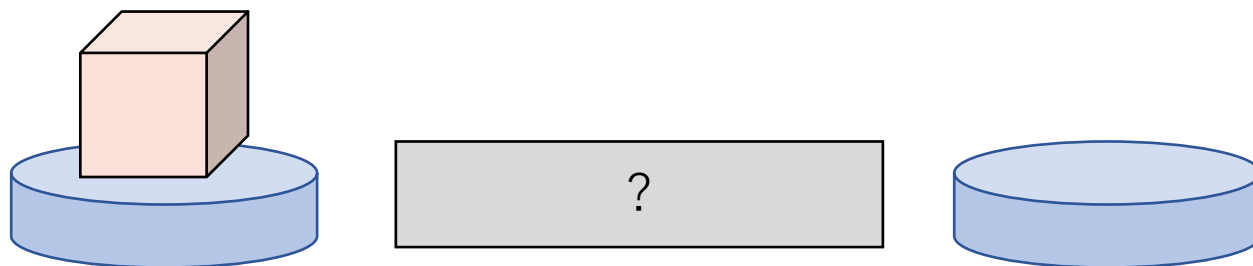


小課題

- ・ (1 点) $N = 2$
- ・ (14 点) $N = 30$
- ・ (10 点) $N = 100\,000$, 入力はパスグラフ
- ・ (75 点) $N = 100\,000$

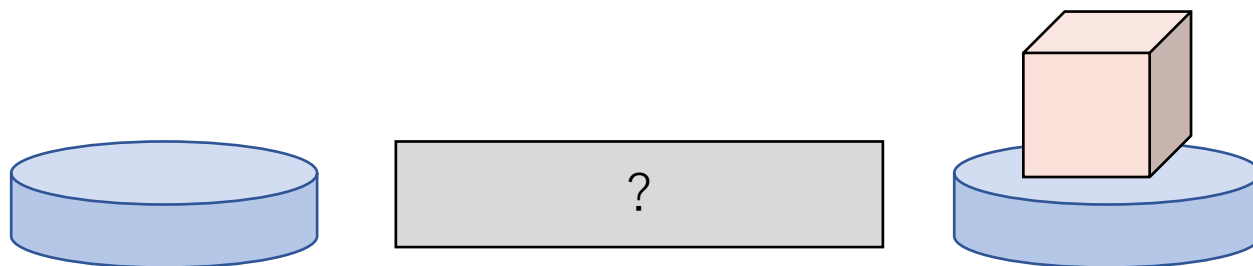
$$N = 2$$

とりあえず置いてみる



$$N = 2$$

とりあえず置いてみる



$$N = 2$$

とりあえず置いてみる



動いたら、その向きに辺が向いていることが確定

$$N = 2$$

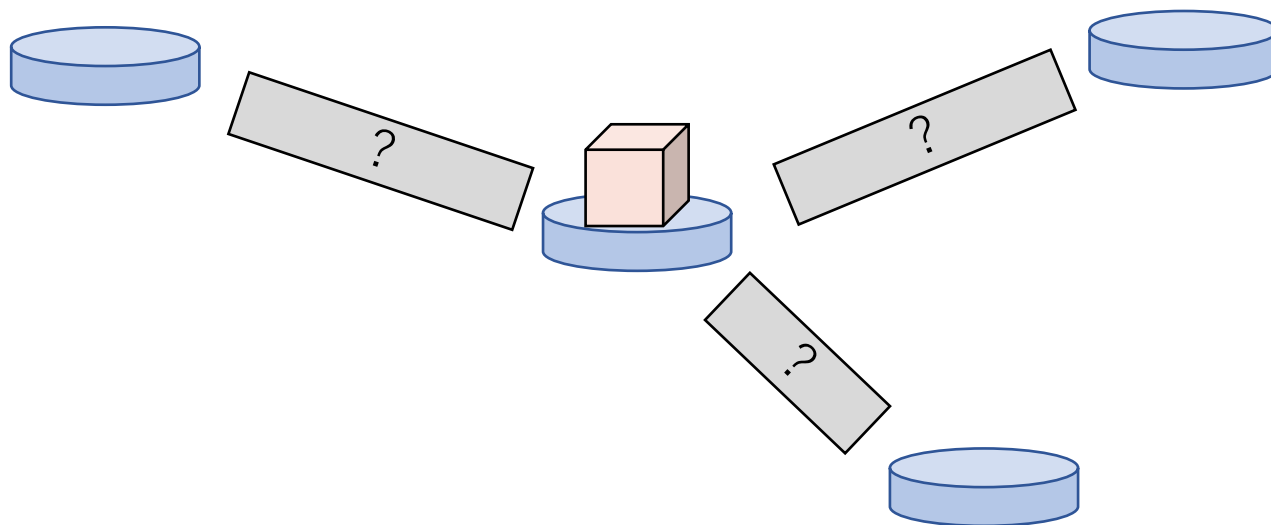
とりあえず置いてみる



留まったら、辺が製品側を向いていることが確定

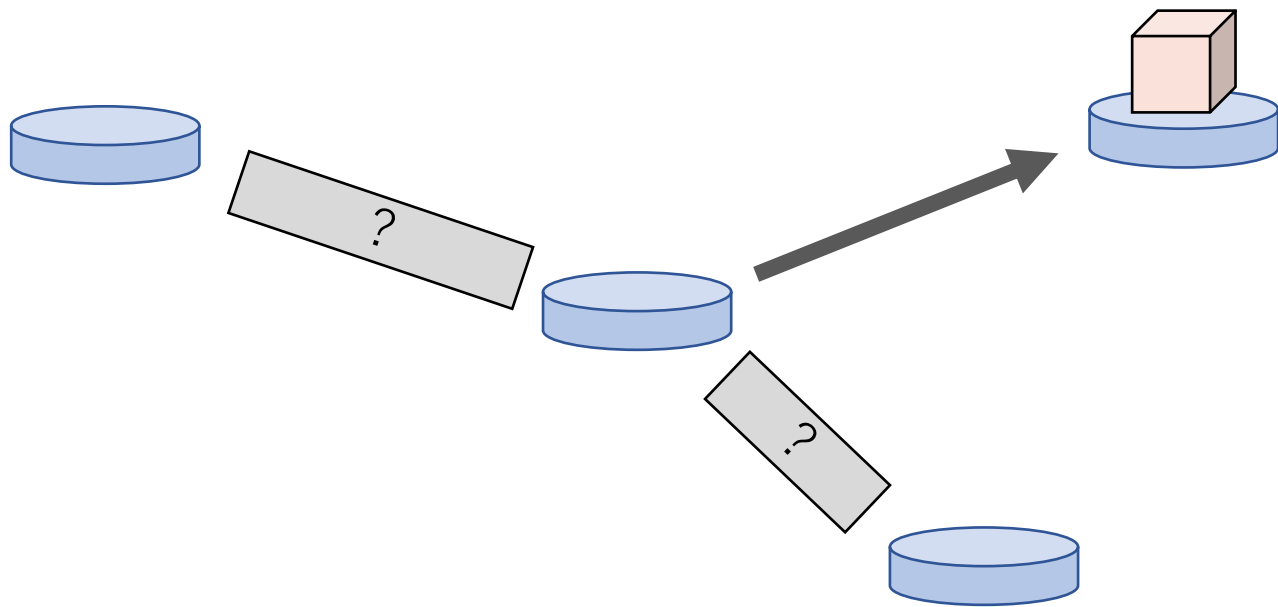
$$N = 30$$

製品を何個も置くと動きがよく分からないので、**1** 個だけ置いてみる



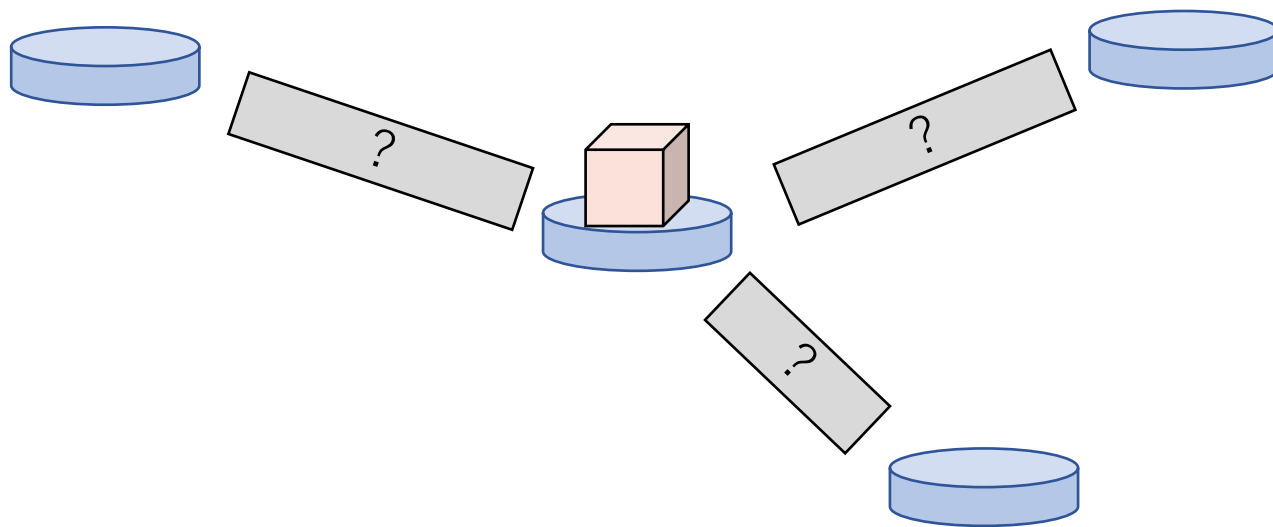
$$N = 30$$

製品を何個も置くと動きがよく分からないので、**1** 個だけ置いてみる



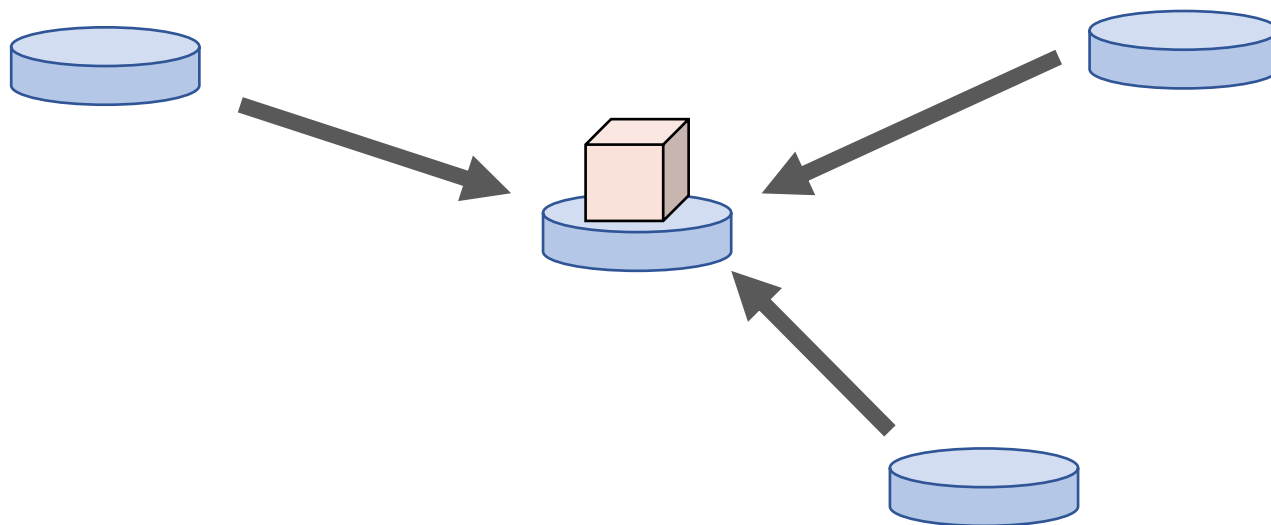
$$N = 30$$

製品を何個も置くと動きがよく分からないので、**1** 個だけ置いてみる



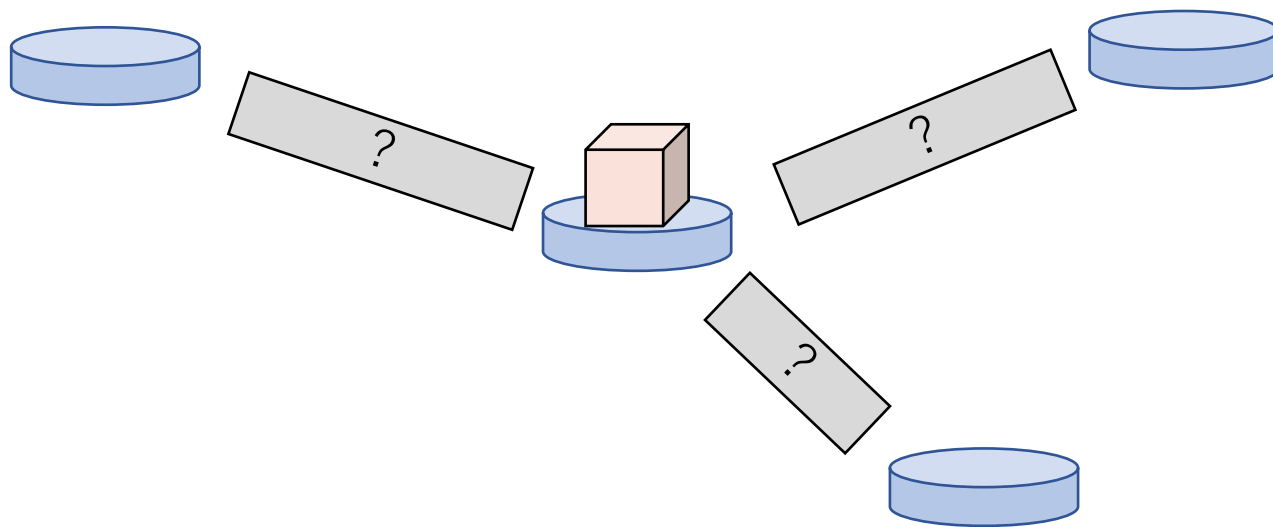
$$N = 30$$

製品を何個も置くと動きがよく分からないので、**1** 個だけ置いてみる



$$N = 30$$

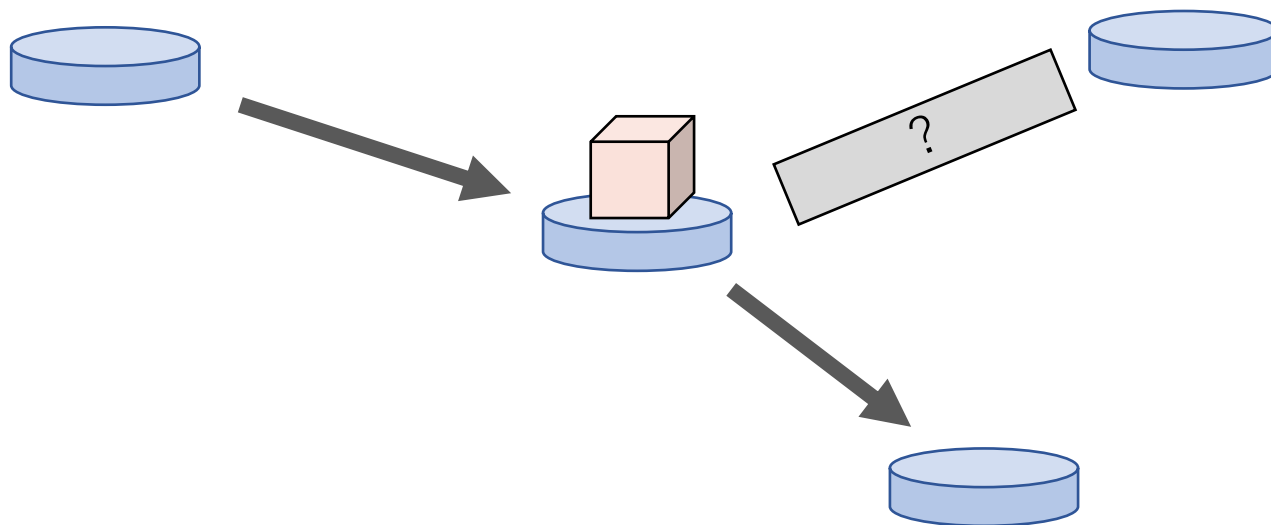
製品が動いても動かなくても、どこかの辺の向きは確定できた。
これを繰り返せば OK → 本当に？



$$N = 30$$

辺の向きが確定しないケース 1

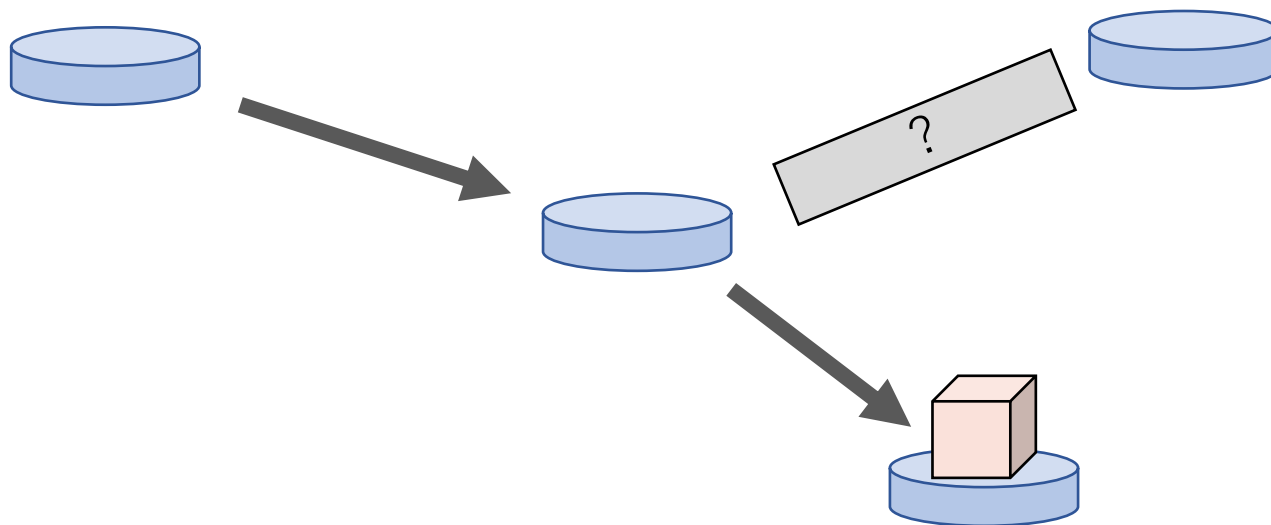
既に確定した辺に沿って流れてしまうと困る



$$N = 30$$

辺の向きが確定しないケース 1

既に確定した辺に沿って流れてしまうと困る

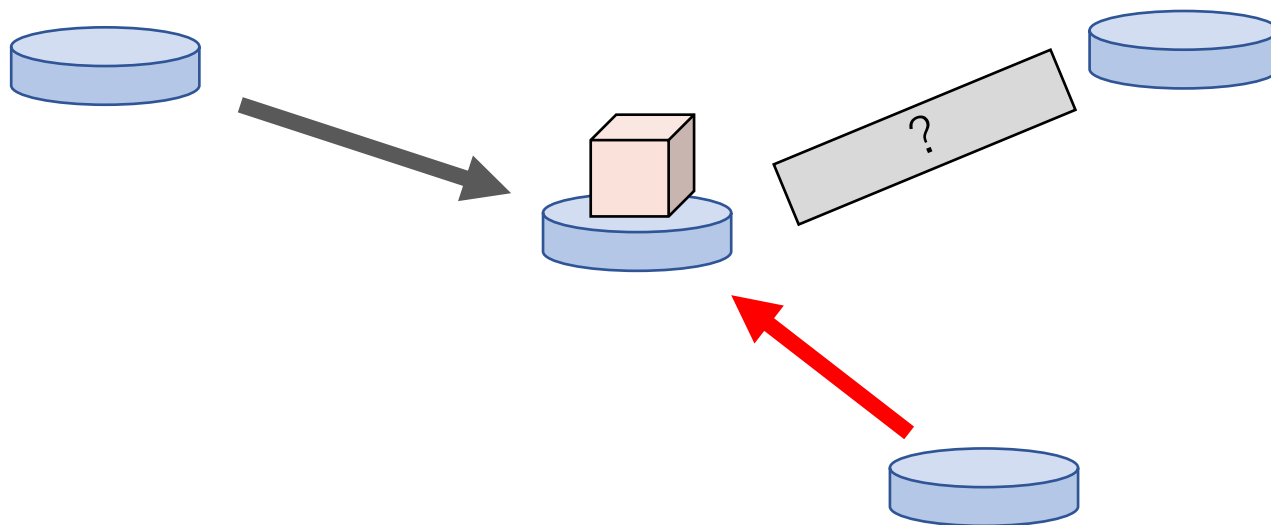


$$N = 30$$

辺の向きが確定しないケース 1

既に確定した辺に沿って流れてしまうと困る

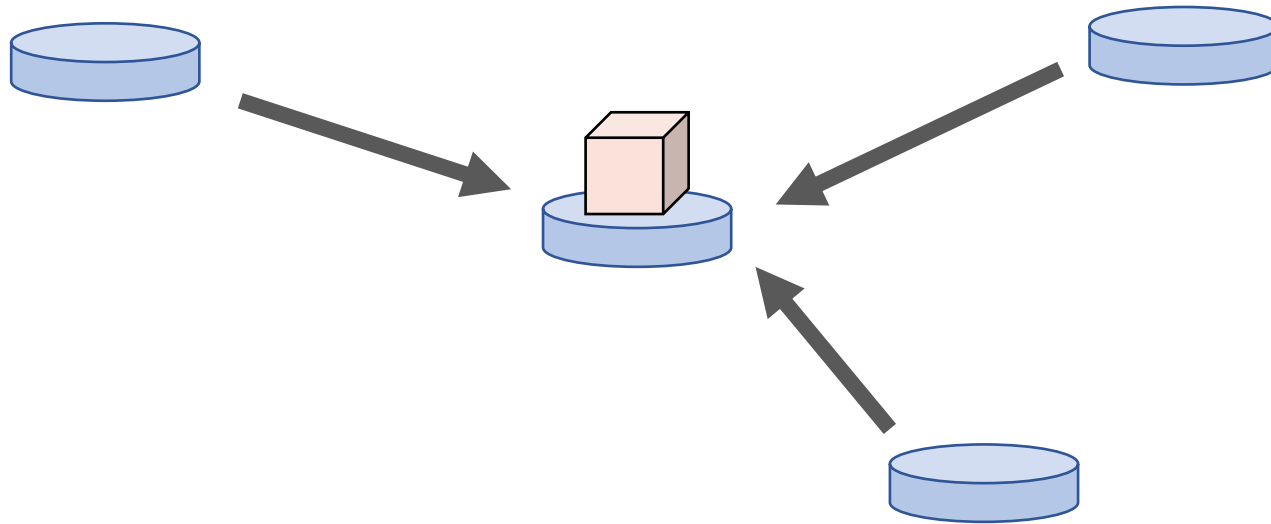
既知の辺は製品側に向けておく



$$N = 30$$

辺の向きが確定しないケース 2

既に確定した辺だけだと困る

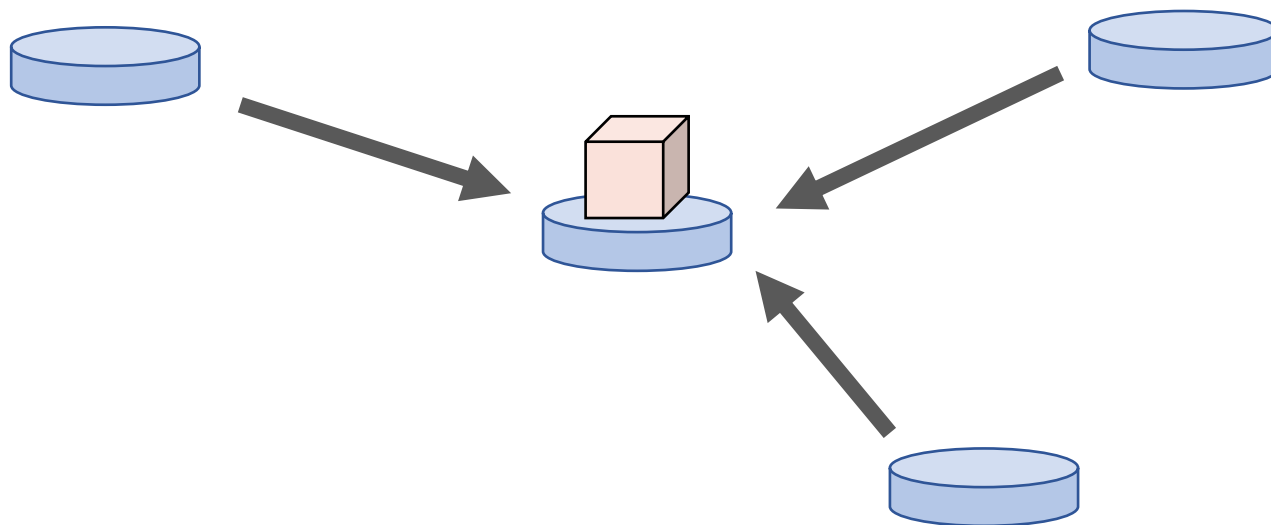


$$N = 30$$

辺の向きが確定しないケース 2

既に確定した辺だけだと困る

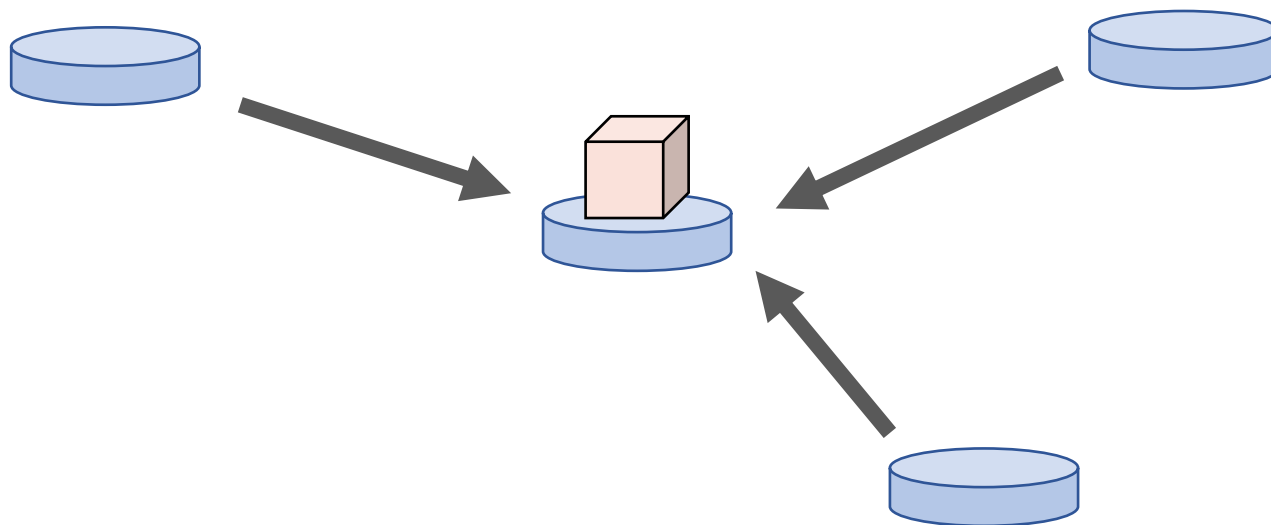
未確定の辺に接続する頂点を選んで製品を置く



$$N = 30$$

クエリの度に、少なくとも **1** 本の辺が確定する

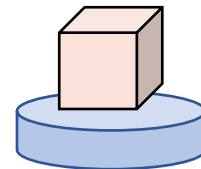
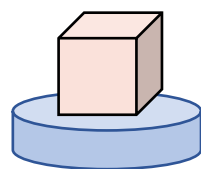
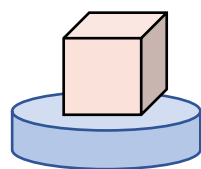
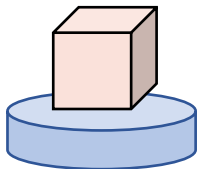
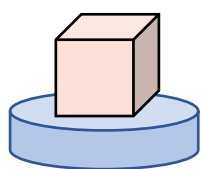
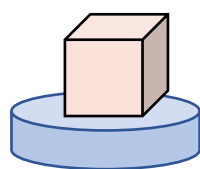
辺は $N - 1 = 29$ 本 $\rightarrow$ クエリは **29** 回以内



$N = 100\,000$, パスグラフ

1 回のクエリで同時に何本もの辺を確定させる必要がある

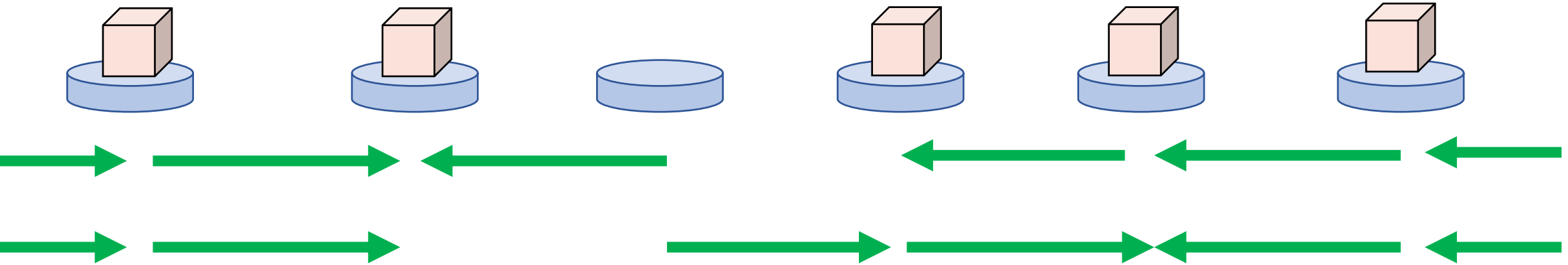
同時に複数の頂点に製品を置きたい



$N = 100\,000$, パスグラフ

1回のクエリで同時に何本もの辺を確定させる必要がある

同時に複数の頂点に製品を置きたい



どの製品がどう動いたのか分からない

$N = 100\,000$, パスグラフ

1 回のクエリで同時に何本もの辺を確定させる必要がある

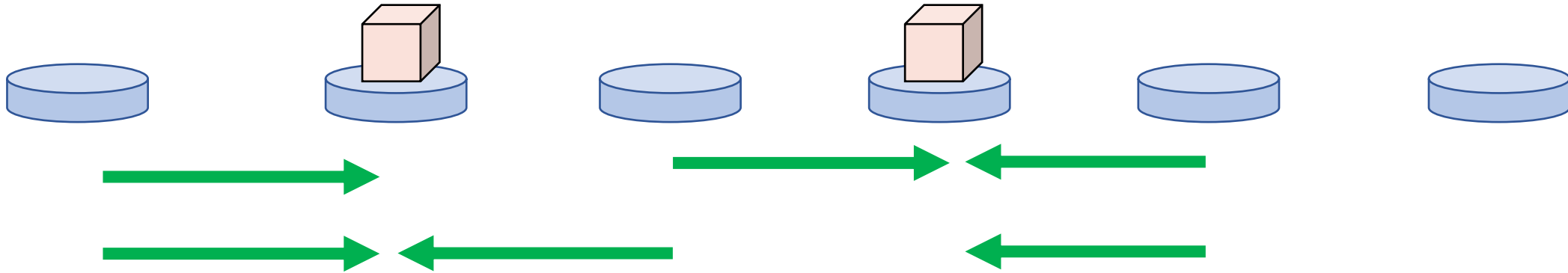
1 個おきに置くとどうか？



$N = 100\,000$, パスグラフ

1 回のクエリで同時に何本もの辺を確定させる必要がある

1 個おきに置くとどうか？

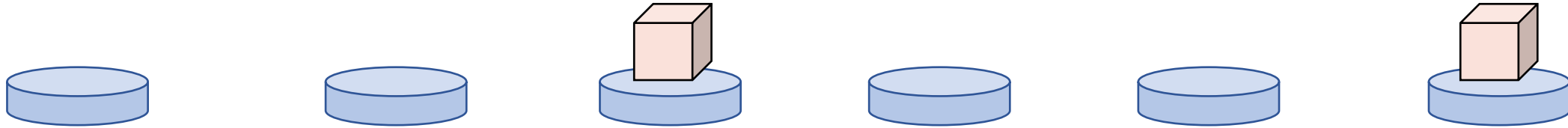


中央の製品がどちらに動いたのか分からない

$N = 100\,000$, パスグラフ

1 回のクエリで同時に何本もの辺を確定させる必要がある

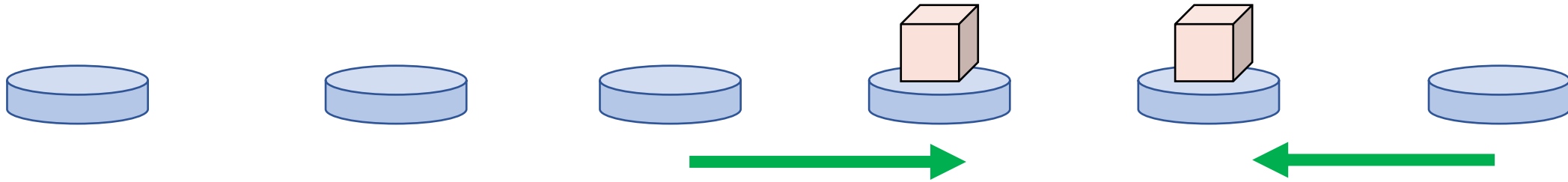
2 個おきに置くとどうか？



$N = 100\,000$, パスグラフ

1 回のクエリで同時に何本もの辺を確定させる必要がある

2 個おきに置くとどうか？

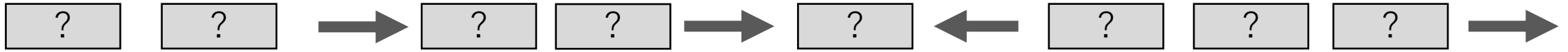


製品同士が干渉しないので、 $N/3$ 本の辺が確定する

$N = 100\,000$, パスグラフ

1 回のクエリで同時に何本もの辺を確定させる必要がある

2 個おきに置くとどうか？



これを繰り返して全ての辺を確定させる

$N = 100\,000$, パスグラフ

1 回のクエリで同時に何本もの辺を確定させる必要がある

2 個おきに置くとどうか？



未確定の辺に接続する頂点を、製品が **2** つおき以上離れるように貪欲に選ぶ

製品が隣接しないから、既知の辺を製品側に向ける処理も問題ない

$N = 100\,000$, パスグラフ

1 回のクエリで同時に何本もの辺を確定させる必要がある

2 個おきに置くとどうか？

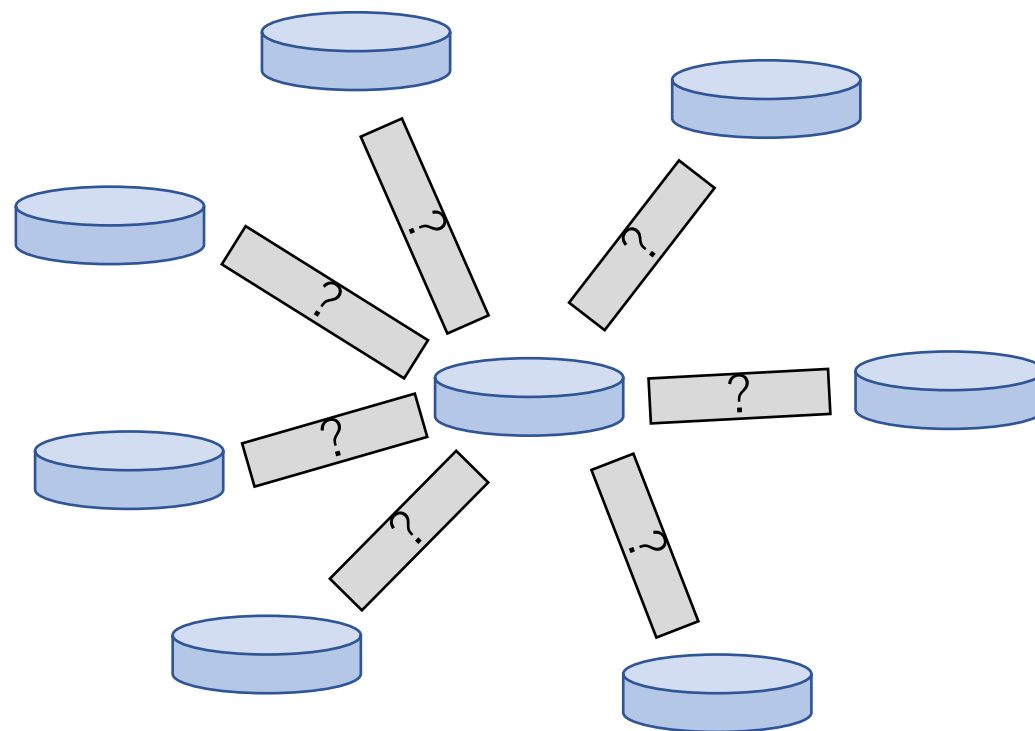


未確定の辺に接続する頂点を、製品が **2** つおき以上離れるように貪欲に選ぶ

4 回以内に全て確定する (証明略)

$$N = 100\,000$$

2 個おきに置く戦略を活用したい

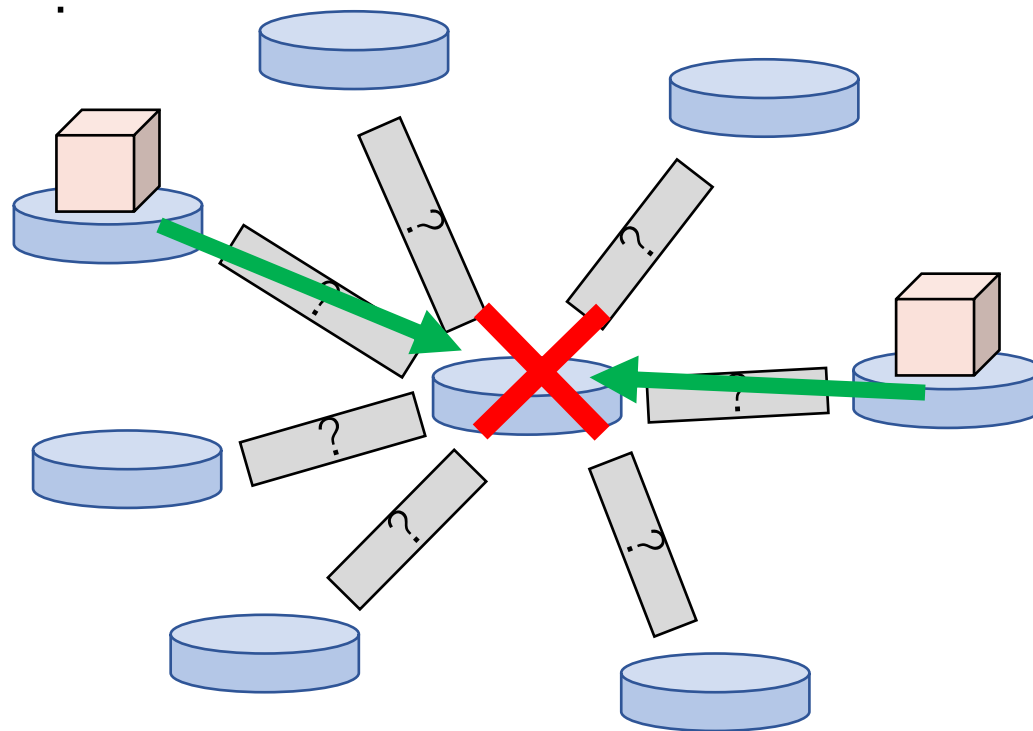


$N = 100\,000$

2 個おきに置く戦略を活用したい

製品の行き先が重ならないようにすると、スターグラフで困る

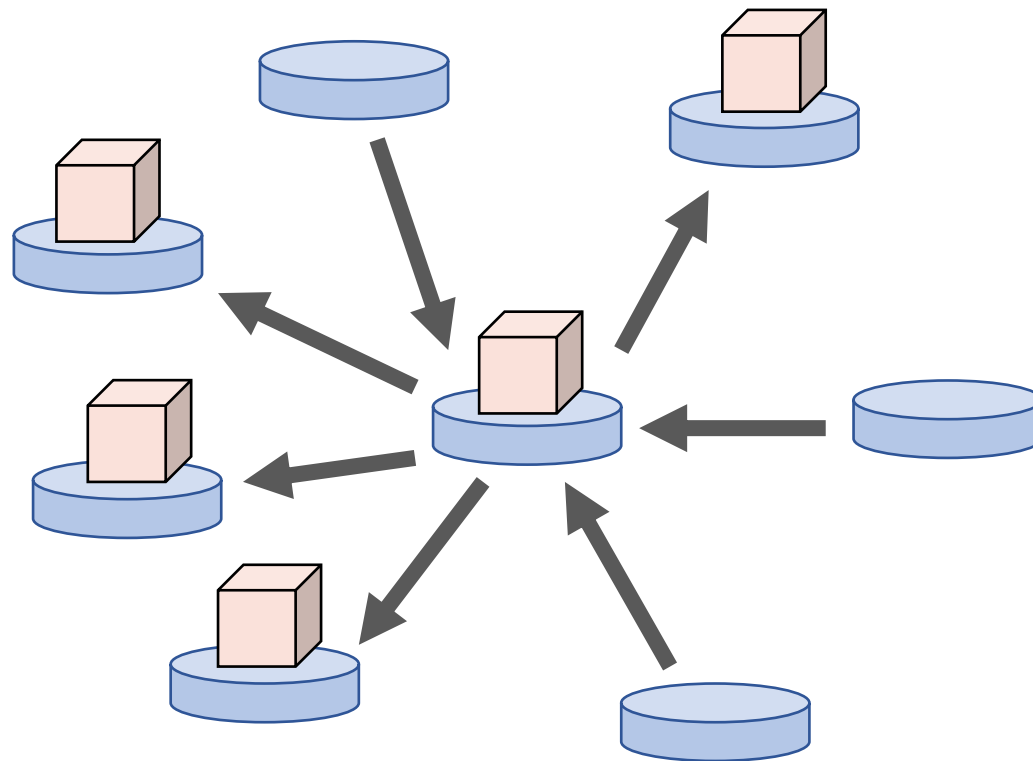
1 個ずつしか置けない！



$$N = 100\,000$$

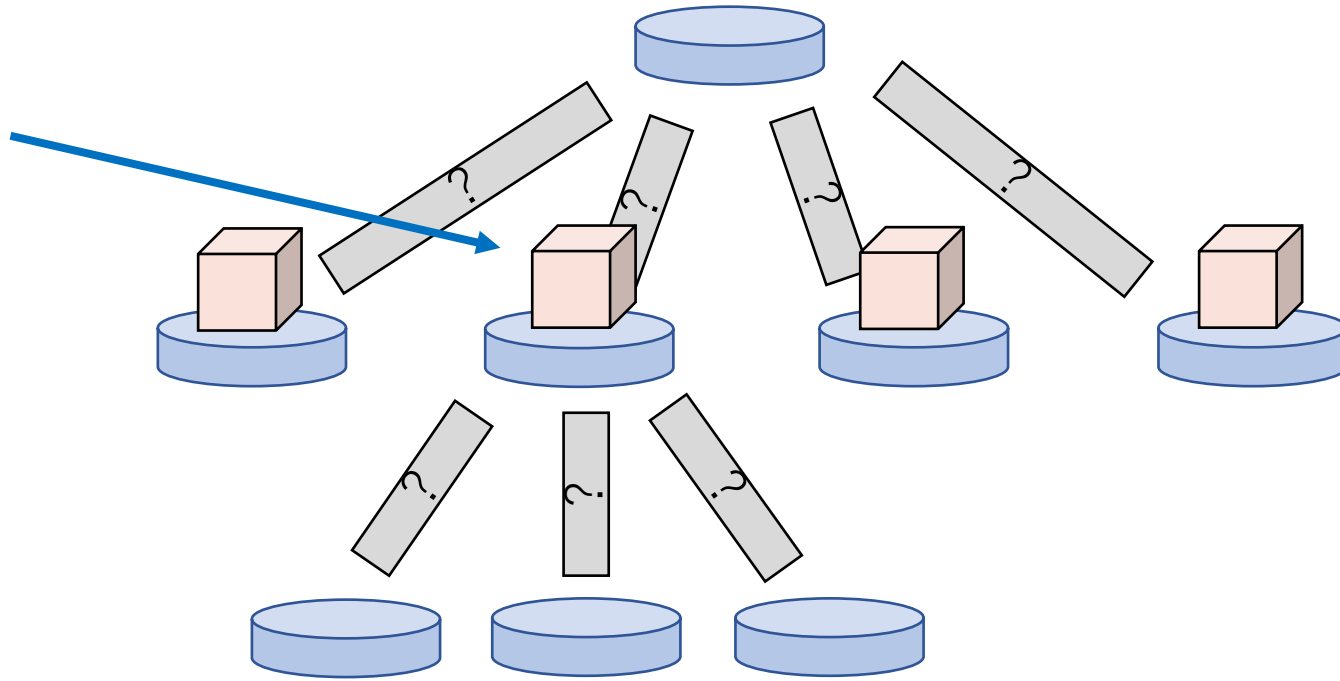
2 個おきに置く戦略を活用したい

スターグラフなら、葉に置いた製品が残っているか見れば、
中心で合流してしまっても大丈夫



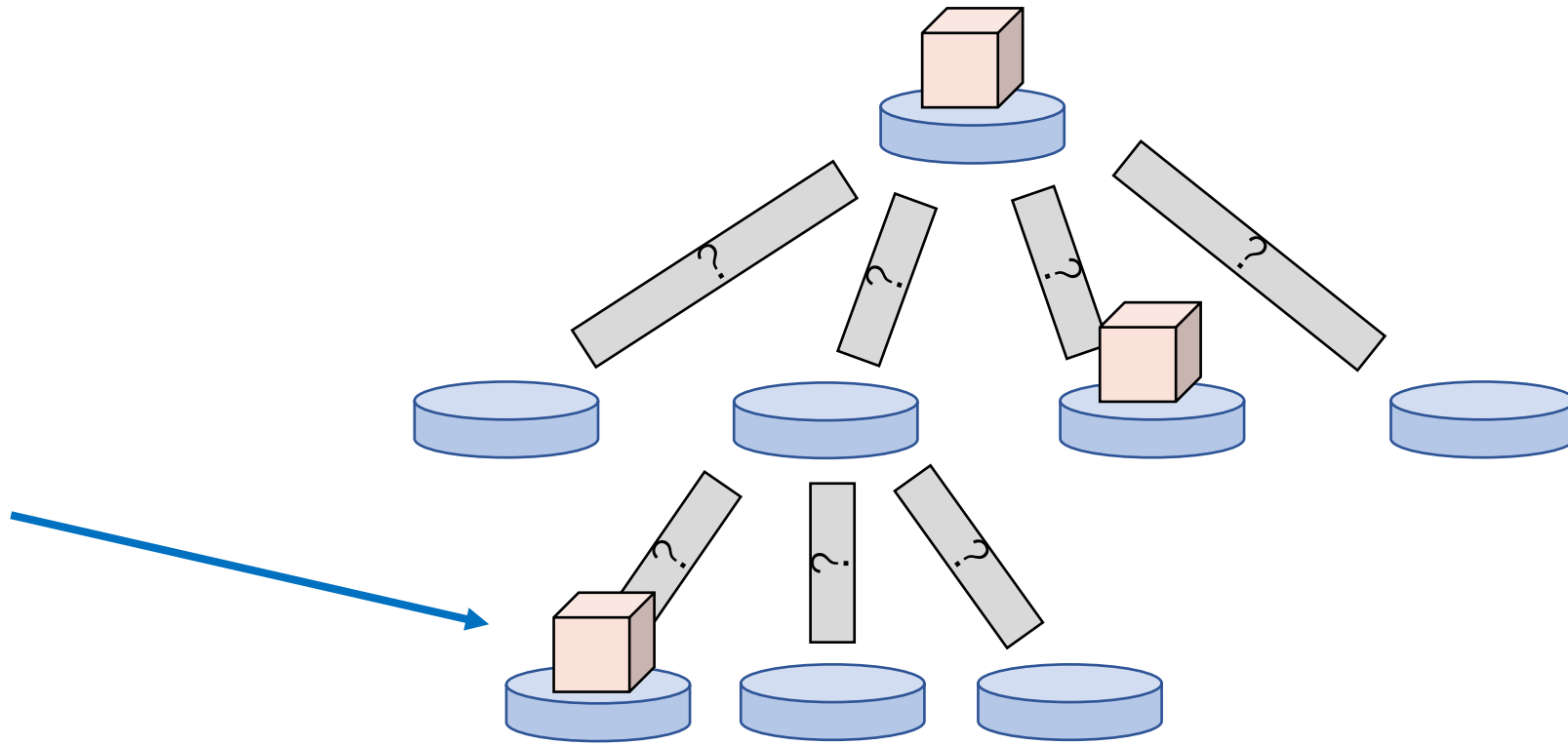
$$N = 100\,000$$

一般の木に対しても、似たような戦略が使えないだろうか？



$$N = 100\,000$$

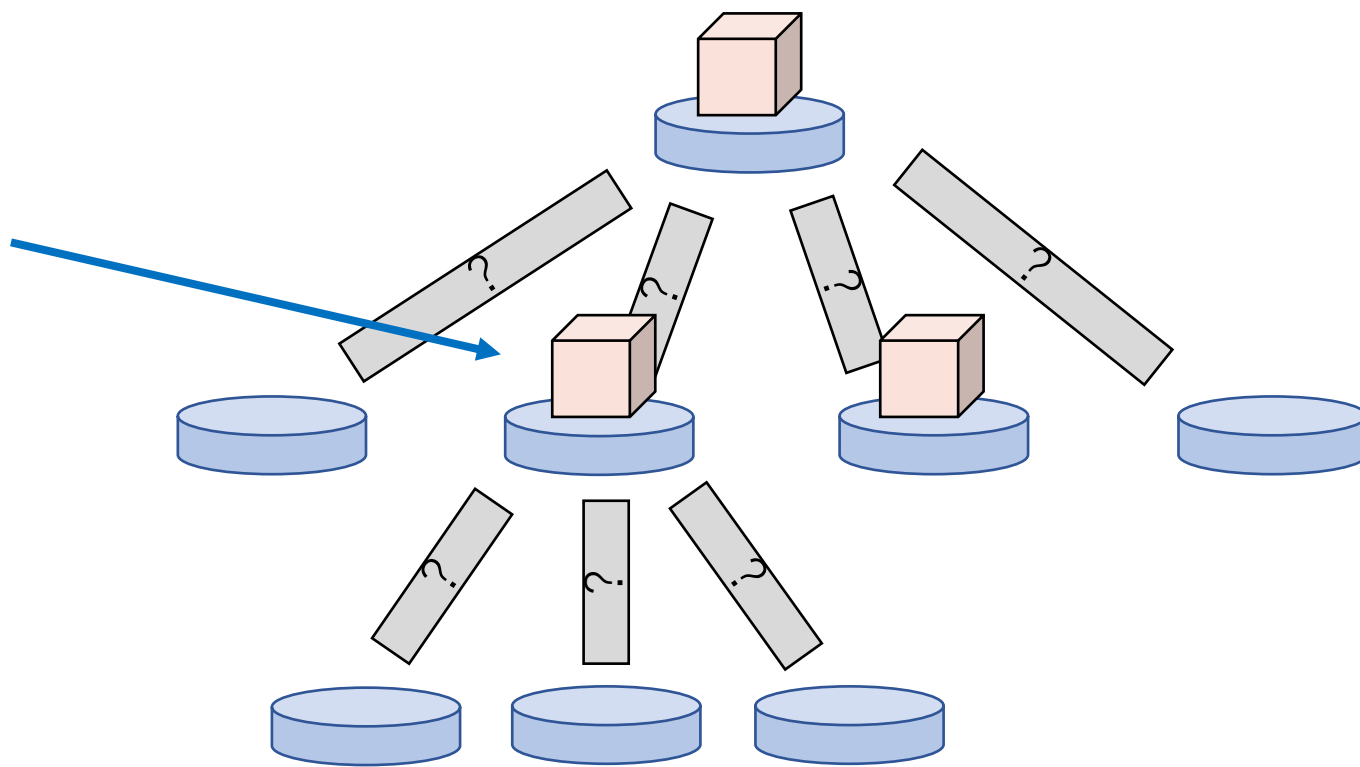
一般の木に対しても、似たような戦略が使えないだろうか？



下の台のどれかに移動していたら分かる。

$$N = 100\,000$$

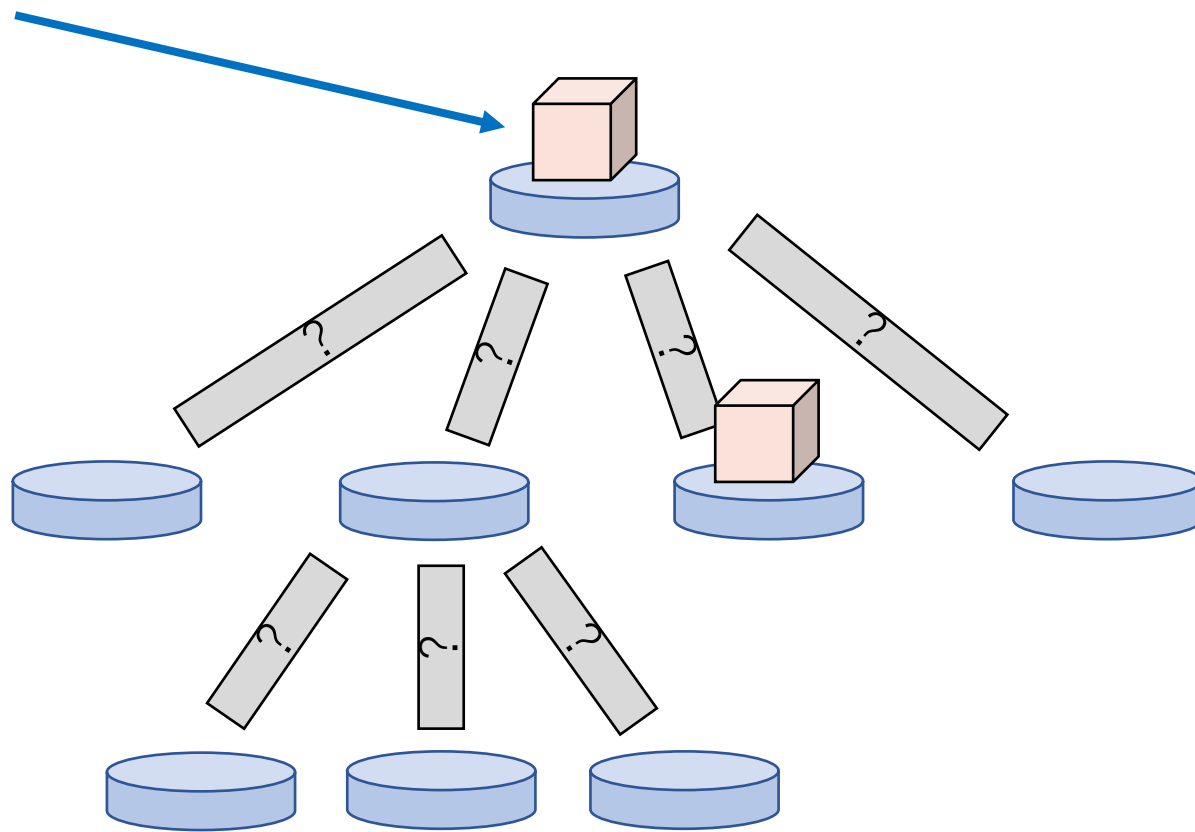
一般の木に対しても、似たような戦略が使えないだろうか？



置いた場所に留まっていた場合も分かる

$$N = 100\,000$$

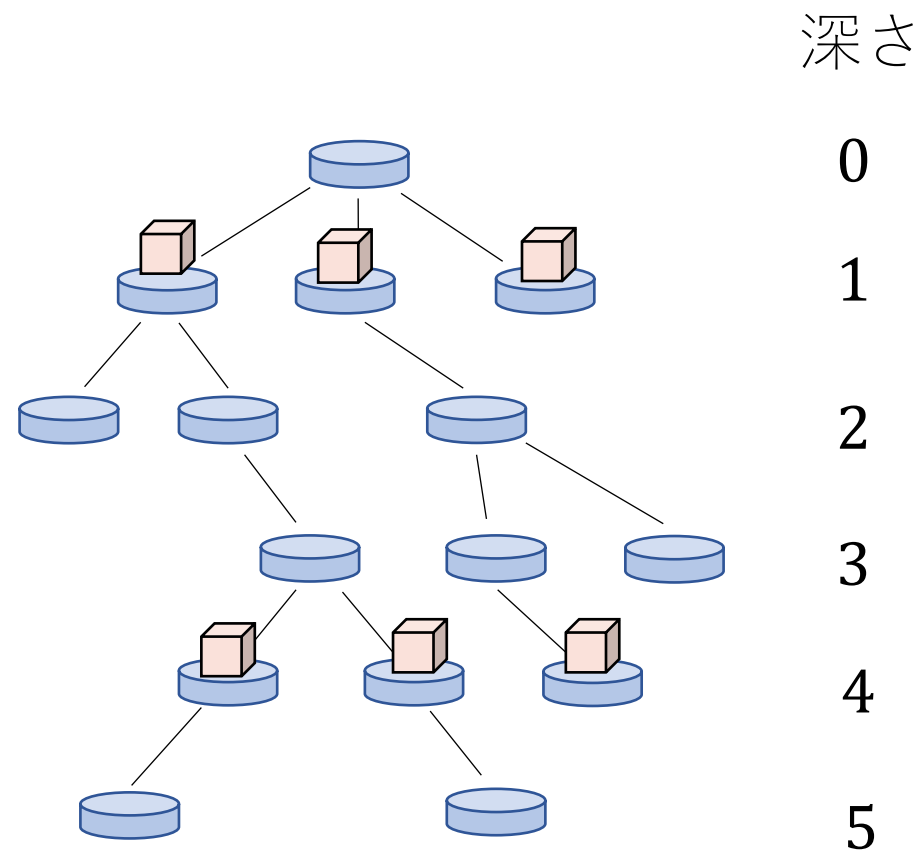
一般の木に対しても、似たような戦略が使えないだろうか？



上の台に行っていた場合、消去法で分かる。

$$N = 100\,000$$

根付き木にして、深さを **2** つおきに選び、製品を置く

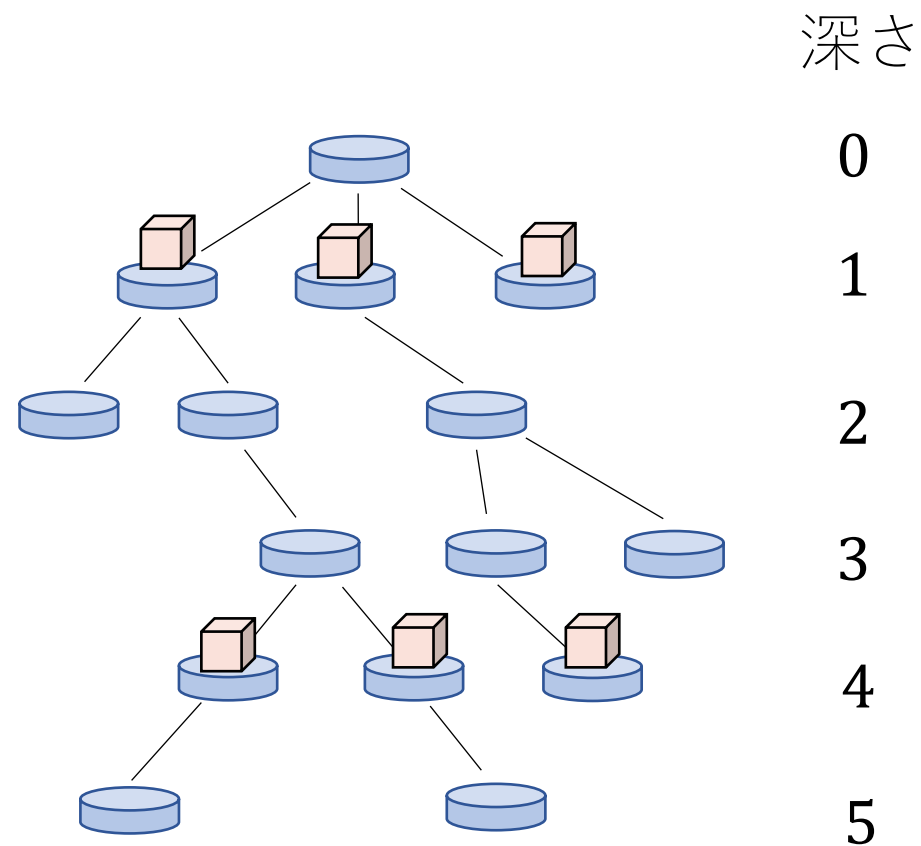


$$N = 100\,000$$

根付き木にして、深さを **2** つおきに選び、製品を置く

深さを mod **3** で分類し、最も大きいグループに製品を置く

$N/3$ 本以上の辺が確定する



$$N = 100\,000$$

未確定の辺に接続する頂点たちを深さを mod 3 で分類し、
最も大きいグループに製品を置く

- ・ 製品は隣接しない
 - ・ 既知の辺は邪魔にならない
 - ・ **1** 製品につき少なくとも **1** 本の辺が確定する
 - ・ 未確定の辺を ***E*** 本とすると、接続する頂点は ***E*** 個以上ある
- 少なくとも ***E*/3** 本の辺が新たに確定する

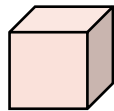
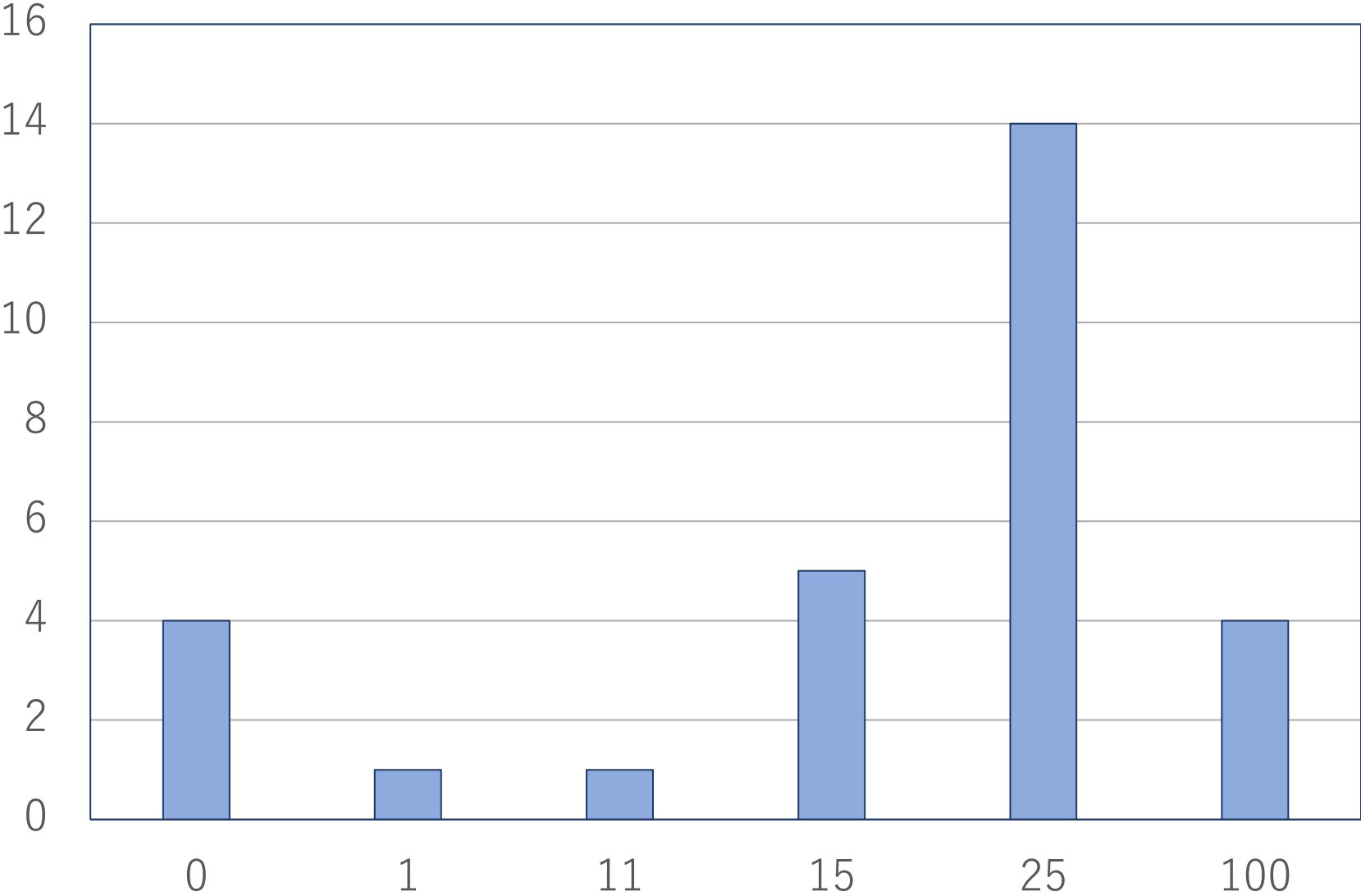
$$N = 100\,000$$

未確定の辺に接続する頂点たちを深さを mod 3 で分類し、
最も大きいグループに製品を置く

- ・ 製品は隣接しない
 - ・ 既知の辺は邪魔にならない
 - ・ 1 製品につき少なくとも 1 本の辺が確定する
- ・ 未確定の辺を E 本とすると、接続する頂点は E 個以上ある
→ 少なくとも $E/3$ 本の辺が新たに確定する

$\log_{1.5} N = 28.39\dots$ だから、30 回以内に全ての辺が確定する

統計情報



JOI 2022/2023 春季トレーニング Day 2

議会 (Council) 解説

解説担当 : tatyam

問題概要

- N 人の人がいて, M 個の案に投票を行う.
- $A_{i,j} = 1$ のとき, 人 i は案 j に賛成票を投じる.
- $i = 1, 2, \dots, N$ について, 以下の問題を解け.
 - $j (\neq i)$ を選び, 人 i と人 j を投票から除外する.
 - 過半数 ($\lfloor N/2 \rfloor$ 票以上) の賛成票を獲得する案の個数の最大値は?

$$N \leq 3 \times 10^5, \text{ **M} \leq \textbf{20}**$$

小課題 (JOI)

番号	配点	制約	計算量の例
1.	8 点	$N \leq 300$	$O(N^3 M)$
2.	8 点	$N \leq 3000$	$O(N^2 M)$
3.	6 点	$M \leq 2$	
4.	19 点	$M \leq 10$	$O(N 2^M)$
5.	15 点	$M \leq 14$	$O(NM + 4^M)$
6.	22 点	$M \leq 17$	$O(NM + 3^M M)$
7.	22 点	$M \leq 20$	$O(NM + 2^M M^2)$

小課題 (JOIG)

番号	配点	制約	計算量の例
1.	13 点	$N \leq 300$	$O(N^3 M)$
2.	17 点	$N \leq 3000$	$O(N^2 M)$
3.	10 点	$M \leq 2$	
4.	18 点	$M \leq 10$	$O(N 2^M)$
5.	12 点	$M \leq 14$	$O(NM + 4^M)$
6.	22 点	$M \leq 17$	$O(NM + 3^M M)$
7.	8 点	$M \leq 20$	$O(NM + 2^M M^2)$

小課題 1 ($N \leq 300$)

- $O(N^3 M)$ くらいの計算量で解けば良い.

小課題 1 ($N \leq 300$)

- $O(N^3 M)$ くらいの計算量で解けば良い.
- 議長 i を全探索: $O(N)$
- 副議長 j を全探索: $O(N)$
- それぞれの案に何票集まったかを集計: $O(NM)$

$\implies$ 全部合わせて $O(N^3 M)$

小課題 2 ($N \leq 3000$)

- 議長 i を全探索 : $O(N)$
- 副議長 j を全探索 : $O(N)$
- **それぞれの案に何票集まったかを集計 : $O(NM)$**

議長と副議長の 2 人しか変わらないのに、毎回集計し直すのは無駄！

小課題 2 ($N \leq 3000$)

- それぞれの案に何票集まっているかを前計算 : $O(NM)$

その後,

- 議長 i を全探索 : $O(N)$
- 副議長 j を全探索 : $O(N)$
- 議長と副議長の分を取り除く : $O(M)$

$\implies$ 全部合わせて $O(N^2M)$

小課題 2 ($N \leq 3000$)

- それぞれの案に何票集まっているかを前計算 : $O(NM)$

その後,

- 議長 i を全探索 : $O(N)$
- 副議長 j を全探索 : $O(N)$
- 議長と副議長の分を取り除く : $O(M)$

$\implies$ 全部合わせて $O(N^2M)$

実装テクニック：整数をビット列として扱おう

- ビット演算 `|` `&` `^` `<<` `>>` を使って、整数をビット列として扱うことができる.
- 議長と副議長を決めたときに、成立する案の個数を $O(M)$ で求めていたが、これを $O(1)$ にできる.

実装テクニック：整数をビット列として扱おう

A をビット列で入力する

```
vector<int> A(N);  
  
for(int i = 0; i < N; i++) for(int j = 0; j < M; j++) {  
    int a;  
    cin >> a;  
    A[i] |= a << j;  
}
```

考察

各案は

- 議長・副議長に関わらず可決
- 議長・副議長のどちらかが反対なら可決
- 議長・副議長の両方が反対なら可決
- 議長・副議長に関わらず否決

の 4 通りに分けられる. そのうち中央の 2 つのみが重要である.

実装テクニック：整数をビット列として扱おう

- 可決票数 + 2 票以上の賛成票を得ている案の集合
- 可決票数 + 1 票の賛成票を得ている案の集合
- 可決票数 + 0 票の賛成票を得ている案の集合

をそれぞれ整数で持つ

```
for(int j = 0; j < M; j++) {  
    if(vote[j] >= N / 2 + 2) two |= 1 << j;  
    if(vote[j] == N / 2 + 1) one |= 1 << j;  
    if(vote[j] == N / 2) zero |= 1 << j;  
}
```

実装テクニック：整数をビット列として扱おう

- 議長 i と副議長 j を決めたら，可決される案の集合が $O(1)$ で求められる

```
int accept = two | one & ~(A[i] & A[j]) | zero & ~(A[i] | A[j]);
```

あとは，これの立っているビットの個数を数えれば良い.

popcount

- C++20

どうして...

function template

std::popcount ⚡ C++20

```
namespace std {  
    template <class T>  
    constexpr int popcount(T x) noexcept;  
}
```

概要

立っているビットを数える。popcountは「population count」の略。 15

popcount

- `std::bitset::count` を使う

```
int popcnt(int x) { return bitset<32>(x).count(); }
```

- `__builtin_popcount` を使う

```
int popcnt(int x) { return __builtin_popcount(x); }
```

これらは通常賢いビット演算をしている `__popcountdi2` にコンパイルされるが、今回はコンパイルオプションに `-march=native` が指定されていたので、`POPCNT` 命令にコンパイルされる 🙌🙌

popcount

- 自分で実装する

```
int popcnt(int x) {  
    int ans = 0;  
    while(x) {  
        ans += x & 1;  
        x >>= 1;  
    }  
    return ans;  
}
```

計算量は $O(M)$

popcount

- 全部前計算しておく

```
int popcnt(int x) {  
    static array<int, 1 << 20> table = []{  
        // 前計算する  
    }();  
    return table[x];  
}
```

計算量は $O(1)$

クイズ

どっちが速い？

[A]

```
int popcnt(int x) {  
    int ans = 0;  
    while(x) {  
        ans += x & 1;  
        x >>= 1;  
    }  
    return ans;  
}
```

計算量は $O(M)$

[B]

```
int popcnt(int x) {  
    static array<int, 1 << 20> table = []{  
        // 前計算する  
    }();  
    return table[x];  
}
```

計算量は $O(1)$

答え

[A]

```
int popcnt(int x) {  
    int ans = 0;  
    while(x) {  
        ans += x & 1;  
        x >>= 1;  
    }  
    return ans;  
}
```

計算量は $O(M)$

- キャッシュの効かないメモリアクセスは、ビット演算や足し算に比べて非常に遅い！！

popcnt

- 賢いビット演算

```
int popcnt(int x) {  
    x = (x & 0x55555555) + (x >> 1 & 0x55555555);  
    x = (x & 0x33333333) + (x >> 2 & 0x33333333);  
    x = (x & 0x0f0f0f0f) + (x >> 4 & 0x0f0f0f0f);  
    x = (x & 0x00ff00ff) + (x >> 8 & 0x00ff00ff);  
    return (x & 0x0000ffff) + (x >> 16);  
}
```

計算量は $O(\log M)$

小課題 3 ($M \leq 2$)

- なんと $M \leq 2$
- $M = 2$ のとき, $(0, 0), (0, 1), (1, 0), (1, 1)$ の 4 種類のタイプの人間がいる.

こんな例を考える.

	案 1	案 2
人 1	0	1
人 2	0	1
人 3	1	0
人 4	1	0
人 5	1	1
合計	3	3

$$N = 5, M = 2$$

可決票数: 2 票

人 1 が議長の場合と人 2 が議長の場合で、状況が全く同じ.

	案 1	案 2
人 2	0	1
人 3	1	0
人 4	1	0
人 5	1	1
合計	3	2

	案 1	案 2
人 1	0	1
人 3	1	0
人 4	1	0
人 5	1	1
合計	3	2

つまり、答えも同じ.

結局, 「議長と副議長が 4 種類のタイプのうちどのタイプの人間か？」が分かれば可決される案も分かる.

副議長が:	(0, 0)	(0, 1)	(1, 0)	(1, 1)
議長が: (0, 0)	2	2	2	2
(0, 1)	2	1	2	1
(1, 0)	2	2	1	1
(1, 1)	2	1	1	0

この $4 \times 4 = 16$ 種類を調べれば良い.

小課題 4, 5 ($M \leq 10, M \leq 14$)

- M が小さめ
- $2^{14} \approx 1.6 \times 10^4 \ll 3 \times 10^5$ なので, 同じタイプの人間を圧縮することで N を小さくできる.
- 小課題 3 で $4 \times 4 = 16$ 通りを調べたのと同様に, $2^M \times 2^M = 4^M$ 通りを全部調べることができる.

計算量は $O(4^M)$

小課題 6 ($M \leq 17$)

- どうにかして計算量を $O(3^M \text{poly}(M))$ にしたい.

小課題 6 ($M \leq 17$)

i, j を決めたとき, 可決される案の集合は以下のように計算された.

```
accept := two | one & ~(A[i] & A[j]) | zero & ~(A[i] | A[j])
```

ここで i を固定すると,

- j によって可決されるかわ変わる案 (可決票数 + 0 票)
- j に関わらず可決される案 (可決票数 + 1 票以上)

の 2 つを持てば良い.

小課題 6 ($M \leq 17$)

i を固定すると,

- j によって可決されるか変わる案 (可決票数 + 0 票)
- j に関わらず可決される案 (可決票数 + 1 票以上)

の 2 つを持てば良い.

```
ONE := two | (one & ~A[i])  
ZERO := (one & A[i]) | (zero & ~A[i])  
accept := ONE | (ZERO & ~A[j])
```

小課題 6 ($M \leq 17$)

```
accept := ONE | (ZERO & ~A[j])
```

- `popcnt(accept)` を最大化したい.
- つまり, `popcnt(ZERO & ~A[j])` を最大化したい.
- 案の全体集合を `ZERO` に絞ったときの, 0 の個数が (人 i を除いて) 最も多い人を探したい.

小課題 6 ($M \leq 17$)

すべての $x \subset \text{ZERO}$ であるような (x, ZERO) の組 (3^M 個) について, 以下が求めれば良い.

- x に含まれる案に反対で, $\text{ZERO} \setminus x$ に含まれる案に賛成であるような人の数 (ZERO に含まれない案については 賛成 / 反対 どちらでもよい)

これは, 高速ゼータ変換 (M 次元累積和) をして, ZERO の部分だけ戻すことを全ての ZERO について行えば, $O(3^M M)$ 時間になる.

小課題 7 ($M \leq 20$)

- 高速ゼータ変換をどうにかして計算量を $O(2^M \text{poly}(M))$ にしたい.

小課題 7 ($M \leq 20$)

```
accept := ONE | (ZERO & ~A[j])
```

→ わかりやすさのため $A[j]$ を反転させておく.

各 ZERO に対し,

$$\max_{j \neq i} |\text{ZERO} \cap A_j|$$

を求める問題である.

小課題 7 ($M \leq 20$)

$$\max_{j \neq i} |\text{ZERO} \cap A_j|$$

$j \neq i$ の制約が面倒なので、一旦忘れると

$$\max_j |\text{ZERO} \cap A_j|$$

となる.

$$\max_j |\text{ZERO} \cap A_j|$$

A_j を ZERO まで減らしたときの集合の大きさ (popcount) の最大値であるので、各 A_j から要素を減らしていったときに現れる集合を前計算しておく。

$$B = \{b \mid \exists j : b \subset A_j\}$$

これは下位集合への累積 OR で $O(2^M M)$ でできる。問題は

$$\max\{|b| \mid b \in B, b \subset \text{ZERO}\}$$

となった。

$$\max\{|b| \mid b \in B, b \subset \text{ZERO}\}$$

今度は、各 b から要素を増やしていった、ZERO まで増やしたときの元々の集合の大きさ (popcount) の最大値という問題である。

したがって、上位集合への chmax をすることで、各 ZERO に対する答えが $O(2^M M)$ で求められる。

まとめ

$j \neq i$ の制約を忘れると,

1. bool 配列 `dp1[a]` を作る
2. 各 A_j に対し, `dp1[A[j]]` に 1 を代入
3. 下位集合に累積 OR
4. int 配列 `dp2[b]` を作る
5. 各 b に対し, `dp1[b] == 1` であれば, `dp2[b]` に `popcnt(b)` を代入
6. 上位集合に累積 chmax

自分自身の分を取り除くには？

自分自身の分を取り除くには？

→ 個数を数えれば良い！

1. **int** 配列 `dp1[a]` を作る
2. 各 A_j に対し, `dp1[A[j]]` に 1 を加算
3. 下位集合に累積和
4. **int** 配列 `dp2[k][b]` を作る (**popcount** の値ごとに個数を数える)
5. 各 b に対し, `dp2[popcnt(b)][b]` に `dp1[b]` を加算
6. **popcount** の値ごとに, 上位集合に累積和

自分自身の分を取り除くには？

1. int 配列 `dp1[a]` を作る
2. 各 A_j に対し, `dp1[A[j]]` に 1 を加算
3. 下位集合に累積和
4. int 配列 `dp2[k][b]` を作る (**popcount** の値ごとに個数を数える)
5. 各 b に対し, `dp2[popcnt(b)][b]` に `dp1[b]` を加算
6. **popcount** の値ごとに, 上位集合に累積和

自分自身の影響は簡単に数えられるので, その分を引けば良い.

計算量は $O(2^M M^2)$

$O(2^M M)$ にもなりますが, 省略します.

統計情報
(JOI)

点数	1	2	3	4	5	6	7	人数	累積人数
100	○	○	○	○	○	○	○	10	10
56	○	○	○	○	○	-	-	5	15
41	○	○	○	○	-	-	-	6	21
33	○	-	○	○	-	-	-	1	22
22	○	○	○	-	-	-	-	1	23
16	○	○	-	-	-	-	-	3	26
0	-	-	-	-	-	-	-	1	27

統計情報
(JOIG)

点数	1	2	3	4	5	6	7	人数	累積人数
58	○	○	○	○	-	-	-	3	3
40	○	○	○	-	-	-	-	2	5
30	○	○	-	-	-	-	-	5	10

水ようかん 2 (Mizuyokan 2) 解説

平木 康傑

問題概要

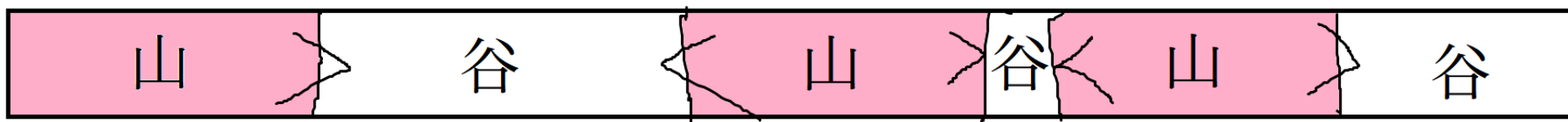
- 長さ N の正の整数列 $L_1, \dots, L_N$ がある.
- 「 L_x を y に更新」 「連続部分列 $L_a, \dots, L_b$ に対して以下の問題を解く」の2つのクエリを処理せよ.
 - 問題: パートごとの総和が**ジグザグ**になるように分割するとき、最大で何個に分割できる？
 - **ジグザグ**とは、(狭義の)増加・減少が交互に起こることとする.

例

- 1, 2, 3, 4, 5, 5, 6, 3 をジグザグに分割する方法を考える.
- $1 / 2 + 3 / 4 / 5 + 5 / 6 + 3$ のように分割すると,
 $1 < 5 > 4 < 10 > 9$ となるので **条件を満たす**.
- $1 + 2 / 3 / 4 / 5 + 5 / 6 + 3$ のように分割すると,
 $3 = 3 > 4 < 10 > 9$ となる. $=$ は狭義の増加・減少ではないので, **条件を満たさない**.

言葉はあった方が便利

- 和が大きい方の部分を「山」，
和が小さい方の部分を「谷」と呼ぶことにする．



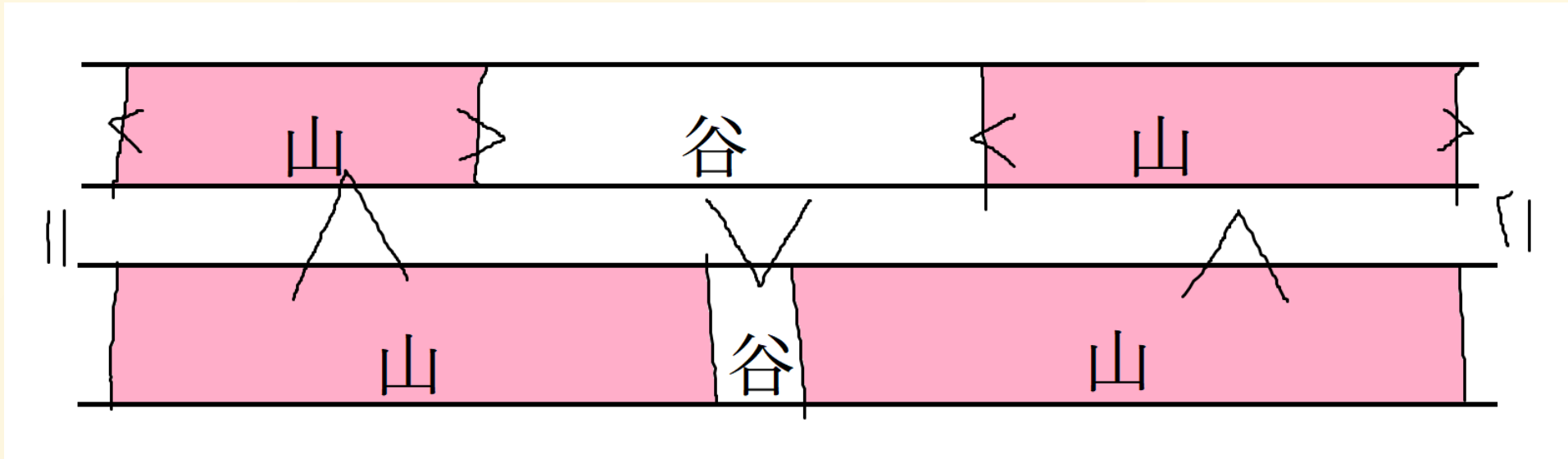
- $1 / 2 + 3 / 4 / 5 + 5 / 6 + 3$ という分割なら
谷，山，谷，山，谷．

小課題

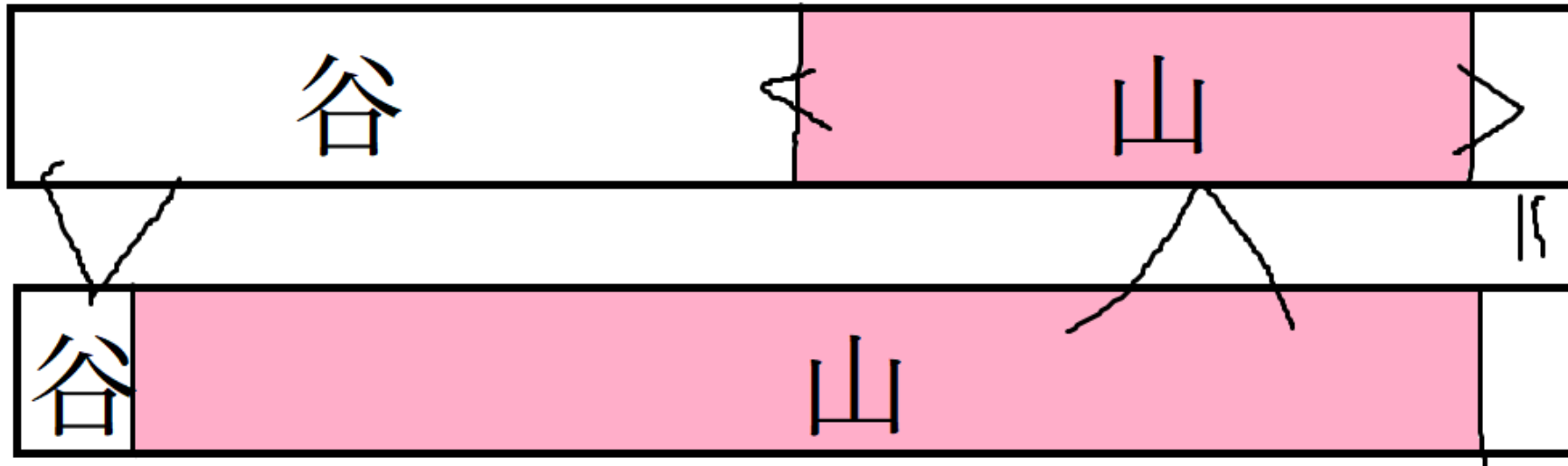
1. $N \leq 200, Q \leq 10$
2. $N \leq 2\,000, Q \leq 10$
3. $Q \leq 10$
4. 更新なし
5. 数列の要素は 120,000 以下
6. 追加の制約はない

- 小課題 1 は区間 DP で、小課題 2 はその高速化で解ける。

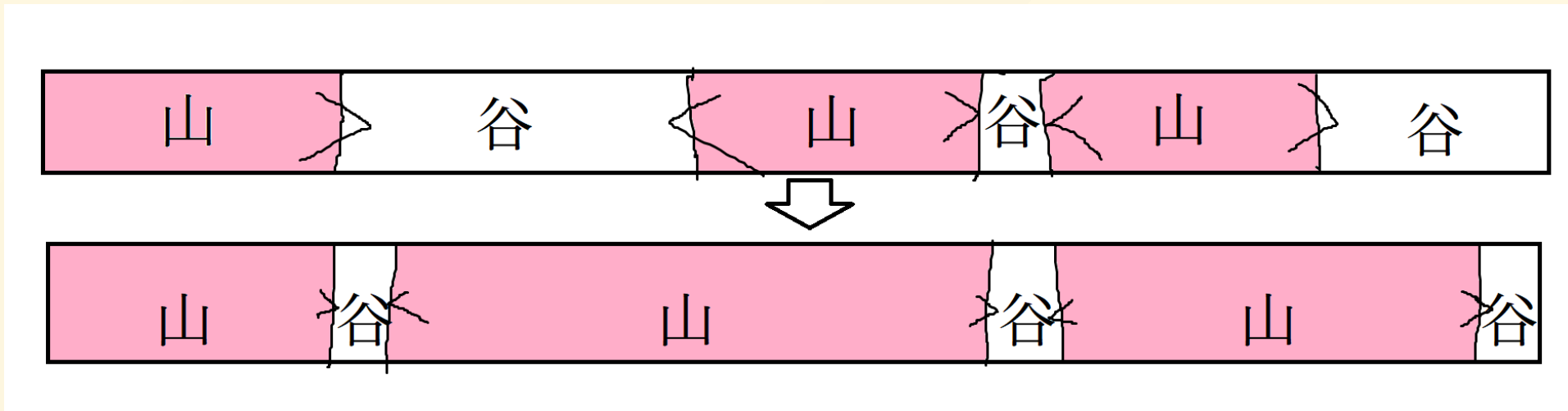
考察 1-a: 2つの山の間にある谷について，両隣の山をのばすことで，谷の長さを1にできる．



考察 1-b: 端にある谷について，隣の山を適切にのばすことで，谷の長さを 1 にできる．



考察 1: 任意の valid な分割に対して，山を適切にのばせば，もとと同じ分割個数で谷の長さがすべて 1 であるような valid な分割が作れる．



- つまり，谷の長さが 1 であるような分割のみを考えれば十分．

考察 1+: 谷の長さが 1 であるような分割のみを考えれば十分.

- 次のような DP をすればよい:
 - $dp[i] := L_i$ が谷であるときの, そこまでの分割個数の最大値.
- $L_i < L_{i+1} + \dots + L_k$ を満たす k は, $k_i \leq k \leq N$ の形で表せる.
- $L_k + \dots + L_{i-1} > L_i$ を満たす k は, $0 \leq k \leq k'_i$ の形で表せる.
- 上の性質から, セグメント木などを使った高速化が効く(詳細略).
 - 小課題 3 ($Q \leq 10$) は通る.

谷と山の両方が絡むとややこしい

- 先ほどは「谷」のみに注目する方針をとった.
- 「山」のみに注目する方針が取れないか？

山のみ注目したい

- 両隣の要素のみを見て判断できるような山の亜種を考える.
- $L_{l-1} < L_l + \dots + L_r > L_{r+1}$ ($L_{-1} = L_{N+1} = -\infty$ とする)を満たすような区間 $[l, r]$ を**和歌山**と呼ぶことにする.
 - 山は必ず和歌山である.



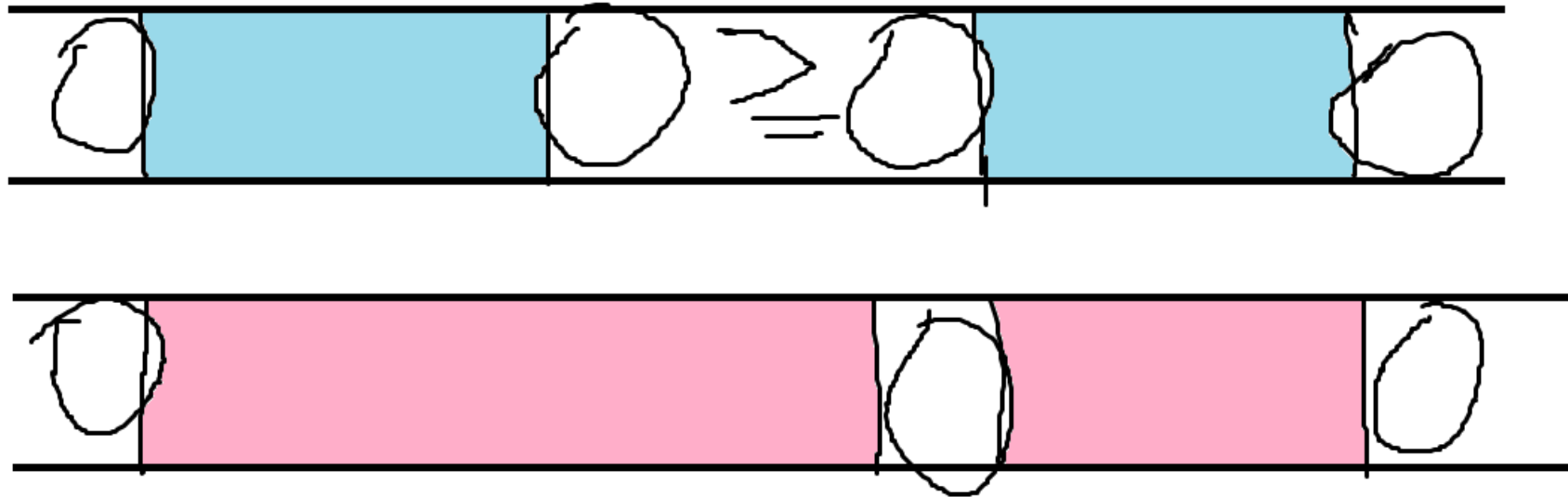
1 要素のみを見る

考察 1: 任意の valid な分割に対して，山を適切にのばせば，もと同じ分割個数で谷の長さがすべて 1 であるような valid な分割が作れる．

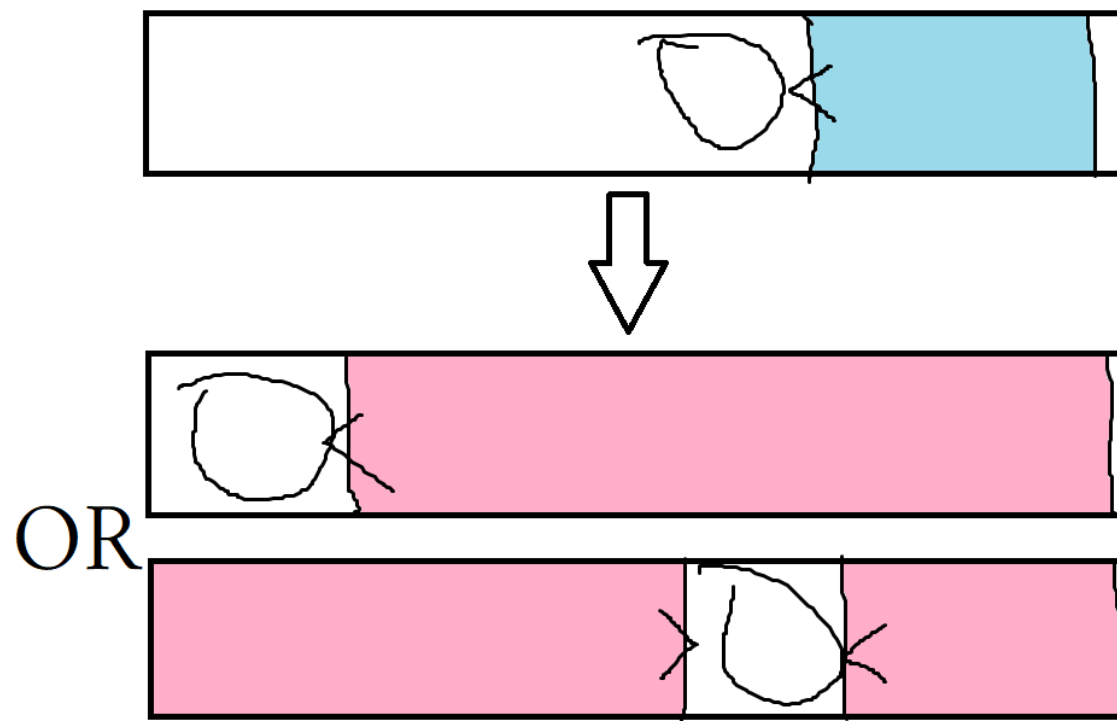
仮説 2: 任意の valid な和歌山分割に対して，和歌山を適切にのばせば，分割個数を減らさずに，谷の長さがすべて 1 であるような valid な分割が作れる．

- ここで和歌山分割は，和歌山 $[l_1, r_1], \dots, [l_k, r_k]$ の取り方であって $l_{k+1} - r_k \geq 2$ を満たすようなものとする．

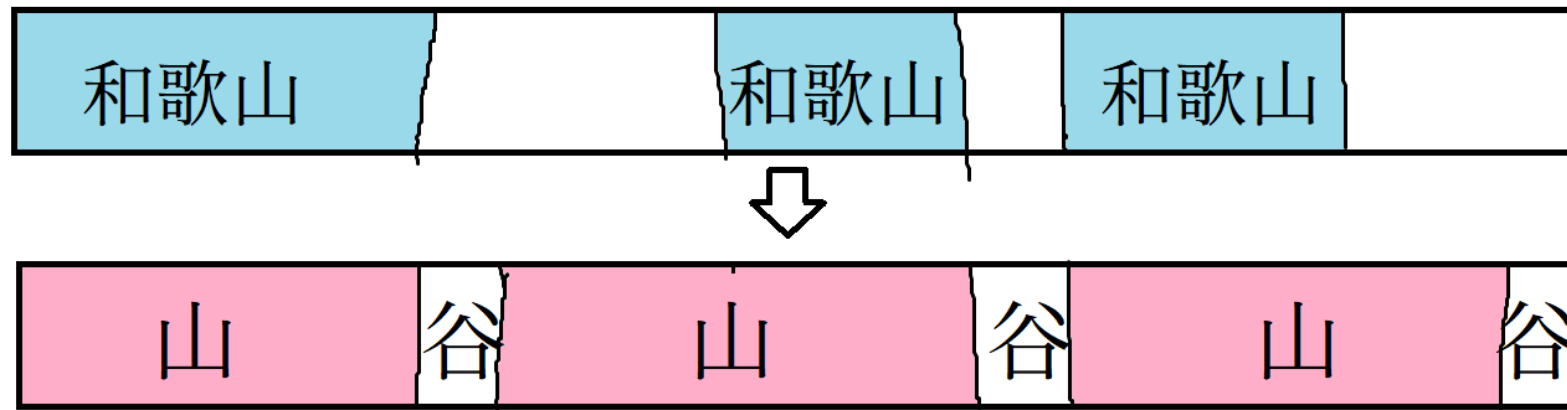
考察 2-a: 2つの和歌山の間の谷モドキについて、どちらかの和歌山をのばすことで、長さ1の谷にできる。



考察 2-b: 端にある谷モドキについて, 隣の和歌山をのばすことで, 長さ 1 の谷にできる.



考察 2: 任意の valid な和歌山分割に対して，和歌山を適切にのばせば，分割個数を減らさずに，谷の長さがすべて 1 であるような valid な分割が作れる．



考察 2+: 和歌山分割の最大個数は，分割の最大個数と一致する．

考察 1+: 谷の長さが 1 であるような分割のみを考えれば十分．

考察 2: 任意の valid な和歌山分割に対して，和歌山を適切にのばせば，分割個数を減らさずに，谷の長さがすべて 1 であるような valid な分割が作れる．

小考察: valid な分割は，和歌山分割としても valid．

考察 2+: 和歌山分割の最大個数は，分割の最大個数と一致する．

- ある区間が山であるかは，前後の山の選び方に依存する．
- ある区間が和歌山であるかは，その区間と両隣の要素の値しか関係しない(そこだけ見れば判断できる！)
- つまり，もともと考えていた分割よりも和歌山分割のほうが扱いやすい．

和歌山分割は

- ある区間が和歌山であるかは，その区間と両隣の要素の値だけが関係する。
 - 和歌山として valid な区間 $[l, r)$ がたくさん与えられるので， $[l, r]$ を被らないように選べばよい。

和歌山分割は区間スケジューリング

- ある区間が和歌山であるかは，その区間と両隣の要素の値だけに関係する.
 - 和歌山として valid な区間 $[l, r)$ がたくさん与えられるので， $[l, r]$ を被らないように選べばよい.
- 区間スケジューリングに帰着された.
- 区間スケジューリングは，選べるもののうち r が最小のものを貪欲に選ぶのが最適解.
 - 和歌山分割に言い換える前の分割では貪欲法は誤りなので注意.

小課題 4: 値は更新されない

- 直前の r から飛べる次の r の最小値を $f(r)$ とする.
 - 次の区間を $[l_{next}, r_{next}]$ とおくと, $l_{next} \geq r + 1$ の場合は $f(r + 1)$ でカバーできるので, $l_{next} = r + 1$ の場合のみを求めればよい.
- $f^x(r) \geq b$ を満たす最大の x を求めればよい.
 - 端の処理に注意. 端の山・谷の組合せを 4 通り試すと確実.

小課題 4: 値は更新されない

- 直前の r から飛べる次の r の最小値を $f(r)$ とする. $f^x(r) \geq b$ を満たす最大の x を求めればよい(端の処理に注意).
- 値が更新されないので, ダブリングによって各クエリが $\Theta(\log N)$ 時間で処理できる.
 - ダブリング: $f^{(2^k)}(r) = f^{(2^{k-1})}(f^{(2^{k-1})}(r))$ を利用して $f^{(2^k)}(r)$ を前計算できるうえ, $f^x(r) \geq a$ を満たす最大の x が二分探索で求まる.

小課題 5, 6: 値は更新される

- 一点更新とはいえ，それを含むすべての区間に影響するので， $f(r)$ が変わる r は多いのではないか？

考察 3: 一度に飛ぶ距離 $f(r) - r$ は, 高々 $2\lfloor \log A_{max} \rfloor + 4$.

- $c := r + \lfloor \log A_{max} \rfloor + 2$ とする.
- $L_k \geq L_{k+1} + \cdots + L_c$ ($r \leq k \leq c - 1$) が成り立つとすると,
 $L_k \geq 2^{c-1-k} L_c \geq 2^{c-1-k}$ (帰納法で示せる) より
 $L_r \geq 2^{c-1-r} = 2^{\lfloor \log A_{max} \rfloor + 1} > A_{max}$ となり, **矛盾**.
したがって, $L_k < L_{k+1} + \cdots + L_c$ が少なくとも 1 ケ所の
 $r \leq k \leq c - 1$ で成り立つ.
- 同様に $L_c + \cdots + L_{k-1} > L_k$ が少なくとも 1 ケ所の
 $c + 1 \leq k \leq 2c - r = r + 2\lfloor \log A_{max} \rfloor + 4$ で成り立つ.

考察 3: 一度に飛ぶ距離 $f(r) - r$ は, 高々 $2\lfloor \log A_{max} \rfloor + 4$.

- $c := r + \lfloor \log A_{max} \rfloor + 2$ とする.
- $L_k < L_{k+1} + \cdots + L_c$ が少なくとも 1 ケ所の $r \leq k \leq c - 1$ で成り立つことが示せた.
- 同様に, $L_c + \cdots + L_{k-1} > L_k$ が少なくとも 1 ケ所の $c + 1 \leq k \leq 2c - r = r + 2\lfloor \log A_{max} \rfloor + 4$ で成り立つことも示せる.
- したがって, $r \leq s, t \leq r + 2\lfloor \log A_{max} \rfloor + 4$ を満たす和歌山の区間 (s, t) が存在する.

これで更新クエリがラクになる

- L_x を一点更新したとき, $f(r)$ が変わりうるのは $x - (2\lfloor \log A_{max} \rfloor + 4) \leq r \leq x$ の範囲のみ.
- 愚直にやっても $O(\log^3 A_{max})$ 時間で済むし, 小課題 4 と同様に $f(r+1)$ を利用する方針でいけば $O(\log^2 A_{max})$ 時間になる.

トドメのセグメント木

- $seg[l, r)[k] := k$ から出発して r を超えないように繰り返し飛ぶとき、最大何回でどこまで飛ぶか
 - 必要な k の範囲は $l \leq k \leq l + 2\lfloor \log A_{max} \rfloor + 4$
- $seg[l, r) = seg[l, m) * seg[m, r)$ の二項演算は $O(\log A_{max})$ 時間.
 - $f(\cdot)$ を利用すれば、 $O(\log A_{max})$ 個ある k それぞれについて $seg[l, r)[k]$ が定数時間で求まるため.
- セグ木の更新と区間取得は、セグ木区間の合成を $O(\log N)$ 回行うのでともに $O(\log A_{max} \log N)$ 時間.

以上より，この問題は

- 谷を長さ 1 につぶす方法を思いつき，

以上より，この問題は

- 谷を長さ 1 につぶす方法を思いつき，
- 谷ではなく山に注目することで区間スケジューリングに帰着し，

以上より，この問題は

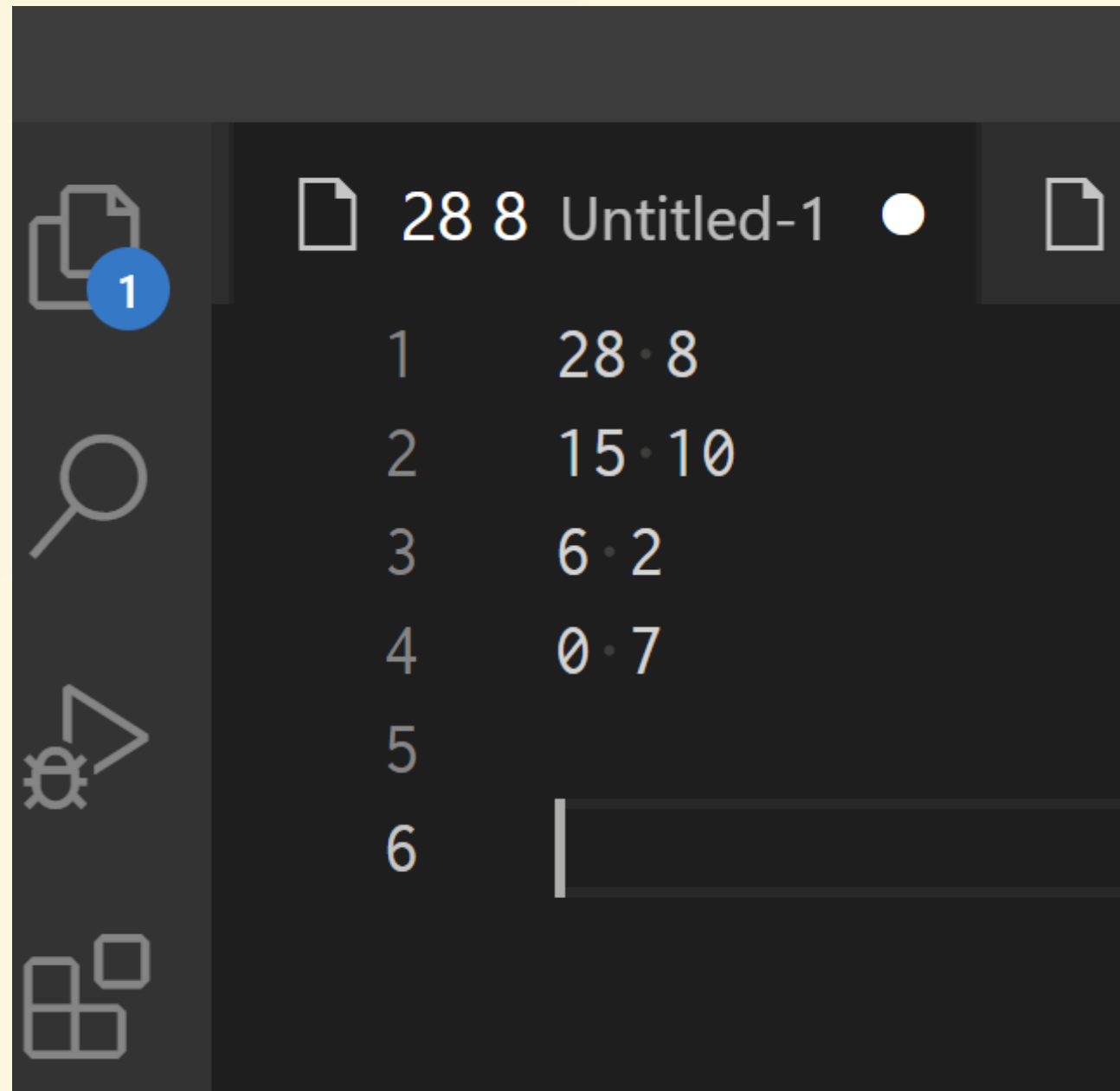
- 谷を長さ 1 につぶす方法を思いつき，
- 谷ではなく山に注目することで区間スケジューリングに帰着し，
- 必要な区間の長さが高々 $2\lfloor \log A_{max} \rfloor + 4$ であることに気付いて，

以上より，この問題は

- 谷を長さ 1 につぶす方法を思いつき，
- 谷ではなく山に注目することで区間スケジューリングに帰着し，
- 必要な区間の長さが高々 $2\lfloor \log A_{max} \rfloor + 4$ であることに気付いて，
- ダブリングの考え方をセグメント木に乗せることで，
- クエリあたり $O(\log A_{max}(\log A_{max} + \log N))$ 時間で解くことができました！

やっべスライド作る の間に合わない

スクショ載せます



Chorus 解説

川崎 理玖

問題概要

- A, B を N 個ずつ含む文字列がある
- 隣接文字のswapを繰り返し、以下の条件を満たしたい
- 条件：AAABBBB みたいな形の部分列 K 個に分解できる
- 最小のswap回数は？

小課題1

- とりあえず swap 0回でOKか判定する
- ちょうど K 個ではなく K 個以下でできるか判定しても同じ
- 条件を満たす部分列に分解するとき場合の最小の個数が分かればよい

小課題1

- 最初に連続する A はまとめて使ってよさそう

AAABBABBB

- バラバラに使う解を適切に変形するとまとめて使う解にできるので OK

小課題1

- 対応する B は先頭から選べばよさそう
A A B B A B A B B
- 後ろのBを使って得することがないので
OK

小課題1

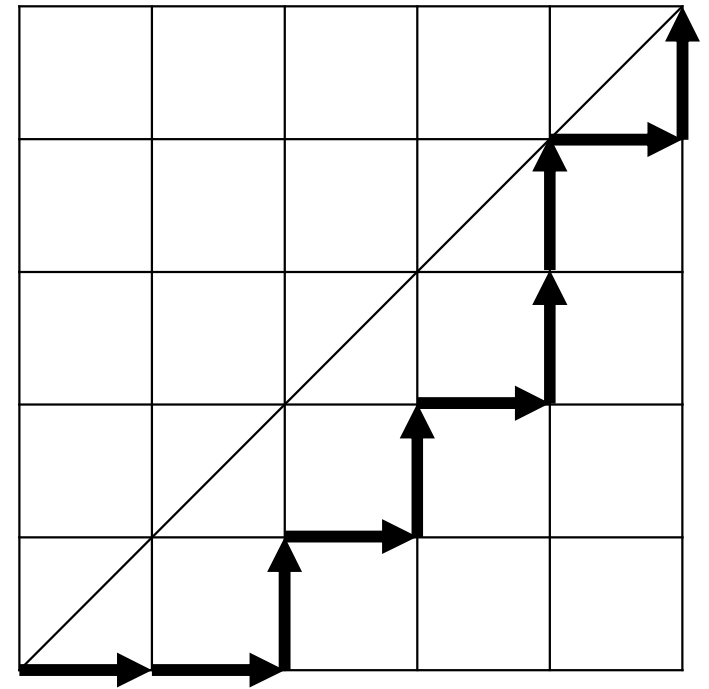
- 結局、前から貪欲に取る操作が最適
- これで分解の最小の個数が求まった

小課題1

- 始状態からswapを繰り返して到達できる文字列をすべて列挙し、OKなものへ到達する最小ステップ数をBFSで求める
- $O(N \cdot 2^{2N})$ で計算できる
- ちなみに判定をbitDPでひたすら頑張るとかでも一応解ける

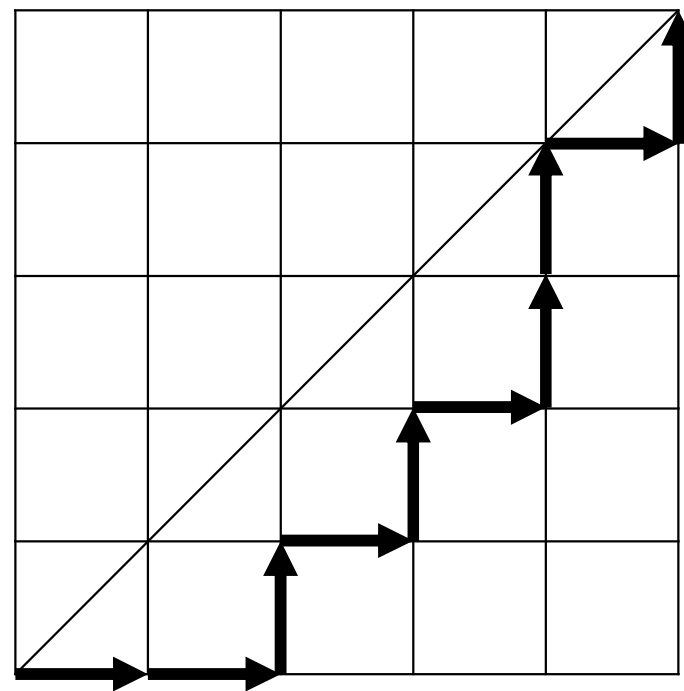
多項式時間解法

- 分割の個数が K 以下である、を巧みに言い換えたい
- グリッドを用意し、 A を $\rightarrow$ 、 B を $\uparrow$ に対応させる



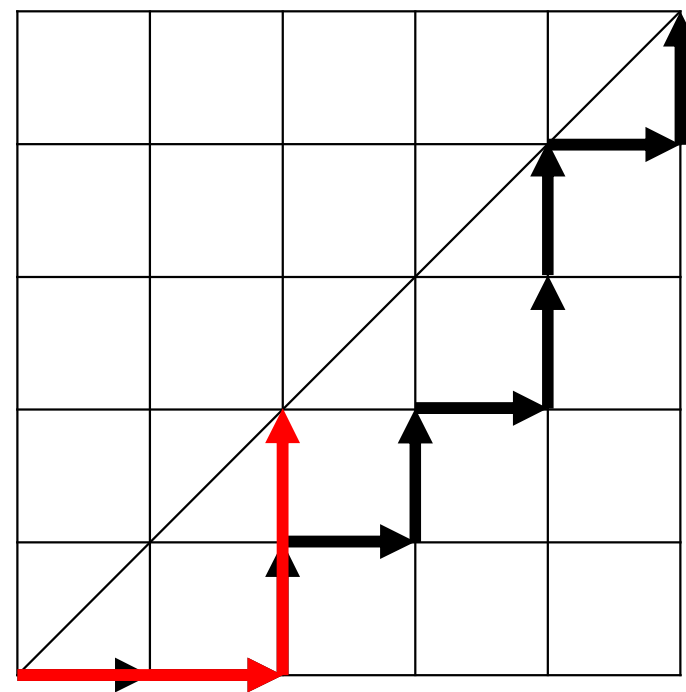
多項式時間解法

- さっきの貪欲を走らせてみる



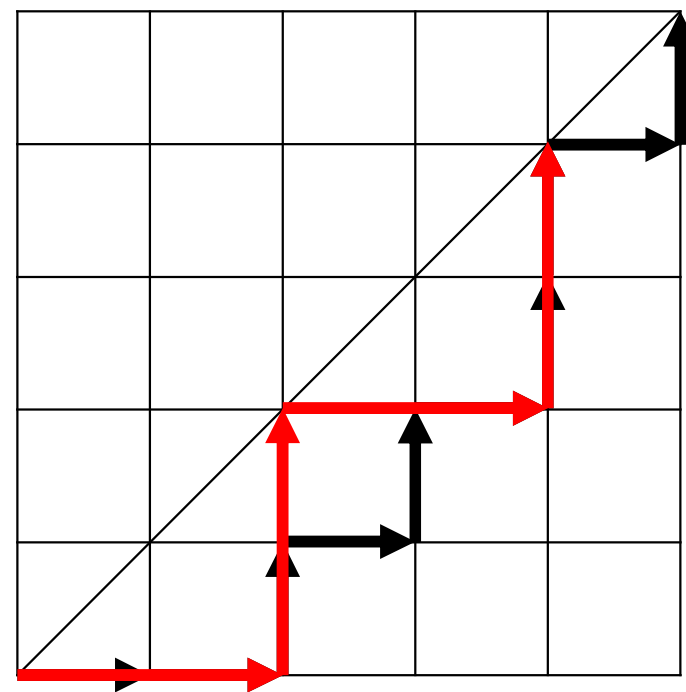
多項式時間解法

- さっきの貪欲を走らせてみる
- AABBBをとる



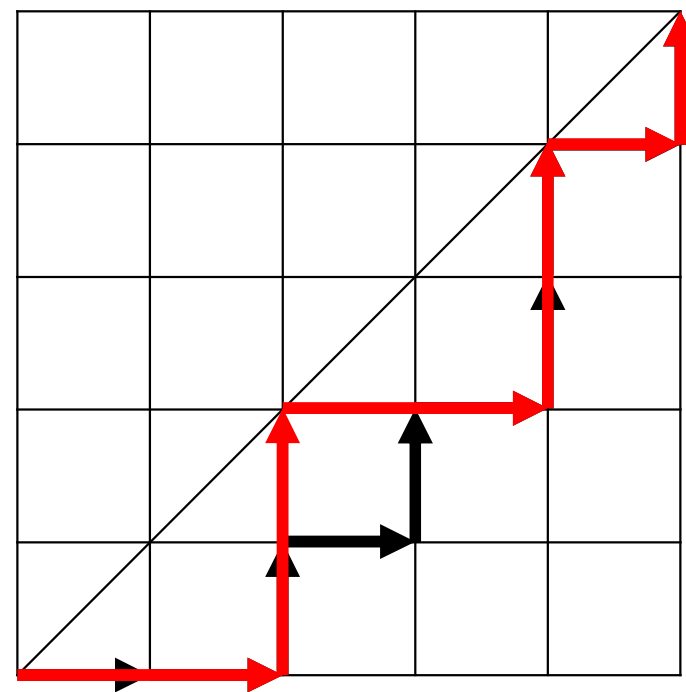
多項式時間解法

- さっきの貪欲を走らせてみる
 - AABBBをとる
 - AABBBをとる
- | | |
|--|--|
| | |
| | |



多項式時間解法

- さっきの貪欲を走らせてみる
 - AABBBをとる
 - AABBBをとる
 - ABをとる
- | | |
|--|--|
| | |
| | |
| | |

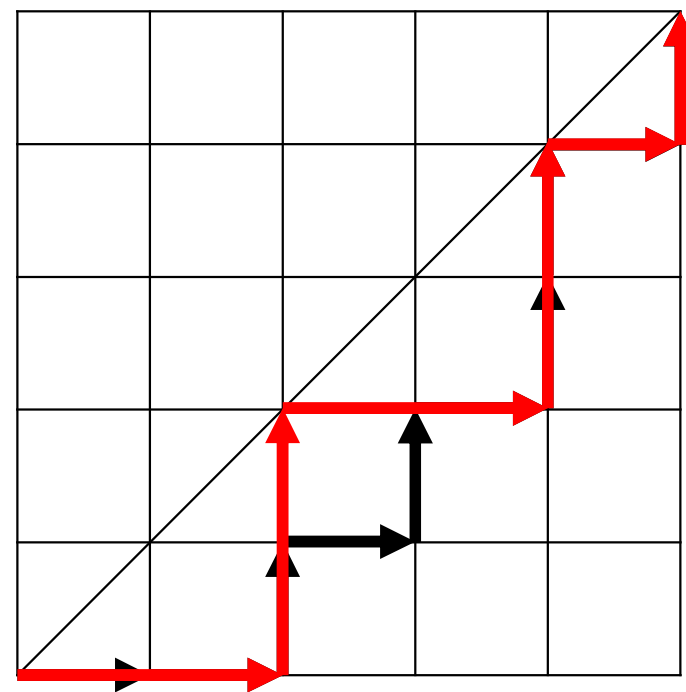


多項式時間解法

- 部分列 K 個に分解できる

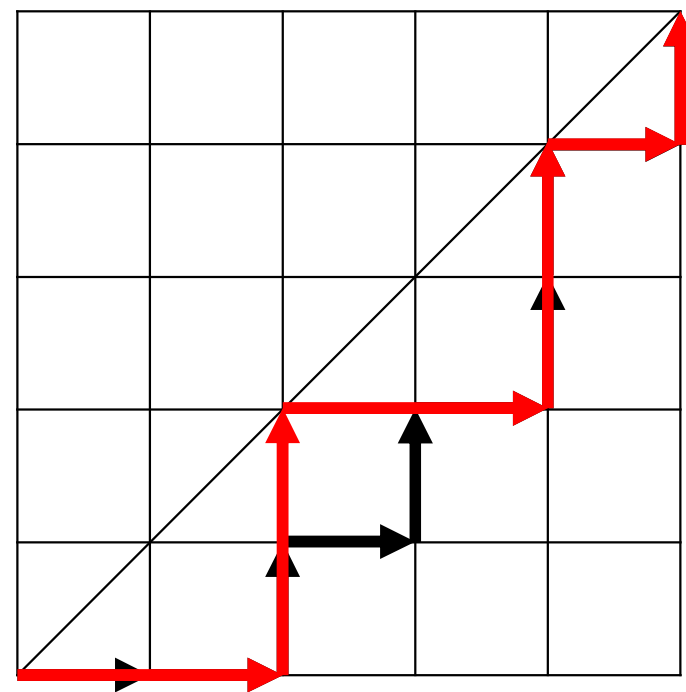
↓

右、上、右、上、と K 回
進む折れ線が
右下領域と重ならない



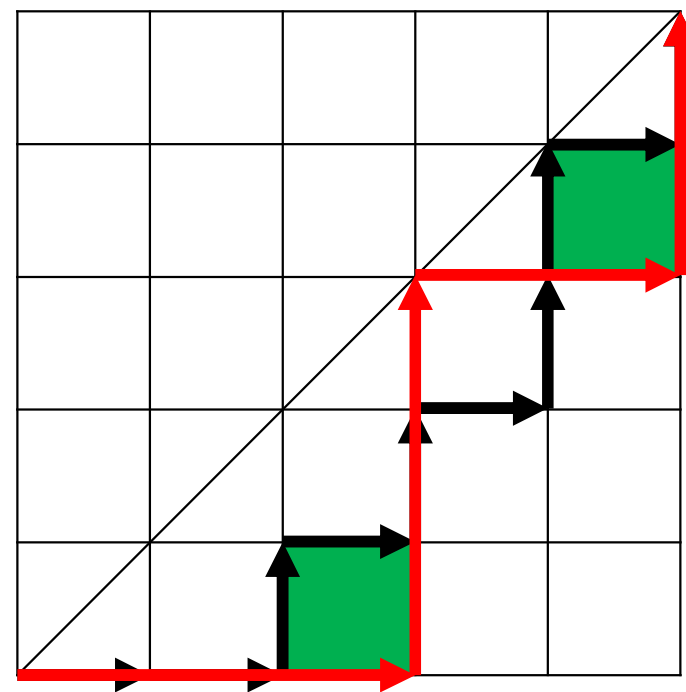
多項式時間解法

- 逆も成立する
- つまり、適切な折れ線があれば、それに対応する分解が作れる



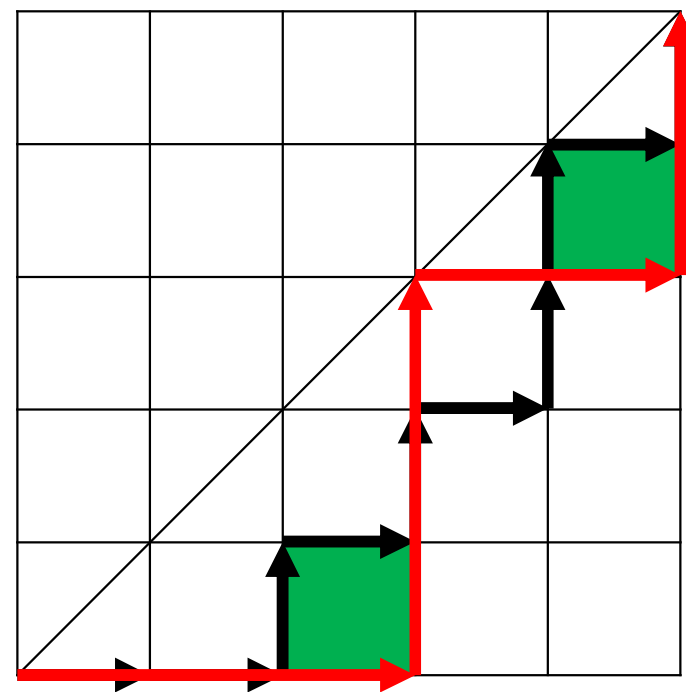
多項式時間解法

- [illegible]



多項式時間解法

- $F(L,R)$ を、座標 (L,L) から (R,R) まで 1 回で進む折れ線の上にはみ出す面積とする
- 例 : $F(0,3)=1, F(3,5)=1$



多項式時間解法

- 問題が以下のように言い換えられる
- 0 から N までジャンプしていく
LからRにジャンプするとコスト $F(L,R)$
- K 回以下のジャンプの最小コストは？

小課題2

- $F(L,R)$ の値は適切な前計算で $O(1)$
- 普通にDPすれば $O(N^2K)$ で間に合う

小課題3

- $F(L,R)$ にいい性質がないと難しそう
- Monge 性がある

小課題3

- $F(L, R+1) - F(L, R) \leq F(L-1, R+1) - F(L-1, R)$
- (最小化か最大化かどうかで不等号が $\leq$ $\geq$ になることがあるのだが、均等に刻んだ方が偏った刻み方よりもうれしい、という気持ち)
- 図示すると証明は簡単

小課題3

- ちょうど s 回のジャンプで場所 x にたどり着くコストを $dp[s][x]$ として、 $dp[s][*]$ から $dp[s+1][*]$ を求めたい
- これをある程度高速にできればOK

小課題3

- Monotone Minima なので分割統治で $O(N \log N)$
- 全体 $O(N^2 \log N)$ で定数は軽い
- 分からない場合は「Monge DP 高速化」等で検索してください

小課題3

- ちなみに SMAWK で $O(N)$ にもなる
- 実装が面倒かつ頑張らないと定数倍が重くて分割統治に負ける
- 覚えなくてよいです

小課題4

- K ステップ全部求めるわけにはいかない
- コストの性質上、K ステップ以下ではなくちょうど K ステップだと思ってよい
- どうするか？

小課題4

- Alien DP と呼ばれるテクニックで解ける
- IOI 2016 Alien で有名になったことが名前の由来
- これも各自で調べてください

小課題4

- 適当な定数 C に対し、 $G(L,R)=F(L,R)+C$ とおき、ジャンプのコスト G の問題を解きたい
- 分割統治（オンラインーオフライン変換）
+ Monotone Minima で
 $O(N \log^2 N)$

小課題4

- 答えが $O(N^2)$ なので、ラグランジュ緩和の二分探索の分で $O(\log N)$ 回走る
- 全体で $O(N \log^3 N)$
- Monotone Minima の $\log$ が軽いので通る

小課題5

- 内側のDP部分をさらに高速化する
- LARSCHを使うと $O(N)$ (全体 $O(N \log N)$) で AC!
- 実装が面倒だしそこまで速くない
- これも覚えなくてよいです

小課題5

- 簡易LARSCHなるものがある
(<https://noshi91.hatenablog.com/entry/2023/02/18/005856>)
- 実装が簡単
- $O(N \log N)$ だけど定数倍がよい
- 全体 $O(N \log^2 N)$ で AC!

小課題5

- 何も知らないとどうなるのか？
- CHT とスライド最小値で DP を高速化すれば $O(N)$
- 全体 $O(N \log N)$ で AC!

得点分布

- 0点 : 4 人
- 16点 : 6 人
- 40点 : 12 人
- 61点 : 3 人
- 87点 : 1 人
- 100点 : 3 人

Cookies 解説

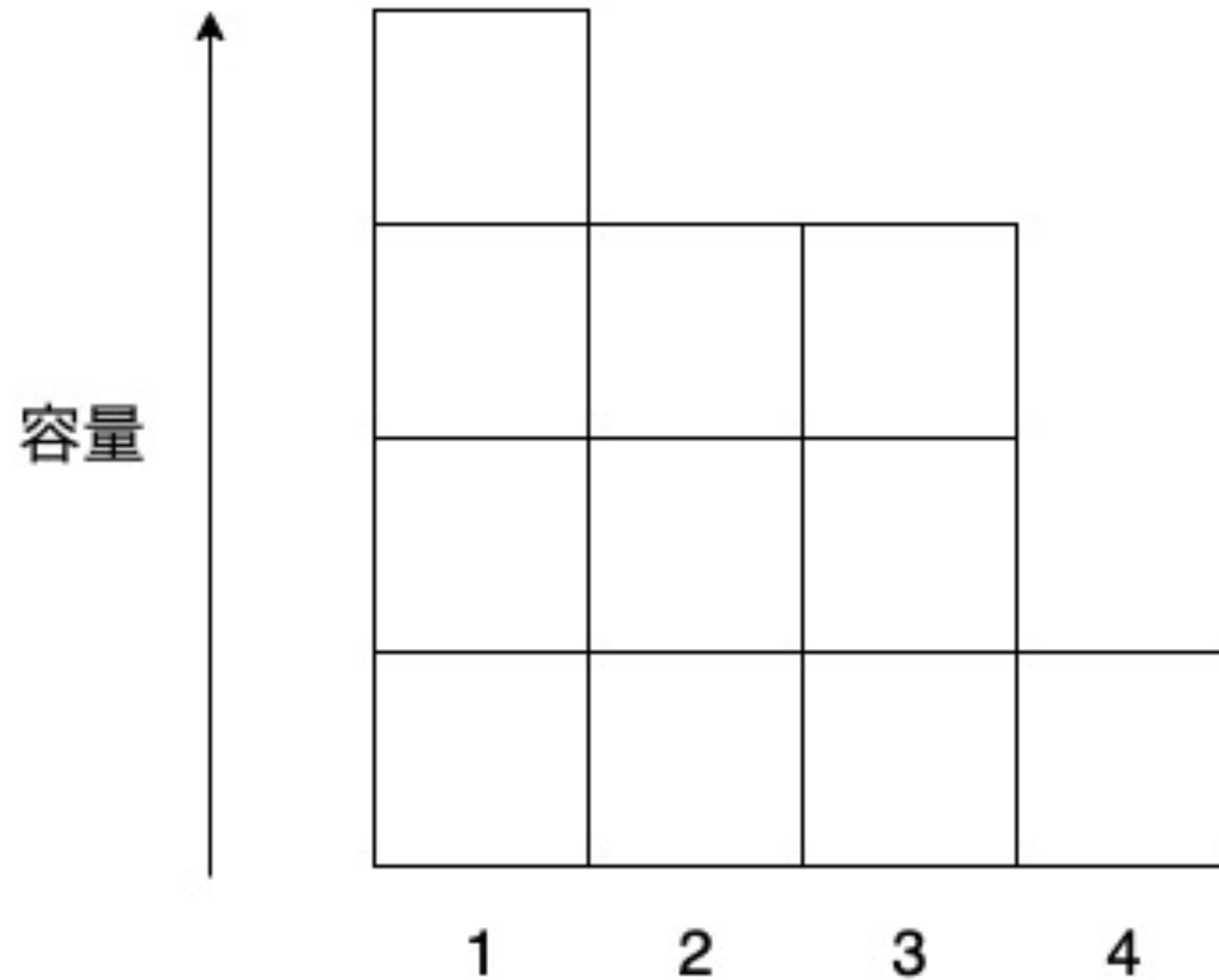
解説 : yuto1115

問題概要

- N 種類のクッキーがあり、 i 種類目は A_i 枚ある
- 今から全てのクッキーを何箱かに分けて箱詰めするが、
 - 同じ箱の中に同じ種類のクッキーがあってはいけない
 - それぞれの箱に入れるクッキーの枚数は $B_1, B_2, \dots, B_M$ のいずれか
- 箱詰めは可能か？可能な場合、使う箱の数が最小になる割り当てを 1 つ構築

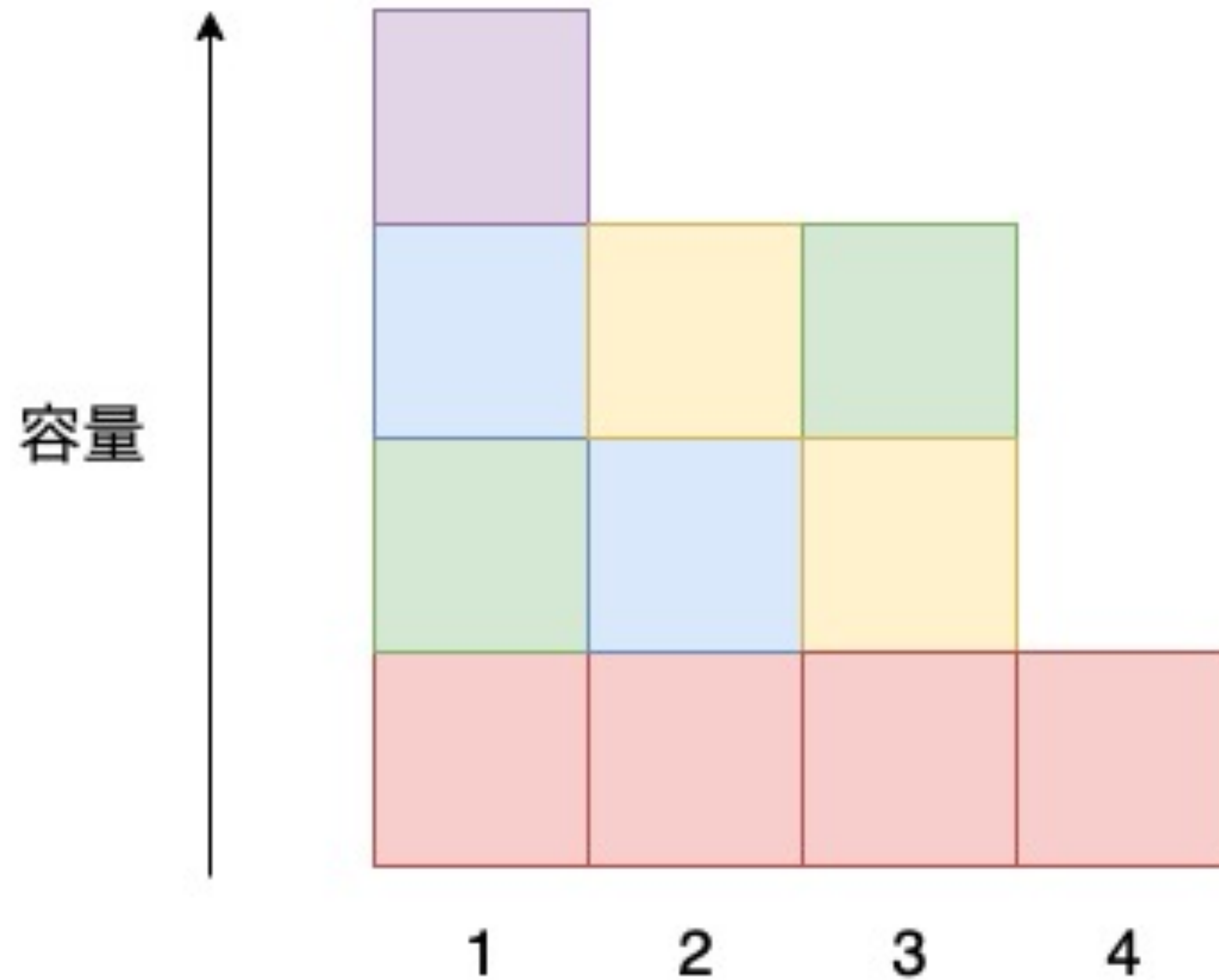
例

$$A = (4, 2, 2, 2, 1), B = (1, 3, 4)$$



例

$$A = (4, 2, 2, 2, 1), B = (1, 3, 4)$$



小課題 1 ($N \leq 500, A_i = 1$)

- クッキーの種類の重複を考えなくていい
- 必要十分条件は $\sum_i A_i = \sum_j B_j$
- 部分和问题なので、DP で $O(N^2)$

ここまでで 6 点

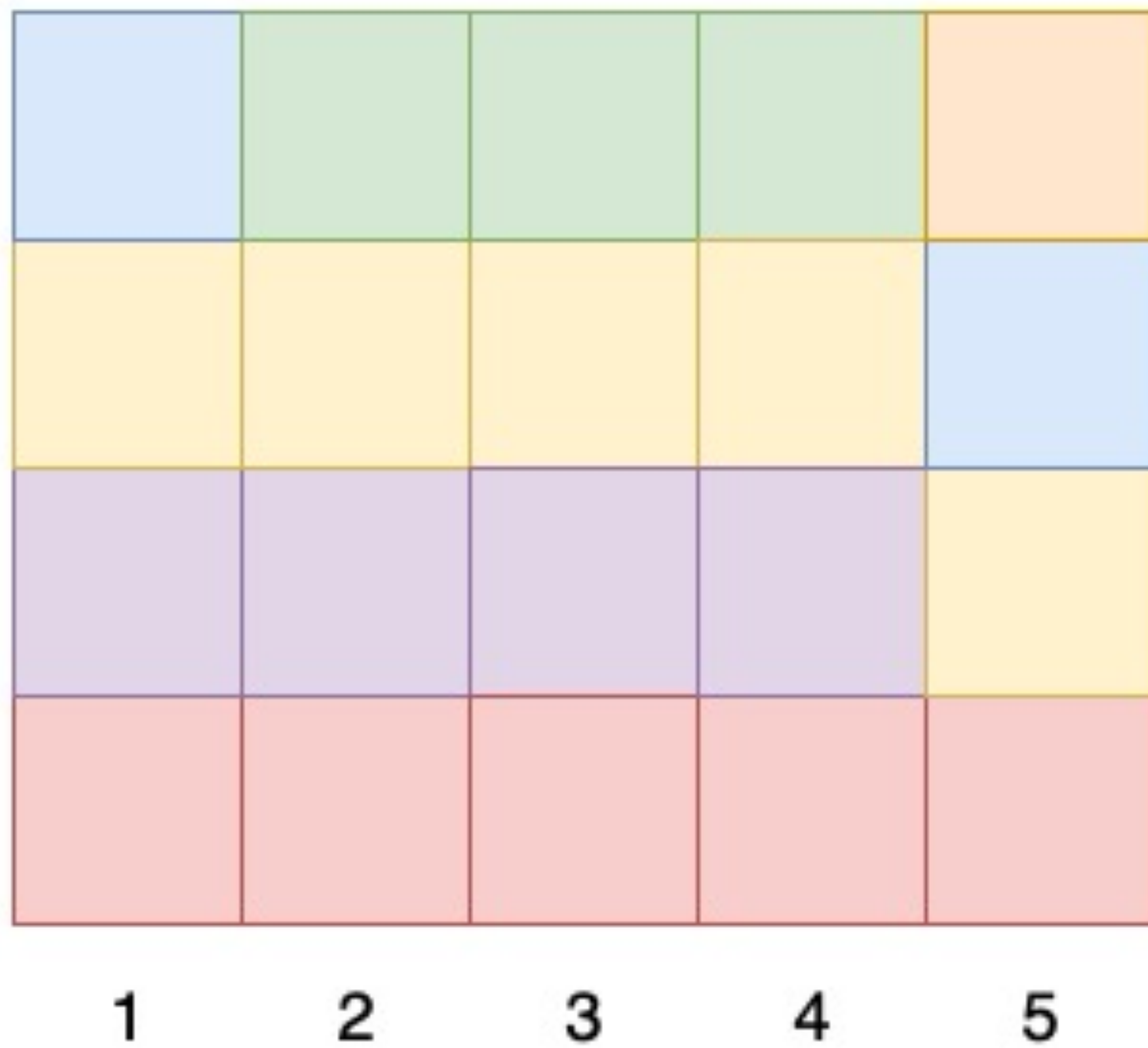
小課題 2 ($N \leq 500, M = 1$)

- 以下、 $S = \sum_i A_i$ とおく
- S が B_1 で割り切れない場合、明らかに割り当て不可能
- そうでないならば、使う箱の数は一意に定まる

小課題 2 ($N \leq 500, M = 1$)

- どんな時に割り当て可能か考えよう
 - $\max A_i$ 箱以上あることは必要条件
 - 逆にこれを満たせば ...?

容量



小課題 2 ($N \leq 500, M = 1$)

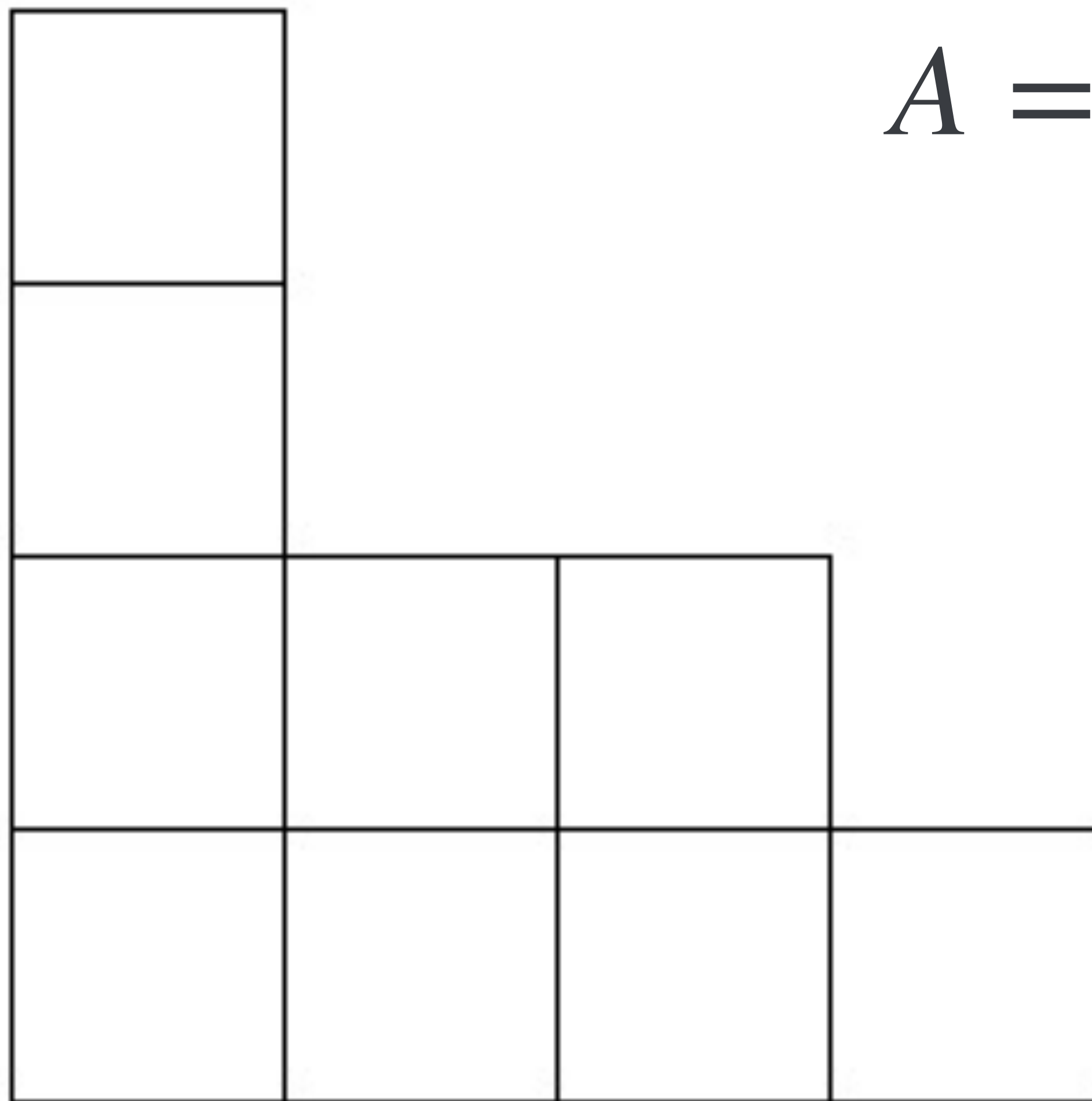
- よって、条件を $O(1)$ で判定可能
- 構築はさっきの図のようにやればいい

ここまでで 13 点

考察

- 一般の場合で、**使う箱が既に決まっているとき**、割り当てを 1 つ構築する or 割り当て不可能と判定する方法を考えたい (箱の容量の和が S と一致していることは前提とする)
- 次のような貪欲法を考える
 - $i = 1, 2, \dots, N$ について以下を行う
 - 残り容量 (元々の容量 $-$ 既に入れたクッキーの枚数) が大きい順に箱を A_i 個選んで、それぞれの箱に種類 i のクッキーを 1 枚ずつ入れる
 - 残り容量が正の箱が A_i 箱未満なら、割り当て不可能と判定する

容量



$$A = (3, 2, 2, 2)$$

1

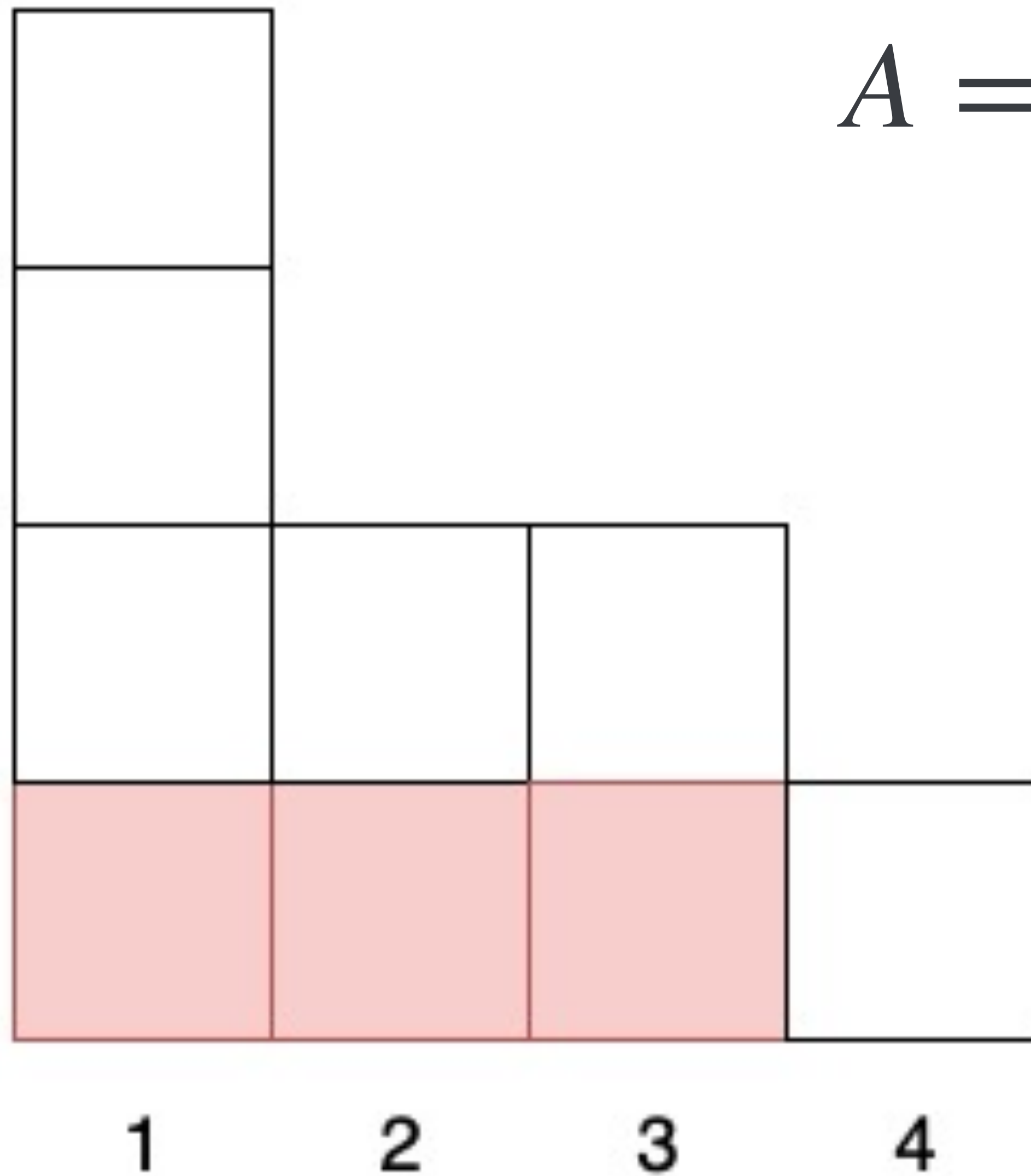
2

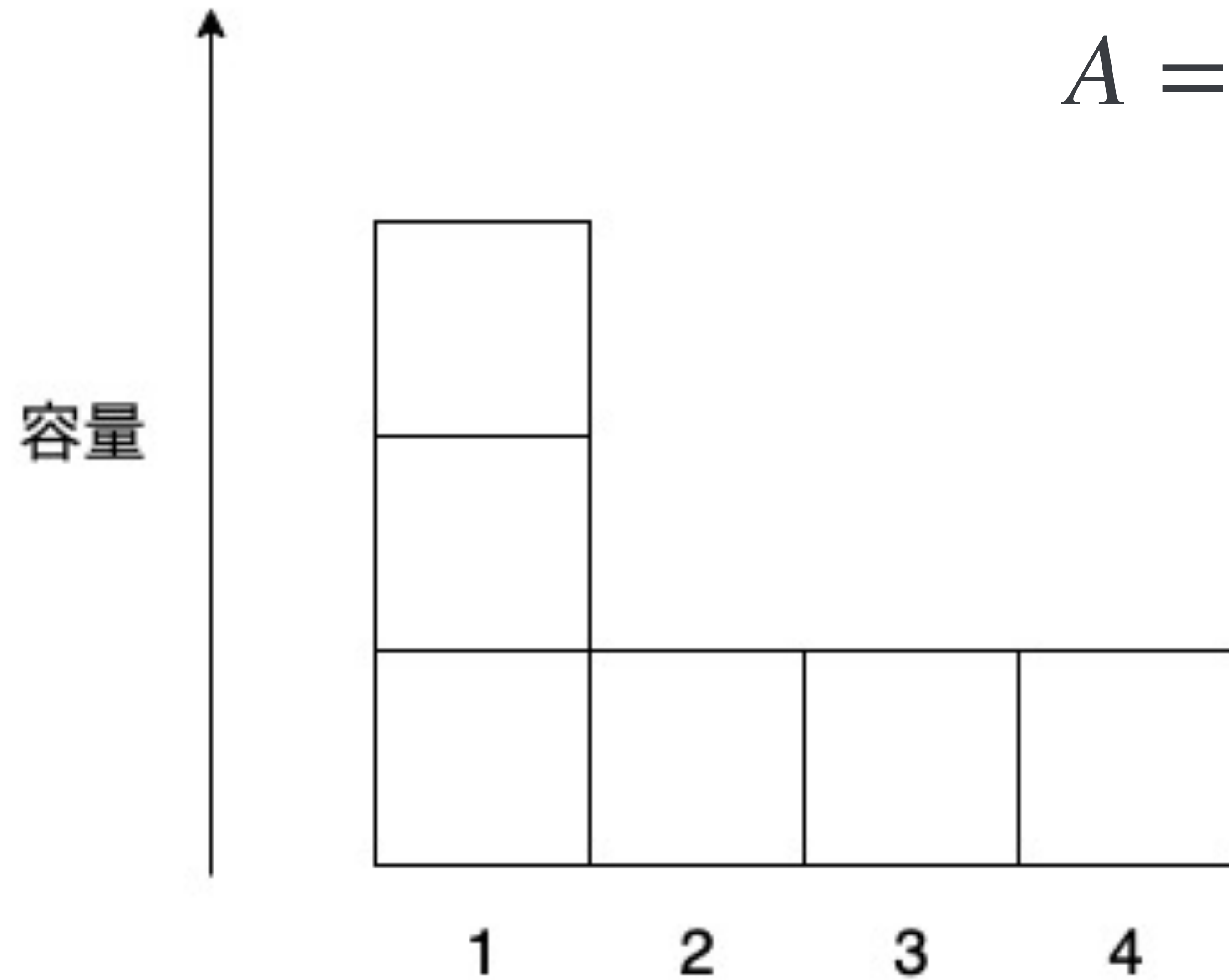
3

4

容量

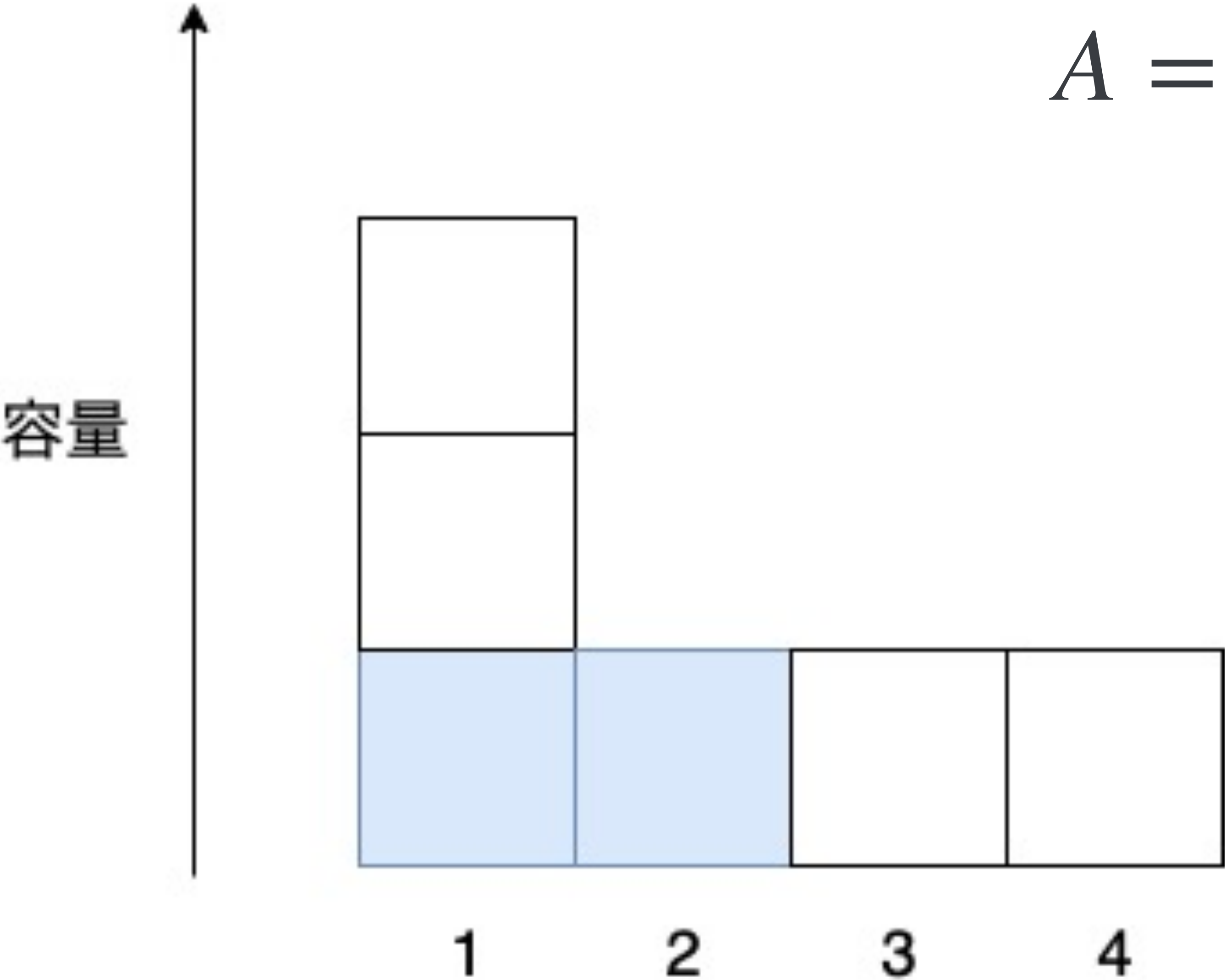
$$A = (3, 2, 2, 2)$$





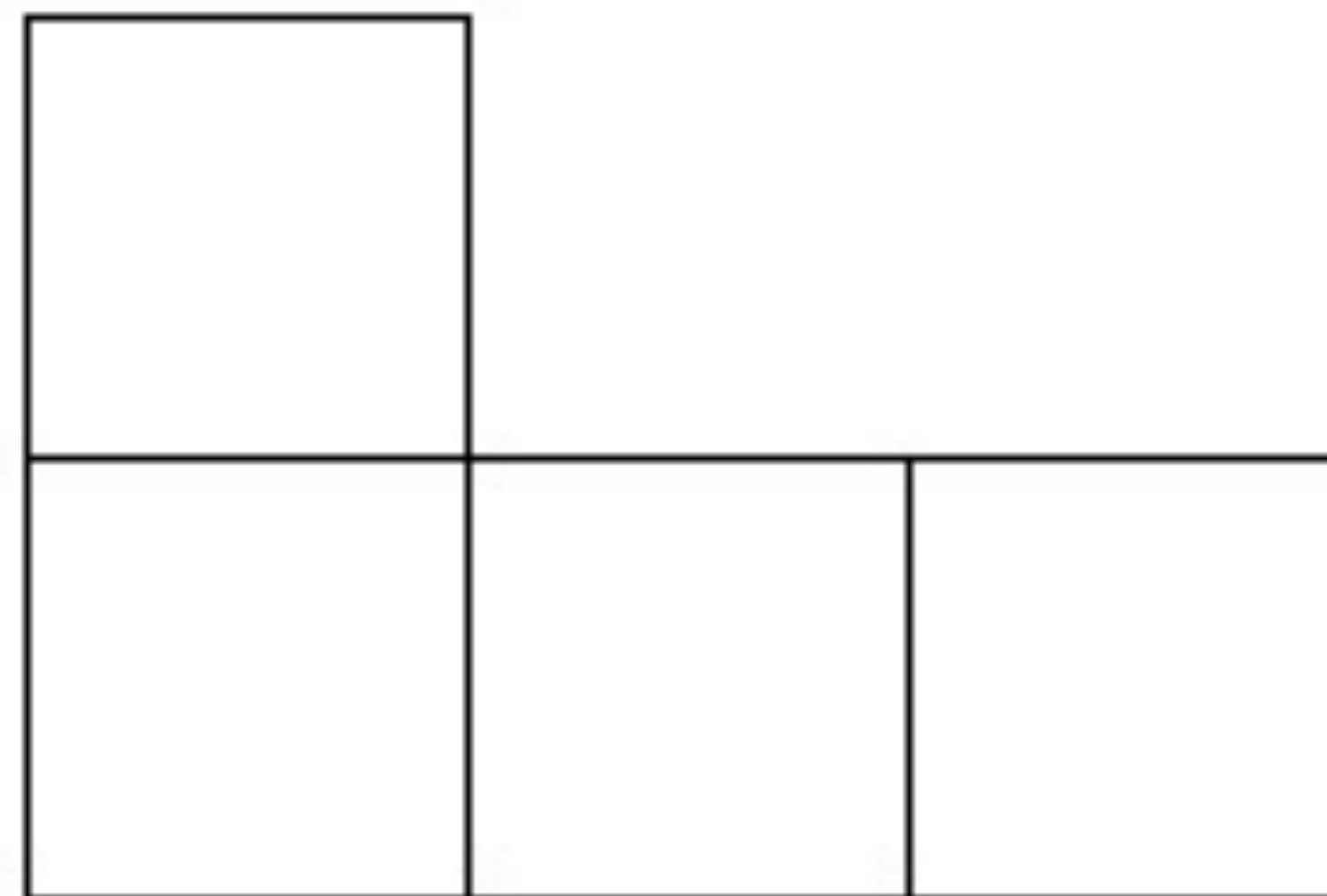
$$A = (3, 2, 2, 2)$$

$A = (3, 2, 2, 2)$



$$A = (3, 2, 2, 2)$$

容量



1

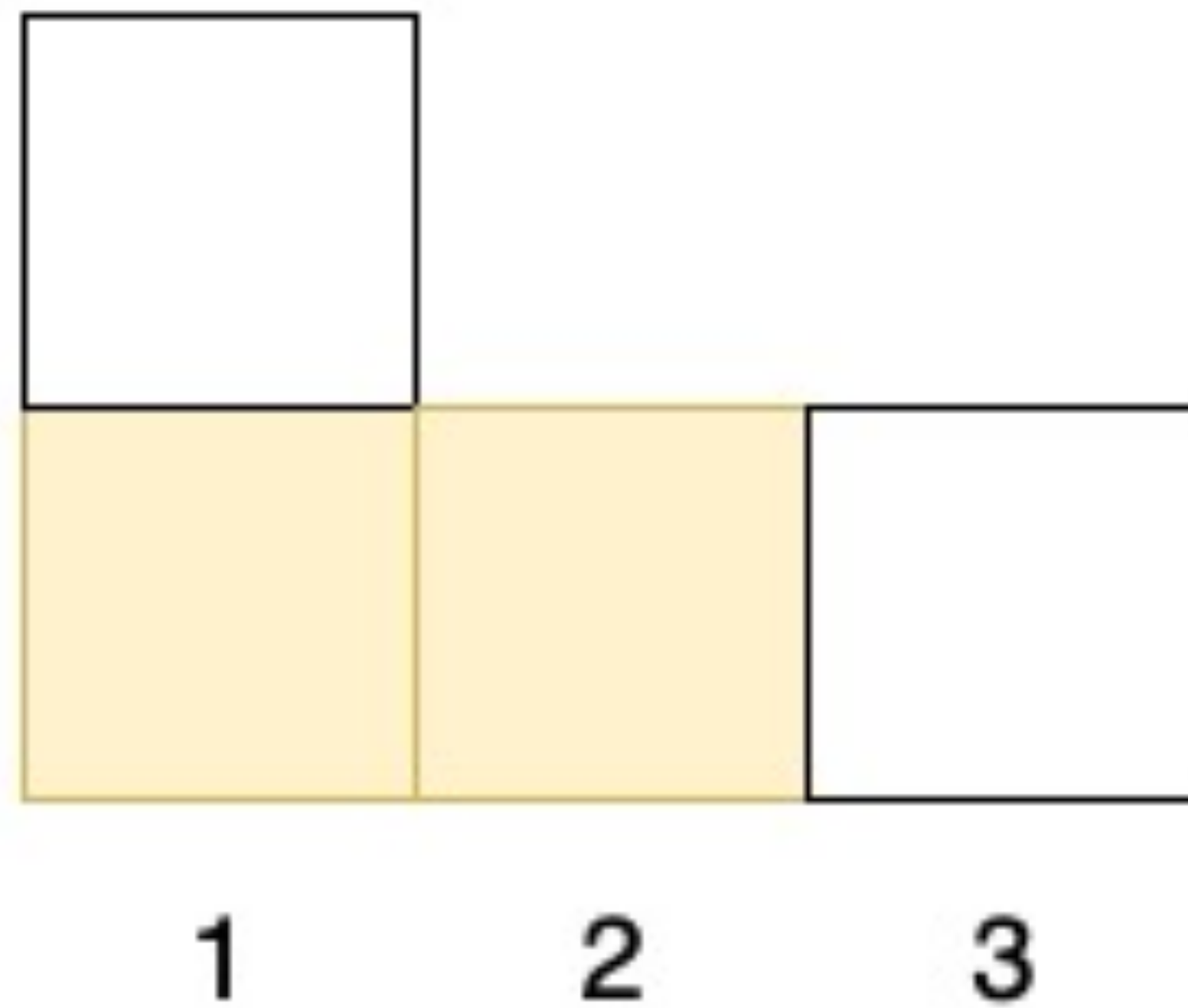
2

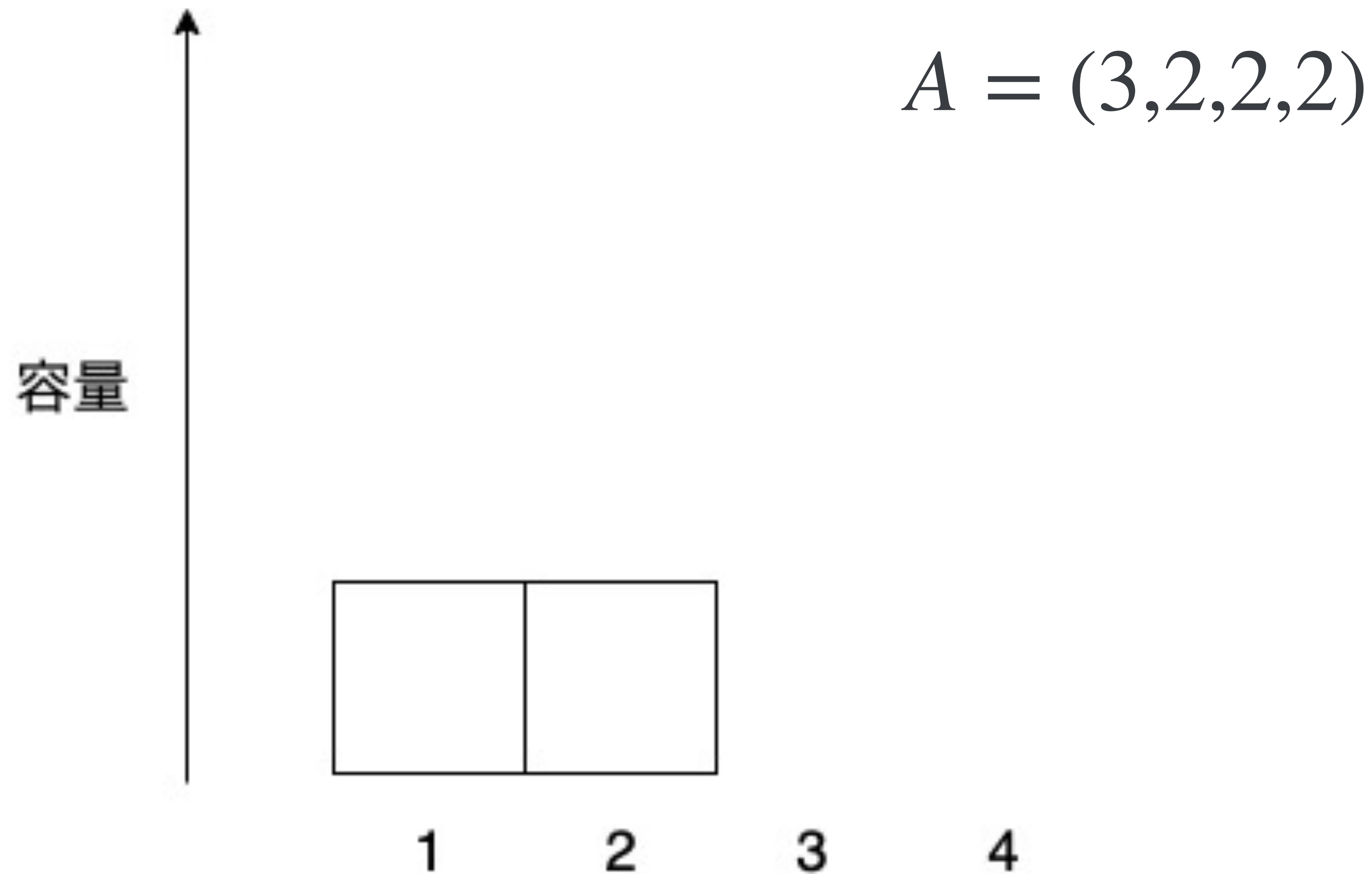
3

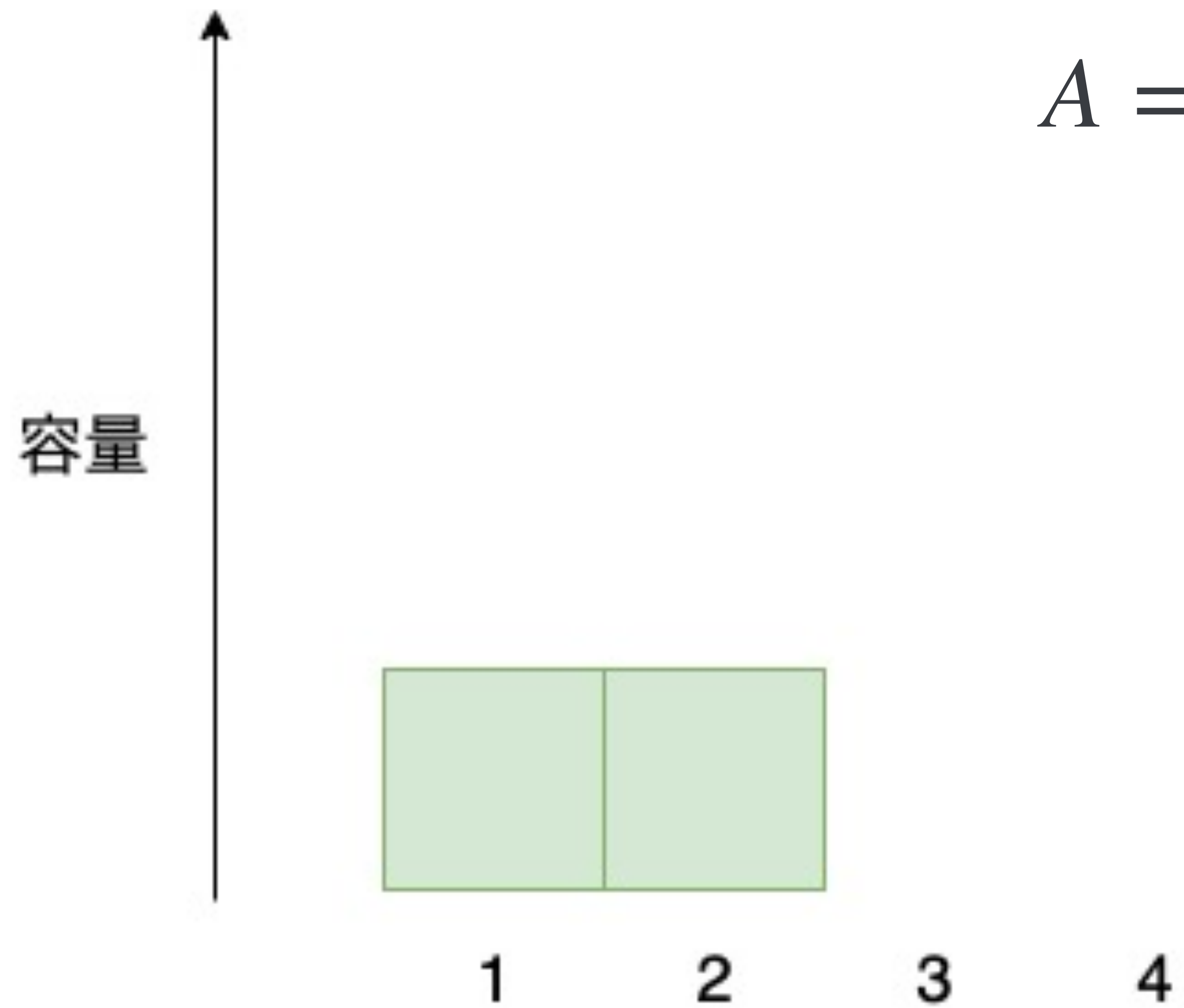
4

$$A = (3, 2, 2, 2)$$

容量







$$A = (3, 2, 2, 2)$$

容量

1

2

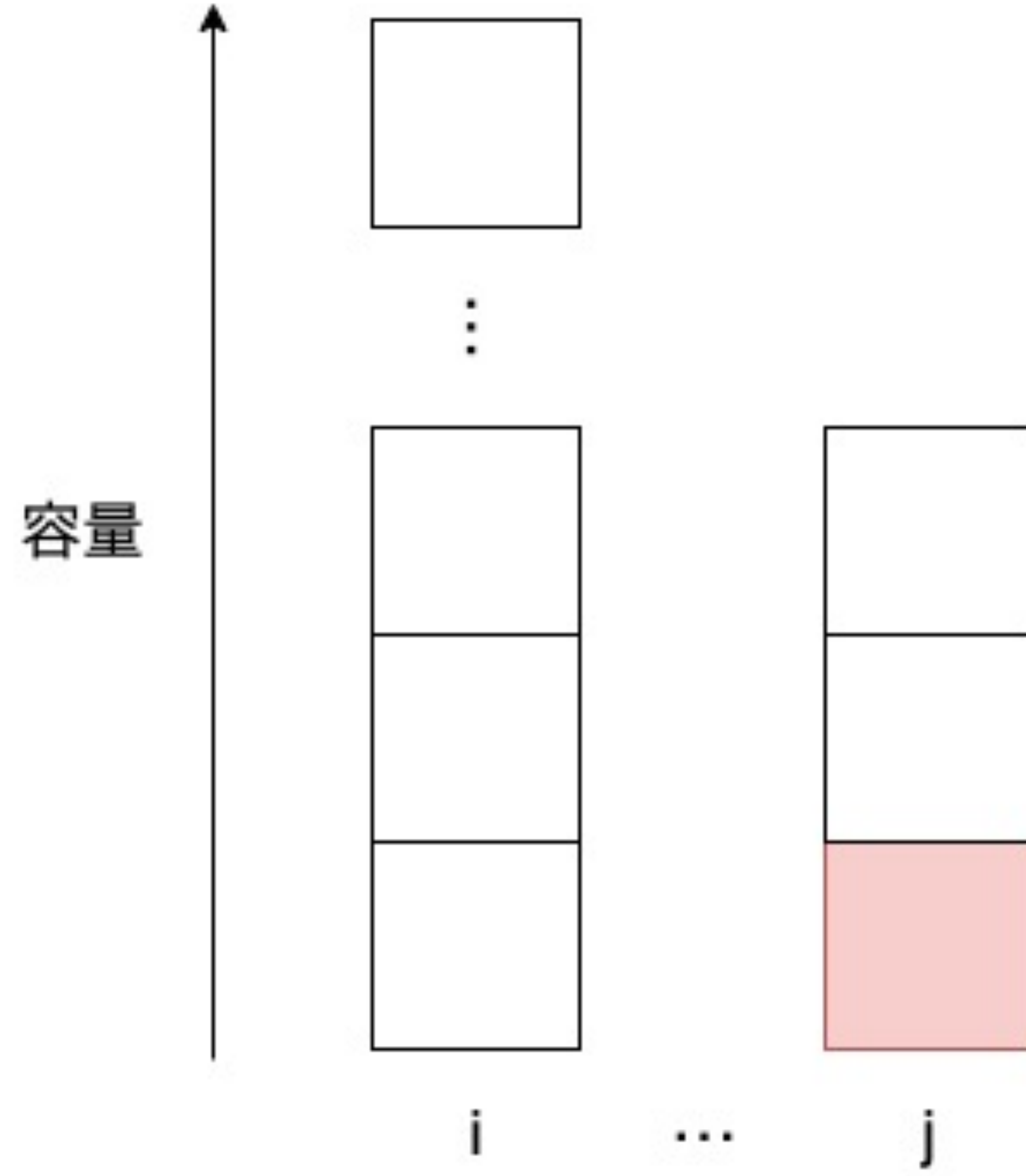
3

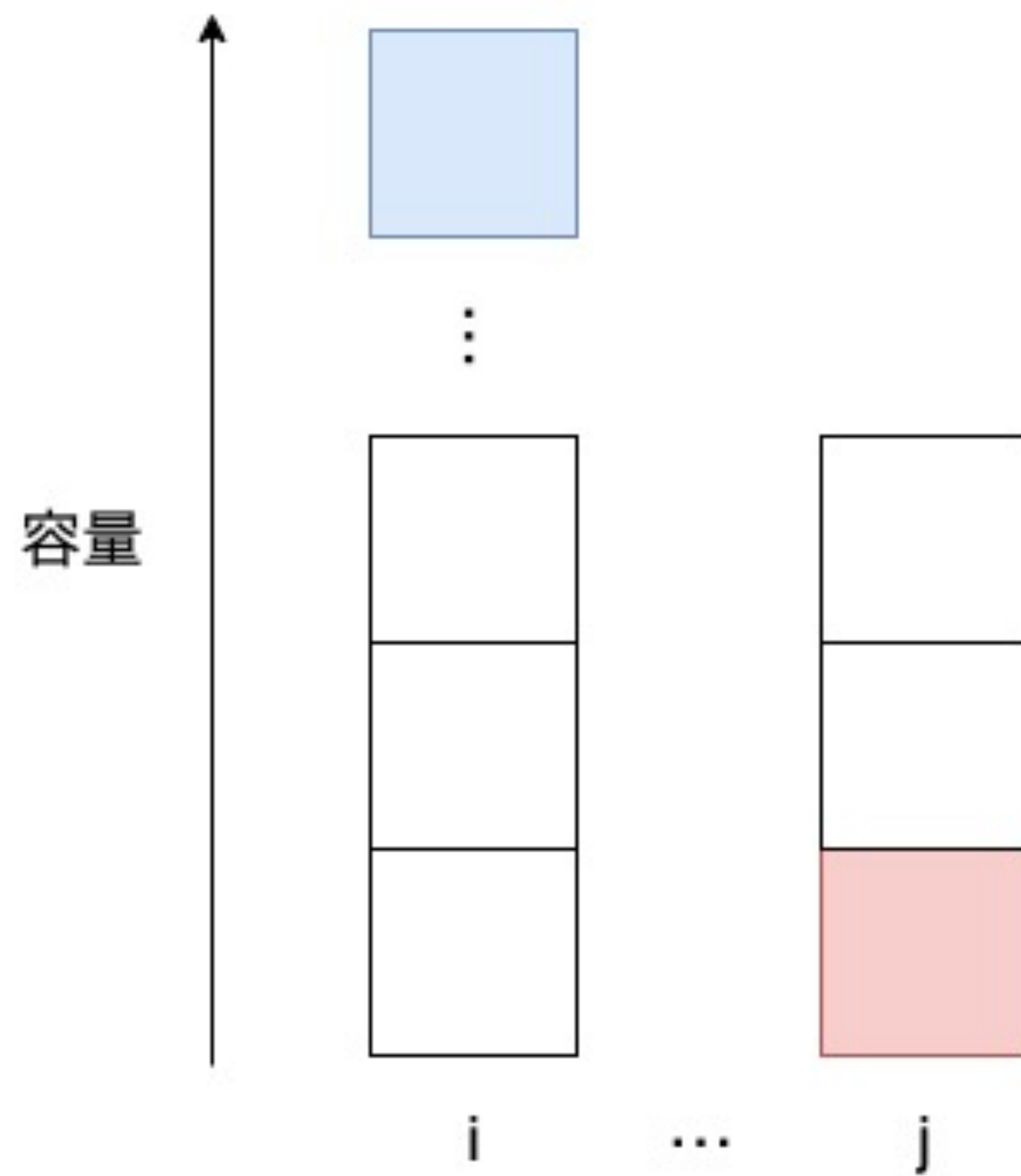
4

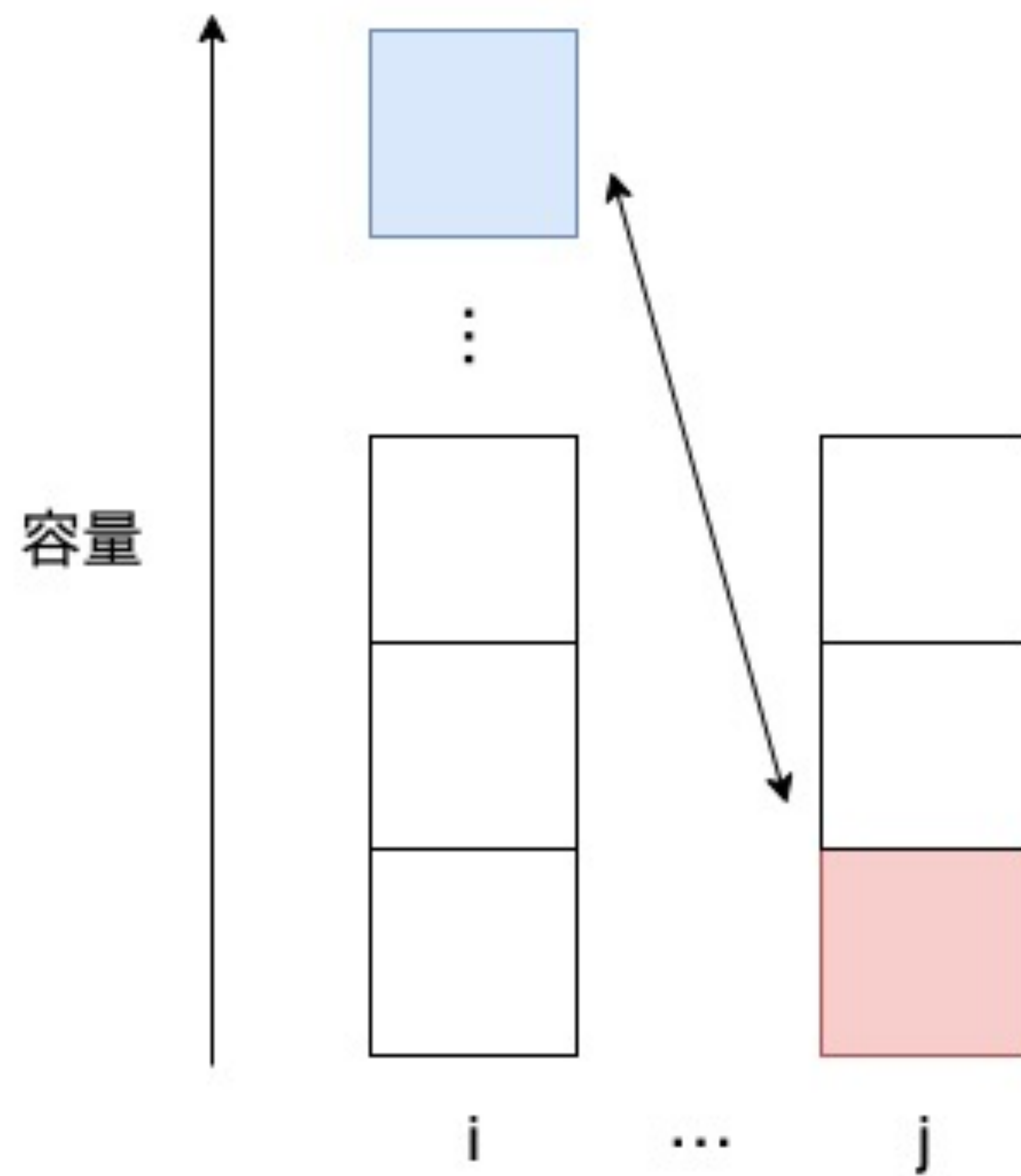
$$A = (3, 2, 2, 2)$$

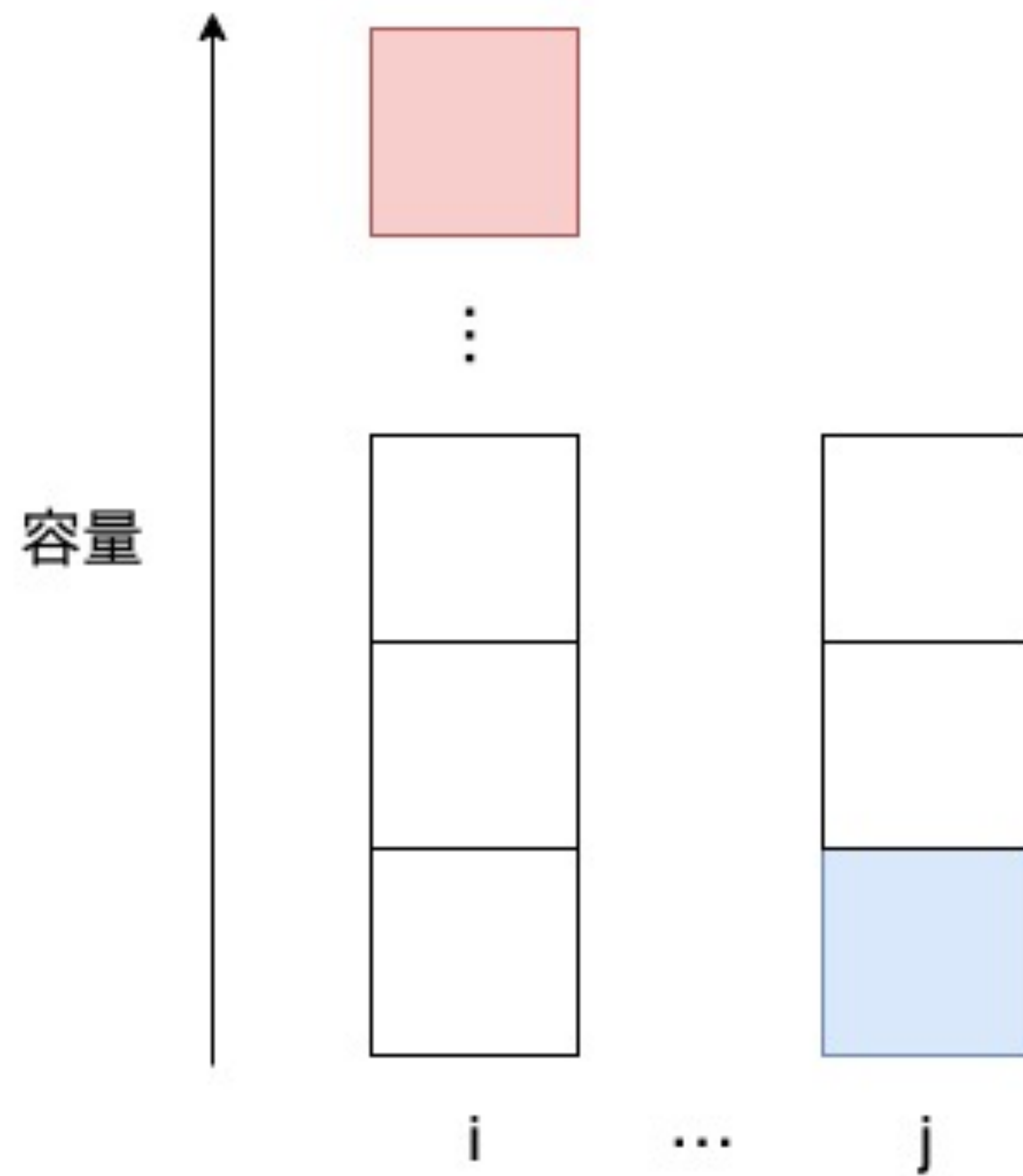
考察

- この貪欲法は正しい（つまり、貪欲法で割り当てを構築できないならば、条件を満たす構築は存在しない）
- 対偶を示す
 - 条件を満たす構築が存在 $\Rightarrow$ 貪欲法で割り当てを構築できる



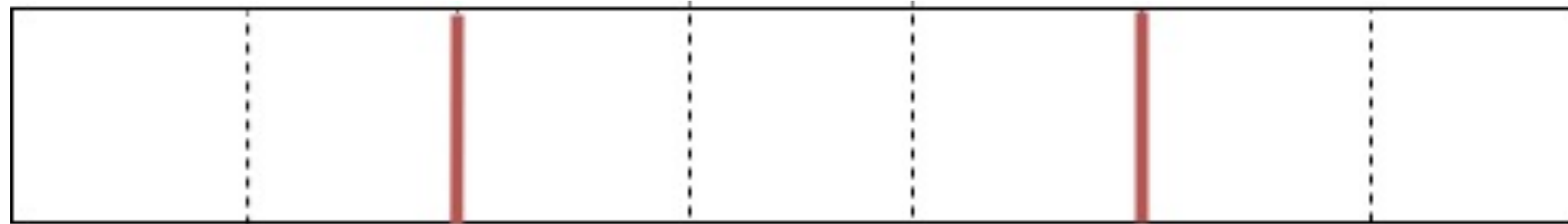






小課題 3 ($S \leq 15$)

- 総和が S になるような箱の選び方は高々 2^{S-1} 通り $\rightarrow$ 全探索できる



S

- それぞれの場合について、割り当て可能かどうかを貪欲法で判定

ここまでで 25 点

小課題 4 ($S \leq 500$)

- 割り当て可能ならば、実際の構築は貪欲法でできる
 - 「貪欲法を実際にやってみて構築できるかどうか」で割り当て可能かどうか判定するのは間に合わない
 - もっと単純な条件を見つけない

小課題 4 ($S \leq 500$)

- 必要条件を列挙してみる 箱の容量を $C_1 \geq C_2 \geq \dots$ として、

- $C_1 \leq N$

- $C_1 + C_2 \leq \sum_{i=1}^N \min(A_i, 2)$

- $C_1 + C_2 + C_3 \leq \sum_{i=1}^N \min(A_i, 3)$

- $\vdots$

小課題 4 ($S \leq 500$)

- 実は十分条件
- 証明は省略（帰納法など）

小課題 4 ($S \leq 500$)

- 条件が単純になったので、容量が大きい順に箱を選んでいく DP ができる
 - $\text{dp}[i][j][k] = i$ 箱選んで、容量が全て j 以上で、容量の和が k の状態にできるか
- 遷移
 - $\text{dp}[i][j][k] \leftarrow \text{dp}[i][j][k] \text{ or } \text{dp}[i][j+1][k]$
 - $\text{dp}[i][j][k] \leftarrow \text{dp}[i][j][k] \text{ or } \text{dp}[i-1][j][k-j]$ (容量 j の箱を使えるとき)
- 計算量 : $O(NS^2)$

ここまでで 70 点

小課題 5 ($S \leq 3\,000$)

- $\text{dp}[i][j][k] = i$ 箱選んで、容量が全て j 以上で、容量の和が k の状態にできるか
- $ij > S$ のとき、容量の和が S を超えてしまうので見ても無意味
- (i, j) のペアとして考えるべきものは $S \log S$ 通り（調和級数）
- 計算量： $O(S^2 \log S)$

ここまでで 85 点

小課題 6 ($S \leq 15\,000$)

- 遷移
 - $\text{dp}[i][j][k] \leftarrow \text{dp}[i][j][k] \text{ or } \text{dp}[i][j+1][k]$
 - $\text{dp}[i][j][k] \leftarrow \text{dp}[i][j][k] \text{ or } \text{dp}[i-1][j][k-j]$ (容量 j の箱を使えるとき)
- $\text{dp}[i][j]$ の値をまとめて bitset で持つと、
 - $\text{dp}[i][j] \leftarrow \text{dp}[i][j] \text{ or } \text{dp}[i][j+1]$
 - $\text{dp}[i][j] \leftarrow \text{dp}[i][j] \text{ or } \text{dp}[i-1][j] \ll j$
- 計算量: $O\left(\frac{S^2 \log S}{w}\right)$

ここまでで 100 点

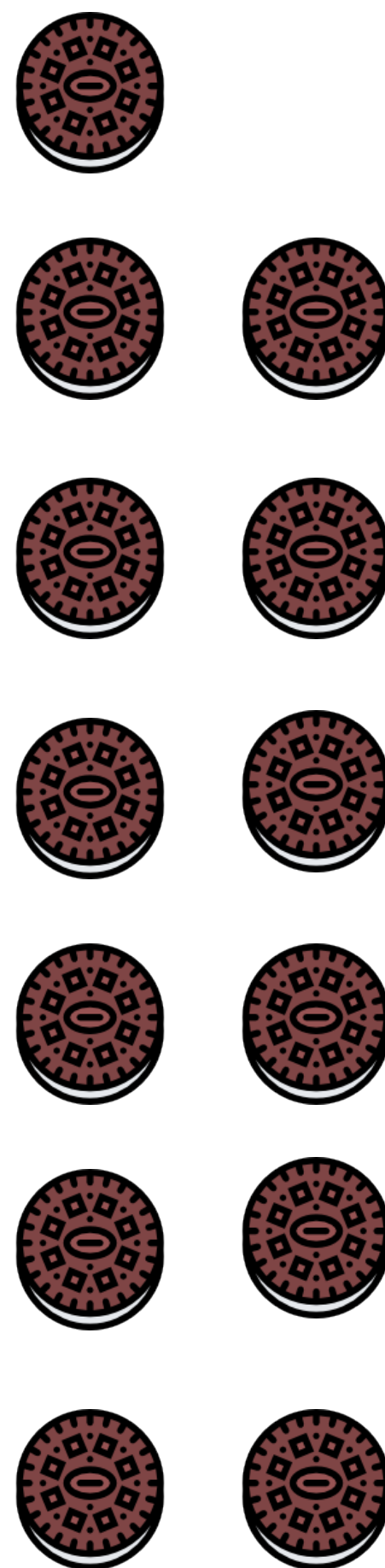
得点分布



100



70



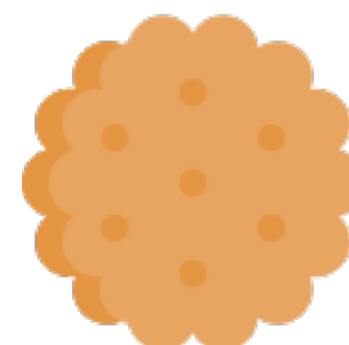
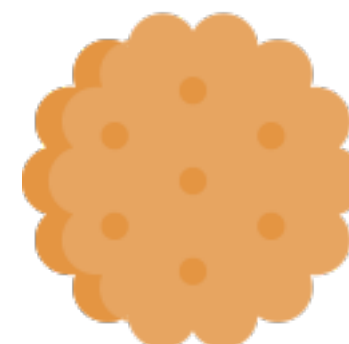
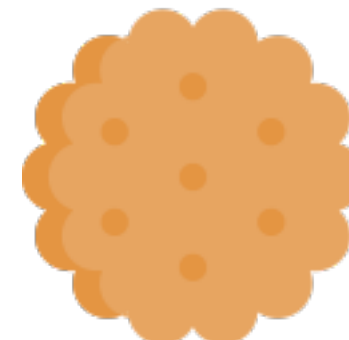
25



19



13



0

JOI 2022/2023 春季トレーニング Day 3

旅行 (Tourism) 解説

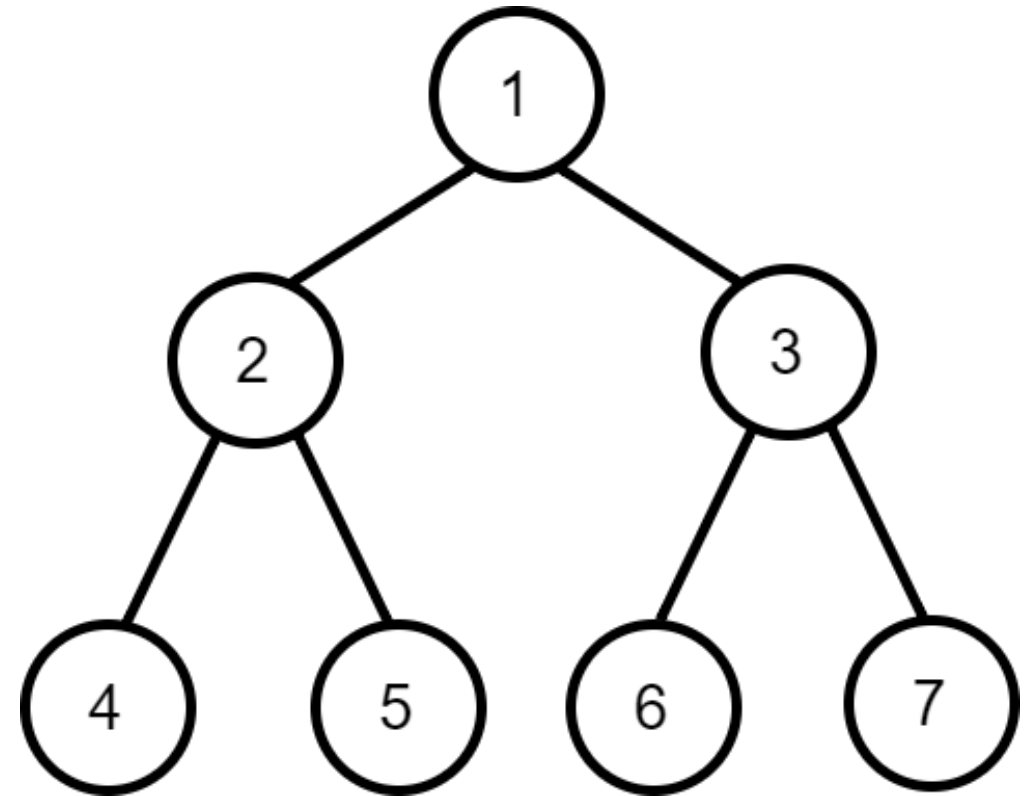
解説担当 : 菅井

問題概要

- N 頂点の木
- 木の頂点の列 $C_1, C_2, \dots, C_M$
- Q クエリ: 各クエリでは L, R が与えられる
- 木上を移動し, 頂点 $C_L, C_{L+1}, \dots, C_R$ をすべて通りたい
- 1 回以上通った頂点の個数の最小値は？
- $N, M, Q \leq 100\,000$

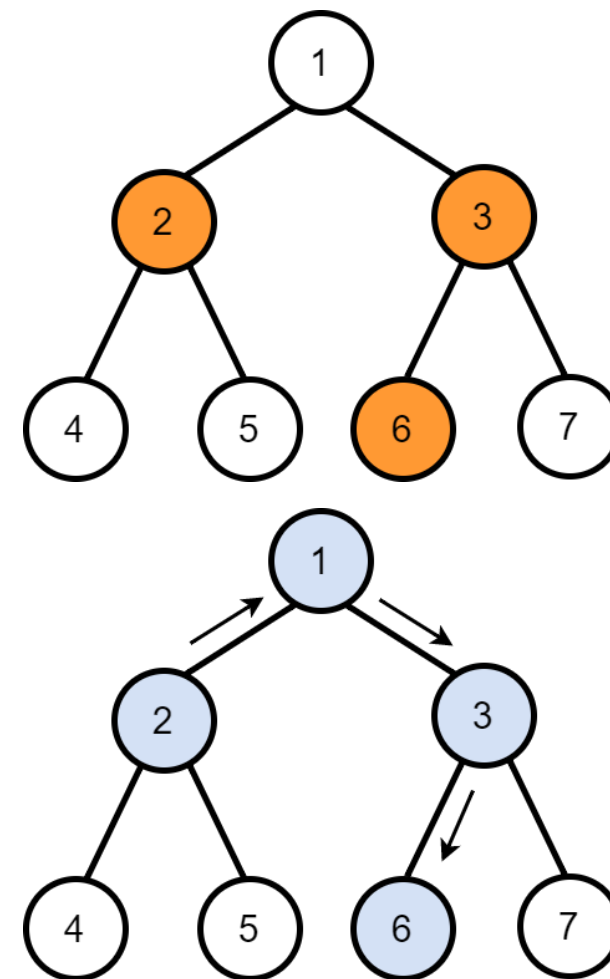
サンプル 1

- $C = [2, 3, 6, 4, 5, 7]$
- $L_1 = 1, R_1 = 3$
- $L_2 = 4, R_2 = 6$



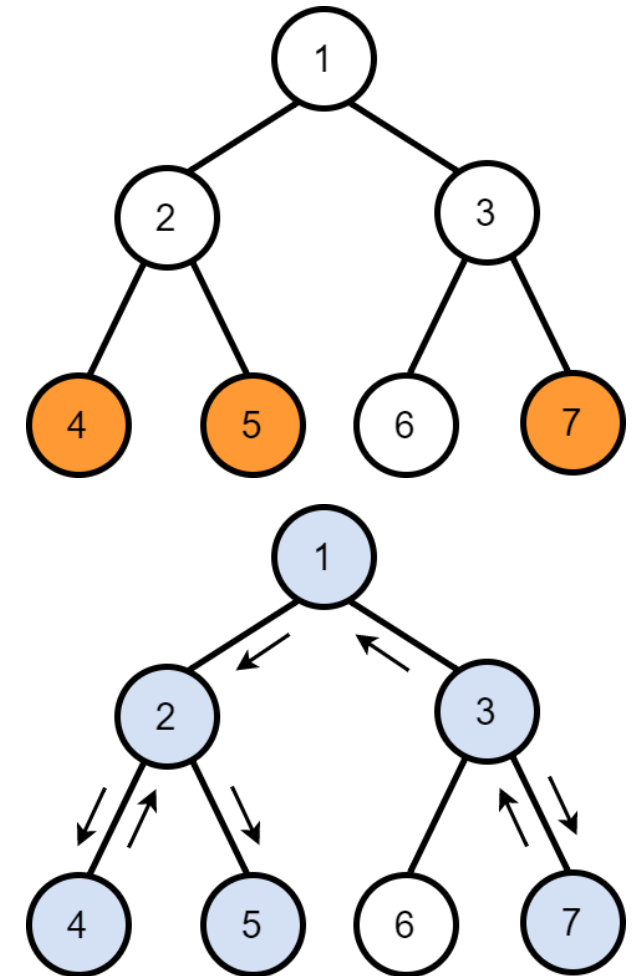
クエリ 1

- 頂点 2, 3, 6 を通りたい
- $2 \rightarrow 1 \rightarrow 3 \rightarrow 6$ と移動
- 4 つの頂点を 1 回以上通る



クエリ 2

- 頂点 4, 5, 7 を通りたい
- $3 \rightarrow 7 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 2 \rightarrow 5$ と移動
- 6 つの頂点を 1 回以上通る



小課題

1. (5 点) $N, M, Q \leq 300$
2. (5 点) $N, M, Q \leq 2\,000$
3. (7 点) パス
4. (18 点) 区間が共通部分を持たない
5. (24 点) 完全二分木 (の一部)
6. (41 点) 追加の制約はない

小課題 1 (5 点)

頂点 $C_L, C_{L+1}, \dots, C_R$ を通りたい

最終的に通る頂点の集合はどのようなになるか？

考察

- $i = L, L + 1, \dots, R - 1$ について, 頂点 C_i と C_{i+1} を結ぶパス上の頂点は必ず通る必要がある
 - パス上のある頂点を通らないと, C_i と C_{i+1} を行き来できないので, 必要な頂点をすべて通れない

$(C_L - C_{L+1} \text{ パス}) \cup (C_{L+1} - C_{L+2} \text{ パス}) \cup \dots \cup (C_{R-1} - C_R \text{ パス})$ に含まれる頂点は必ず通らなければならない

考察

逆に, これらの頂点だけ通る方法が存在する

- C_L から開始
- C_L から C_{L+1} へ最短経路で移動
- C_{L+1} から C_{L+2} へ最短経路で移動
- $\vdots$
- C_{R-1} から C_R へ最短経路で移動

小課題 1 (5 点) 解法

- $(C_L - C_{L+1} \text{ パス}) \cup (C_{L+1} - C_{L+2} \text{ パス}) \cup \dots \cup (C_{R-1} - C_R \text{ パス})$ に含まれる頂点の個数を求めればよい
- BFS/DFS など $\rightarrow$ 経路を復元する
- コーナーケース: $L = R$ の場合に注意 (答えは 1)
- 計算量 $O(NMQ)$

小課題 2 (5 点)

- 最終的に通る頂点は木をなす
- 頂点を数えるかわりに、辺を数えることにしてもよい (最後に 1 を足しておく)
- ある辺を通らなければならない条件は？

考察

- ある辺を切ると 2 つの木に分かれる
- 2 つの木どちらにも通りたい頂点があるとき, またそのときのみ, その辺を通る必要がある

典型: 根付き木にする

- 問題が根付き木でなくても，根を勝手に定めてしまうと実装が楽になることが多い
- BFS を使うと，各頂点の親・子・深さなどを簡単に計算できる
- BFS 順も持っておくと便利

```
std::queue<int> Q;  
Q.push(0);  
while (!Q.empty()){  
    int v = Q.front();  
    Q.pop();  
    bfs.push_back(v);  
    for (int w : E[v]){  
        if (w != parent[v]){  
            parent[w] = v;  
            child[v].push_back(w);  
            depth[w] = depth[v] + 1;  
            Q.push(w);  
        }  
    }  
}
```

考察

- 2つの木どちらにも通りたい頂点があるとき、またそのときのみ、その辺を通る必要がある
- 辺が頂点 u, v を結ぶとし、 u が v の親だとすると、 v の部分木に通りたい頂点がないか、通りたい頂点がすべてあるなら、その辺は通らない
- そうでないなら、その辺は通る

小課題 2 (5 点) 解法

- 各頂点について、部分木に通りたい頂点が何個あるか木 DP で求める
- $\text{dp}[v] = 0$ または $\text{dp}[v] = R - L + 1$ なら、その辺は通らない
- そうでないなら、その辺は通る
- 計算量 $O((N + M)Q)$

小課題 3 (7 点)

- パスグラフ
- 通る頂点は区間になる
- $C_L, C_{L+1}, \dots, C_R$ のうち最小の頂点から最大の頂点まで通る

小課題 3 (7 点) 解法

- 区間 $[L, R]$ の最小値と最大値をそれぞれ求めればよい
- Segment Tree を 2 本作る
- 計算量 $O(N + M + Q \log M)$

小課題 4 (18 点)

- クエリの区間に共通部分がない
- クエリに含まれる頂点数に比例する計算量で解くことができればよい

考察

- 結局, 頂点 $C_L, C_{L+1}, \dots, C_R$ を含む木の頂点数/辺数の最小値を求めればよい
- これは「最小シュタイナー木問題」で, 一般のグラフでは NP 困難であることが知られている
- 木上の最小シュタイナー木といえは...?

典型: 木上の最小シュタイナー木

頂点 $X_1, X_2, \dots, X_k$ を含む木の辺数の最小値は, 以下のように求められる

- 前計算で DFS し, 木の頂点を行きがけ順に並べる
- $X_1, X_2, \dots, X_k$ を行きがけ順で現れる順にソートする
- $\frac{1}{2} \{d(X_1, X_2) + d(X_2, X_3) + \dots + d(X_{k-1}, X_k) + d(X_k, X_1)\}$ が答え. ただし, $d(u, v)$ は頂点 u, v の距離
- 類題: [yukicoder #901 K-ary extreme](#)

小課題 4 (18 点) 解法

- 前計算で頂点を行きがけ順に並べる: $O(N)$
- 各クエリで頂点をソート: $O((R - L) \log(R - L))$
- 距離を $O(R - L)$ 回求める
- 合計で $O(N + M(\log N + \log M))$ など

小課題 5 (24 点)

- 完全二分木

考察

- $(C_L - C_{L+1} \text{ パス}) \cup (C_{L+1} - C_{L+2} \text{ パス}) \cup \dots \cup (C_{R-1} - C_R \text{ パス})$ に含まれる頂点の個数を求めたい
- $L = l$ と固定すると, R を増やしたときに集合から辺が取り除かれることはないので, それぞれの辺が含まれるかどうかは R について単調性がある
- 境界の位置を求めることを考える

考察

- $C_l - C_{l+1}$ パスに含まれる辺は, $R \geq l + 1$ のときに含まれる
- それ以外の辺で, $C_{l+1} - C_{l+2}$ パスに含まれる辺は, $R \geq l + 2$ のときに含まれる
- それ以外の辺で, $C_{l+2} - C_{l+3}$ パスに含まれる辺は, $R \geq l + 3$ のときに含まれる
- $\vdots$
- 最後まで選ばれなかった辺は R によらず含まれない

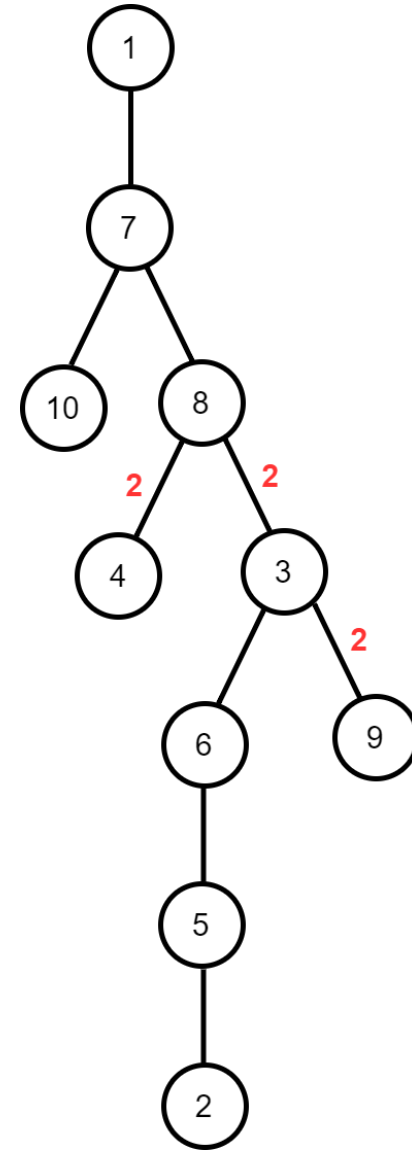
考察

それぞれの辺に, 「その辺が含まれるような R の最小値」を書きたい. (絶対に含まれないなら ∞)

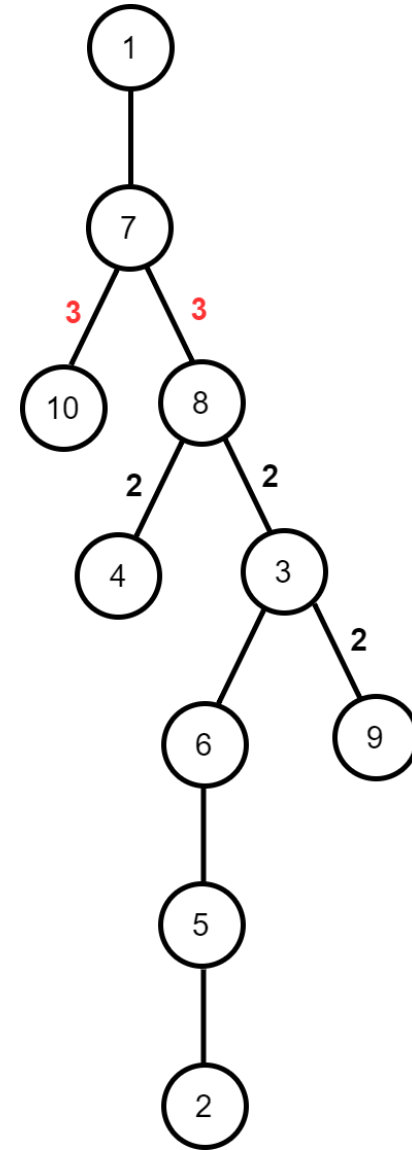
これは以下のような手順でできる

- $i = l + 1, l + 2, \dots, R$ の順に, $C_{i-1} - C_i$ パスに含まれるまだ何も書いていない辺に i と書く
- 最後まで何も書いていない辺は ∞ と書く

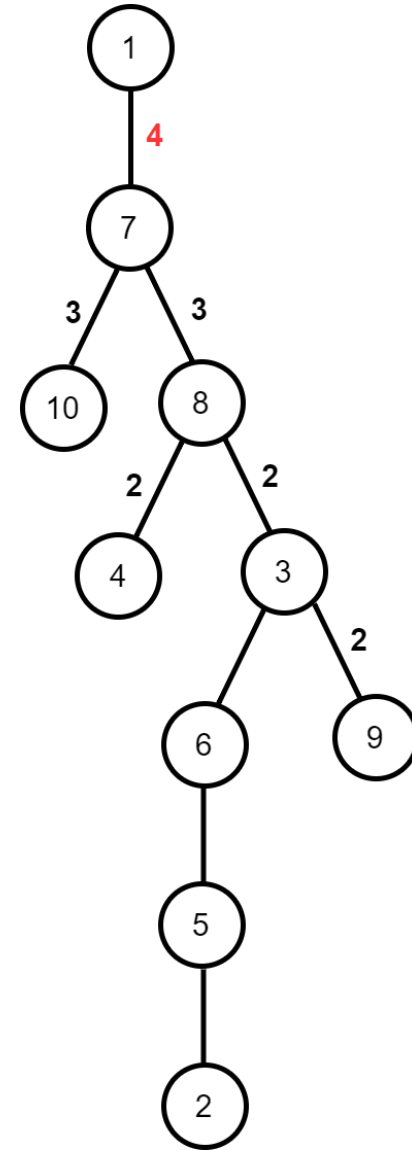
- $C = [9, 4, 10, 1, 10, 7, 6]$
- $l = 1$
- $(C_1 =) 9 - (C_2 =) 4$ パスに含まれる辺に 2 と書く



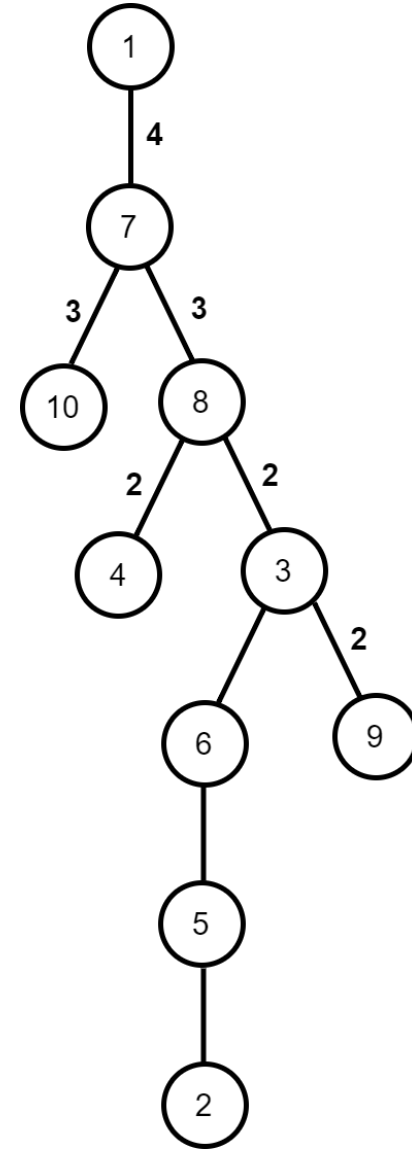
- $C = [9, 4, 10, 1, 10, 7, 6]$
- $l = 1$
- $(C_2 =) 4 - (C_3 =) 10$ パスに含まれるまだ何も書いていない辺に **3** と書く



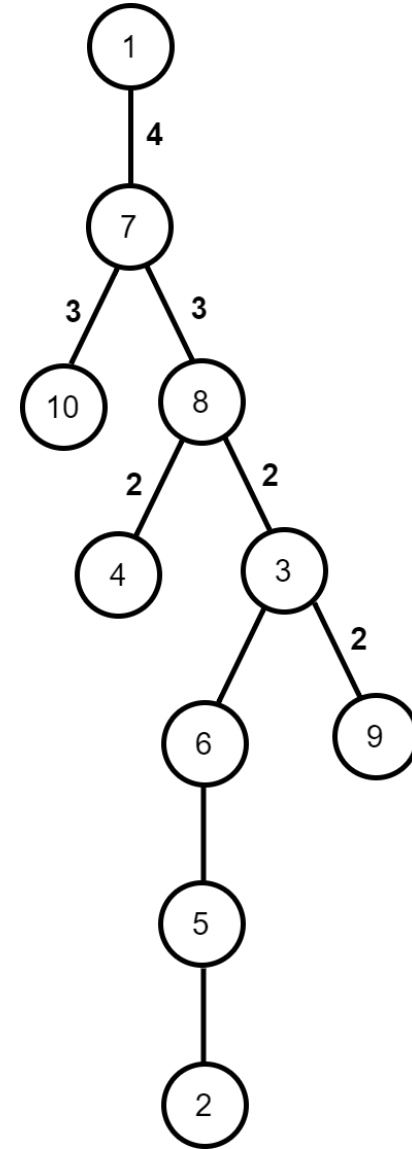
- $C = [9, 4, 10, 1, 10, 7, 6]$
- $l = 1$
- $(C_3 =) 10 - (C_4 =) 1$ パスに含まれるまだ何も書いていない辺に 4 と書く



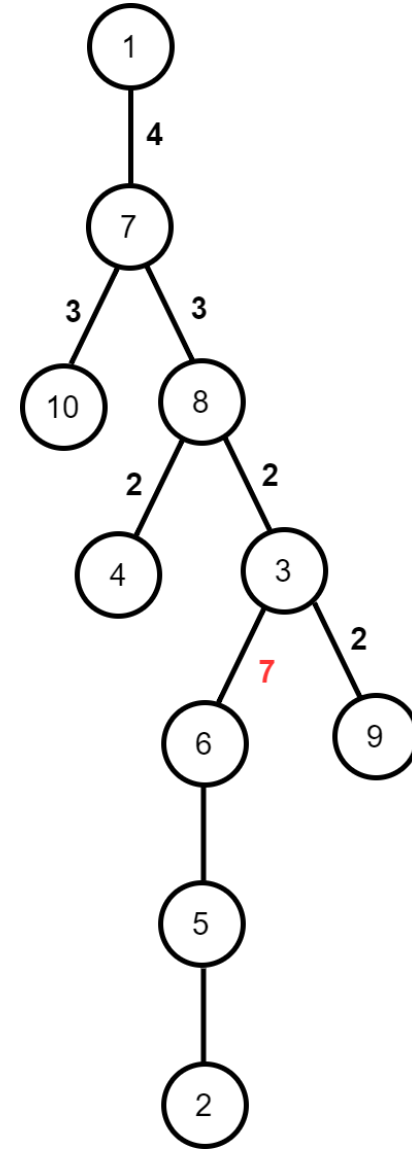
- $C = [9, 4, 10, 1, 10, 7, 6]$
- $l = 1$
- $(C_4 =) 1 - (C_5 =) 10$ パスに含まれるまだ何も書いていない辺に 5 と書く



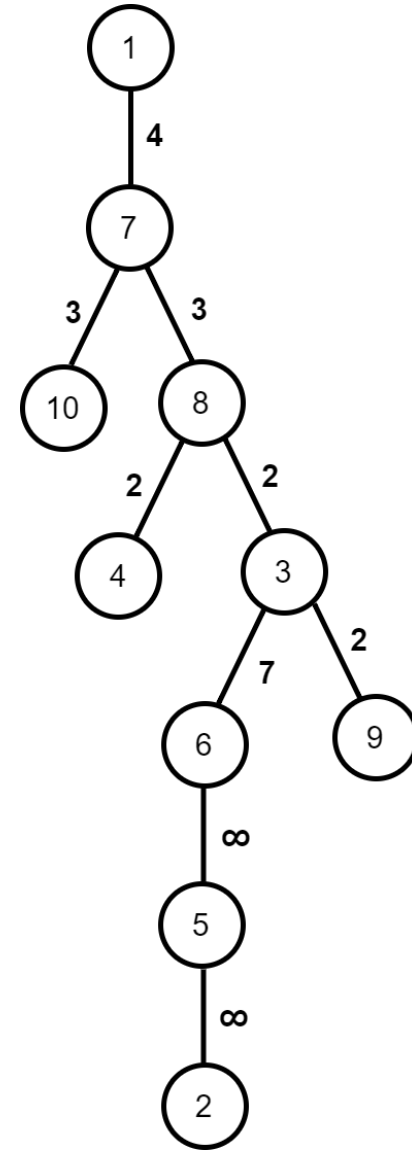
- $C = [9, 4, 10, 1, 10, 7, 6]$
- $l = 1$
- $(C_5 =) 10 - (C_6 =) 7$ パスに含まれるまだ何も書いていない辺に 6 と書く



- $C = [9, 4, 10, 1, 10, 7, 6]$
- $l = 1$
- $(C_6 =) 7 - (C_7 =) 6$ パスに含まれるまだ何も書いていない辺に 7 と書く



- $C = [9, 4, 10, 1, 10, 7, 6]$
- $l = 1$
- まだ何も書いていない辺に ∞ と書く
- $(R = r \text{ のときの答え}) = (r \text{ 以下の値が書かれた辺の本数}) + 1$



考察

- L が固定の場合は解けそう
- 実際は L が各クエリで異なるため、このままでは解けない
- 「まだ何も書いていない辺」は l によって異なるため、難しそう

典型: 更新クエリ

- 「区間を〇〇に更新する」というクエリは、逆から見ると「区間に〇〇と書き、それ以降変更されないようにする」クエリになる
- 「区間に〇〇と書き、それ以降変更されないようにする」は「区間のまだ何も書いていないところに〇〇と書く」と同じ
- 今回は逆に、更新クエリに置き換えて考える

言い換え前

- $i = l + 1, l + 2, \dots, R$ の順に, $C_{i-1} - C_i$ パスに含まれるまだ何も書いていない辺に i と書く
- 最後まで何も書いていない辺は ∞ と書く

言い換え後

- 最初にすべての辺に ∞ と書く
- $i = R, R - 1, \dots, l + 1$ の順に, $C_{i-1} - C_i$ パスに含まれるすべての辺に i と書く

この言い換えをしたうえで, l が変化したときどのように状況が変化するか考える

$l = k$ のとき

- すべての辺に ∞ と書く
- $C_{R-1} - C_R$ パスに R と書く
- $C_{R-2} - C_{R-1}$ パスに $R - 1$ と書く
- $\vdots$
- $C_k - C_{k+1}$ パスに $k + 1$ と書く

$l = k - 1$ のとき

- すべての辺に ∞ と書く
- $C_{R-1} - C_R$ パスに R と書く
- $C_{R-2} - C_{R-1}$ パスに $R - 1$ と書く
- $\vdots$
- $C_k - C_{k+1}$ パスに $k + 1$ と書く
- $C_{k-1} - C_k$ パスに k と書く

- $l = k$ の状態から $l = k - 1$ の状態を得るには, $C_{k-1} - C_k$ パスに k と書くだけでよい
- クエリを先読みし, L の降順に見る
- 最初それぞれの辺には ∞ と書かれている
- 以下の 2 種類のクエリを処理することに帰着
 - クエリ 1: 頂点 u, v と整数 x が与えられるので, $u - v$ パスのそれぞれの辺に x と書く
 - クエリ 2: 整数 k が与えられるので, k 以下の数が書かれた辺の本数を答える

考察

- 小課題 5 では完全二分木なので, パスに含まれる辺の本数は $O(\log N)$
- 前半: パスに含まれる辺を列挙する
- 後半: それぞれの辺ごとに更新処理

前半

- パスに含まれる辺が少ないことがわかっているとき、それらを具体的に列挙したい
- 完全二分木なので、2 進数を用いてビット演算で計算することもできる
- 今回はもっと汎用的な方法を紹介する

典型: パスに含まれる辺の列挙

- 根付き木と考える
- 前計算で、各頂点の親と深さを求めておく
- 与えられる頂点を u, v とすると、 $u = v$ になるまで以下を繰り返す
 - u と v のうち深さが大きいほうについて、自分とその親の間の辺を答え、親に置き換える (深さが同じ場合はどちらでもよい)
- $O(\text{パスの長さ})$ で列挙できる

後半

- $A[x] = (x \text{ が書かれた辺の本数})$ を管理する
- 1 本の辺に数を書くと, A の要素高々 2 つが変化する
- k 以下の数が書かれた辺の本数は, $A[1] + A[2] + \dots + A[k]$ に等しい
- 結局, A 上の一点加算・区間和に帰着する
- Binary Indexed Tree を使えばよい

小課題 5 (24 点) 解法

- クエリを先読みし, L の大きい順に答える
- L を 1 減らすとき, $C_{L-1} - C_L$ パスに含まれる辺を列挙し, BIT を更新する
- L がクエリの箇所まで来たら, $A[1] + A[2] + \dots + A[R]$ を答える
- 計算量は $O(N + M \log N \log M + Q \log M)$

小課題 6 (41 点)

- 小課題 5 の考察から, 以下の 2 種類のクエリが処理できればよい
 - クエリ 1: 頂点 u, v と整数 x が与えられるので, $u - v$ パスのそれぞれの辺に x と書く
 - クエリ 2: 整数 k が与えられるので, k 以下の数が書かれた辺の本数を答える

考察

まずパスグラフの場合を考える

- 辺に書かれた数を並べ、数列を作る
- クエリ 1: l, r, x が与えられるので、区間 $[l, r)$ を x に更新する
- クエリ 2: 整数 k が与えられるので、 k 以下の要素の個数を答える

考察

- 隣接する 2 つの要素が異なる箇所に仕切りを入れる
- 数列を連長圧縮して扱くと、区間に入っている仕切りの個数が少なければ高速に処理できる
- いわゆる「区間を `set` で持つテク」

実装

- (区間の開始位置, 値) の `pair` を `set` に入れる
- 両端に番兵を置くと実装しやすい
- 区間が消えるときは, その値の出現数が区間の長さだけ減る
- 最後に x の出現数が $r - l$ だけ増える
- 小課題 5 と同様に, それぞれの数の出現回数を BIT で管理

計算量解析

- クエリ 1 で新たに最大 2 箇所仕切りが入る
- 区間が消える回数は、区間が作られる回数で上から抑えられる $\rightarrow$ 区間が消える回数は $O(M)$ 回
- 全体で計算量は $O(N + (M + Q) \log M)$

考察

- パスの場合は解けたので, どうにかして問題をパスの場合に帰着させたい
- 木をパスに変換するアルゴリズム
- Heavy Light Decomposition

満点解法

- クエリを先読みし, L が大きい順に処理
- HLD して各 $C_{i-1} - C_i$ パスを $O(\log N)$ 個の区間に変換し, 区間ごとに処理
- 各数の出現回数を管理する BIT を更新
- 計算量 $O(N + M \log N \log M + Q \log M)$

別解 1: 分割統治

- 数列 C 上で分割統治
- 区間 $[l, r)$ を見ているとすると, $m = \lfloor \frac{l+r}{2} \rfloor$ とし m が含まれるクエリだけ考えればよい
- $((C_L - C_{L+1}) \cup \dots \cup (C_{m-1} - C_m)) \cup ((C_m - C_{m+1}) \cup \dots \cup (C_{R-1} - C_R))$ に含まれる辺の本数を求めたい
- $C_l, C_{l+1}, \dots, C_r$ について木を圧縮し, $O(r - l)$ 頂点の木を作る

- それぞれの辺が $(C_L - C_{L+1}) \cup \dots \cup (C_{m-1} - C_m)$ に含まれるかどうかは L について単調性がある (R についても同様)
- 各辺 e について, 「 $L \leq x$ または $y \leq R$ 」の条件と重みがあり, 満たしている条件の重みの総和を求めることに帰着
- 平面走査と BIT でできる
- 計算量 $O(N + M \log^2 M + Q \log M)$ など

別解 2: Mo's Algorithm

- 小課題 4 で, DFS 順の位置を管理して, 1 つ追加・削除を処理する
- 直前・直後の頂点の取得, 距離の計算で $\log$ が付く
- 計算量 $O(N + M\sqrt{Q} \log N + Q \log Q)$: TLE
- Rollback Mo + 連結リストで直前・直後の頂点を取得し, Sparse Table で距離を計算すると $\log$ が落ちて満点

得点分布

得点	小課題	人数	累積人数
100	123456	1	1
59	12345	4	5
41	123 5	1	6
35	1234	6	12
31	3 5	1	13
28	12 4	1	14
17	123	7	21
12	1 3	3	24
5	1	1	25
0		2	27



最後の戦い 解説

情報オリンピック春季トレーニング 4日目

2023 年 3 月 22 日

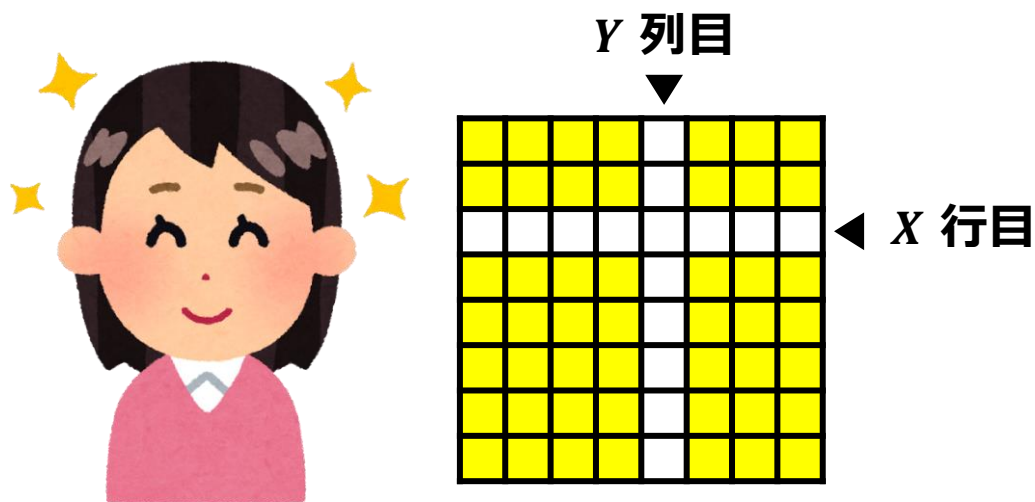
米田優峻 (@e869120)



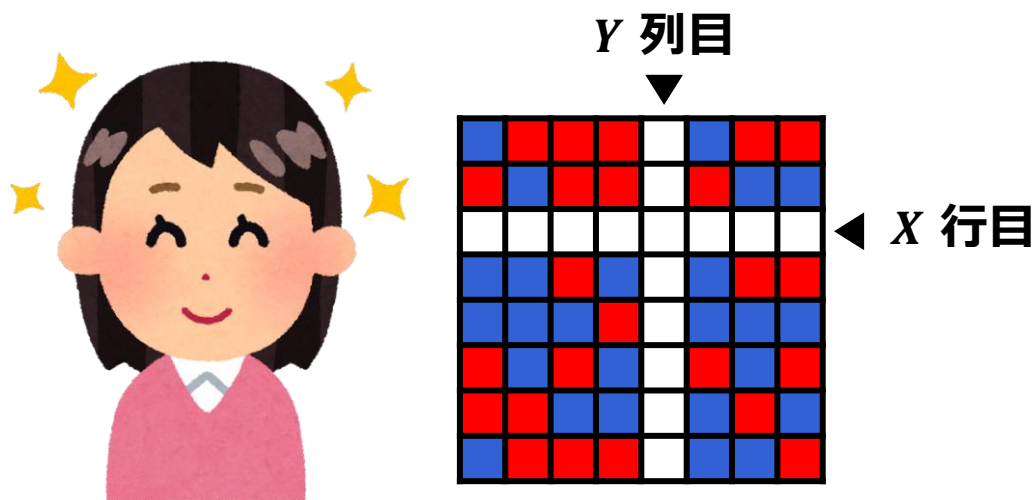
AAAABBBBBBAB...



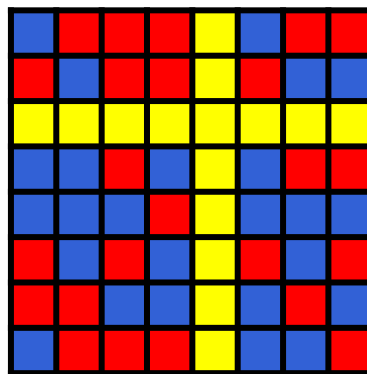
Anna は Bruno に 「長さ N の AB 文字列」 を伝えたいです
ただし、決められた方法で伝えなければなりません



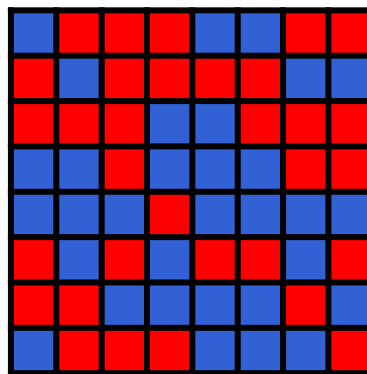
まず、Anna は 8×8 のマス目の X 行目・ Y 列目以外の
49 個のマスに、青か赤で色を塗ります



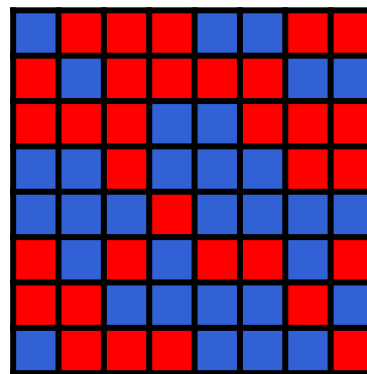
まず、Anna は 8×8 のマス目の X 行目・ Y 列目以外の
49 個のマスに、青か赤で色を塗ります



次に、ビ太郎が残りの 15 マスに青か赤で色を塗ります
(Anna にも Bruno にも見えない場所で塗ります)



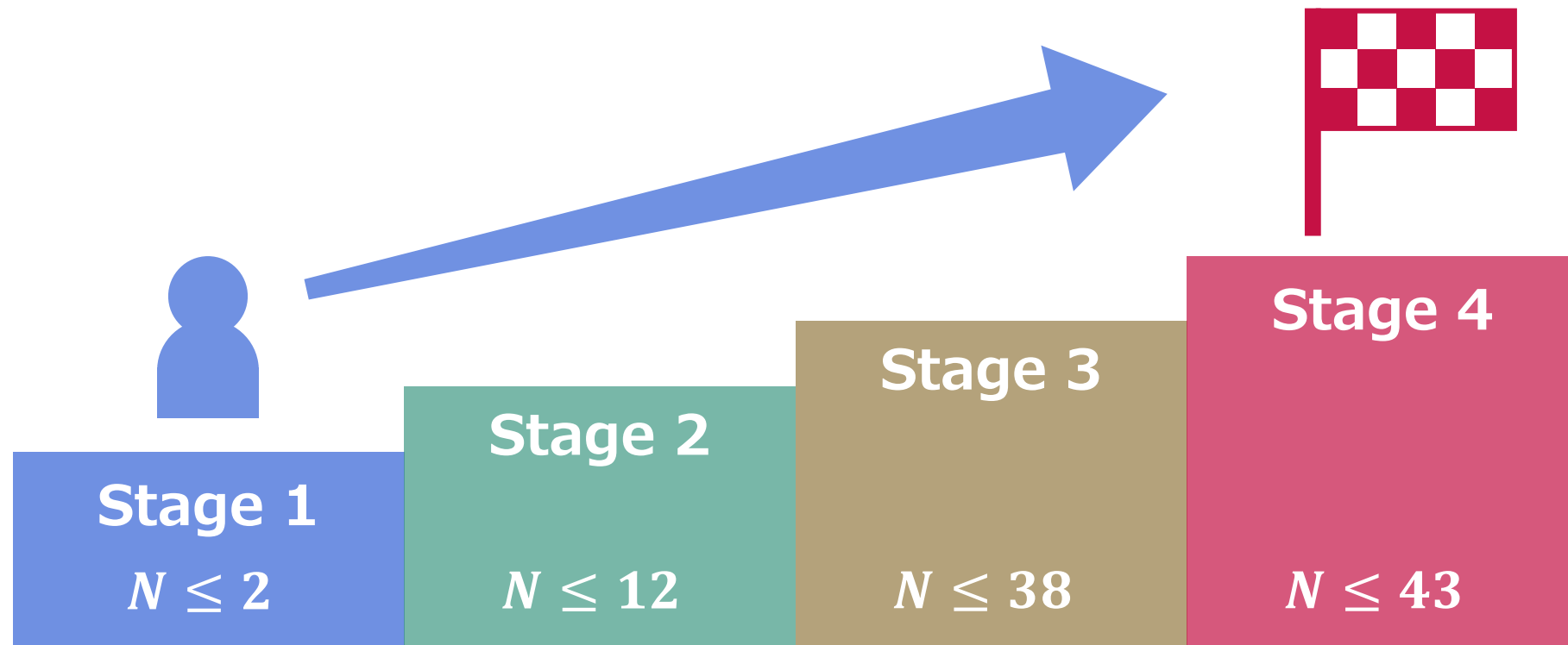
**次に、ビ太郎が残りの 15 マスに青か赤で色を塗ります
(Anna にも Bruno にも見えない場所で塗ります)**



最後に、Bruno がマス目を見ます (x, y の値は見えません)
長さ N の文字列を正しく当ててください

より長い文字列でも当てられた場合、高得点が得られます

あなたはどこまで行けますか？



Stage 1

アイデア：赤の個数で識別する

Anna

- 文字列が A ならば、全部青で塗る
- 文字列が B ならば、全部赤で塗る

Bruno

- 赤が 32 個以下のときに限り「A」と判定

1

$N = 1$ の解法：具体例

12 / 105

例

文字列 A を伝えたい場合

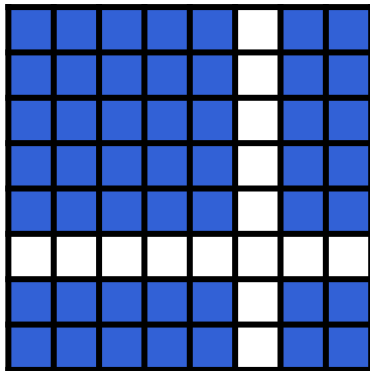
1

$N = 1$ の解法 : 具体例

13 / 105

例

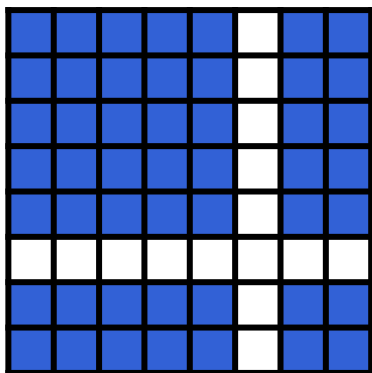
文字列 A を伝えたい場合



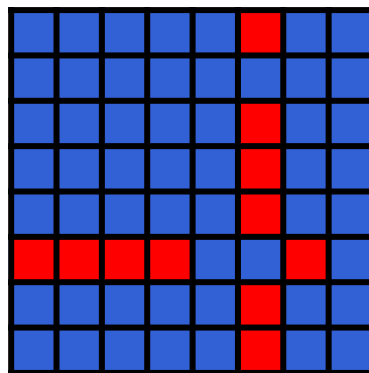
Annaは全部
青で塗る

例

文字列 A を伝えたい場合



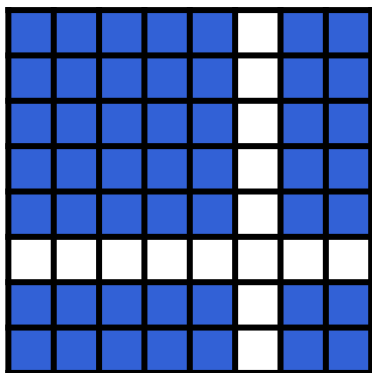
Annaは全部
青で塗る



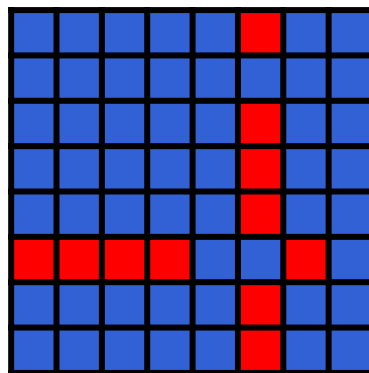
ビ太郎が残りの
マスを塗る

例

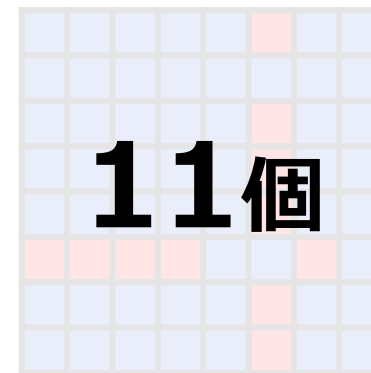
文字列 A を伝えたい場合



Annaは全部
青で塗る



ビ太郎が残りの
マスを塗る



赤の個数 ≤ 32
Brunoは「A」と判定

1

$N = 1$ の解法：具体例

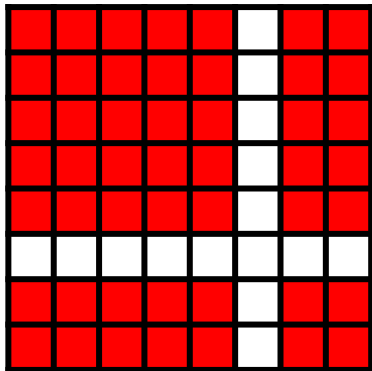
16 / 105

例

文字列 B を伝えたい場合

例

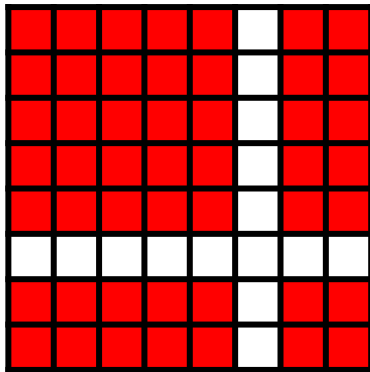
文字列 B を伝えたい場合



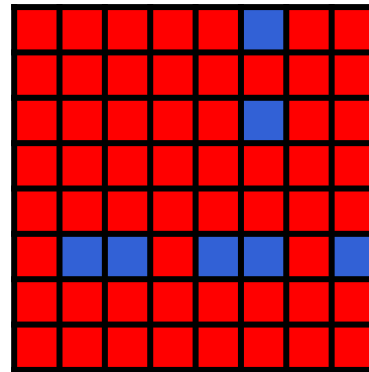
Annaは全部
赤で塗る

例

文字列 B を伝えたい場合



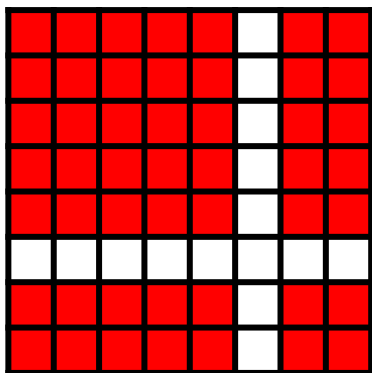
Annaは全部
赤で塗る



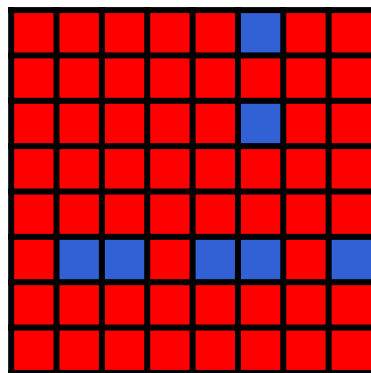
ビ太郎が残りの
マスを塗る

例

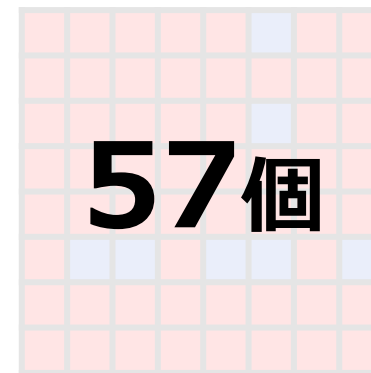
文字列 B を伝えたい場合



Annaは全部
赤で塗る

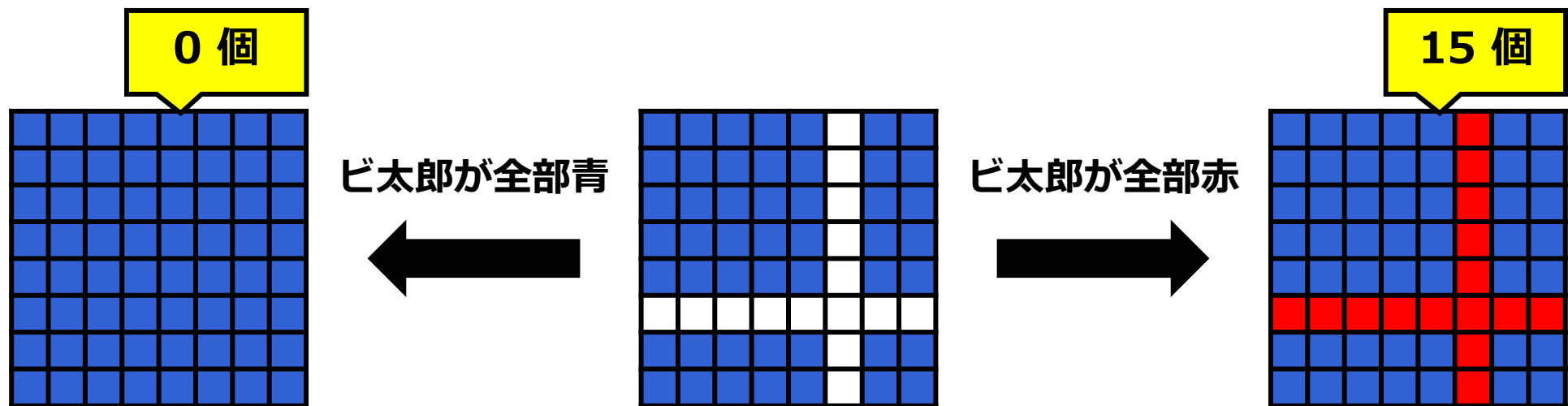


ビ太郎が残りの
マスを塗る



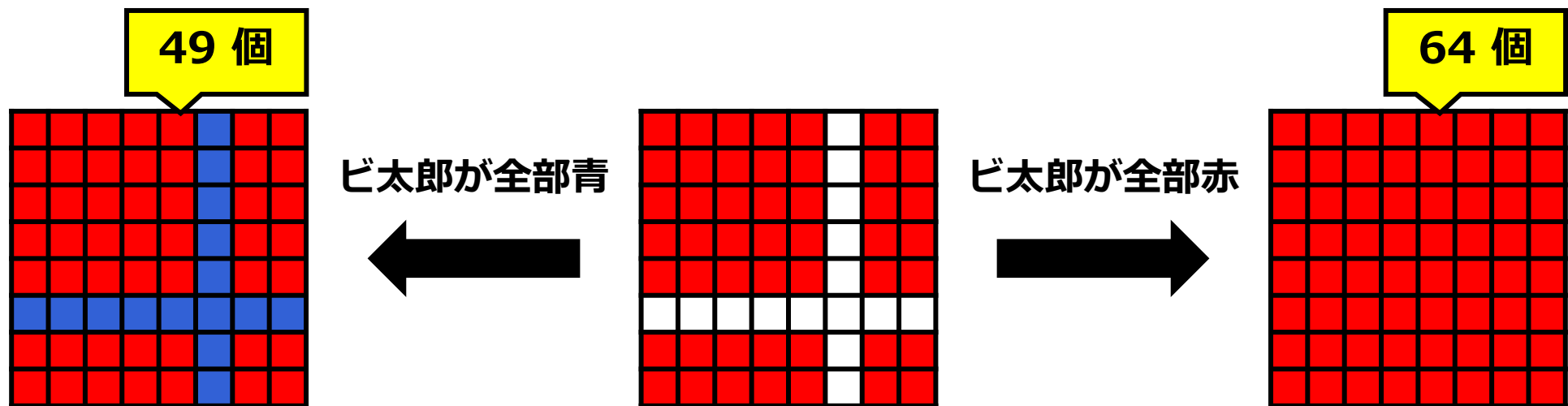
赤の個数 > 32
Brunoは「B」と判定

なぜこの方法が上手くいくか？



Anna が塗れるマスを全部青で塗った場合

赤の個数は 0~15 個の範囲になる (つまり絶対に 32 個以下)



Anna が塗れるマス全部赤で塗った場合

赤の個数は 49~64 個の範囲になる (つまり絶対に 32 個より多い)

この方法を少し工夫すると
 $N = 2$ でも解ける

Anna

以下の数だけ赤で塗る

- AA : 0個
- AB : 16個
- BA : 32個
- BB : 48個

Bruno

赤の個数で場合分け

- 0~15個 : AA
- 16~31個 : AB
- 32~47個 : BA
- 48~63個 : BB

※Anna は、自分が赤で塗るマスの個数を 0~49 個の範囲で調整できる

1

$N = 2$ の解法：具体例

25 / 105

例

文字列 AB を伝えたい場合

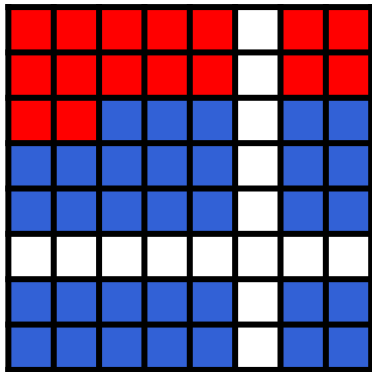
1

$N = 2$ の解法 : 具体例

26 / 105

例

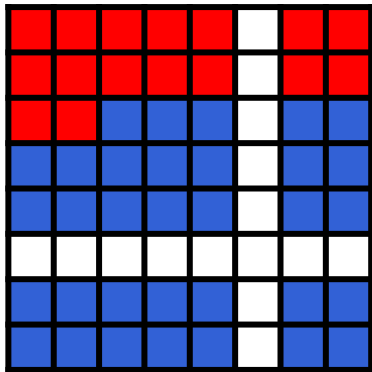
文字列 AB を伝えたい場合



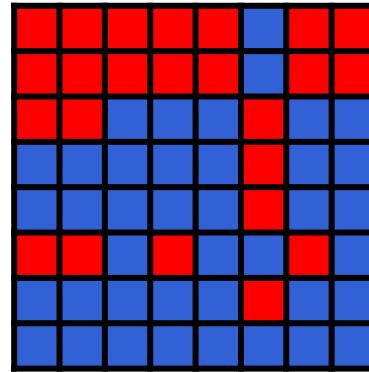
Annaは16マス
赤で塗る

例

文字列 AB を伝えたい場合



Annaは16マス
赤で塗る



ビ太郎が残りの
マスを塗る

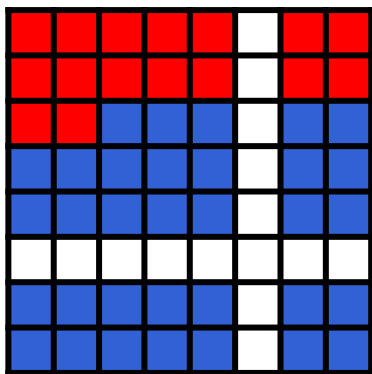
1

 $N = 2$ の解法：具体例

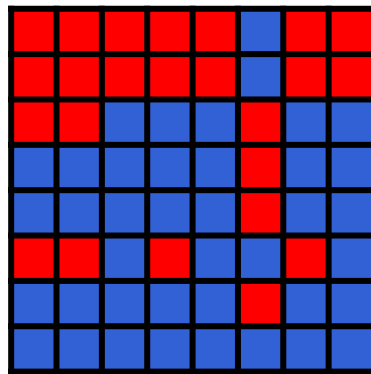
28 / 105

例

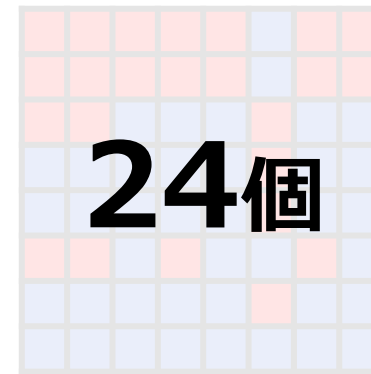
文字列 AB を伝えたい場合



Annaは16マス
赤で塗る



ビ太郎が残りの
マスを塗る



24個

$16 \leq \text{赤の個数} \leq 31$
Brunoは「AB」と判定

1

$N = 2$ の解法：具体例

29 / 105

例

文字列 BB を伝えたい場合

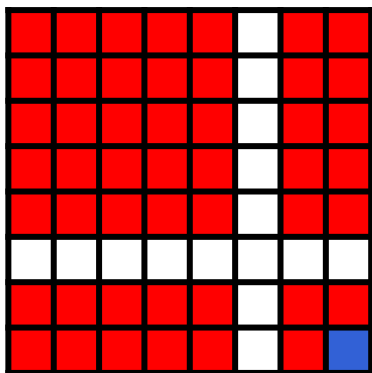
1

$N = 2$ の解法：具体例

30 / 105

例

文字列 BB を伝えたい場合



Annaは48マス
赤で塗る

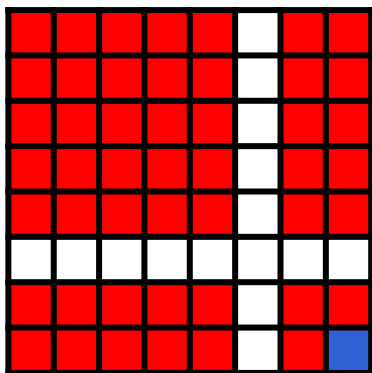
1

$N = 2$ の解法 : 具体例

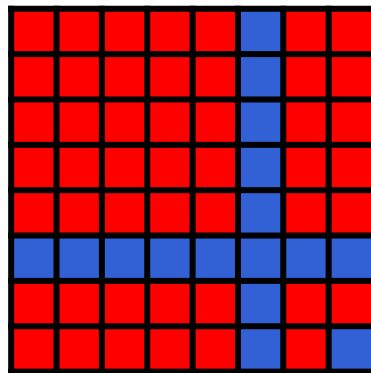
31 / 105

例

文字列 BB を伝えたい場合



Annaは48マス
赤で塗る



ビ太郎が残りの
マスを塗る

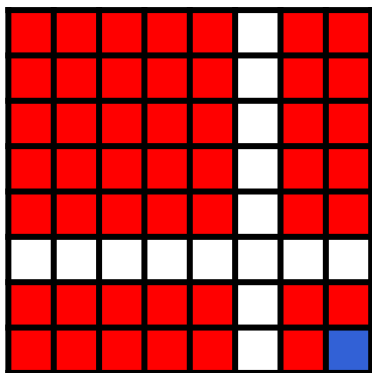
1

 $N = 2$ の解法：具体例

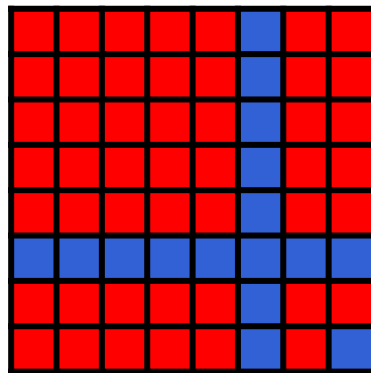
32 / 105

例

文字列 BB を伝えたい場合



Annaは48マス
赤で塗る



ビ太郎が残りの
マスを塗る



$48 \leq \text{赤の個数} \leq 63$
Brunoは「BB」と判定

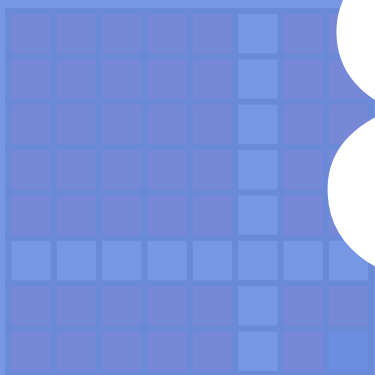
1

$N = 2$ の解法：具体例

33 / 105

例

文字列 BB を伝えたい場合



Annaは48マス
赤で塗る

8



ビ太郎が残りの
マスを塗る

点獲得!

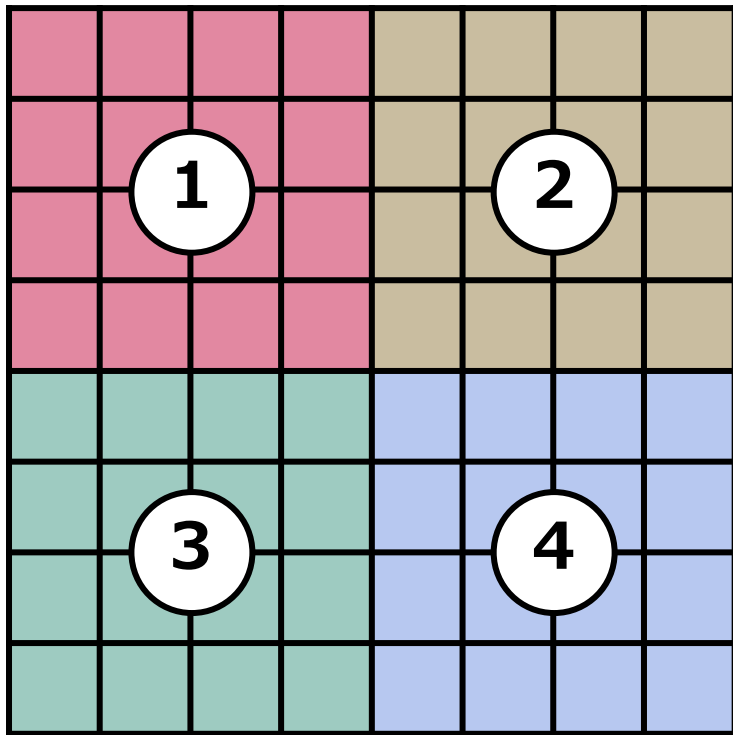
48個



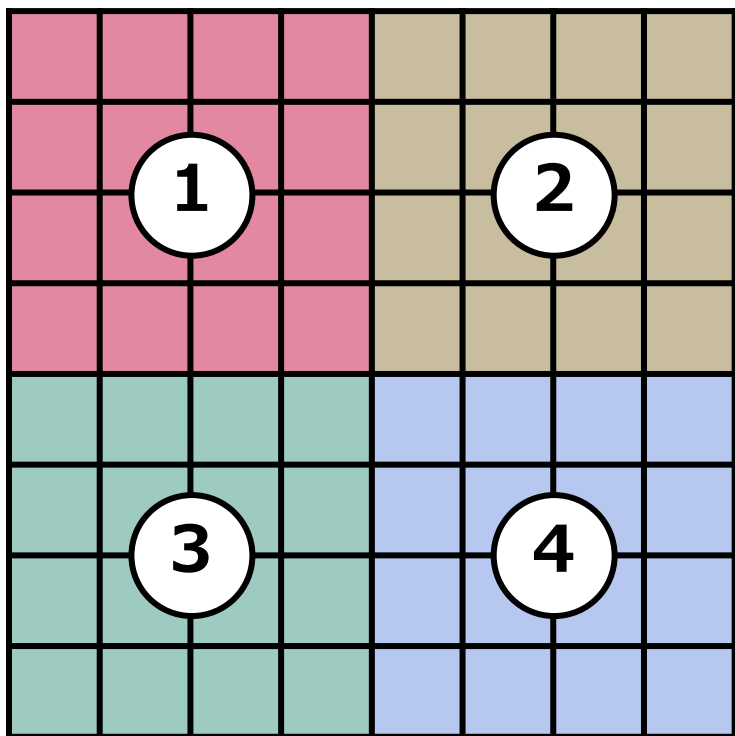
$48 \leq \text{赤の個数} \leq 63$
Brunoは「BB」と判定

Stage 2

アイデア：区画ごとに多数決



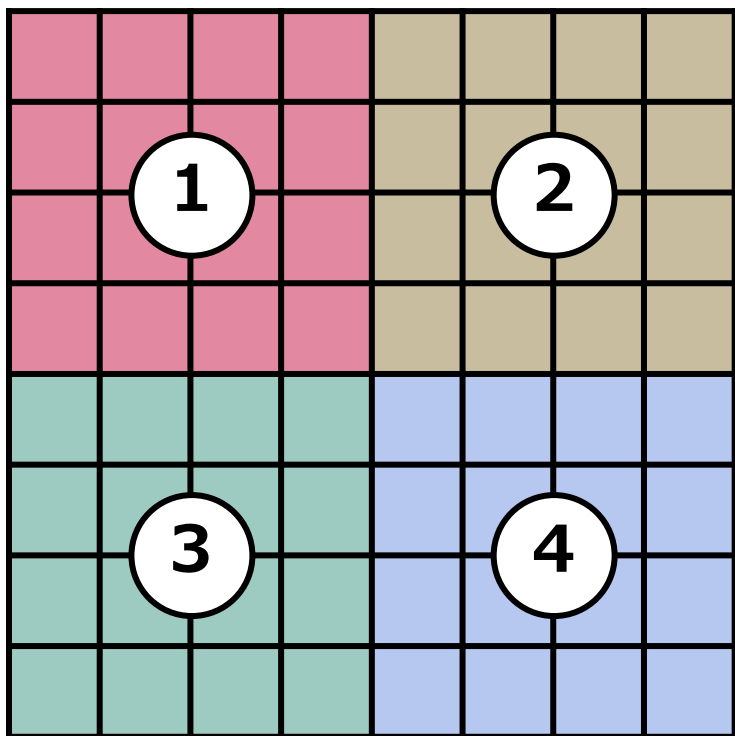
マス目を左図のように 4 分割する



マス目を左図のように 4 分割する



i 文字目が A のとき、Anna は区画 i の塗れる
マスを青で塗る (B のとき赤)



マス目を左図のように 4 分割する



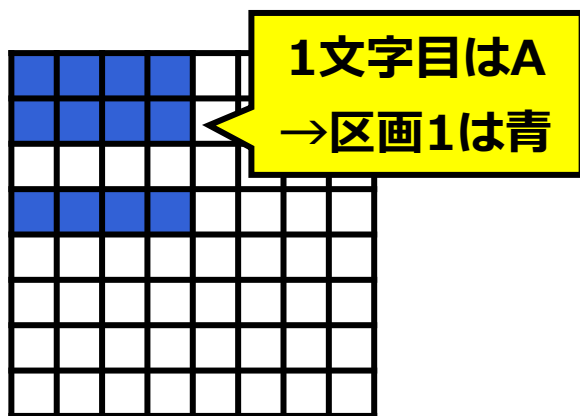
i 文字目が A のとき、Anna は区画 i の塗れる
マスを青で塗る (B のとき赤)



区画 i 内で青の方が赤より多い場合、Bruno は
 i 文字目を A だと判断する

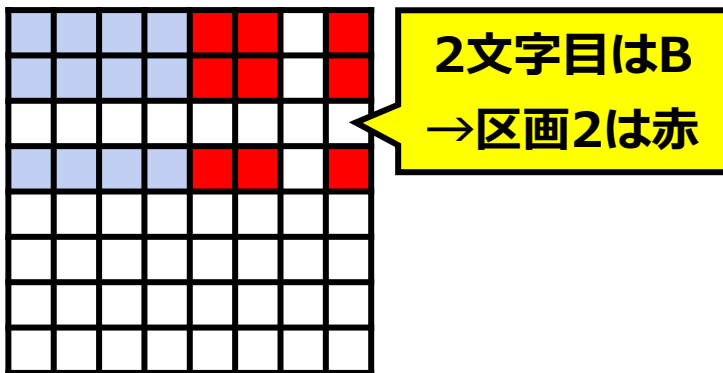
例**文字列 ABBA を伝えたい場合**

例

文字列 **ABBA** を伝えたい場合

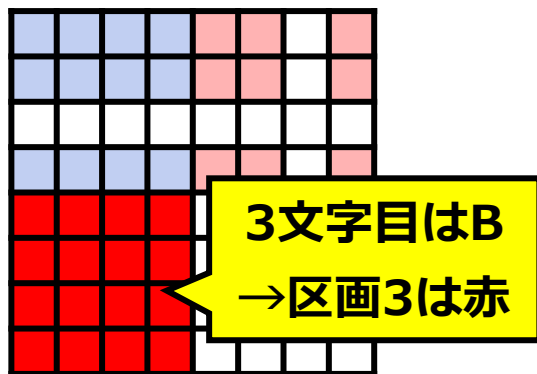
Annaは各区画に
色を塗る

例

文字列 **AB**BA を伝えたい場合

Annaは各区画に
色を塗る

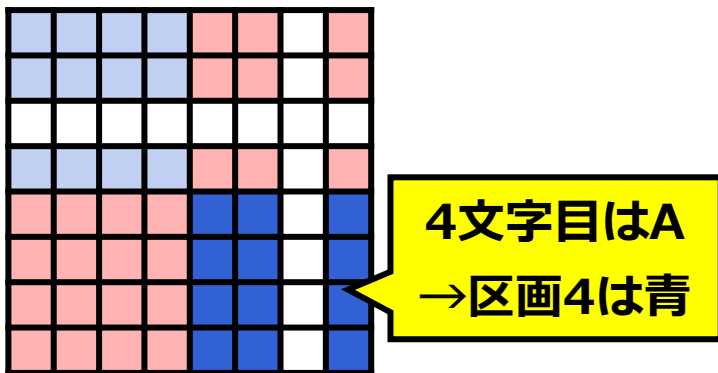
例

文字列 **ABBA** を伝えたい場合

Annaは各区画に
色を塗る

例

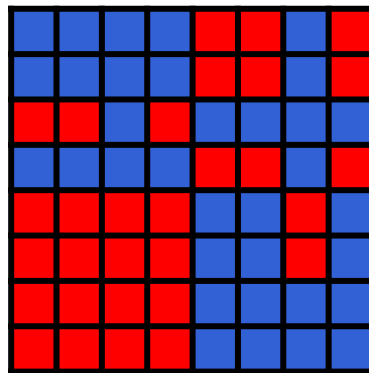
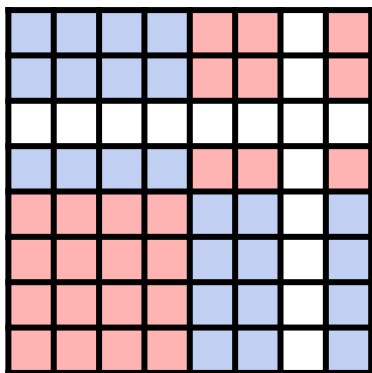
文字列 ABBA を伝えたい場合



Annaは各区画に
色を塗る

例

文字列 ABBA を伝えたい場合

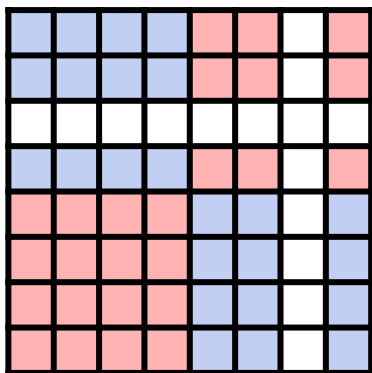


Annaは各区画に
色を塗る

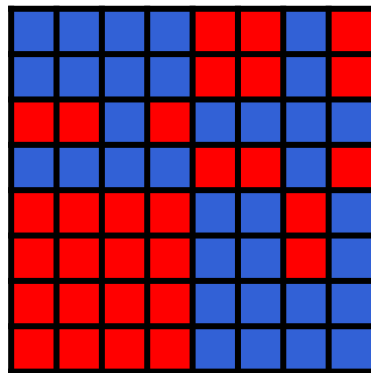
ビ太郎が残りの
マスを塗る

例

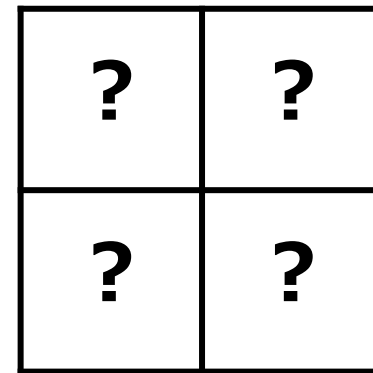
文字列 ABBA を伝えたい場合



Annaは各区画に
色を塗る



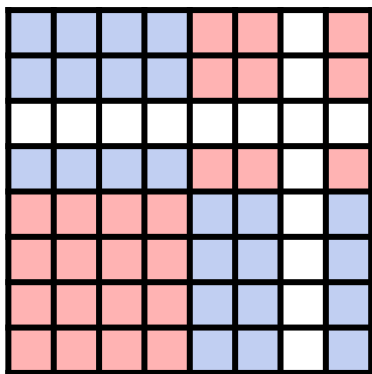
ビ太郎が残りの
マス进行塗る



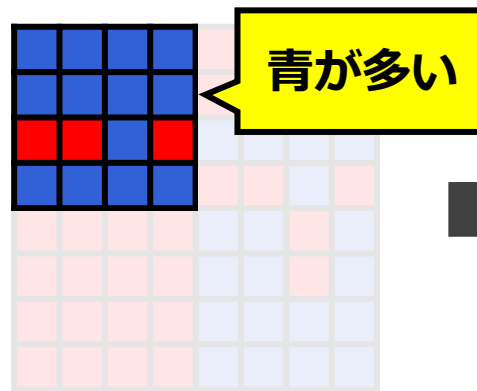
Brunoは区画ごとの
多数决を取る

例

文字列 ABBA を伝えたい場合



Annaは各区画に
色を塗る



ビ太郎が残りの
マスを塗る

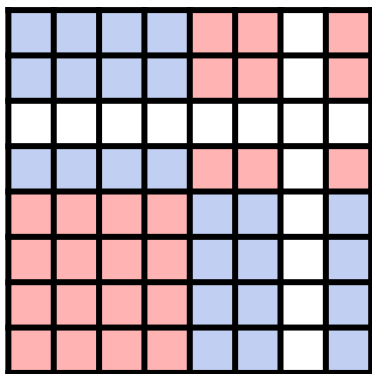


A	?
?	?

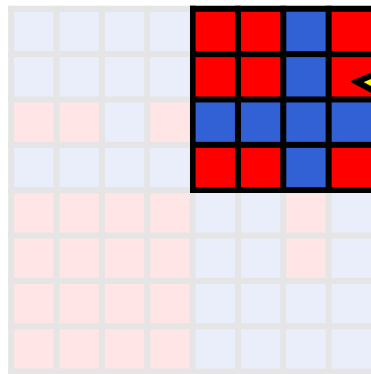
Brunoは区画ごとの
多数決を取る

例

文字列 ABBA を伝えたい場合



Annaは各区画に
色を塗る



ビ太郎が残りの
マスを塗る

赤が多い

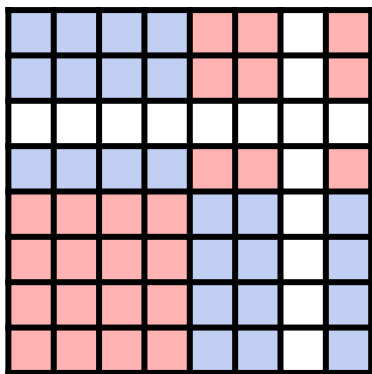


A	B
?	?

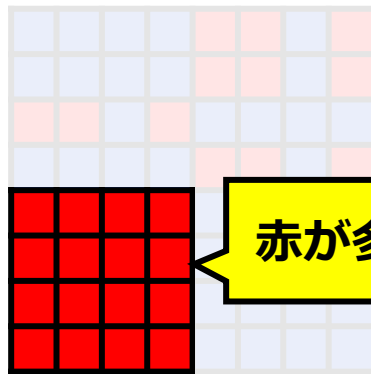
Brunoは区画ごとの
多数決を取る

例

文字列 ABBA を伝えたい場合



Annaは各区画に
色を塗る



ビ太郎が残りの
マス塗る

赤が多い

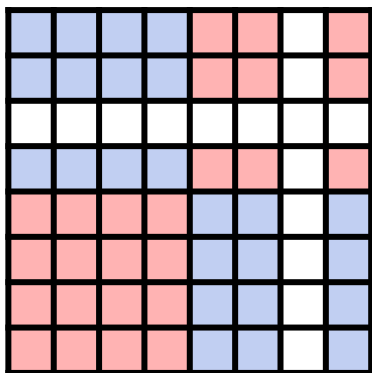


A	B
B	?

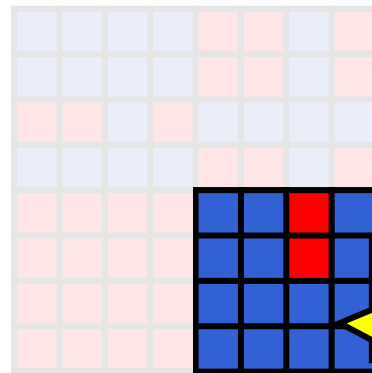
Brunoは区画ごとの
多数決を取る

例

文字列 ABBA を伝えたい場合



Annaは各区画に
色を塗る



ビ太郎が残りの
マスを塗る

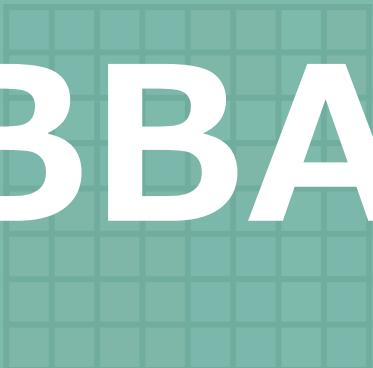


A	B
B	A

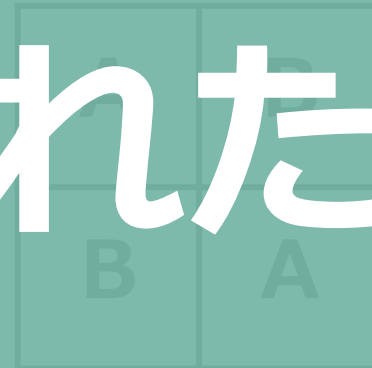
Brunoは区画ごとの
多数決を取る

例

文字列 ABBA を伝えたい場合



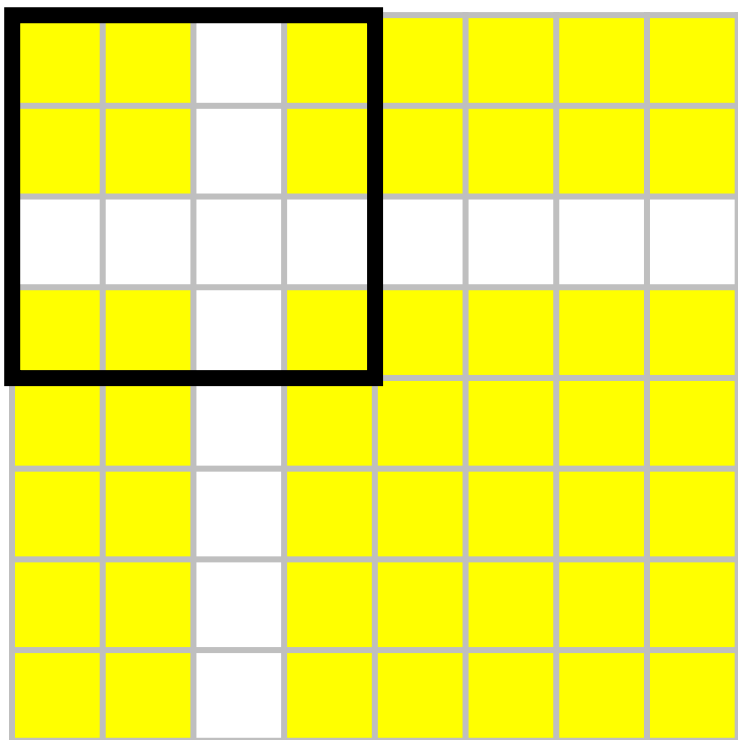
ABBA が当てられた！



Annaは各区画に
色を塗る

ビ太郎が残りの
マス进行塗る

Brunoは区画ごとの
多数決を取る



どんな場合でも、Anna が塗れる黄色マスは
各区画に (16 個中) 9 個以上存在する



Anna が過半数をコントロールできる !

この方法を少し工夫すると
 $N = 8$ でも解ける

1	1	1	1	2	2	2	2
1	1	1	1	2	2	2	2
1	1	1	1	2	2	2	2
1	1	1	1	2	2	2	2
3	3	3	3	4	4	4	4
3	3	3	3	4	4	4	4
3	3	3	3	4	4	4	4
3	3	3	3	4	4	4	4

これまでは、区画が正方形だったが...

1	8	7	6	5	4	3	2
2	1	8	7	6	5	4	3
3	2	1	8	7	6	5	4
4	3	2	1	8	7	6	5
5	4	3	2	1	8	7	6
6	5	4	3	2	1	8	7
7	6	5	4	3	2	1	8
8	7	6	5	4	3	2	1

これまでは、区画が正方形だったが…
斜めに分けるとどうなるか？



どの区画も (8 個中) 6 個以上 Anna が塗れる
過半数獲得！

この方法をさらに工夫すると
 $N = 12$ でも解ける

1	12	11	9	8	6	5	4
2	1	12	11	9	8	7	5
4	2	1	12	11	10	8	7
5	4	3	1	-	11	10	8
7	6	4	3	1	-	11	10
9	7	6	4	3	2	-	12
10	9	7	6	5	3	2	-
12	10	9	8	6	5	3	2

左図のようになると...

3/5 以上のマスに Anna が塗れるような
12 個の区画に分割できる！

1	12	11	9	8	6	5	4
2	1	12	11	9	8	7	5
4	2	1	12	11	8	7	
5	4	3	1	-	10		
7	6	4	3	-	11	10	
9	7	6	4	3	2	-	12
10	9	7	6	5	3	2	-
12	10	9	8	6	5	3	2

24

左図のようになると...

3/5 以上のマスに Anna が塗れるような
12 個の区画に分割できる！

点獲得！

Stage 3

アイデア : X, Y を読めるようにする

もし Bruno が x, y の値を見ることができたら
Anna が塗った場所もわかる



かなりの文字数を読み取れそう

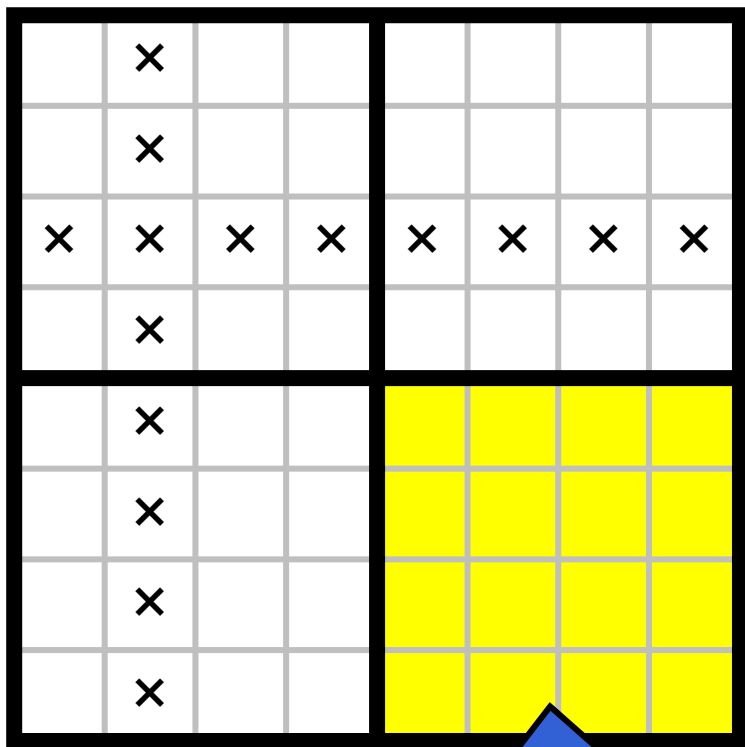
でも、Bruno は直接 X, Y の値を
見ることができない

一体どうすれば、 X, Y の値を
読み取ることができるのか？

3

 $N = 36$ の解法 (1/3)

61 / 105



全マス置ける

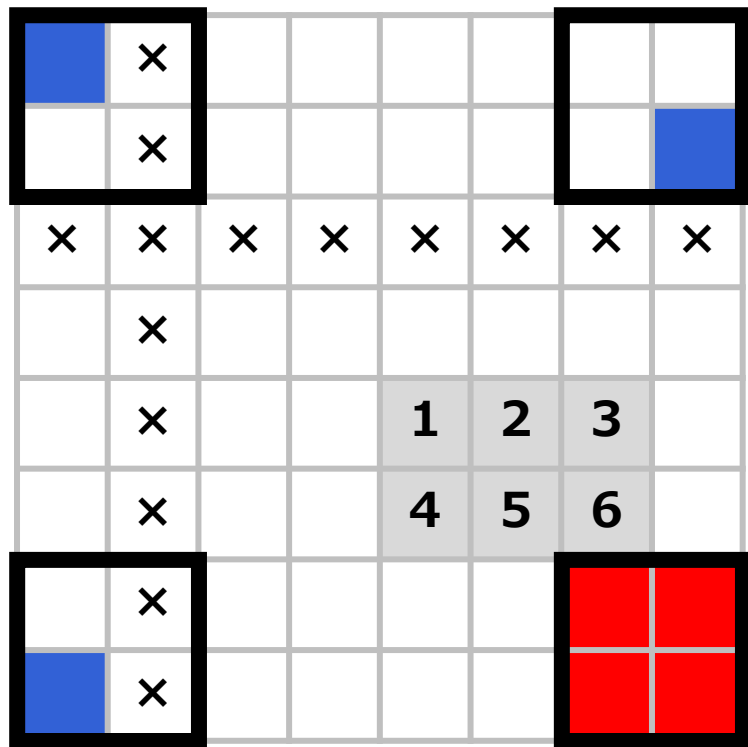
まず、 4×4 の区画の中のどれか 1 つは
全マスに Anna が置くことができる

※Anna が塗れないマス を × 印で示している

	×						
	×						
×	×	×	×	×	×	×	×
	×						
	×			1	2	3	
	×			4	5	6	
	×						
	×						

そこで、全マス置ける区画の適当な場所に
 X, Y の情報を記録することを考える

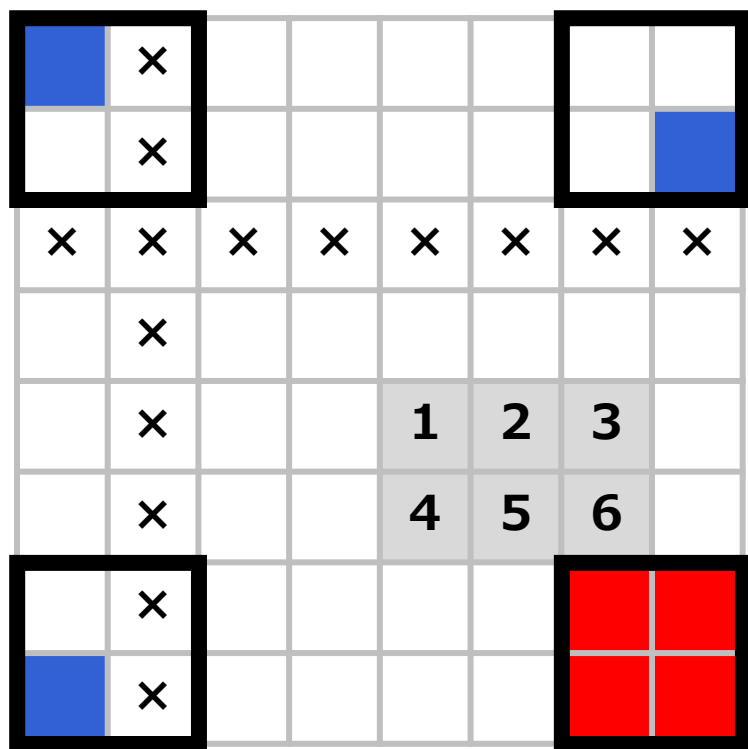
(64 通りあるので $\log_2 64 = 6$ マス必要)



さらに目印として、全マス置ける区画の 2×2 隅を赤で塗ることを考える

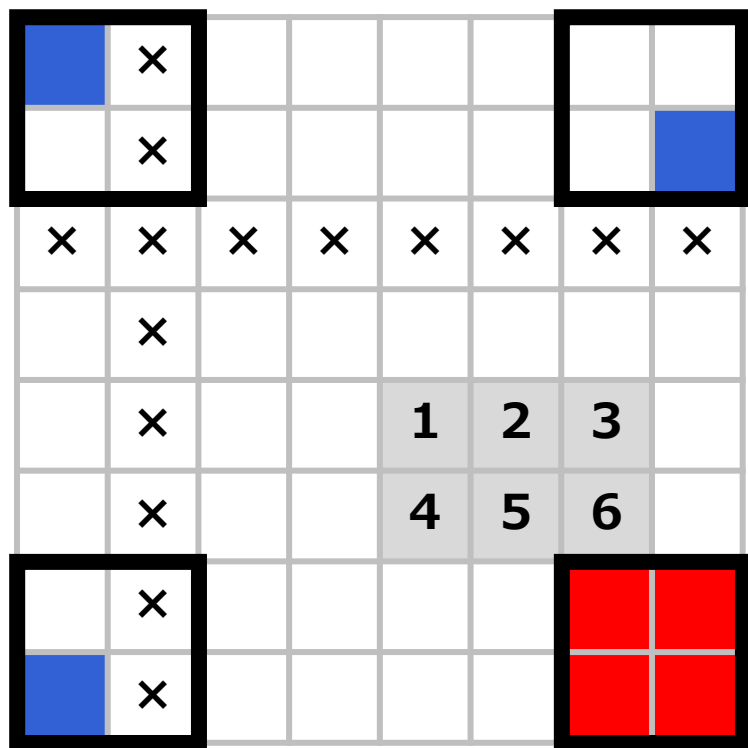
(他の区画の 2×2 隅はどれか 1 つを青で塗る)

このとき、Bruno は
 X, Y の値を読み取れるのか？



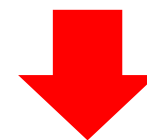
2×2 の隅のうち、全部赤になっているものは
必ず 1 つだけ存在する

(しかもこの周辺に x, y が書き込まれている)



2×2 の隅のうち、全部赤になっているものは
必ず 1 つだけ存在する

(しかもこの周辺に x, y が書き込まれている)



Bruno は隅を見れば良い！

X, Y を読み取った後は？

■	×	■	■	■	■	■	■
■	×	■	■	■	■	■	■
×	×	×	×	×	×	×	×
■	×	■	■	■	■	■	■
■	×	■	■	■	■	■	■
■	×	■	■	■	■	■	■
■	×	■	■	■	■	■	■
■	×	■	■	■	■	■	■

Anna が塗れるマスの中で
現時点で使われていないものは 36 個

	×	1	2	3	4	5	6
7	×	8	9	10	11	12	
×	×	×	×	×	×	×	×
13	×	14	15	16	17	18	19
20	×	21	22				23
24	×	25	26				27
28	×	29	30	31	32		
	×	33	34	35	36		

Anna が塗れるマスの中で
現時点で使われていないものは 36 個



左上から順に 1, 2, ..., 36 文字目を記録すれば、 $N = 36$ で解ける

具体例を考えてみよう

例：AAAAAAAAAAAAAAAAAAAAAAAABBBBBBBBBBBBBBBBBBBB を伝えたい

(ただし $X = 4, Y = 7$)

3 $N = 36$ の解法：具体例 (Anna)

71 / 105

							×
							×
							×
							×
×	×	×	×	×	×	×	×
							×
							×
							×

まず、左上の 4×4 隅が完全に空いている

3

 $N = 36$ の解法 : 具体例 (Anna)

72 / 105

							×
							×
	X	X	X				×
	Y	Y	Y				×
×	×	×	×	×	×	×	×
							×
							×
							×

まず、左上の 4×4 隅が完全に空いている



この中の適切な場所に

$X=4$ (赤青青)、 $Y=7$ (赤赤赤) を記録

※赤青青は 2 進法の 100 [=4]、赤赤赤は 2 進法の 111 [=7] に対応。

3 $N = 36$ の解法：具体例 (Anna)

73 / 105

★	★					★	×
★	★						×
							×
							×
×	×	×	×	×	×	×	×
							×
							×
★						★	×

まず、左上の 4×4 隅が完全に空いている



この中の適切な場所に
 $X=4$ (赤青青)、 $Y=7$ (赤赤赤) を記録

次に、左上の 2×2 隅を全部赤で塗り

それ以外の隅を 1 マス青で塗る

3 $N = 36$ の解法：具体例 (Anna)

74 / 105

		1	2	3	4		×
		5	6	7	8	9	×
10				11	12	13	×
14				15	16	17	×
×	×	×	×	×	×	×	×
18	19	20	21	22	23	24	×
25	26	27	28	29	30	31	×
	32	33	34	35	36		×

その後、残った塗れるマスに
左上から順に「文字に対応する色」で塗る
(A は青、B は赤)

※AA…ABB…B なので、18 文字目までが青、19 文字目からが赤

3 $N = 36$ の解法 : 具体例 (Anna)

75 / 105

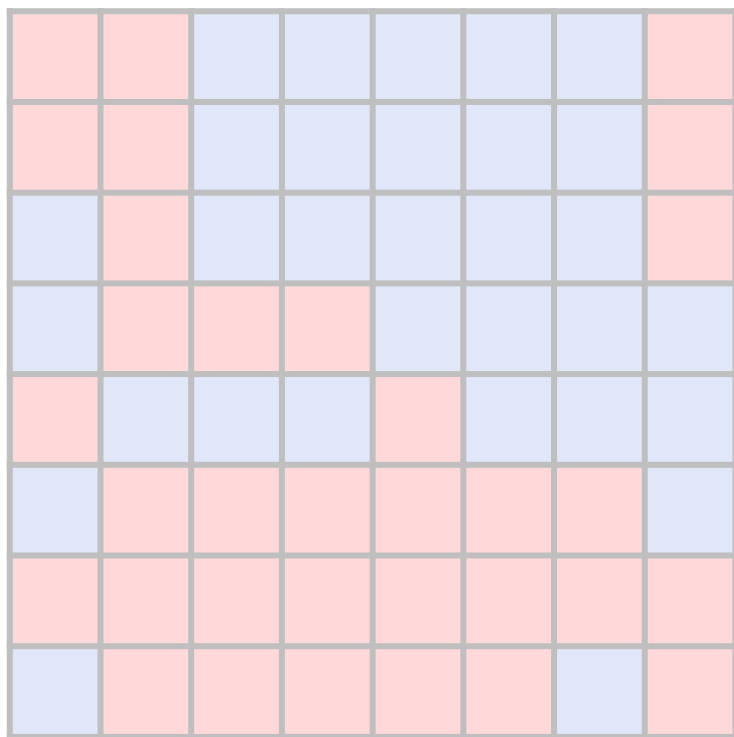
							×
							×
							×
							×
×	×	×	×	×	×	×	×
							×
							×
							×

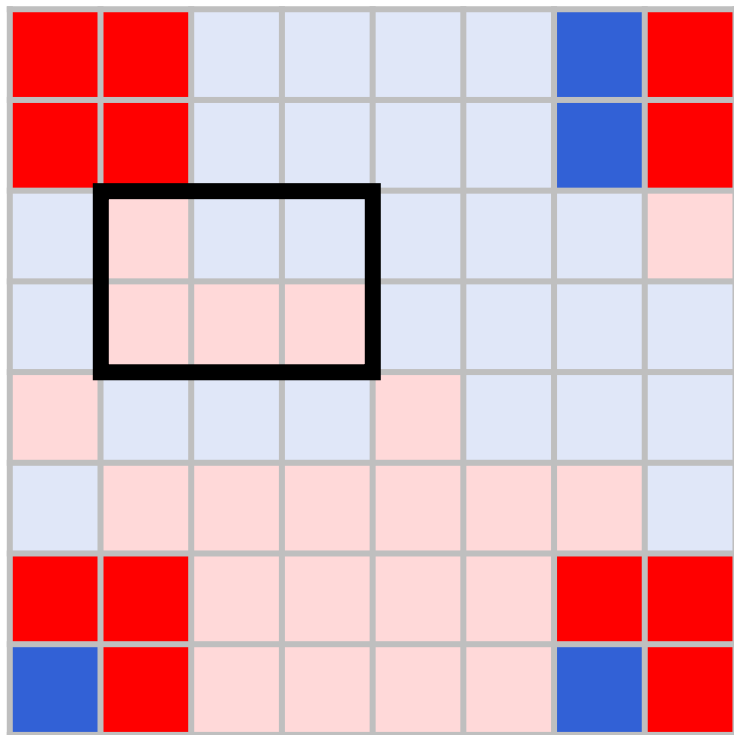
**このとき、Bruno は文字列を
復元できるか？**

3

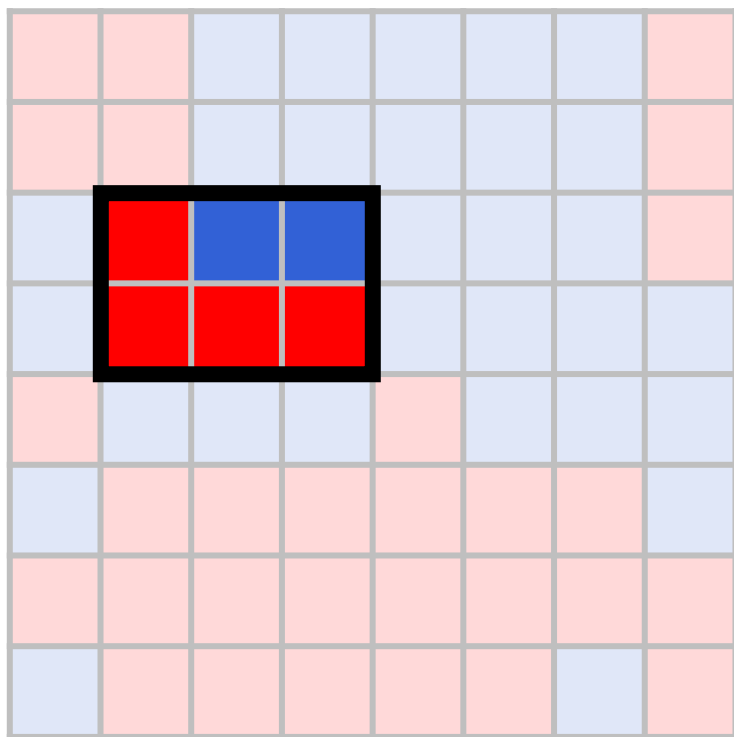
$N = 36$ の解法 : 具体例 (Bruno)

77 / 105





まず、 2×2 の隅の中で全部赤のものは左上だけ
→太枠の中に X, Y の情報が書かれている



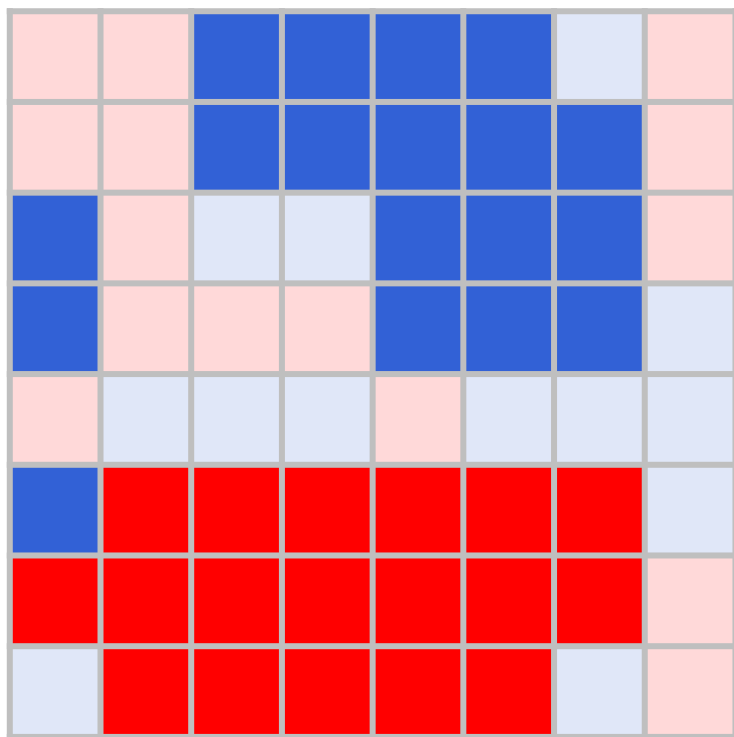
まず、 2×2 の隅の中で全部赤のものは左上だけ
→太枠の中に X, Y の情報が書かれている

情報を読むと、 $(X, Y) = (4, 7)$

3

 $N = 36$ の解法：具体例 (Bruno)

80 / 105



まず、 2×2 の隅の中で全部赤のものは左上だけ
→太枠の中に X, Y の情報が書かれている

情報を読むと、 $(X, Y) = (4, 7)$

そして (X, Y) が決まれば、文字列の情報がどこに
書かれているかがわかる

→答え「AAAA...ABBBB...B」が復元できる



36 文字を正しく

答えることができた！

まず、 2×2 の隅の中で全部赤のものは左上だけ

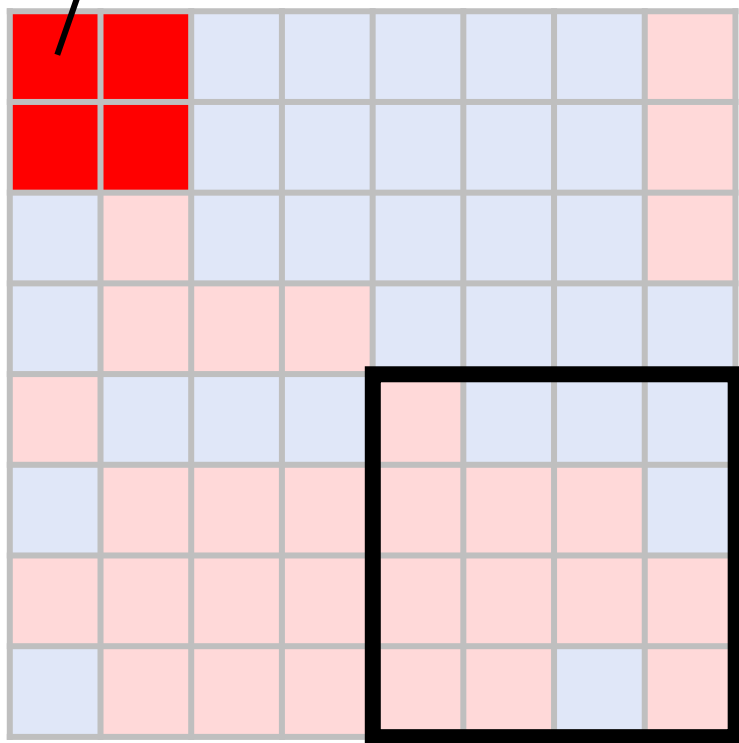
→大枠の中に X, Y の情報が書かれている

そして、 1 が残ったのは 1 文字列の情報がどこに
書かれているかがわかる

→答え「AAAA...ABBBB...B」が復元できる

この方法を少し工夫すると
 $N = 38$ でも解ける

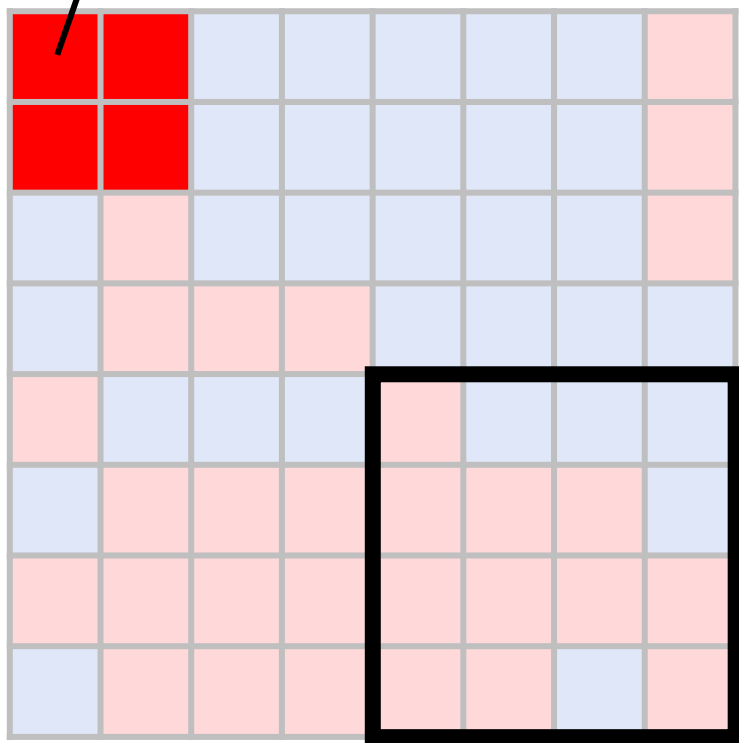
2×2 隅



元々は、 (X, Y) を記録するのに 6 マス使ったが...

2×2 隅が全部赤になっている部分がわかれば
 (X, Y) は太枠の 16 通りに絞れる

2×2 隅



元々は、 (X, Y) を記録するのに 6 マス使ったが...

2×2 隅が全部赤になっている部分がわかれば
 (X, Y) は太枠の 16 通りに絞れる



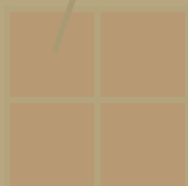
(X, Y) の記録が 4 マスで済み
2 文字分多く記録できる！

3

$N = 38$ の解法

85 / 105

2×2 隅



68

点獲得！

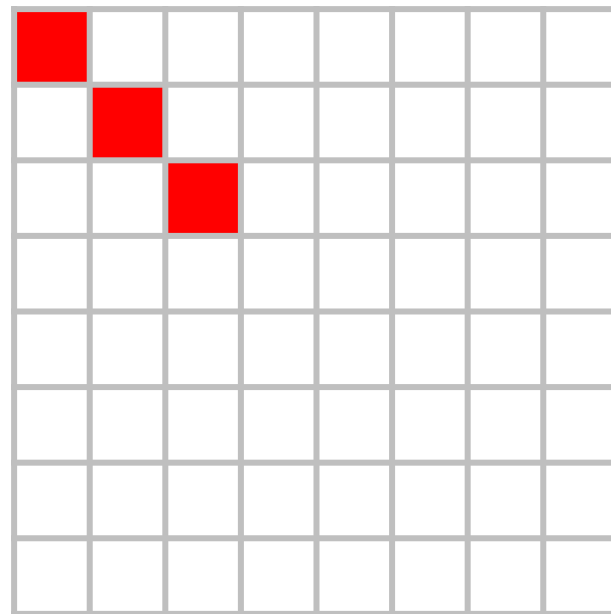
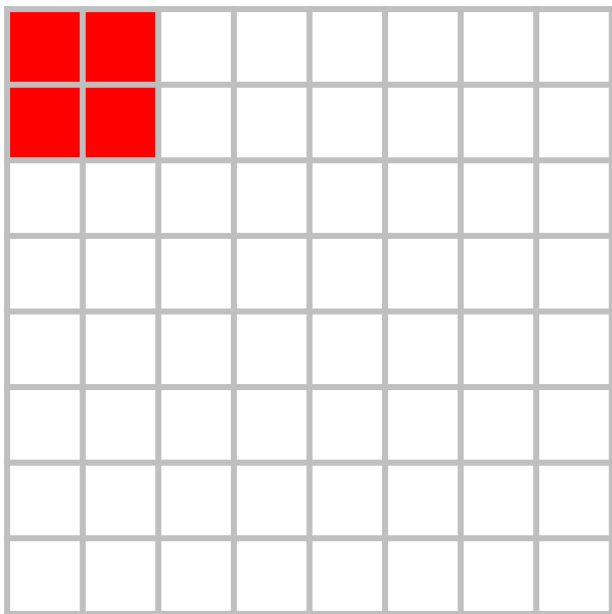
元々は、 (X, Y) を記録するのに 6 マス使ったが...

2×2 隅が全部赤になっている部分がわかれば

(X, Y) は太枠の 16 通りに絞れる

(X, Y) の記録が 4 マスで済み

2 文字分多く記録できる！

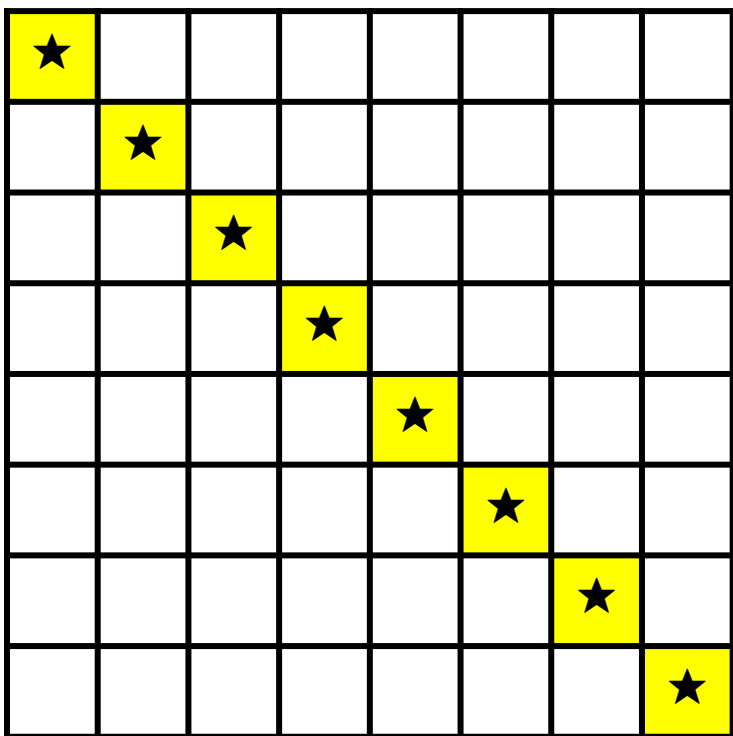


実は、識別は 2×2 隅ではなく斜め 3 つでも出来る

→ 1 マス減るので $N = 39$ が解ける (71 点)

Stage 4

アイデア：対角線を活用する



対角線の 8 マスだけで

(X, Y) の情報を読み取ることができる

×	×	×	×	×	×	×	×
×	★						
×		★					
×			★				
×				★			
×					★		
×						★	
×							★

42 マス使える

対角線の 8 マスだけで

 (X, Y) の情報を読み取ることを考える

もしこれが可能であれば...

- $X = Y$ のとき 42 文字分記録できる
- $X \neq Y$ のとき 43 文字分記録できる

★	×						
	×						
	×	★					
	×		★				
	×			★			
×	×	×	×	×	×	×	×
	×					★	
	×						★

43 マス使える

対角線の 8 マスだけで

(X, Y) の情報を読み取ることを考える

もしこれが可能であれば...

- $X = Y$ のとき 42 文字分記録できる
- $X \neq Y$ のとき 43 文字分記録できる

つまり、対角線だけで (X, Y) を読めれば $N = 42$ で解ける

でも、本当にできるのか？

×							
×	×	×	×	×	×	×	×
×		★					
×			★				
×				★			
×					★		
×						★	
×							★

$X \neq Y$ の場合、★の部分は決められるが

対角線の 2 マスはビ太郎が塗る

→対角線の色の並びとしては $2^2=4$ 通りがあり得る

×							
×	×	×	×	×	×	×	×
×		赤					
×			赤				
×				青			
×					青		
×						赤	
×							青

$X \neq Y$ の場合、★の部分は決められるが
対角線のうち 2 マスはビ太郎が塗る

→対角線の色の並びとしては $2^2=4$ 通りがあり得る

例：★印を「赤赤青青赤青」にする場合…

赤赤赤赤青青赤青、赤青赤赤青青赤青、

青赤赤赤青青赤青、青青赤赤青青赤青の 4 つ

★			×				
	★		×				
		★	×				
×	×	×	×	×	×	×	×
			×	★			
			×		★		
			×			★	
			×				★

$X = Y$ の場合、★の部分は決められるが

対角線のうち 1 マスはビ太郎が塗る

→対角線の色の並びとしては $2^1=2$ 通りがあり得る

つまり、対角線の色の並びとしては（重複含め）

- $X \neq Y$ の場合 : $56 \times 4 = 224$ 通り
- $X = Y$ の場合 : $8 \times 2 = 16$ 通り

240 通りが存在する！

そして、この並びは全部異ならなければならない※
つまり $2^8=256$ 通りの中に 240 通りを埋め込む必要がある

こんなこと出来るのか？

※もし対角線の色の並びに同じものがあった場合、 (X, Y) を正しく読み取ることができない

そして、この並びは全部異ならなければならない※

つまり $2^8=256$ 通りの中に 240 通りを埋め込む必要がある

焼きなまし法を使えば、構成できる

こんなこと出来るのか？

※もし対角線の色の並びに同じものがあった場合、 (X, Y) を正しく読み取ることができない

×	×	×	×	×	×	×	×
×	★						
×		★					
×			★				
×				★			
×					★		
×						★	
×							★

42 マス使える

$X = Y$ の場合、対角線以外で
Anna が書けるマスが 42 個しかない

このままでは 42 文字しか記録できないので
対角線に X, Y だけでなく「最後の 1 文字」も
記録することを考える

すると、対角線の色の並びの数は…

- $X \neq Y$ の場合 : $56 \times 4 = 224$ 通り
- $X = Y$ の場合 : $8 \times 2 \times 2 = 32$ 通り

の合計 256 通りになる !

そこで、 $2^8=256$ 通りの中に 256 通りを埋め込むことは可能か？

一見厳しそうに思えるが...

焼きなまし法を高速化すれば
構成できる！

※解説担当者の方針では、1億～10億回程度のループが必要になるので
数分以内に実行を終わらせるには、スコア計算を $O(1)$ 時間で終わらせる必要がある

4

$N = 43$ の解法

102 / 105

100点獲得！

解法のまとめ

Stage 1

赤の個数で文字列を識別

Stage 2

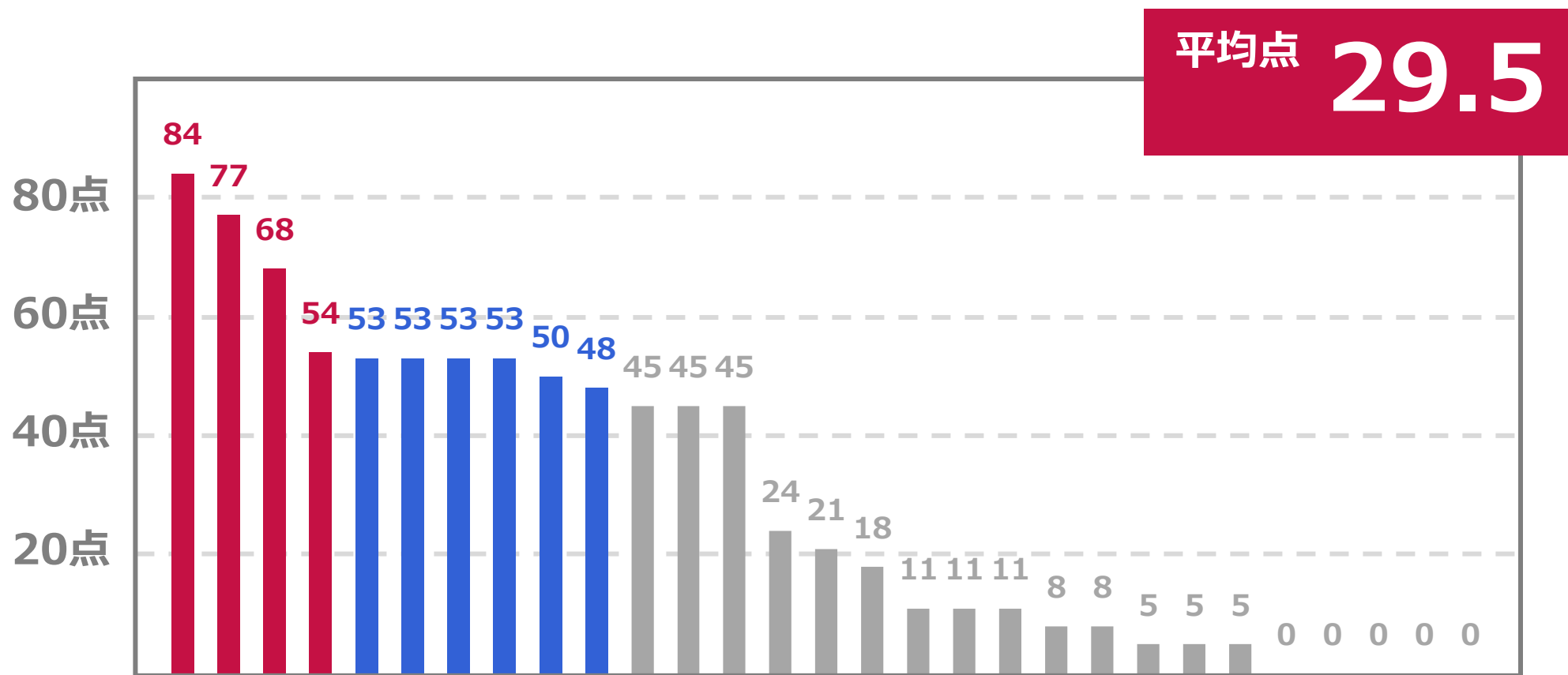
区画ごとに分けて多数決をとる

Stage 3

Bruno が x, y の値を読み取れるようにする

Stage 4

対角線に x, y を記録 + 焼きなまし法で埋め込み法を構成



警備員 (Security Guard)

解説 nxteru (戸高空)

はじめに

この解説の多くの部分で証明を飛ばしています
[チューターのnoshi91さんによるありがたい証明](#)

問題概要

- N 頂点のグラフ
- ある2頂点に停泊できそれらを行き来する M 隻の船

セキュリティ条件

頂点 v に停泊する船には常に $S[v]$ 人以上の警備員がいる

問題概要

小問題

- 最初、いくつかの船を廃止、残っている船をどちらかに停泊させ好きな人数の警備員を乗せる
- 次の条件を満たすために必要な警備員の最小人数を求めよ

問題概要

条件

どの頂点 u, v についてもセキュリティ条件を満たしながら次の行動を繰り返して客を u から v まで輸送できる

- 停泊する船と停泊する頂点の間で警備員または客が乗り降りする
- ある頂点に停泊する船がもう一方に移動し停泊

問題概要

小問題を次の $k=0,1,2,\dots,Q$ の場合について解け

- 初めに船を新たに k 隻増やすことができる
- ただしそれらの船が行き来する2つの頂点は自由に選ぶことができる

制約

- $N \leq 200000$
- $M \leq 400000$
- $Q \leq 200000$
- $1 \leq S[v] \leq 10^9 \quad (1 \leq v \leq N)$

小課題

1. (12点) $Q=0$, パス, $S[v] \leq 2$
2. (13点) $Q=0$, パス
3. (12点) $Q=0$, 木
4. (13点) $Q=0$
5. (8点) $N \leq 16$
6. (18点) $N \leq 3000$
7. (24点) 追加の制約はない

まずは $Q=0$ (小問題)までを解く

1. (12点) $Q=0$, パス, $S[v] \leq 2$
2. (13点) $Q=0$, パス
3. (12点) $Q=0$, 木
4. (13点) $Q=0$

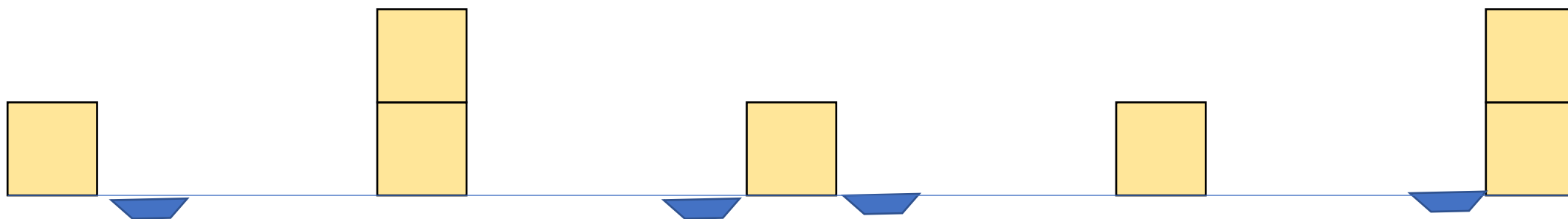
まずは $Q=0$ (小問題)までを解く

小課題3の木を解くまでの方針を1つ紹介
 $s[v]$ の最小値で全部埋めて消すを繰り返す

$s[v]$ の最小値で全部埋めて消すを繰り返す

小課題1. $Q=0$, パス, $S[v] \leq 2$

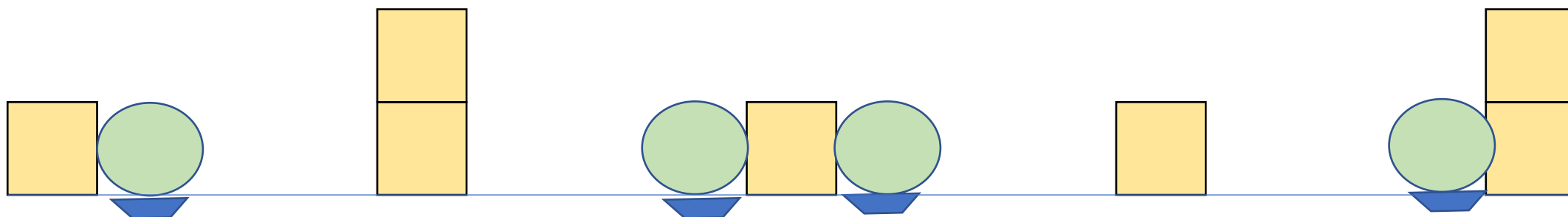
- $S[v]$ は1か2



$s[v]$ の最小値で全部埋めて消すを繰り返す

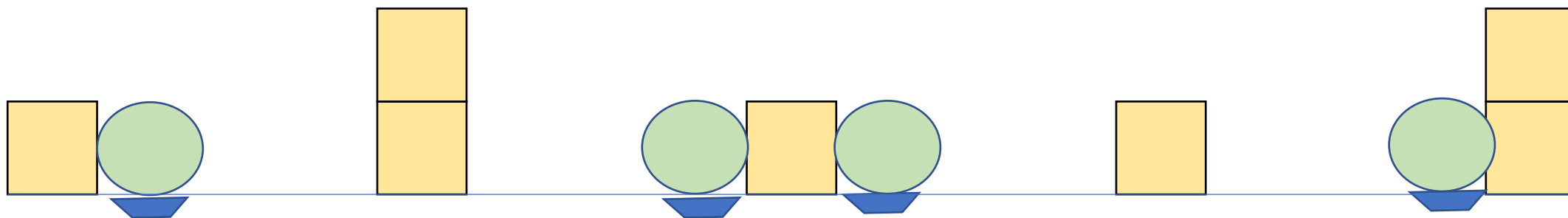
小課題1. $Q=0$, パス, $S[v] \leq 2$

- $S[v]$ は1か2
- とりあえずすべての船に1人は必ず必要



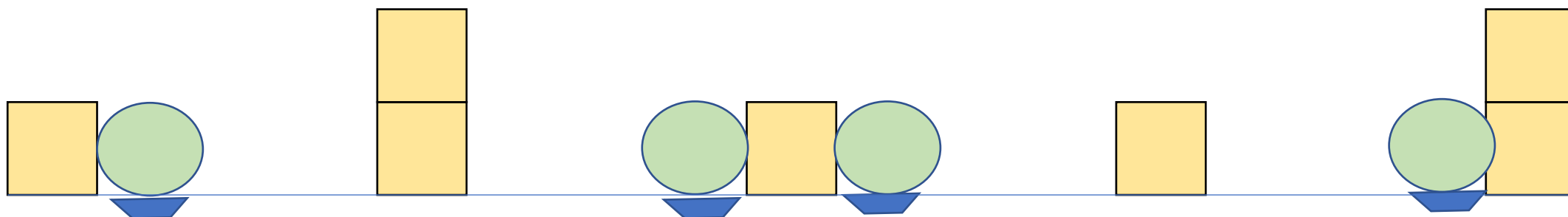
$s[v]$ の最小値で全部埋めて消すを繰り返す

- このときその1人はずっとその船に常駐していると
してよい



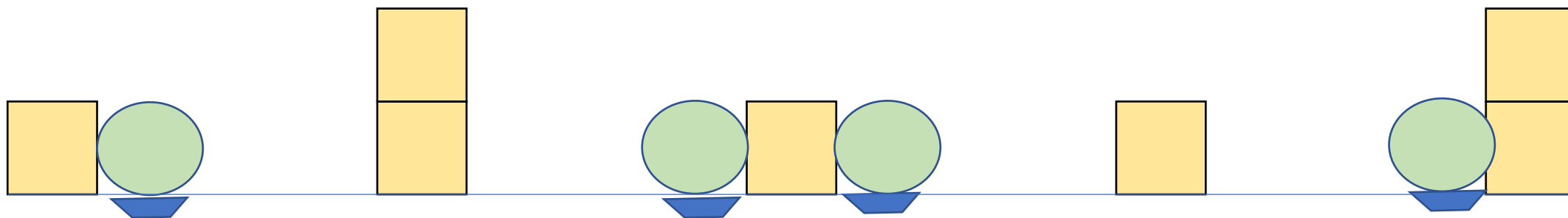
$s[v]$ の最小値で全部埋めて消すを繰り返す

- このときその1人はずっとその船に常駐していると
してよい
- すべての船について1人は必ずいる状態



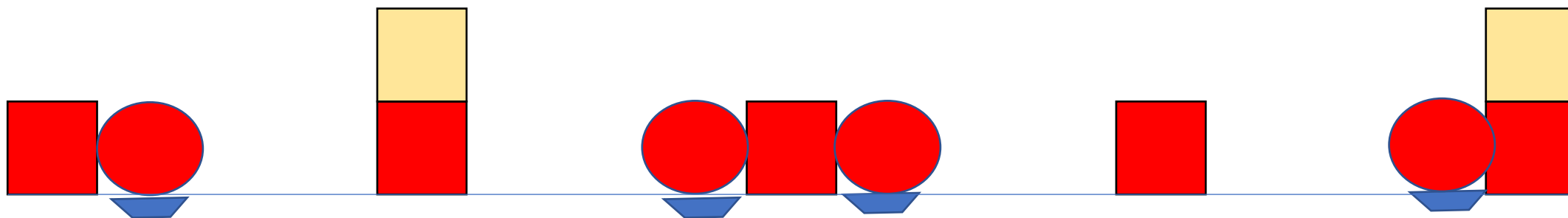
$s[v]$ の最小値で全部埋めて消すを繰り返す

- 全体の $S[v]$ が1下がったと考えることができる



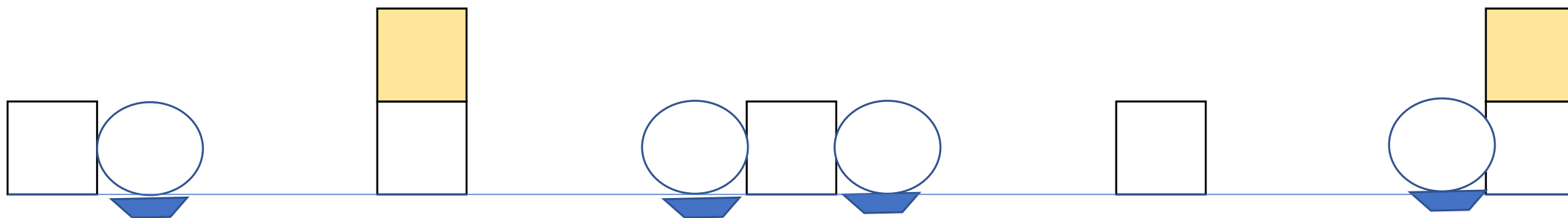
$s[v]$ の最小値で全部埋めて消すを繰り返す

- 全体の $S[v]$ が1下がったと考えることができる



$s[v]$ の最小値で全部埋めて消すを繰り返す

- 全体の $S[v]$ が1下がったと考えることができる



$s[v]$ の最小値で全部埋めて消すを繰り返す

- 全体の $S[v]$ が1下がったと考えることができる



$s[v]$ の最小値で全部埋めて消すを繰り返す

- $S[v]$ が0になったところはその上を警備員が自由に移動できるので、その間の船は合わせて1つとみなす



$s[v]$ の最小値で全部埋めて消すを繰り返す

- $S[v]$ が0になったところはその上を警備員が自由に移動できるので、その間の船は合わせて1つとみなす



$s[v]$ の最小値で全部埋めて消すを繰り返す

- これをあと1回すればよい



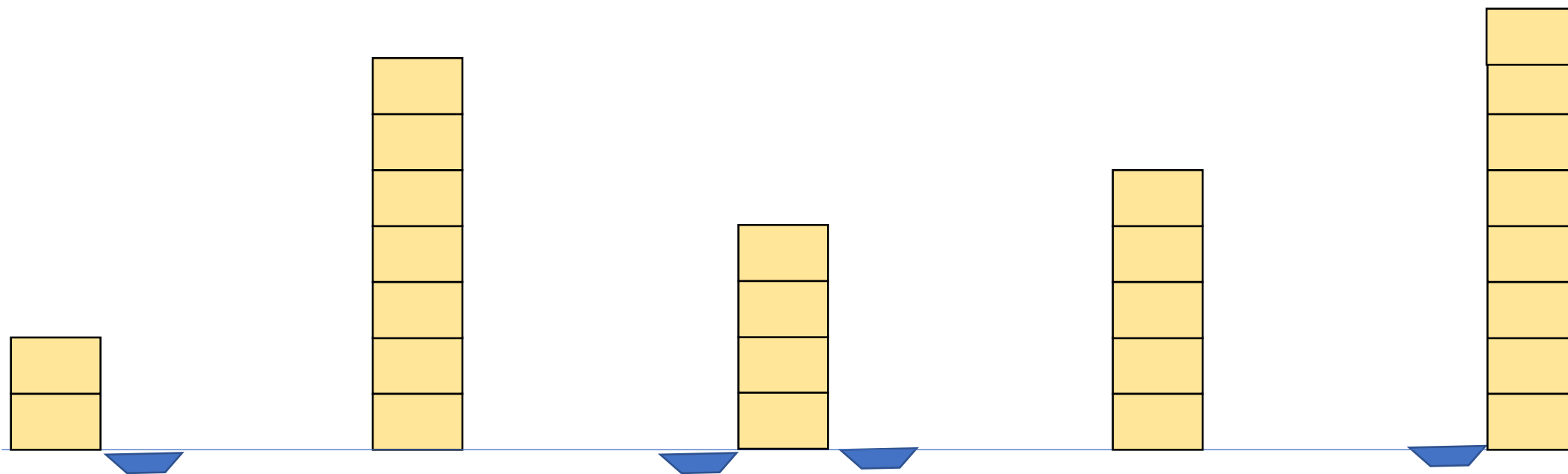
$s[v]$ の最小値で全部埋めて消すを繰り返す

- これで求められる人数以上は必ず必要
- またこれで求められた人数がいれば自由に輸送できる



$s[v]$ の最小値で全部埋めて消すを繰り返す

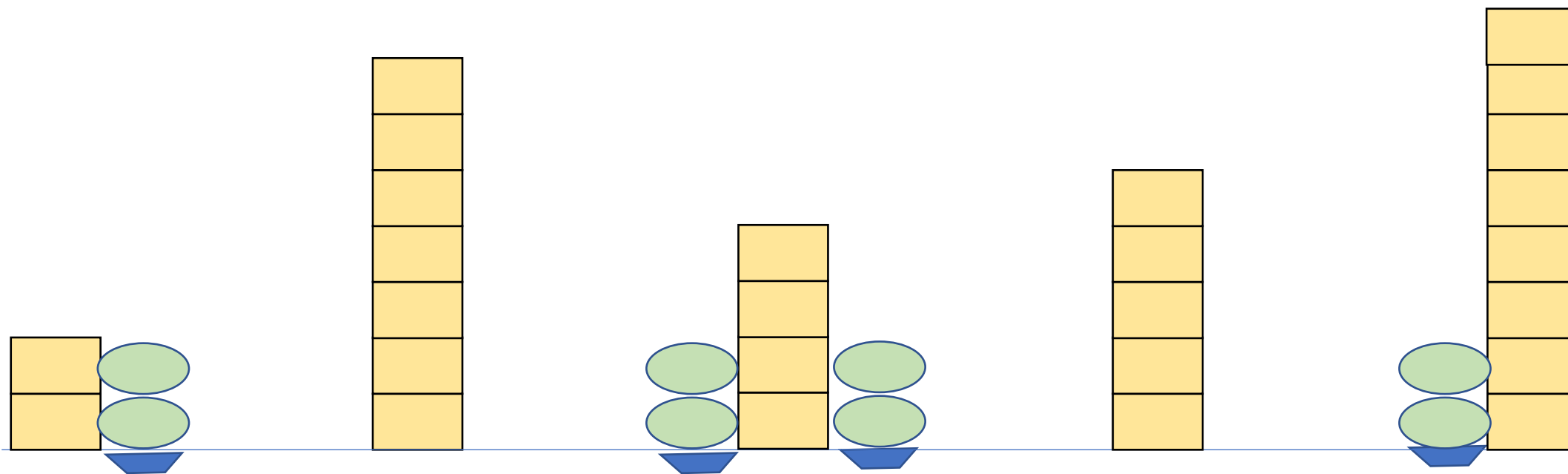
小課題2. $Q=0$, パス



$s[v]$ の最小値で全部埋めて消すを繰り返す

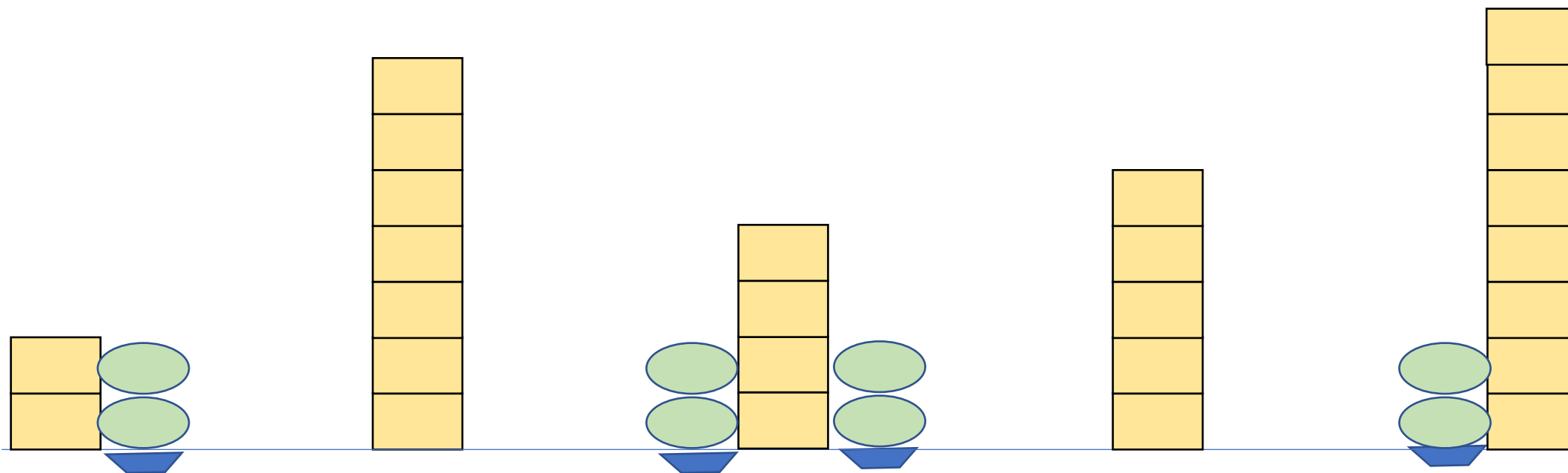
小課題2. $Q=0$, パス

- とりあえずすべての船に S の最小値の人数は必要



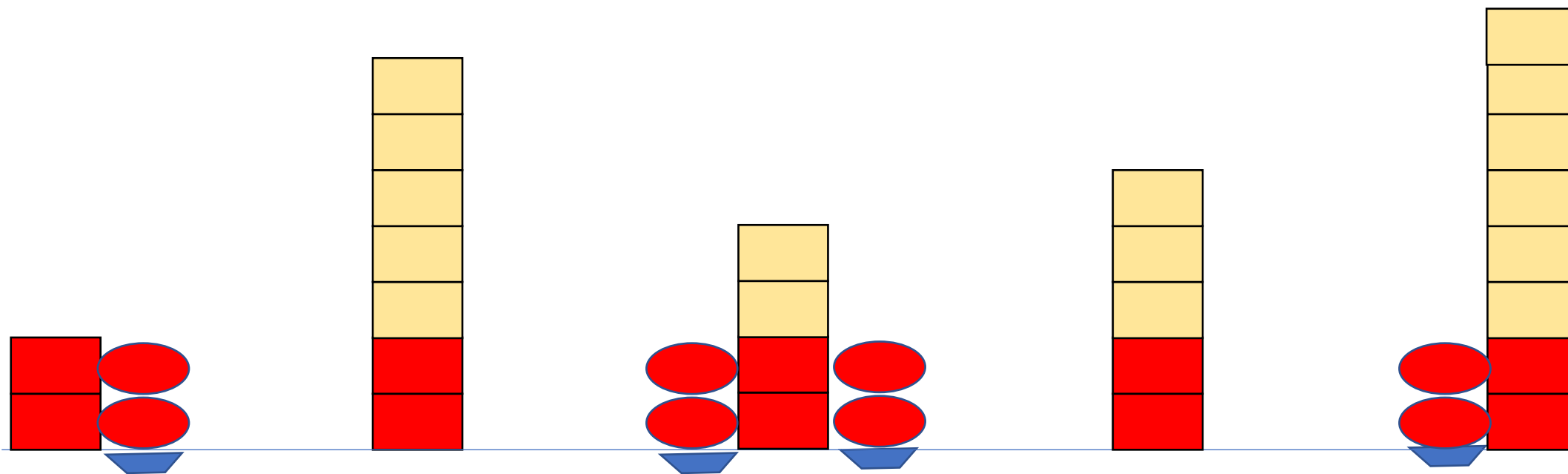
$s[v]$ の最小値で全部埋めて消すを繰り返す

- その人はずっとその船に常駐しているとしてよい
- すべての船について最小値人は必ずいる状態



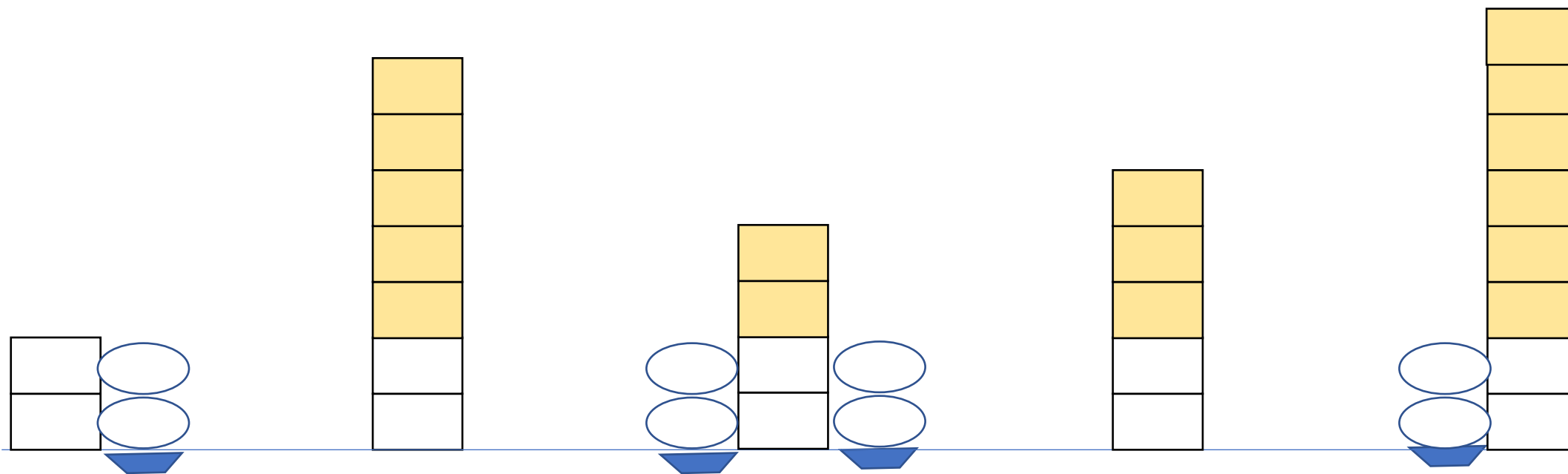
$s[v]$ の最小値で全部埋めて消すを繰り返す

- 全体のSが下がったと考える



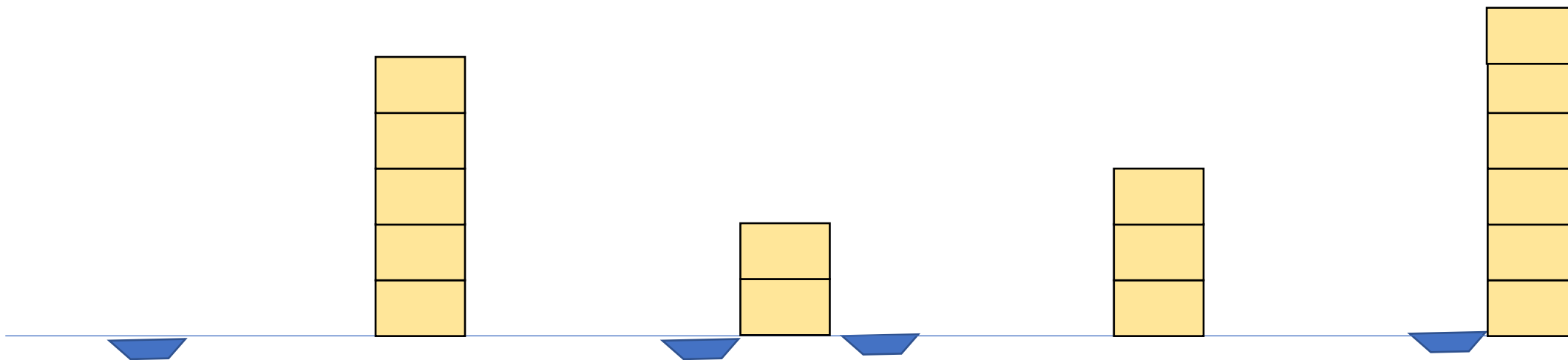
$s[v]$ の最小値で全部埋めて消すを繰り返す

- 全体のSが下がったと考える



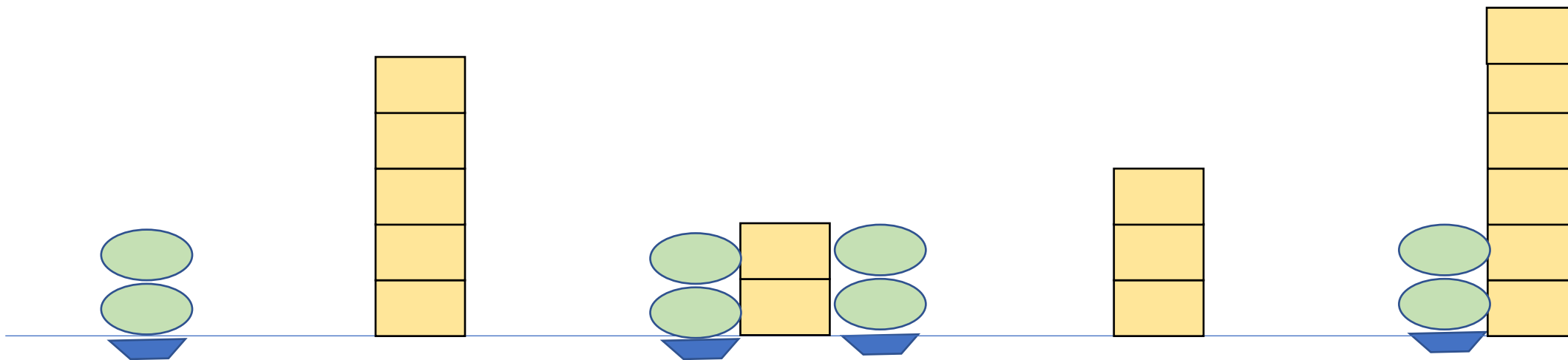
$s[v]$ の最小値で全部埋めて消すを繰り返す

- 全体のSが下がったと考える



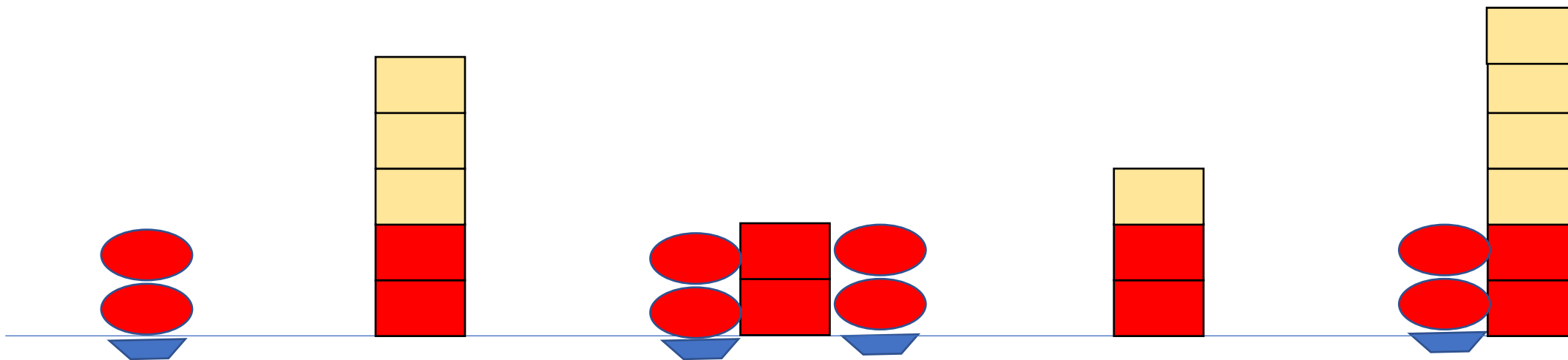
$s[v]$ の最小値で全部埋めて消すを繰り返す

- これをくりかえす



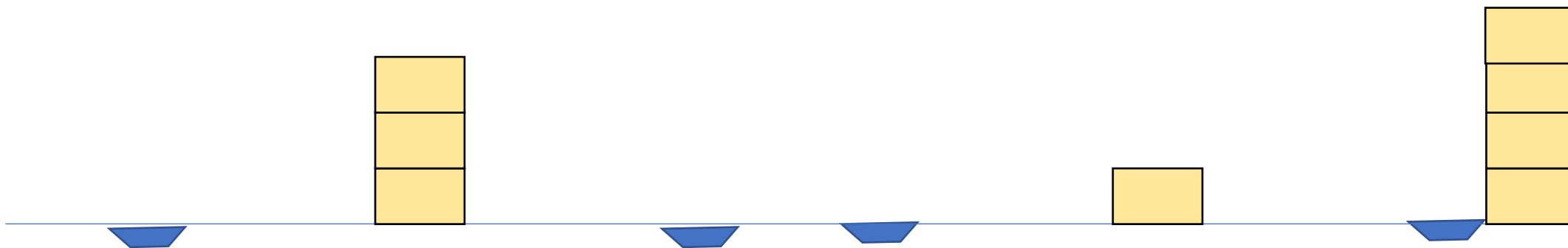
$s[v]$ の最小値で全部埋めて消すを繰り返す

- これをくりかえす



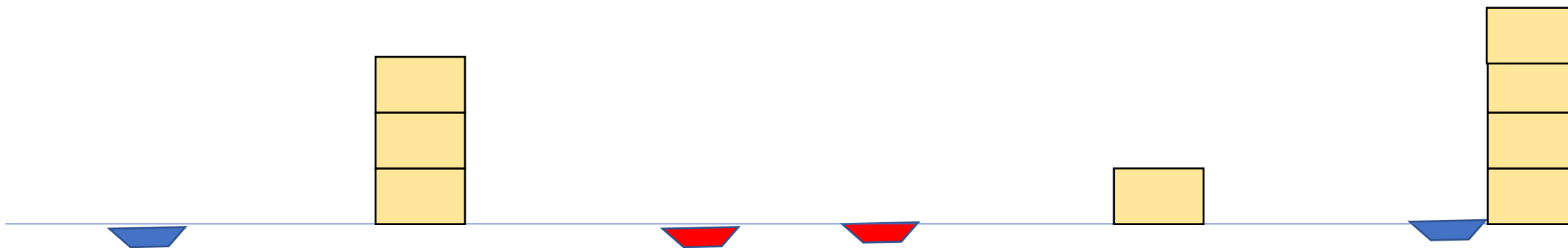
$s[v]$ の最小値で全部埋めて消すを繰り返す

- これをくりかえす



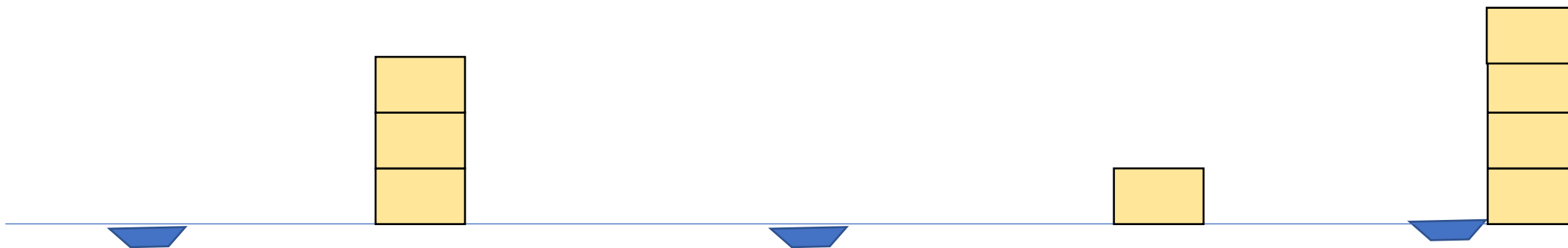
$s[v]$ の最小値で全部埋めて消すを繰り返す

- これをくりかえす
- 頂点がなくなったら船を合わせる



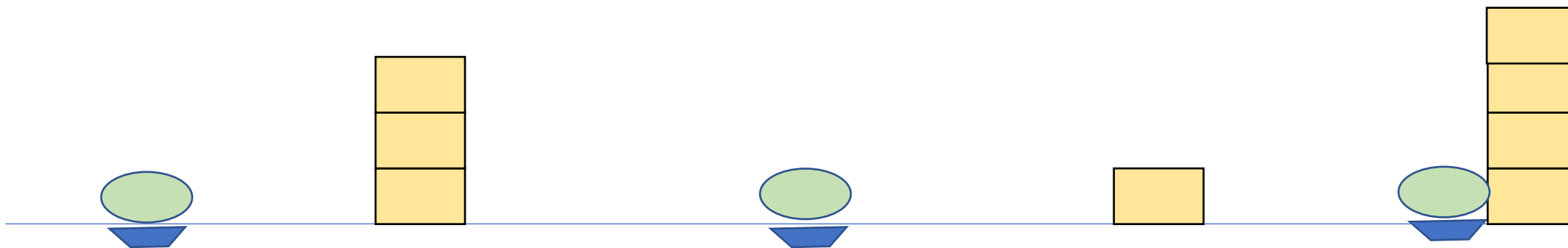
$s[v]$ の最小値で全部埋めて消すを繰り返す

- これをくりかえす
- 頂点がなくなったら船を合わせる



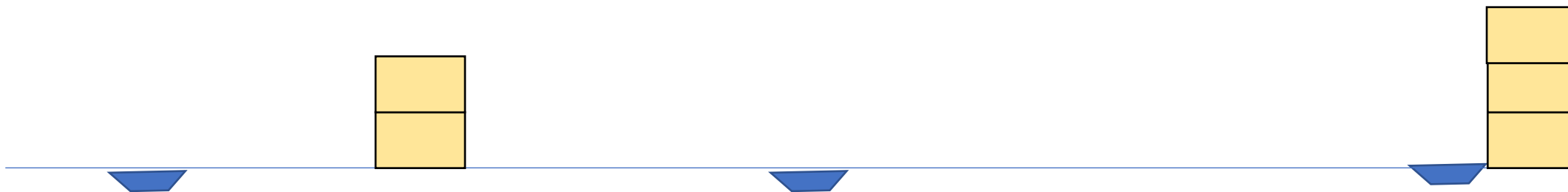
$s[v]$ の最小値で全部埋めて消すを繰り返す

- これをくりかえす
- 頂点がなくなったら船を合わせる



$s[v]$ の最小値で全部埋めて消すを繰り返す

- これをくりかえす
- 頂点がなくなったら船を合わせる



$s[v]$ の最小値で全部埋めて消すを繰り返す

- これをくりかえす
- 頂点がなくなったら船を合わせる



$s[v]$ の最小値で全部埋めて消すを繰り返す

- これをくりかえす
- 頂点がなくなったら船を合わせる



$s[v]$ の最小値で全部埋めて消すを繰り返す

- これをくりかえす
- 頂点がなくなったら船を合わせる



$s[v]$ の最小値で全部埋めて消すを繰り返す

- これをくりかえす
- 頂点がなくなったら船を合わせる



$s[v]$ の最小値で全部埋めて消すを繰り返す

- ここまでに足した警備員の数の合計を答えればよい



$s[v]$ の最小値で全部埋めて消すを繰り返す

- シミュレーションしてもよいが整理するとこの人数は次のように定式化できる

$$S[2] + S[3] + \cdots + S[N-1] + \max(S[1], S[2], \cdots, S[N])$$

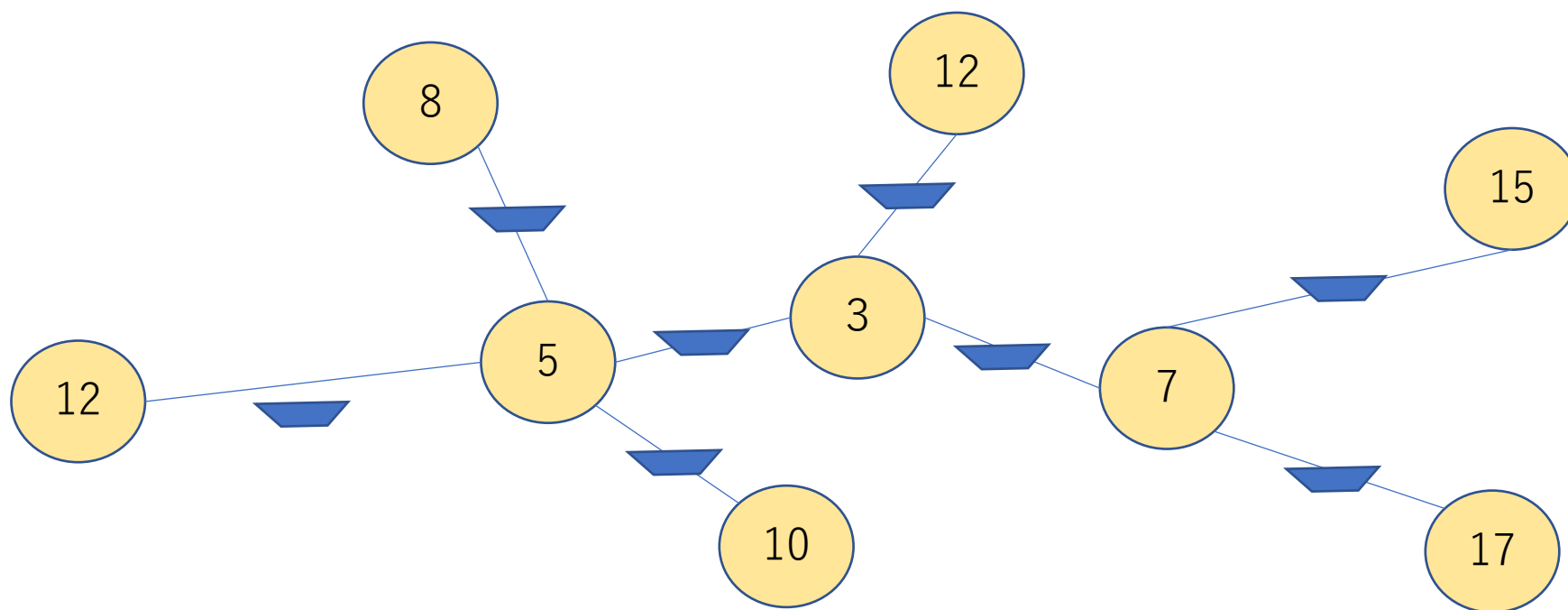
$s[v]$ の最小値で全部埋めて消すを繰り返す

小課題3. $Q=0$, 木

- この考え方を木に拡張する

船の数 : 8

合計 : 0



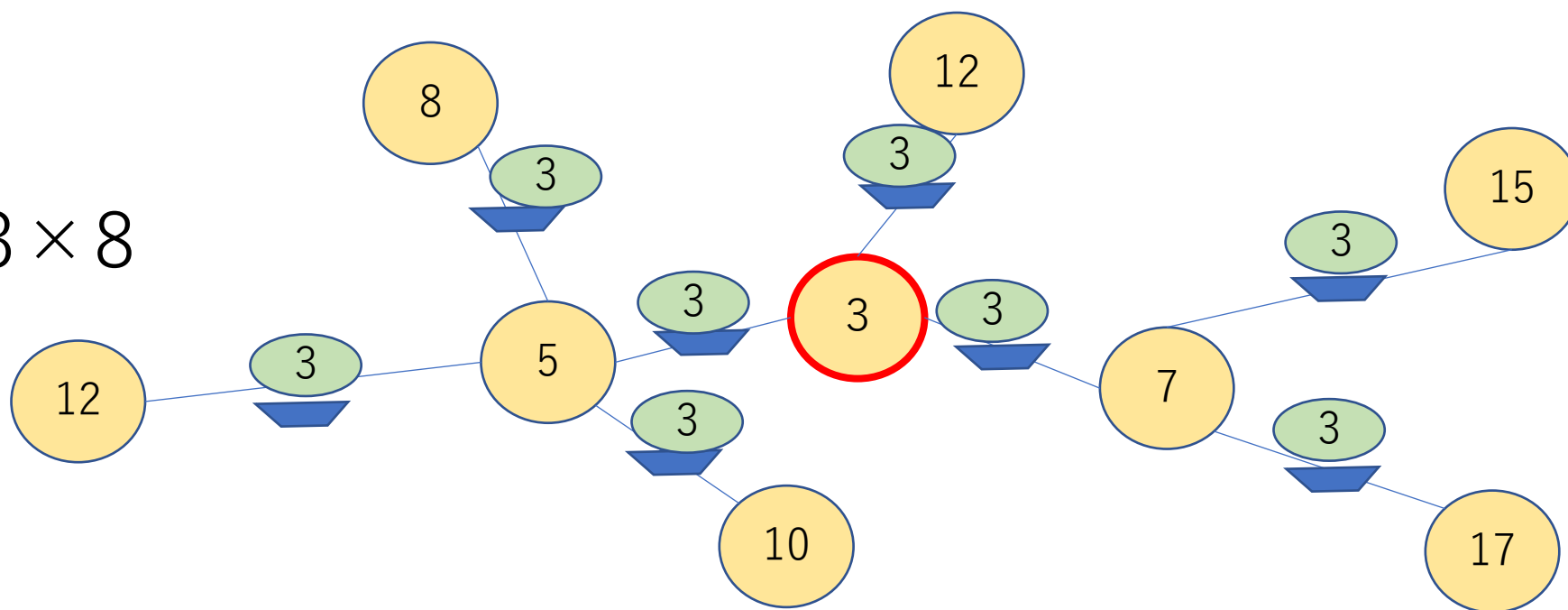
$s[v]$ の最小値で全部埋めて消すを繰り返す

小課題3. $Q=0$, 木

- この考え方を木に拡張する

船の数 : 8

合計 : $0 + 3 \times 8$



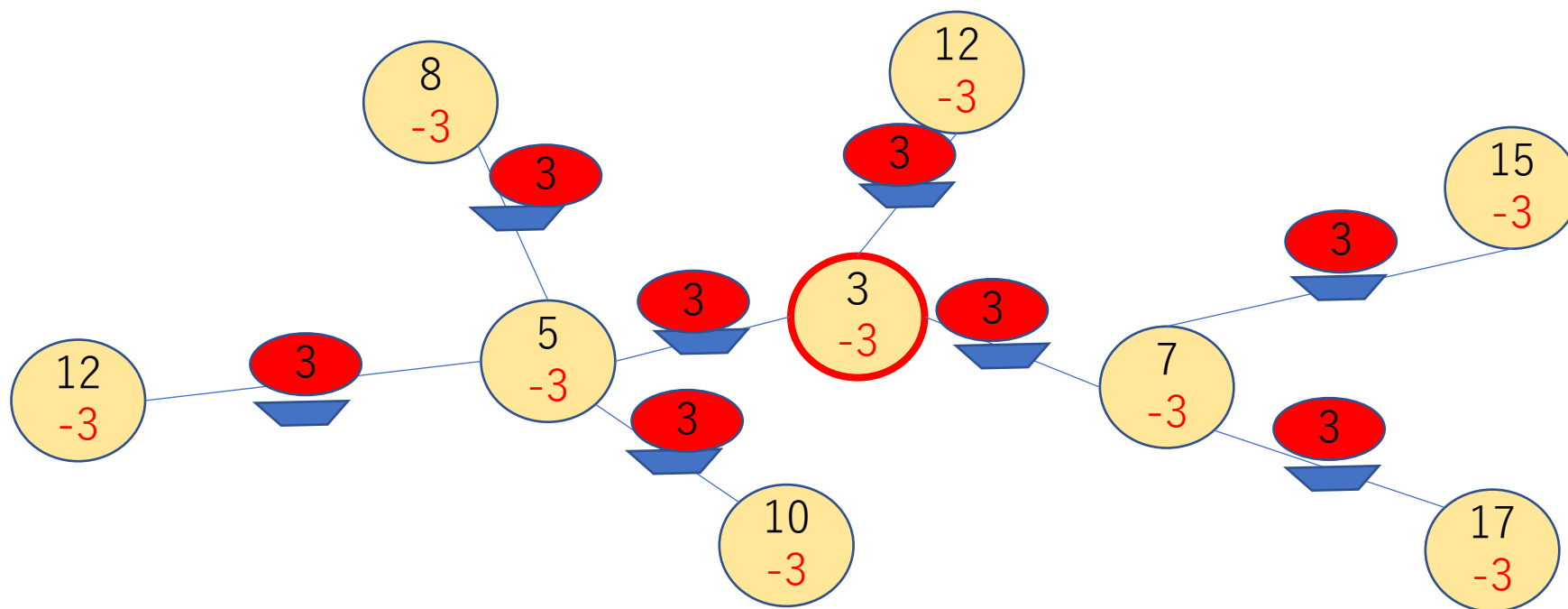
s[v]の最小値で全部埋めて消すを繰り返す

小課題3. $Q=0$, 木

- この考え方を木に拡張する

船の数：8

合計：24



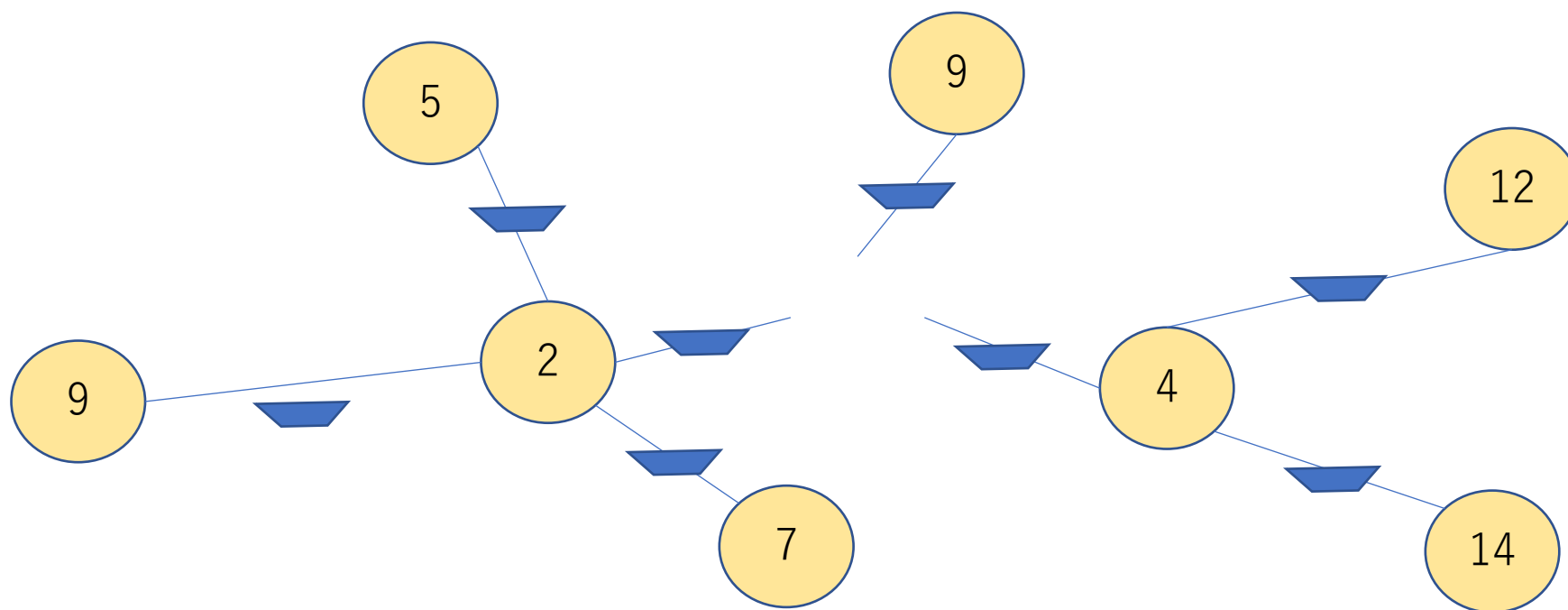
$s[v]$ の最小値で全部埋めて消すを繰り返す

小課題3. $Q=0$, 木

- この考え方を木に拡張する

船の数：8

合計：24



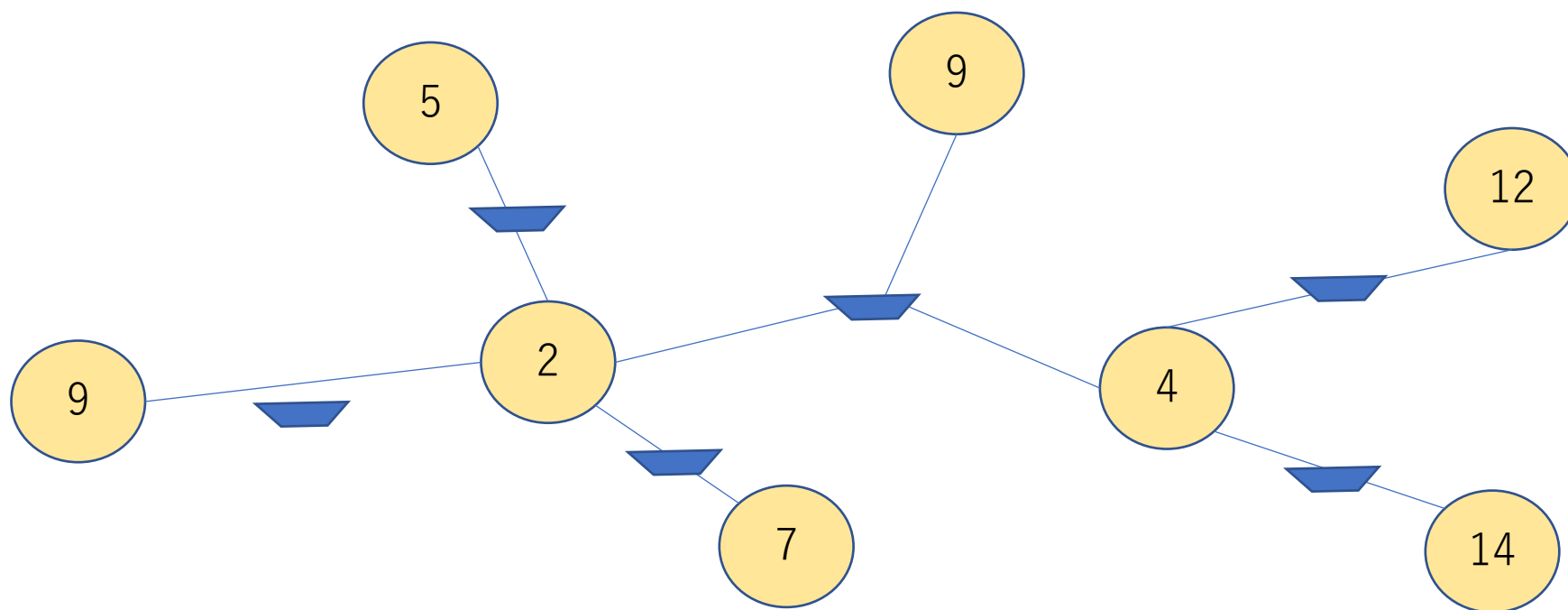
$s[v]$ の最小値で全部埋めて消すを繰り返す

小課題3. $Q=0$, 木

- この考え方を木に拡張する

船の数：6

合計：24



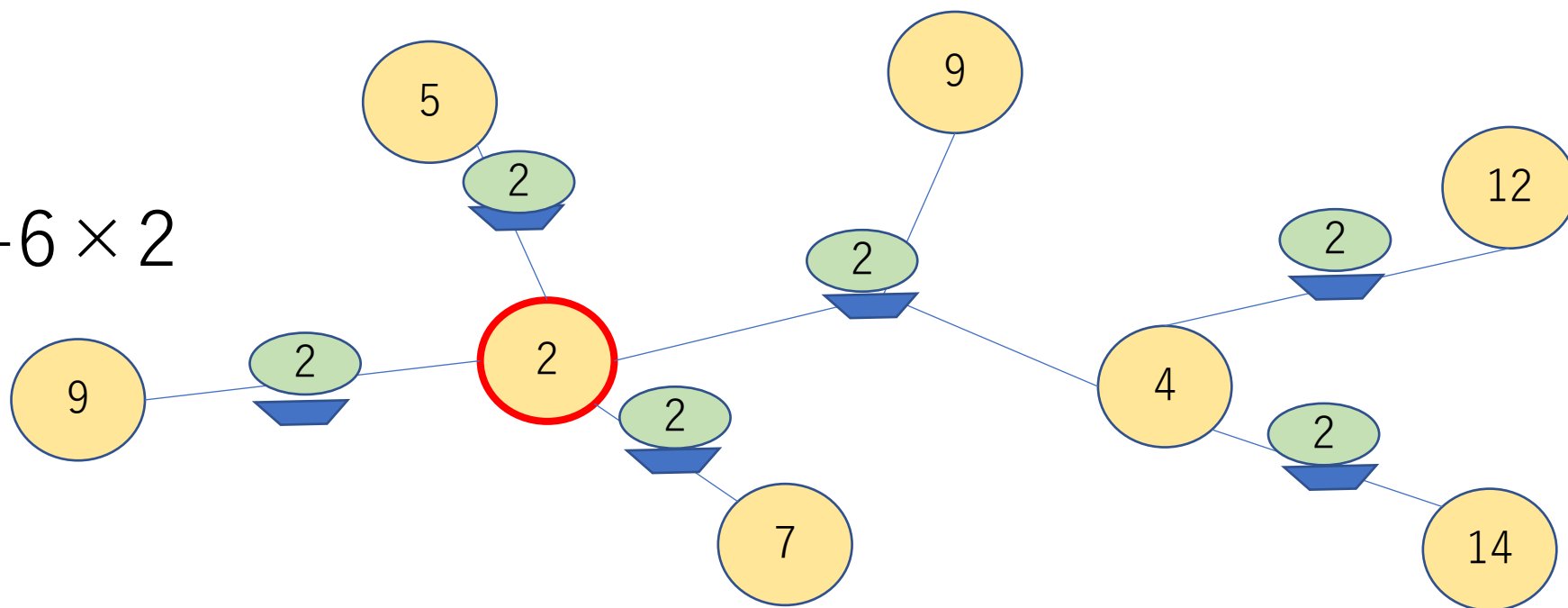
$s[v]$ の最小値で全部埋めて消すを繰り返す

小課題3. $Q=0$, 木

- この考え方を木に拡張する

船の数：6

合計： $24 + 6 \times 2$



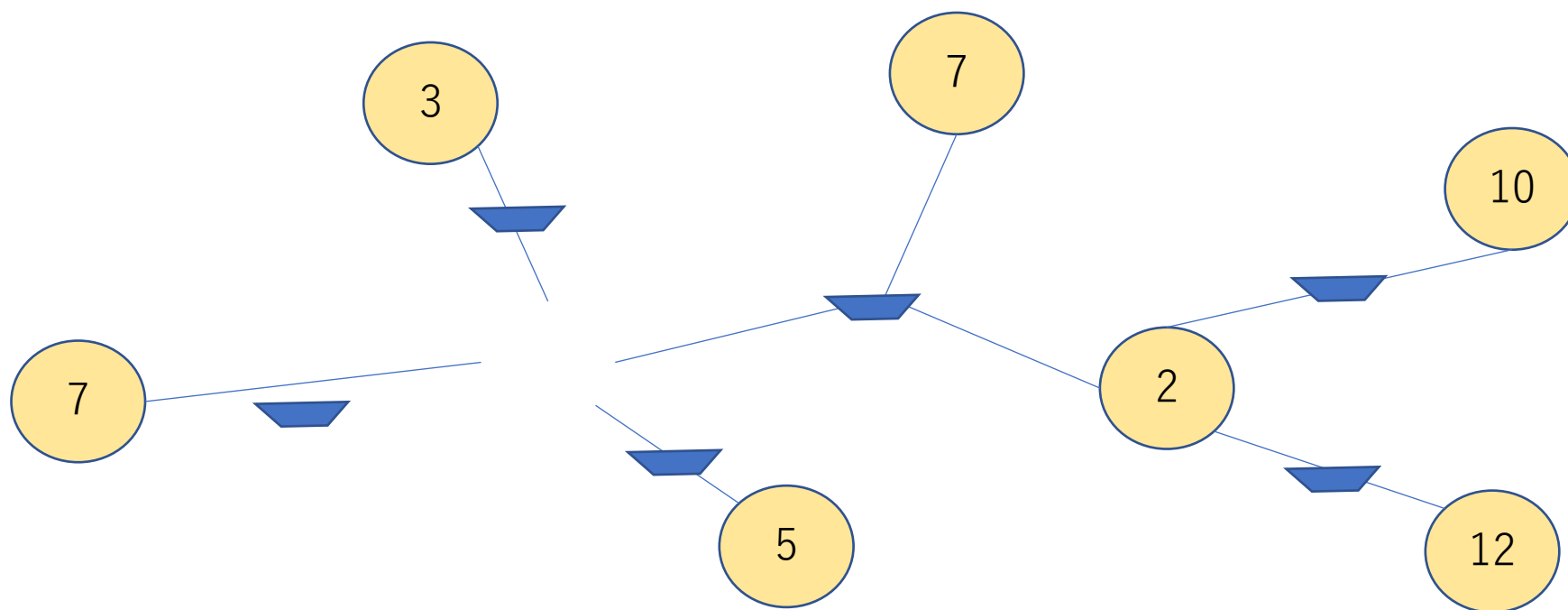
$s[v]$ の最小値で全部埋めて消すを繰り返す

小課題3. $Q=0$, 木

- この考え方を木に拡張する

船の数：6

合計：36



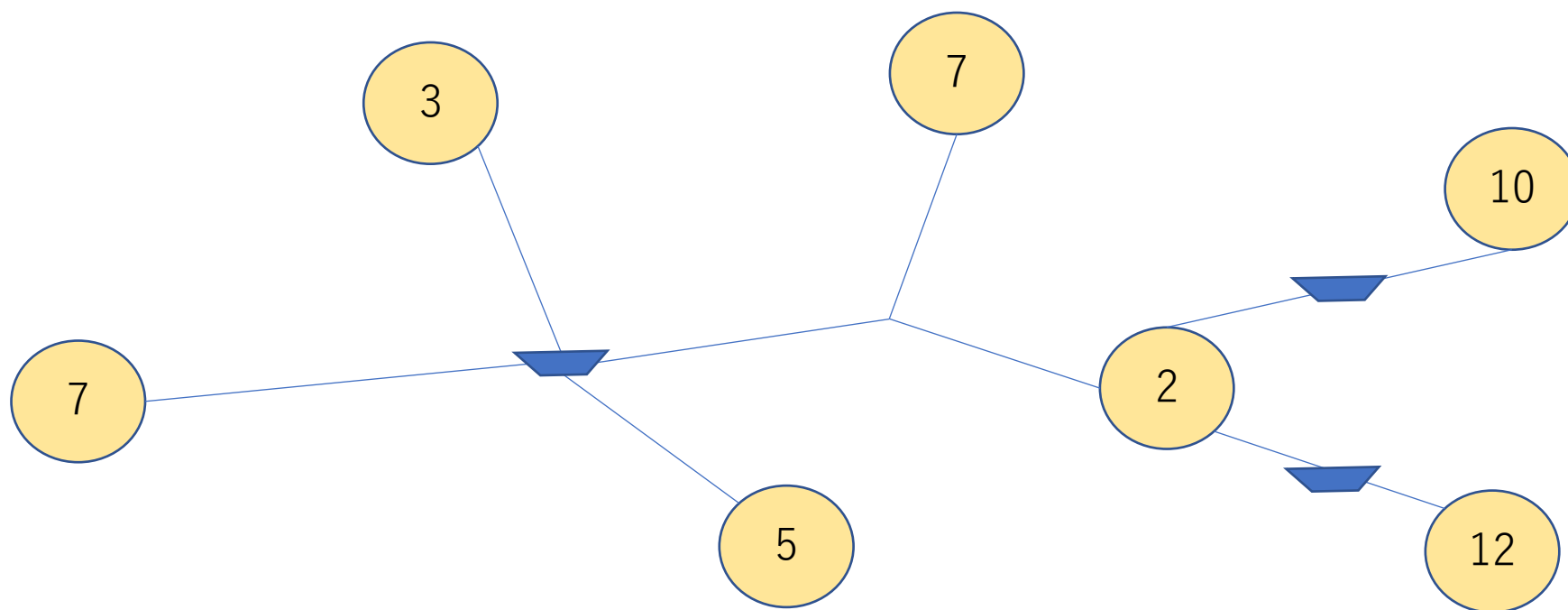
$s[v]$ の最小値で全部埋めて消すを繰り返す

小課題3. $Q=0$, 木

- この考え方を木に拡張する

船の数 : 3

合計 : 36



$s[v]$ の最小値で全部埋めて消すを繰り返す

- 頂点を S でソート、小さい順に消す
- i 番目に小さい頂点を消すとき
 - 警備員の合計に $+(S[i]-S[i-1]) \times (\text{船の数})$
 - 船の数を $\text{deg}[i]$ だけ減らす ($\text{deg}[i]$ は頂点 i の次数)
- 木の形は関係なく $\text{deg}[i]$ と $S[i]$ だけで計算できる

s[v]の最小値で全部埋めて消すを繰り返す

- さきほどの計算を整理すると次のように定式化できる

$$\sum S[i] \times (\deg[i]-1) + \max(S)$$

小課題4 $Q=0$ （一般のグラフ）

- 木の制約がなくなって一般のグラフになった
- **使う辺が木であるような最小値を達成するやり方が存在する**
- 証明は省略

小課題4 $Q=0$ (一般のグラフ)

- 木を定めたときのコスト $\sum S[i] \times (\deg[i]-1) + \max(S)$
- 上の値が最小になるような木を選ぶ問題
- 上の式 $= \sum_{\{(x, y) \in E\}} S[x] + S[y] - \sum S[i] + \max(S)$
(各辺についてコストを両端のSの和としたとき、辺のコストの合計-Sの合計+ $\max(S)$)
となるから辺 (x, y) のコストを $S[x] + S[y]$ とした最小全域木を求めればよい

小課題4 (Q=0)までまとめ

- 各辺 (x,y) についてコストが $S[x]+S[y]$ の最小全域木を求める

小課題5 以降

- 辺 (x,y) のコストが $S[x]+S[y]$ のときの最小全域木を次の $k=0,1,2,\dots,Q$ の場合について求める
 - あらたに好きな2頂点を選んで辺を追加することを k 回できる

小課題5 以降

- 5. (8点) $N \leq 16$
- 6. (18点) $N \leq 3000$
- 7. (24点) 追加の制約はない

小課題5 以降

- もともとあった辺で使うのは $k=0$ の時求めた最小全域木の辺のみ
- 新たに追加した辺は必ず使う
- **もとの最小全域木から k 本消して、新たに k 本つなぐと
してよい**
- もとの最小全域木の辺を**元辺**、新たに追加する辺を**新
辺**と呼ぶ

小課題5 以降

- **新たに追加する辺はすべてS最小の頂点を端点とする**
- 消すk本の辺を決め打ったとき，新たにk本追加してすべて連結にするコスト最小化になる
- 各連結成分ごとにS最小の頂点のみ考える
- S最小の頂点と各連結成分をつなぐのが最適

小課題5 (8点) $N \leq 16$

- 新たに追加する辺はすべてS最小の頂点を端点とする
- 新たに追加する辺の候補が $N-1$ 本に絞られたので追加の仕方 $2^{(N-1)}$ 通り全探索

小課題6 (18点) $N \leq 3000$

- $k=x+1$ のときの最適な木について、 $k=x$ で辺の差が2本（つまり元辺1本、新辺1本の差）であるような最適な木が存在する
- 証明は略

小課題6 (18点) $N \leq 3000$

- $k=0,1,2,\dots,Q$ について順番に元辺を1本消して, 新辺を1本追加する操作を繰り返していく
- このときそれぞれのタイミングでコストが最小になるような操作を貪欲に選んでいけばよい

小課題6 (18点) $N \leq 3000$

- $k=x$ が求まっていたとき $k=x+1$ を $O(N)$ で求める
- S 最小の頂点を根とした根付き木にする
- 消す元辺を全探索する $O(N)$
- 消す元辺を決め打ったとき, 追加する新辺は部分木で S 最小の頂点と根を結ぶ辺
- 部分木の S 最小値を前計算で求めておくと $O(1)$

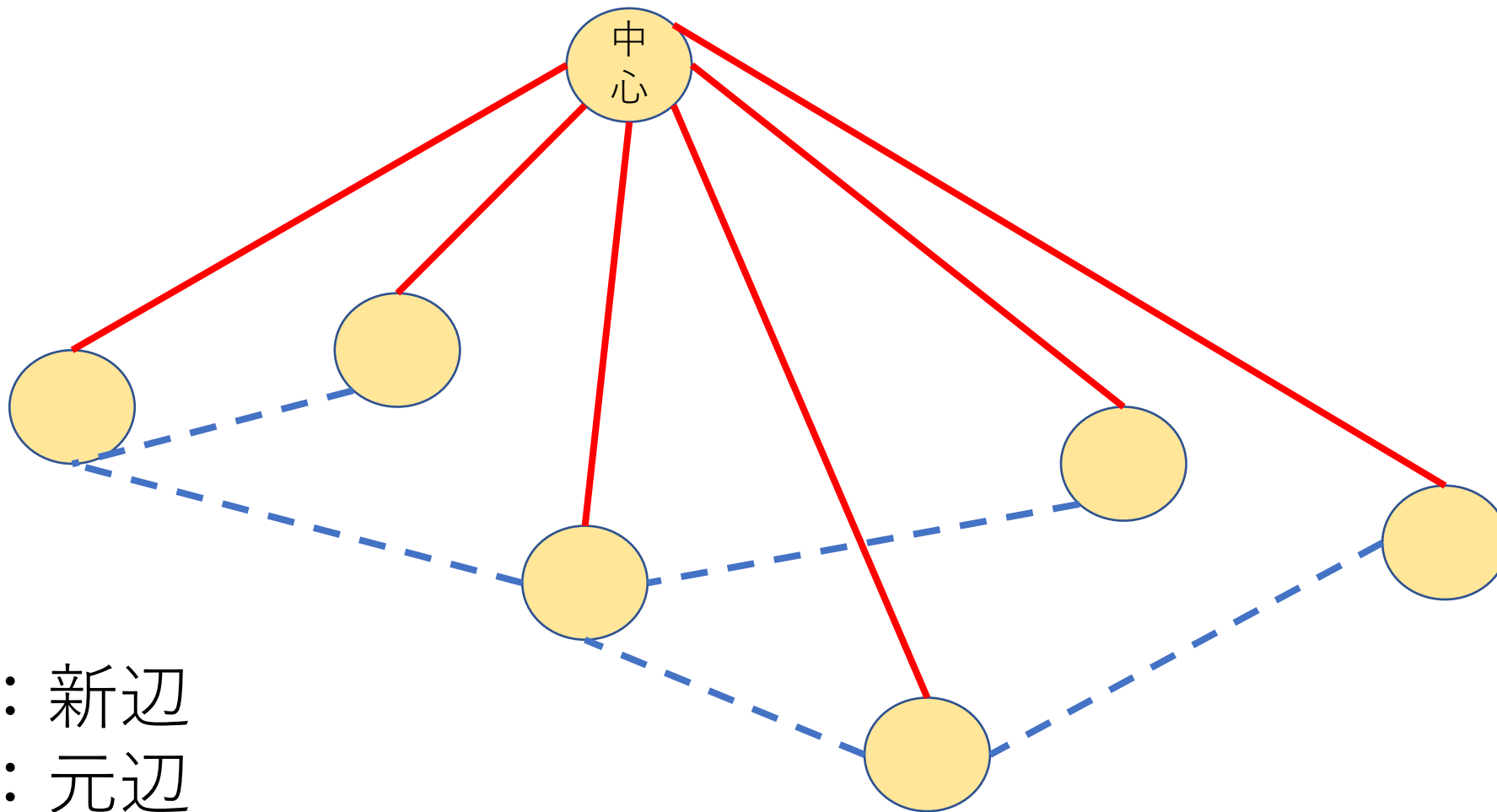
小課題7 (24点)追加の制約なし($N \leq 200000$)

- 辺は消すより追加する方が簡単
- 操作を逆から考える
- S最小の頂点を中心とするスターグラフからスタート
- 最初はすべて中心と各頂点を結ぶ新辺
- 連結性が保たれるように新辺を1本削除して元辺を1本追加するという操作を繰り返す

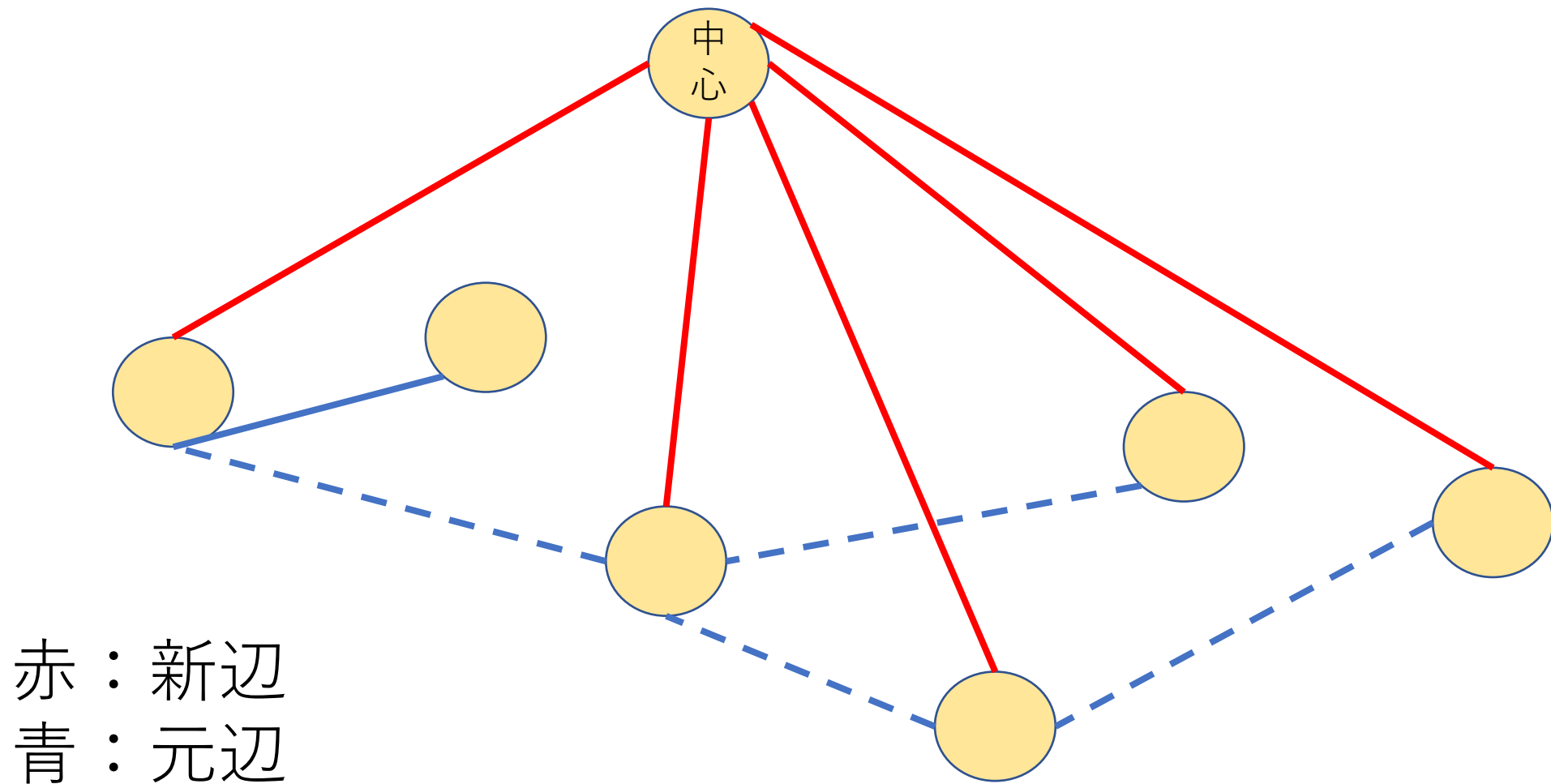
小課題7 (24点)追加の制約なし($N \leq 200000$)

- この場合でも各操作でコストが最小になるものを貪欲に選んでいけばよい

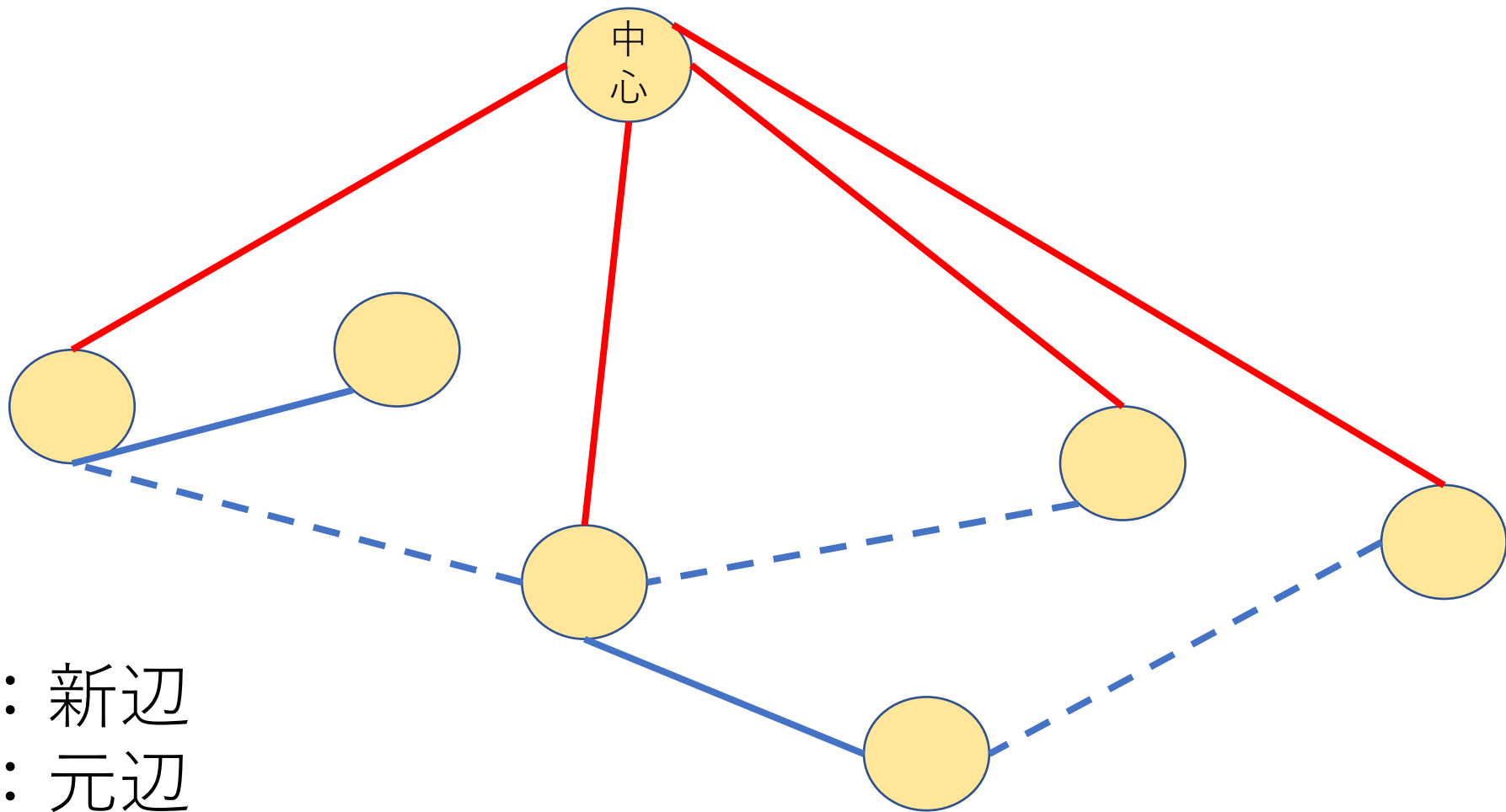
小課題7 (24点)追加の制約なし($N \leq 200000$)



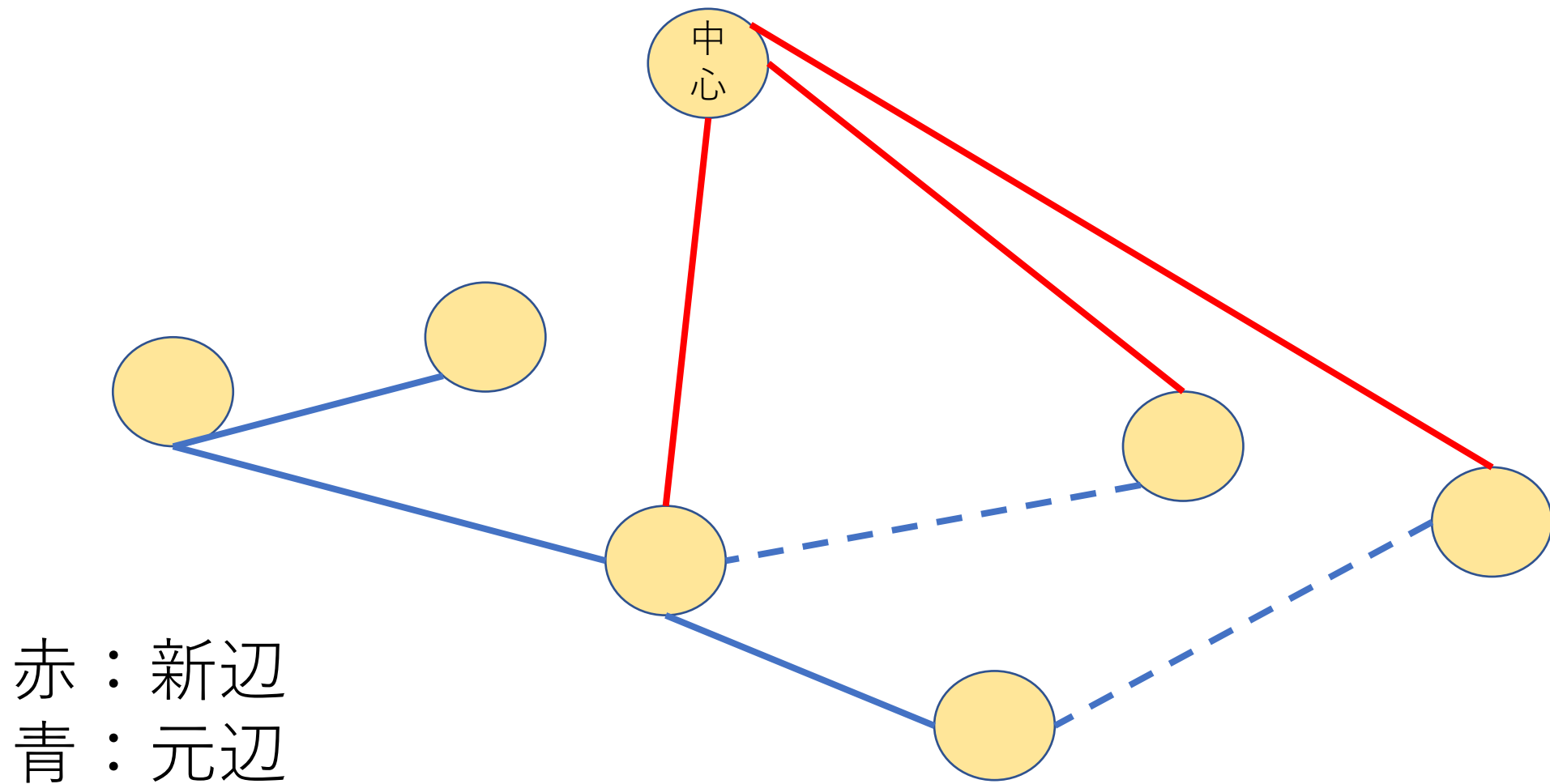
小課題7 (24点)追加の制約なし($N \leq 200000$)



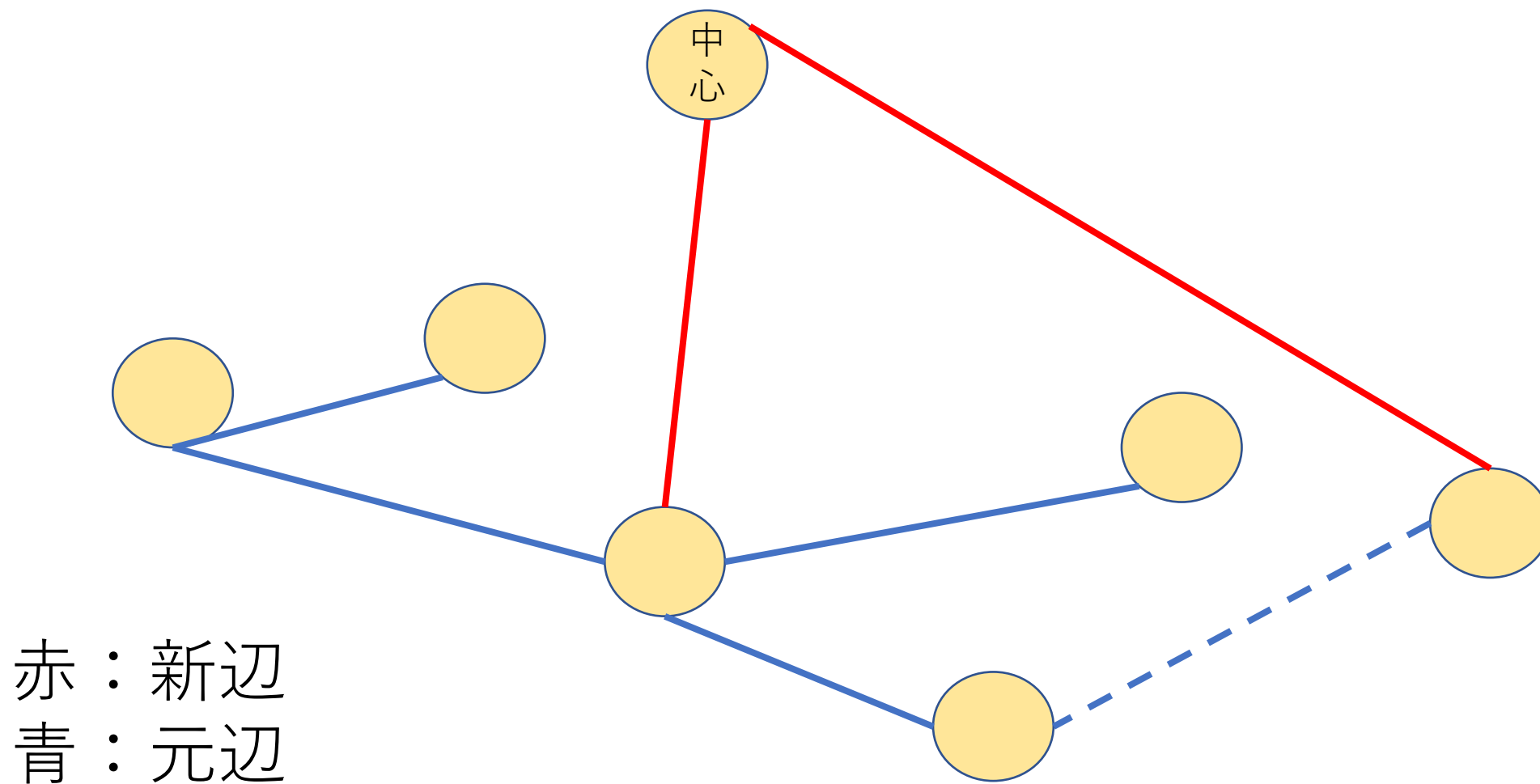
小課題7 (24点)追加の制約なし($N \leq 200000$)



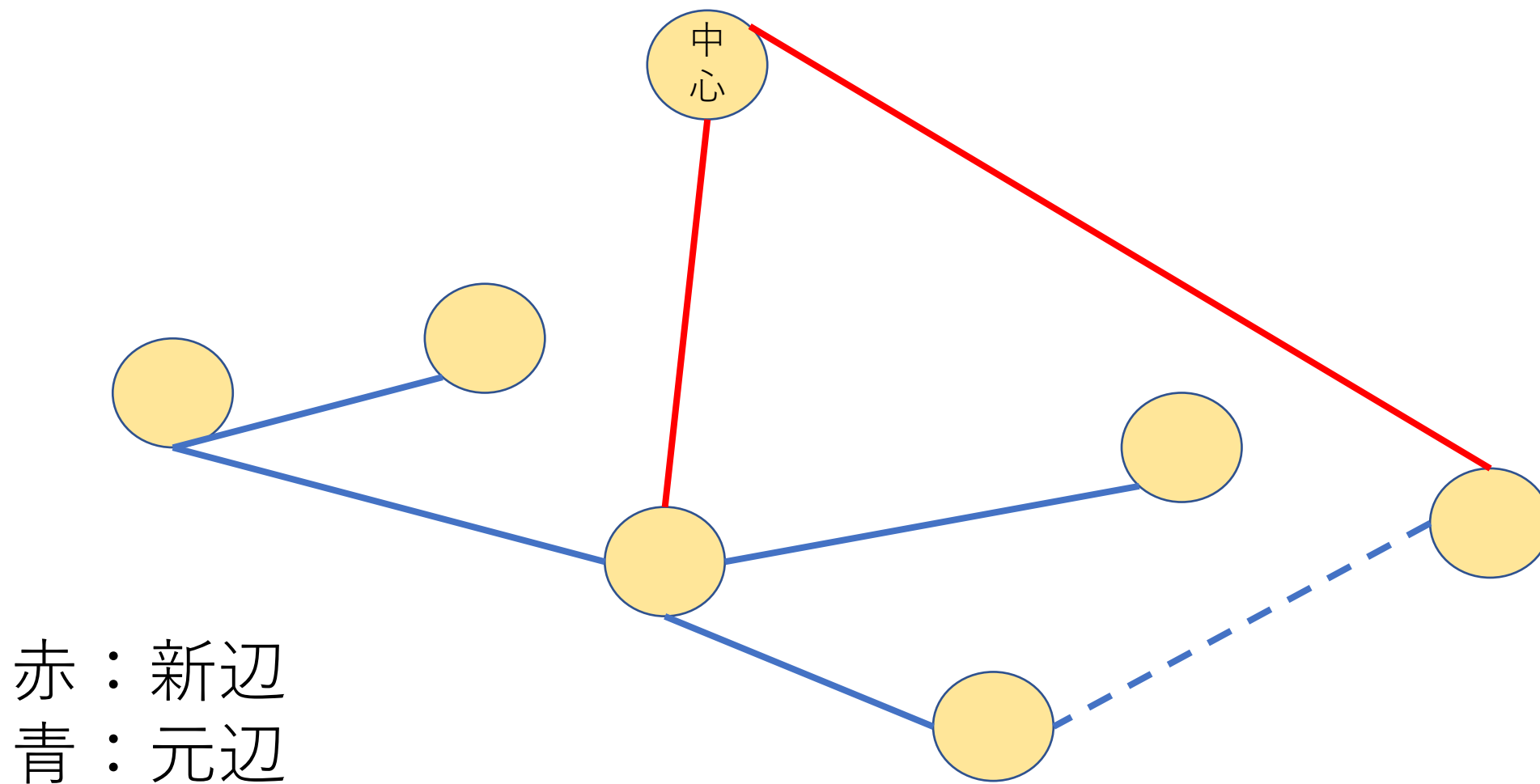
小課題7 (24点)追加の制約なし($N \leq 200000$)



小課題7 (24点)追加の制約なし($N \leq 200000$)



小課題7 (24点)追加の制約なし($N \leq 200000$)



小課題7 (24点)追加の制約なし($N \leq 200000$)

- 各新辺について、それを消すときに追加する元辺は、その新辺がつなぐ連結成分からでている元辺でコスト最小のものとしてよい
- 各新辺を消すとしたときのコストを管理する

小課題7 (24点)追加の制約なし($N \leq 200000$)

- 新辺を消して元辺を追加すると、2つの連結成分がマージされる
- このとき連結成分のもつ元辺がマージされ、その連結成分について新辺を消すときのコストが更新される（その連結成分のもつコスト最小の元辺が変わるから）
- 逆にこれに関わらない新辺はそれを消すときのコストは変化しない

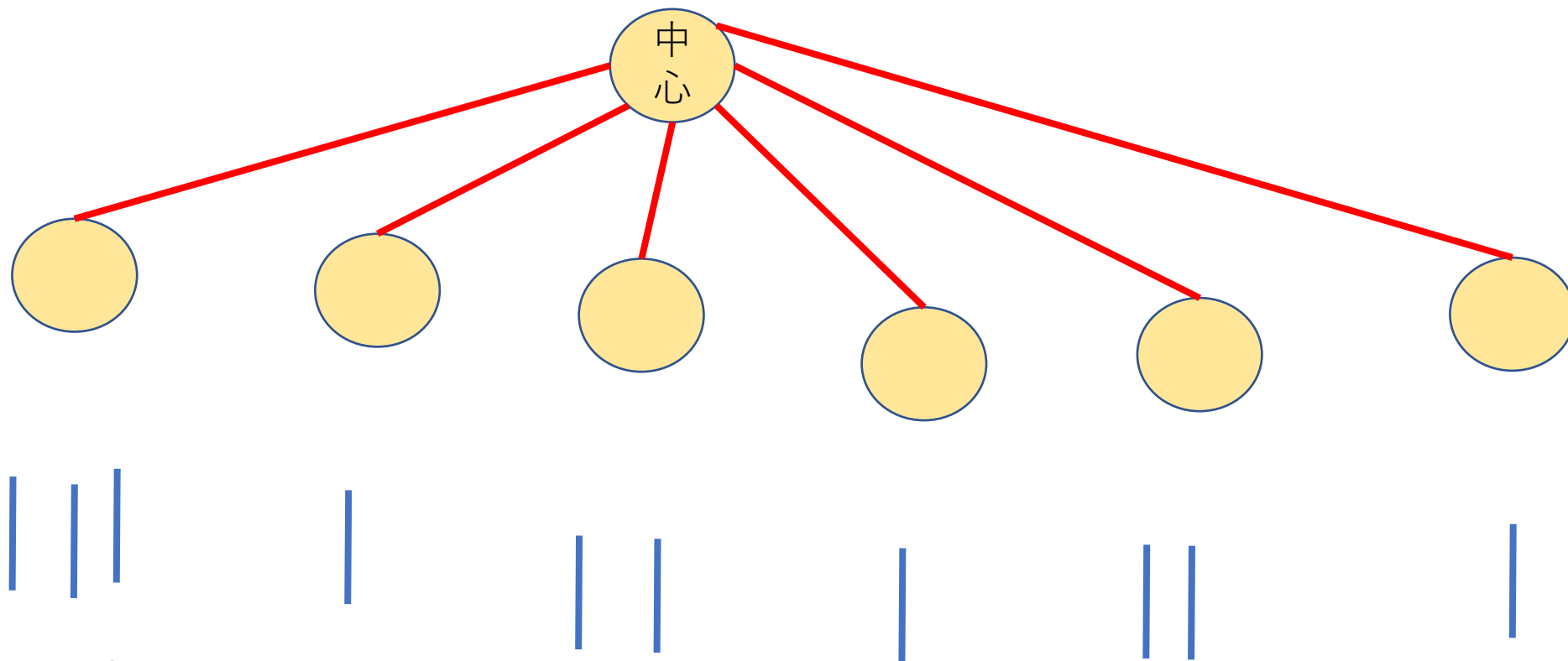
小課題7 (24点)追加の制約なし($N \leq 200000$)

- 全体で各新辺を消したときのコストをpriority_queueとかsetとかで管理
- 次の処理を繰り返す
- 全体から取り出しコスト最小の操作をする。つまり1本新辺を消して元辺を追加、連結成分をマージ
- 新たにできた連結成分について新辺を消したときのコストを求めて全体に入れる

小課題7 (24点)追加の制約なし($N \leq 200000$)

- このマージのとき各連結成分はまだ使っていない元辺をもって、そのコスト最小値を求める必要がある (priority queueとかsetとかで管理)
- マージテクを使って辺の数が少ない方を大きい方に追加するようにすると $O(N \log^2 N)$ で満点

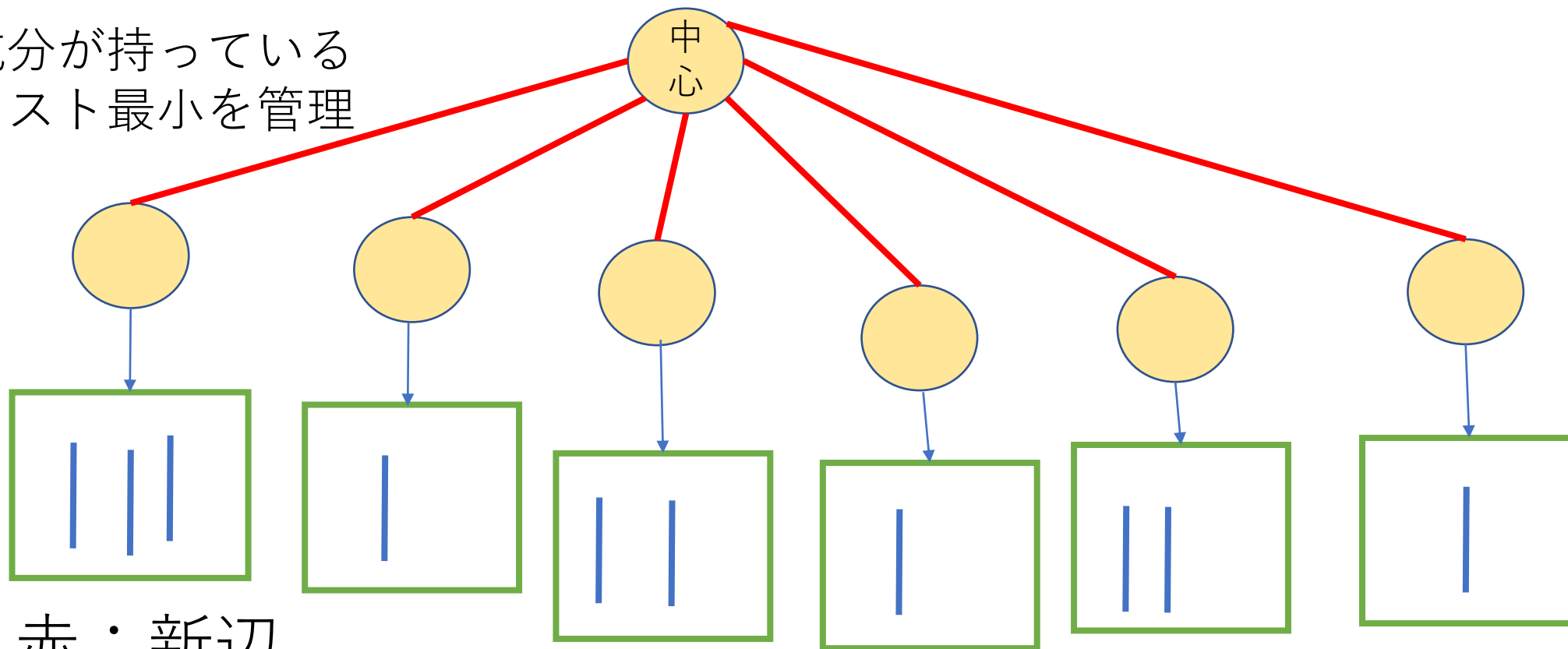
小課題7 (24点)追加の制約なし($N \leq 200000$)



赤：新辺
青：元辺

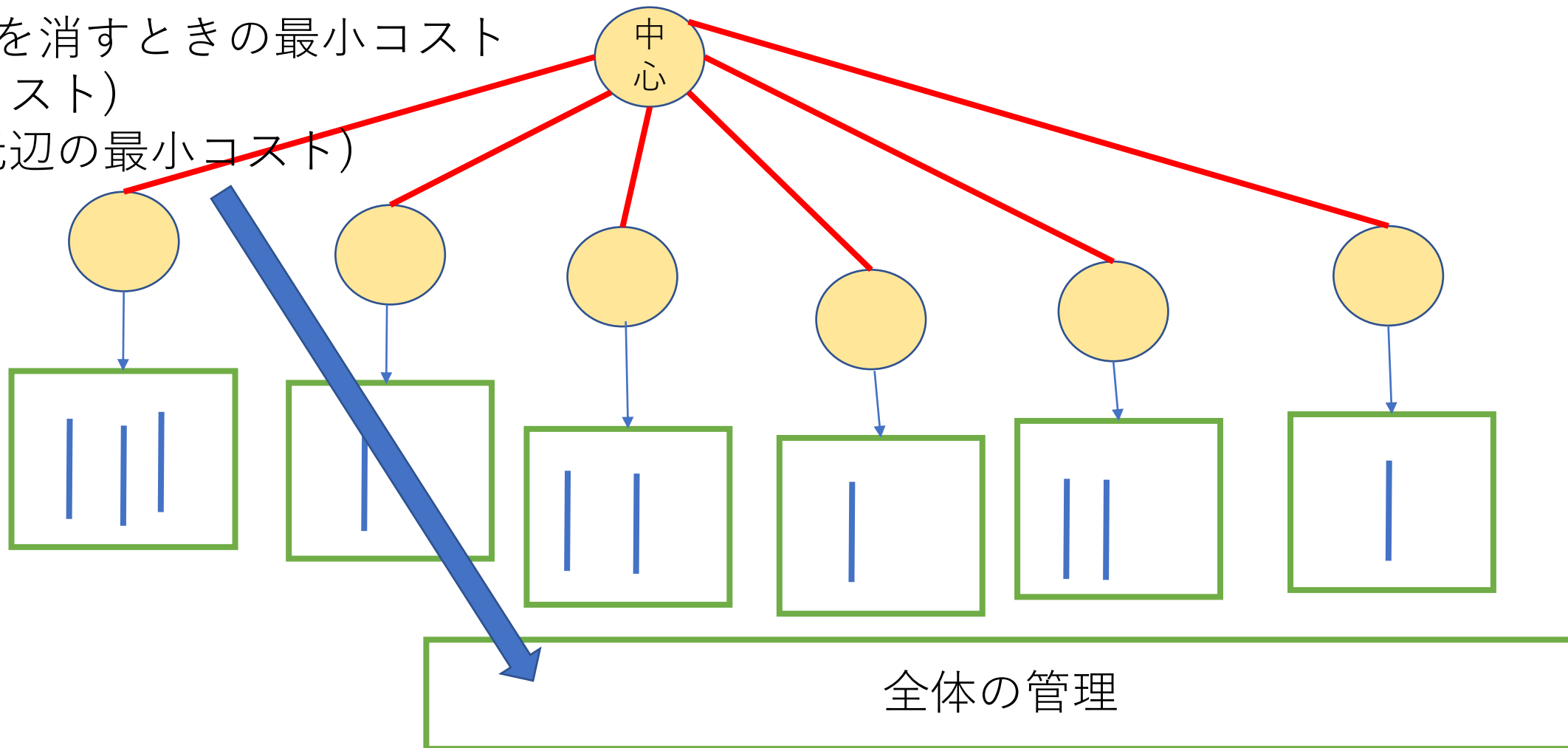
小課題7 (24点)追加の制約なし($N \leq 200000$)

各連結成分が持っている
元辺のコスト最小を管理



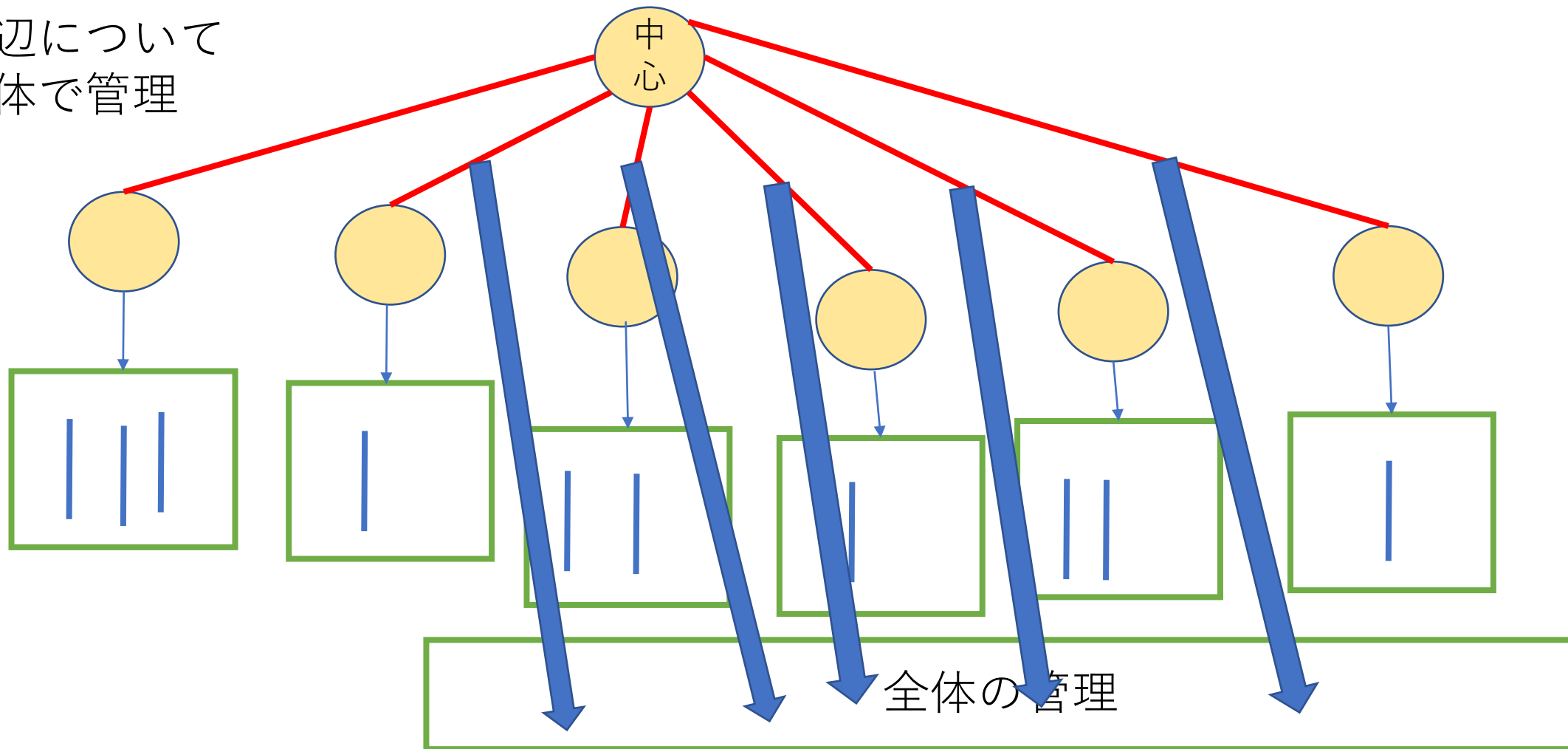
小課題7 (24点)追加の制約なし($N \leq 200000$)

この赤辺を消すときの最小コスト
- (赤辺コスト)
+ (持つ元辺の最小コスト)
を入れる

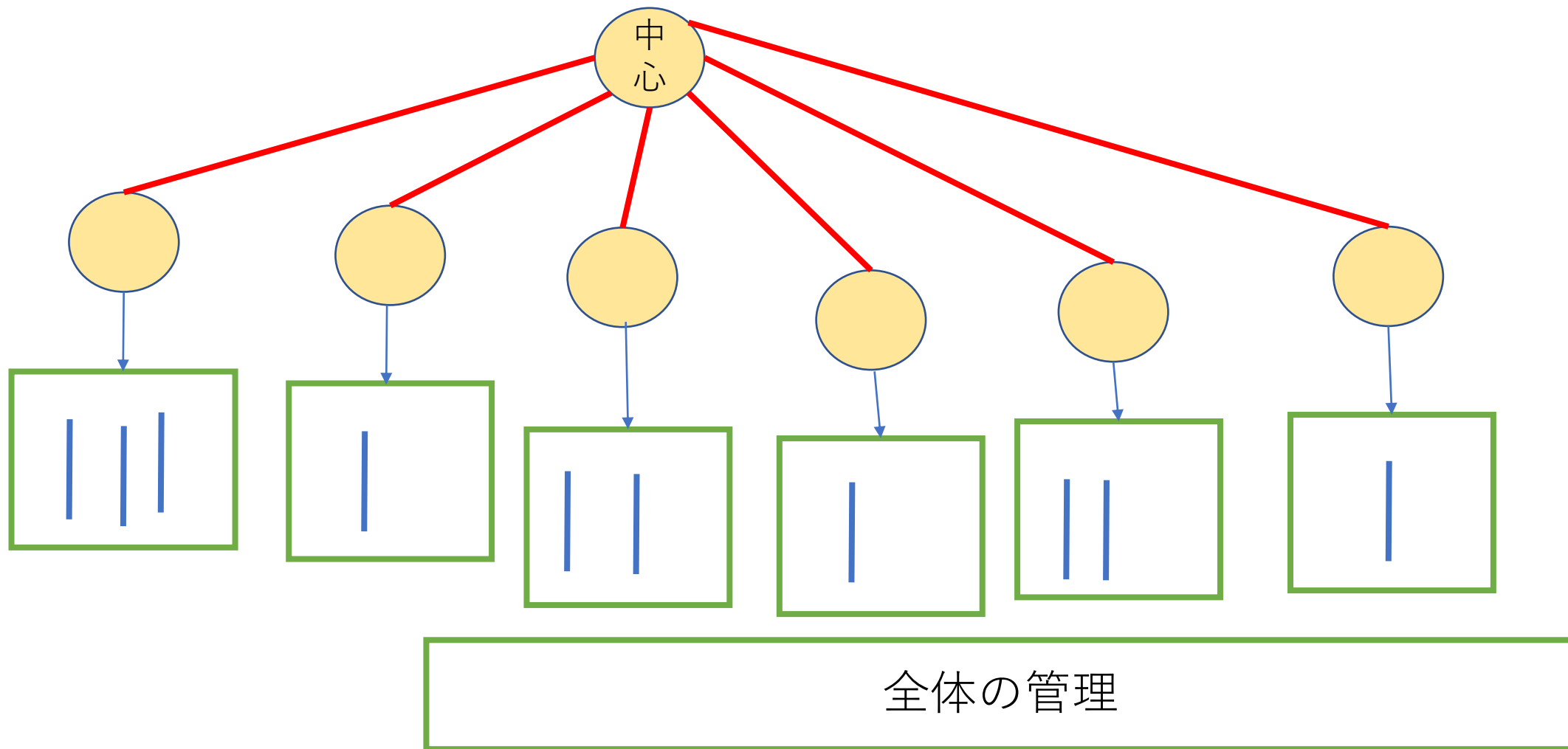


小課題7 (24点)追加の制約なし($N \leq 200000$)

全部の赤辺について
入れて全体で管理

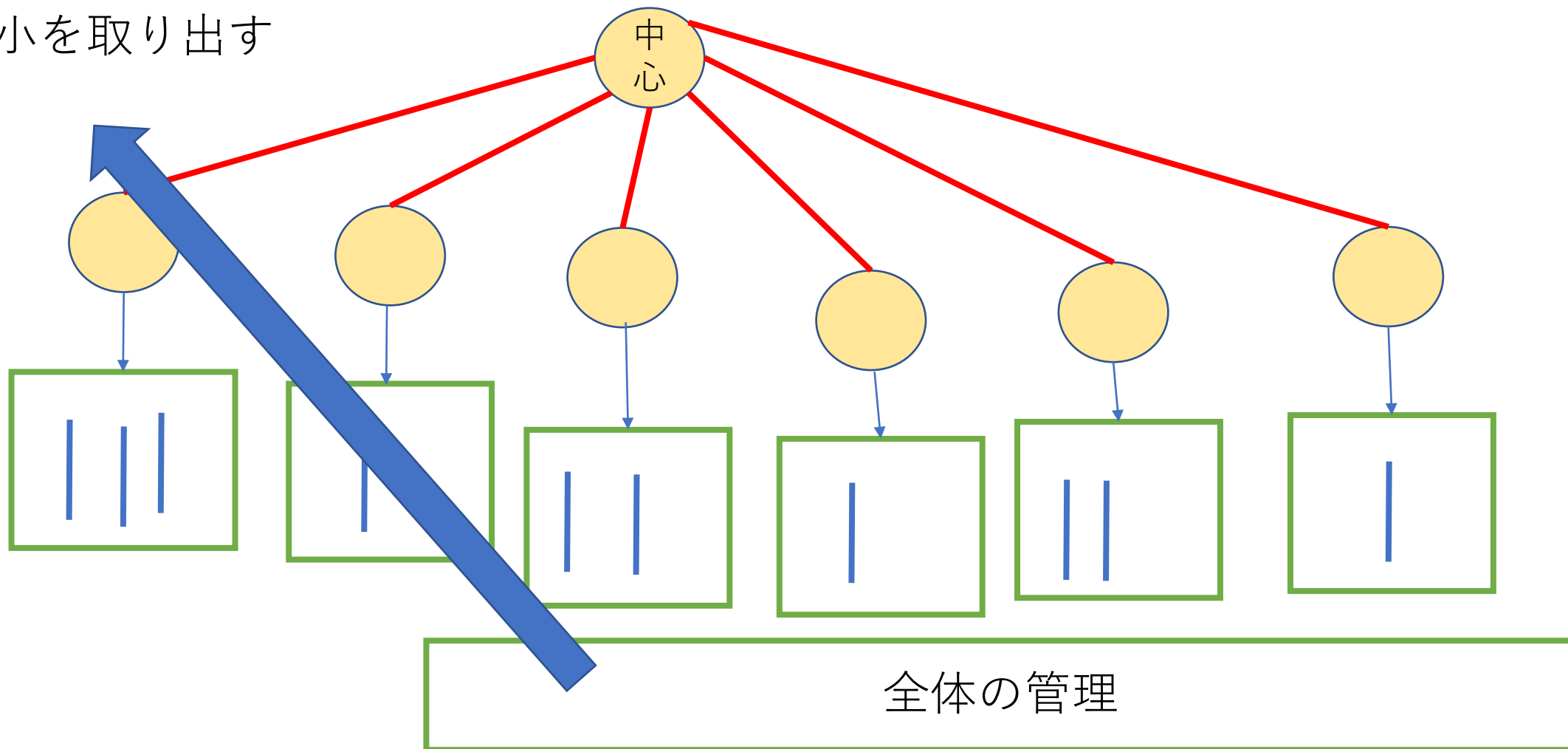


小課題7 (24点)追加の制約なし($N \leq 200000$)



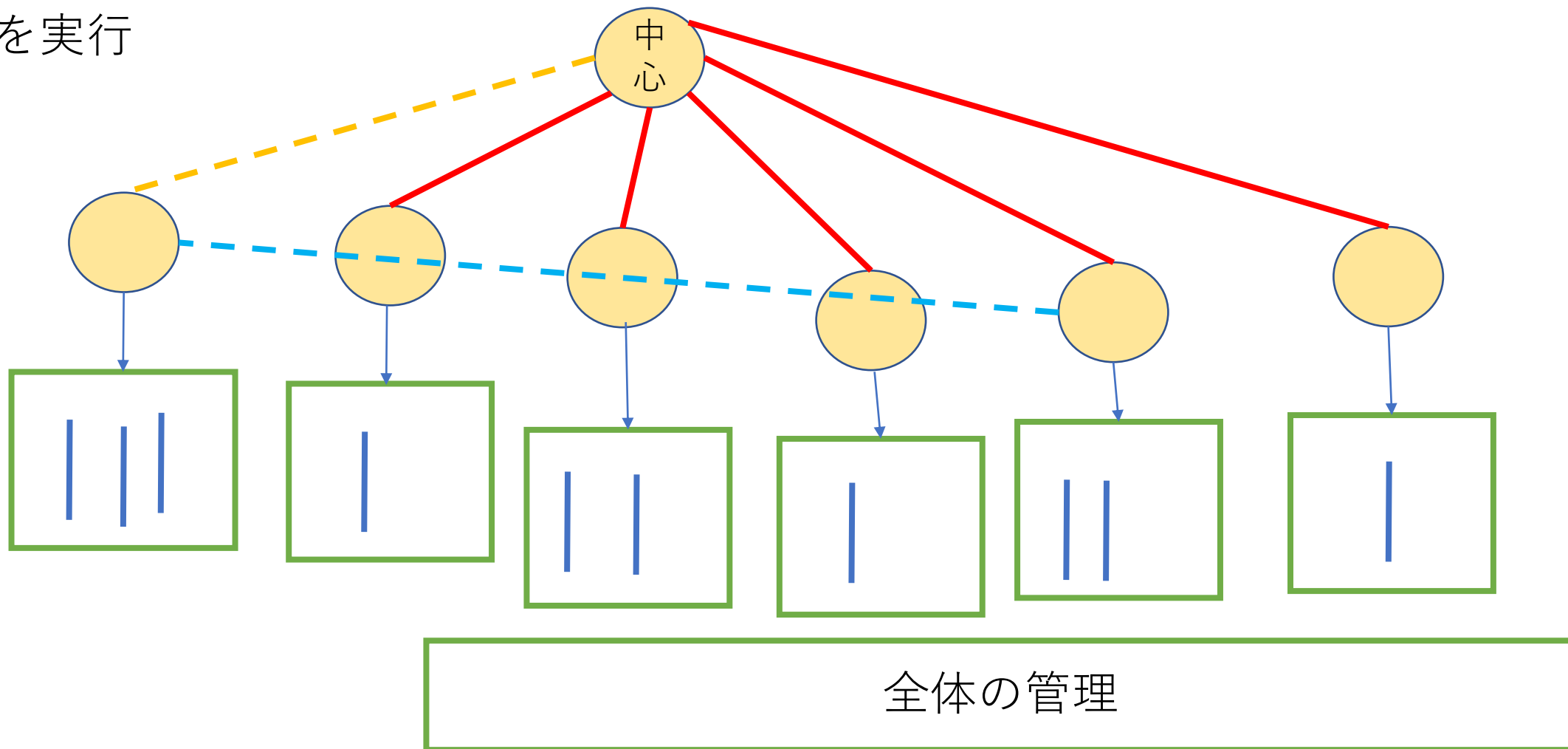
小課題7 (24点)追加の制約なし($N \leq 200000$)

全体の最小を取り出す



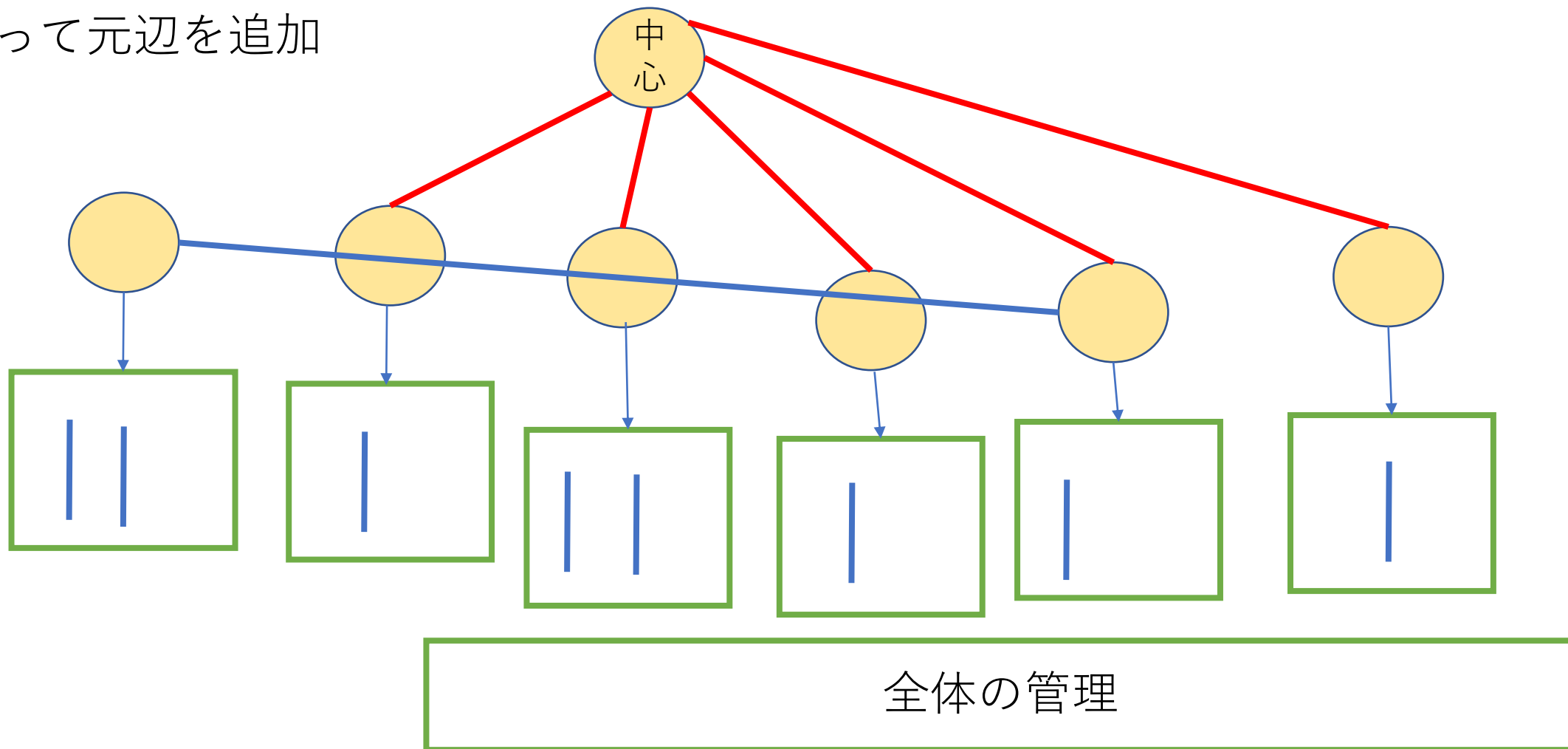
小課題7 (24点)追加の制約なし($N \leq 200000$)

その操作を実行



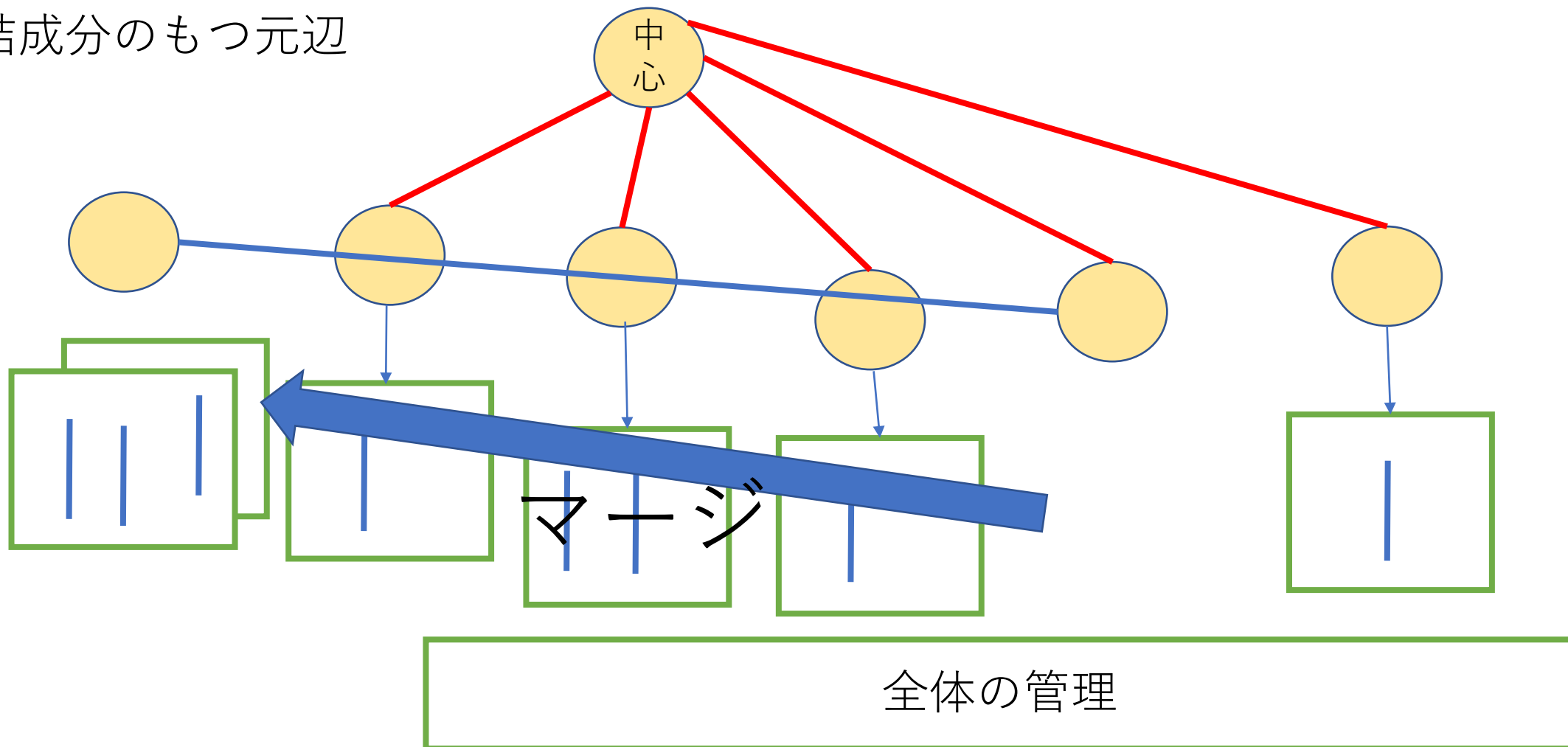
小課題7 (24点)追加の制約なし($N \leq 200000$)

新辺を切って元辺を追加

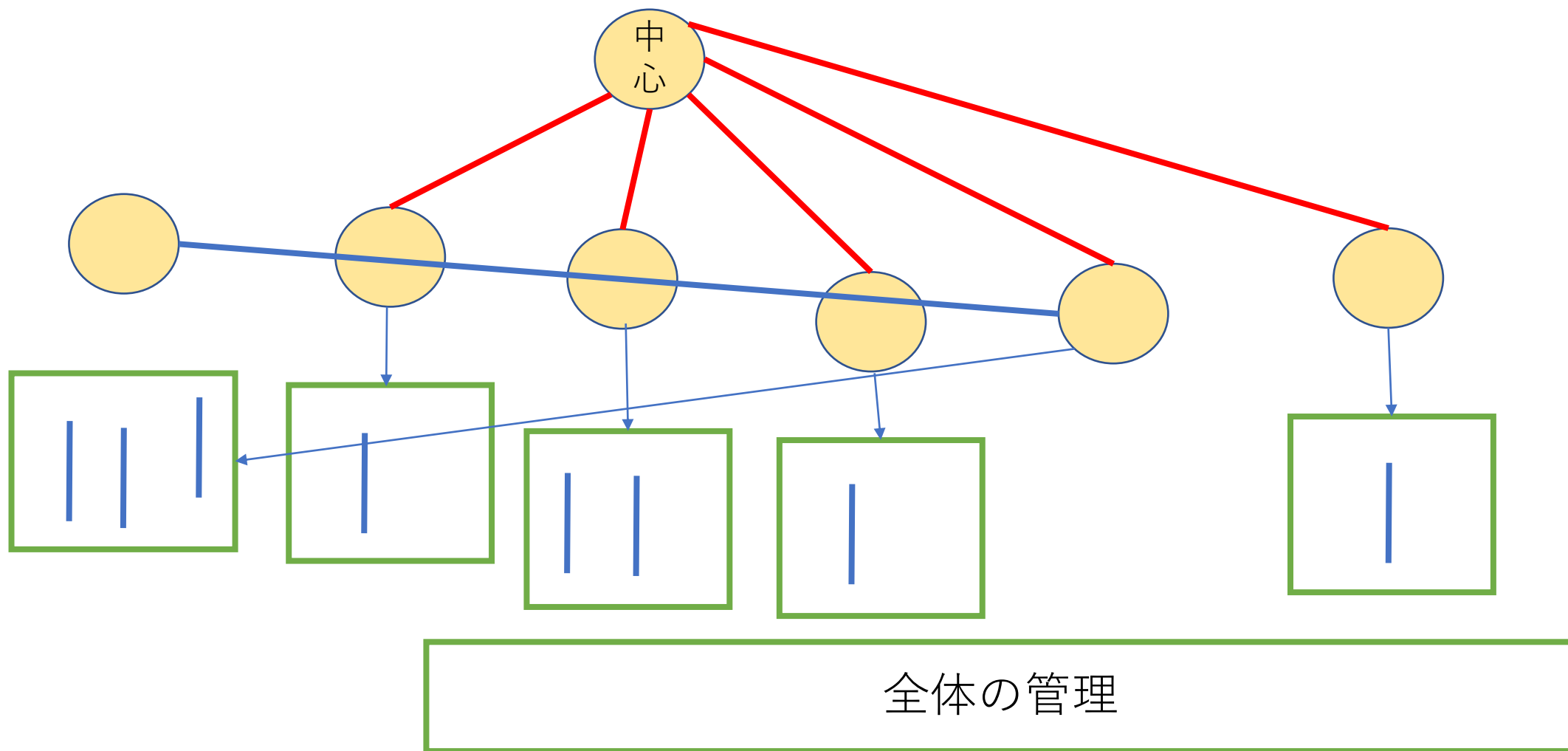


小課題7 (24点)追加の制約なし($N \leq 200000$)

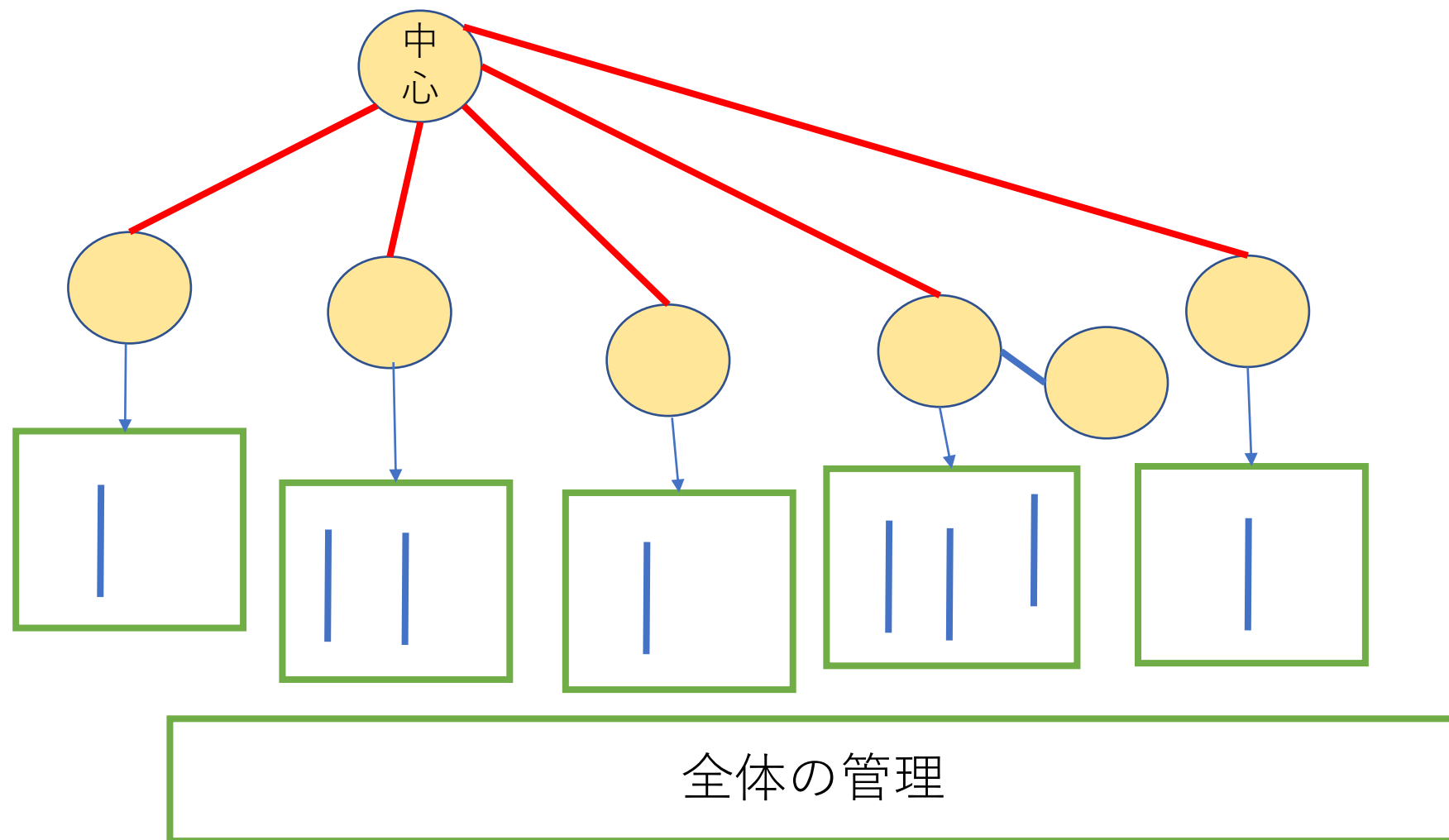
2つも連結成分のもつ元辺
をマージ



小課題7 (24点)追加の制約なし($N \leq 200000$)

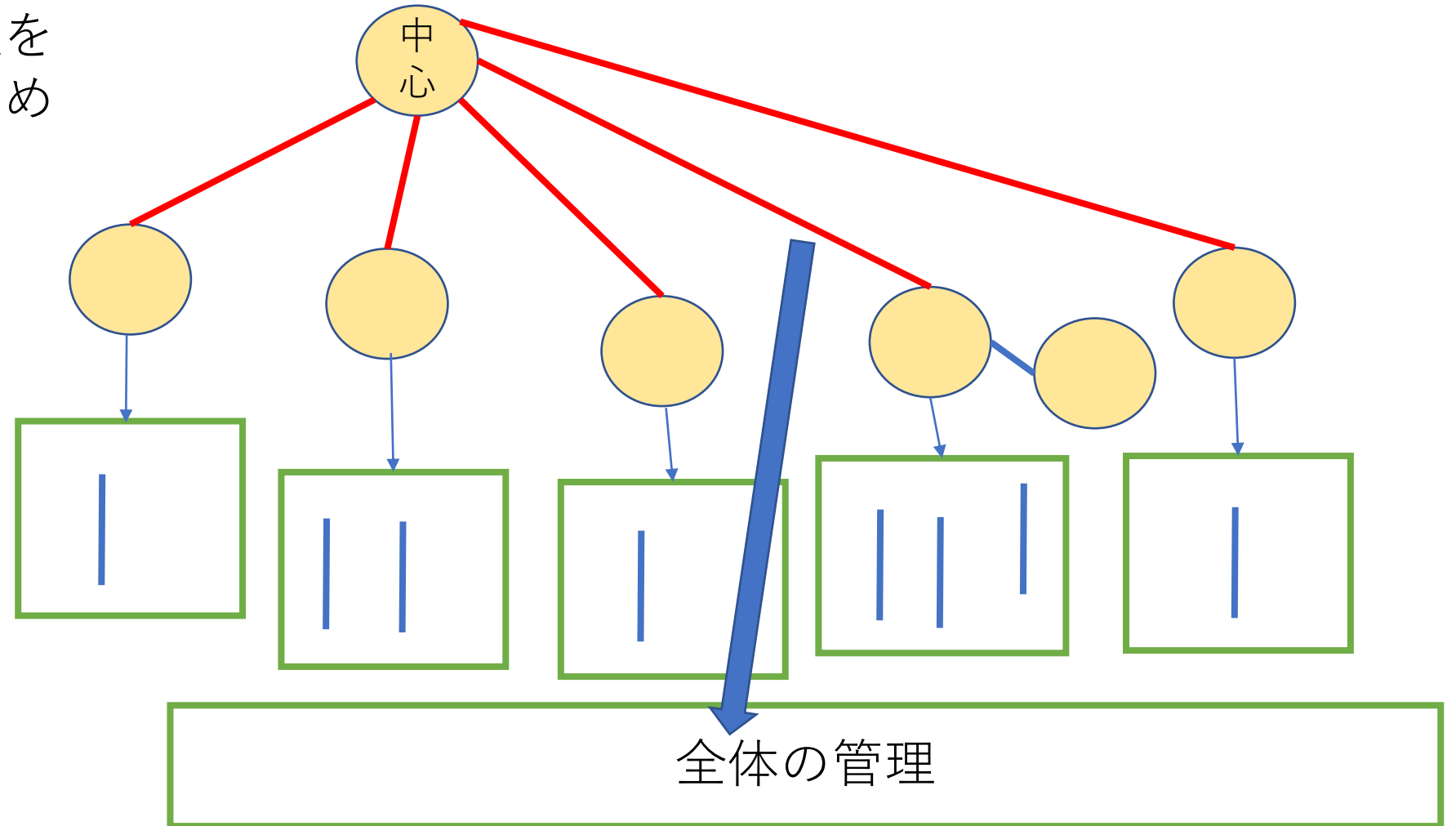


小課題7 (24点)追加の制約なし($N \leq 200000$)



小課題7 (24点)追加の制約なし($N \leq 200000$)

新しい連結成分の赤辺を
消すときのコストを求め
てくれる



得点分布

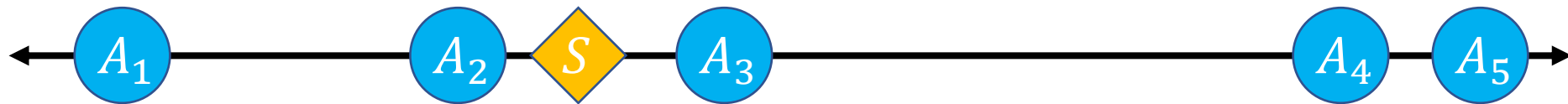
- 76点 1人
- 58点 3人
- 50点 1人
- 45点 1人
- 37点 4人
- 25点 5人
- 12点 3人
- 0点 10人

Bitaro's Travel 解説

ynymxiaolongbao

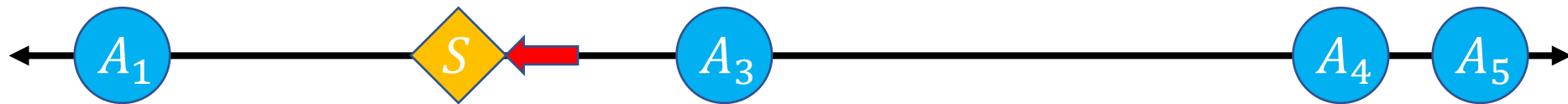
問題概要

- 数直線上に N 個の点 $A_1, A_2, \dots, A_N$ がある
- S からスタートして、「最も近い点に移動して、その点を消す」ことを点がなくなるまで繰り返す
- 移動距離の総和はいくつか？
- これを色々な S について Q クエリ答える



問題概要

- 数直線上に N 個の点 $A_1, A_2, \dots, A_N$ がある
- S からスタートして、「最も近い点に移動して、その点を消す」ことを点がなくなるまで繰り返す
- 移動距離の総和はいくつか？
- これを色々な S について Q クエリ答える



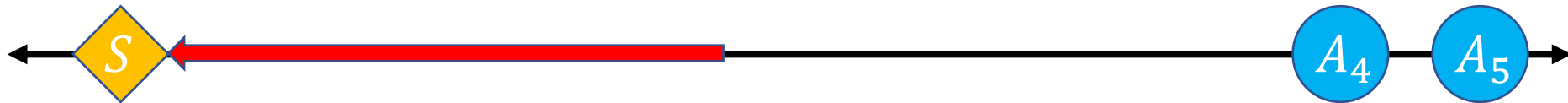
問題概要

- 数直線上に N 個の点 $A_1, A_2, \dots, A_N$ がある
- S からスタートして、「最も近い点に移動して、その点を消す」ことを点がなくなるまで繰り返す
- 移動距離の総和はいくつか？
- これを色々な S について Q クエリ答える



問題概要

- 数直線上に N 個の点 $A_1, A_2, \dots, A_N$ がある
- S からスタートして、「最も近い点に移動して、その点を消す」ことを点がなくなるまで繰り返す
- 移動距離の総和はいくつか？
- これを色々な S について Q クエリ答える



問題概要

- 数直線上に N 個の点 $A_1, A_2, \dots, A_N$ がある
- S からスタートして、「最も近い点に移動して、その点を消す」ことを点がなくなるまで繰り返す
- 移動距離の総和はいくつか？
- これを色々な S について Q クエリ答える



問題概要

- 数直線上に N 個の点 $A_1, A_2, \dots, A_N$ がある
- S からスタートして、「最も近い点に移動して、その点を消す」ことを点がなくなるまで繰り返す
- 移動距離の総和はいくつか？
- これを色々な S について Q クエリ答える



小課題 1 ($N \leq 2000, Q = 1$)

- 点が全てなくなるまで、毎回一番近い点を $O(N)$ で求める
- $O(N^2Q)$



小課題 2 ($N \leq 200000, Q = 1$)

- 途中の状態を見してみる
- 左からいくつかと、右からいくつかの点だけが残っている
- 左側で一番右にある点を A_l 、右側で一番左にある点を A_r とする
- S から一番近いのは A_l か A_r $\rightarrow$ これだけ見ればok
- $O(NQ)$



小課題 2 ($N \leq 200000, Q = 1$)

- 途中の状態を見してみる
- 左からいくつかと、右からいくつかの点だけが残っている
- 左側で一番右にある点を A_l 、右側で一番左にある点を A_r とする
- S から一番近いのは A_l か A_r → これだけ見ればok
- $O(NQ)$



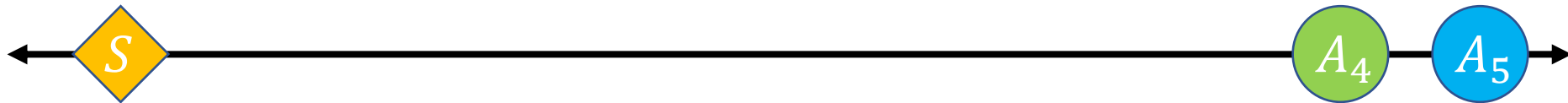
小課題 2 ($N \leq 200000, Q = 1$)

- 左からいくつかと、右からいくつかの点だけが残っている
- **左側で一番右にある点**を A_l 、右側で一番左にある点を A_r とする
- 「左側で一番右にある点」はいつも存在するんですか？



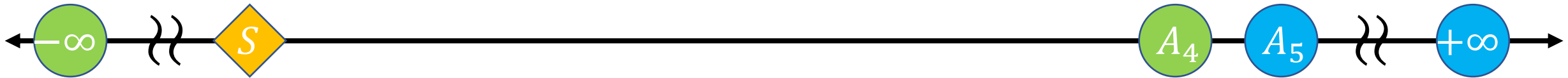
小課題 2 ($N \leq 200000, Q = 1$)

- 左からいくつかと、右からいくつかの点だけが残っている
- **左側で一番右にある点**を A_l 、右側で一番左にある点を A_r とする
- 「左側で一番右にある点」はいつも存在するんですか？
- → **No**



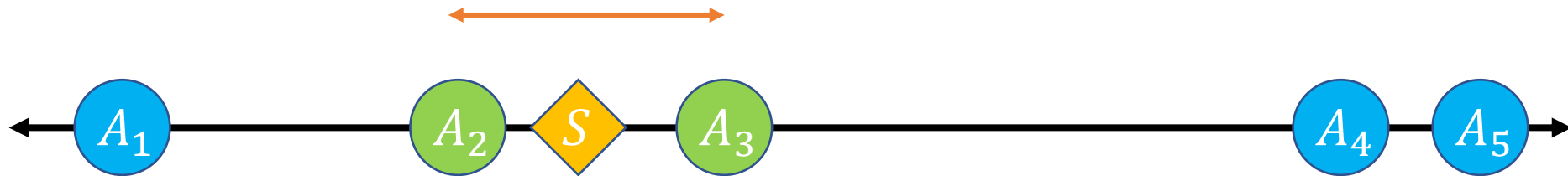
小課題 2 ($N \leq 200000, Q = 1$)

- $-\infty$ と $+\infty$ に点を追加する (**番兵**)
- これらの点は必ず最後に取りられる
- これらの点を取る直前に終了すればよい



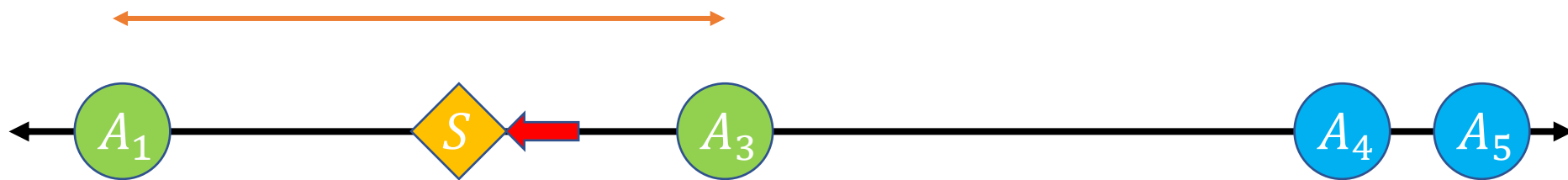
小課題 3 ($A_{i+1} - A_i \leq 100$)

- もう一度経過を見てみる
- A_l と A_r の距離は大きくなっていく



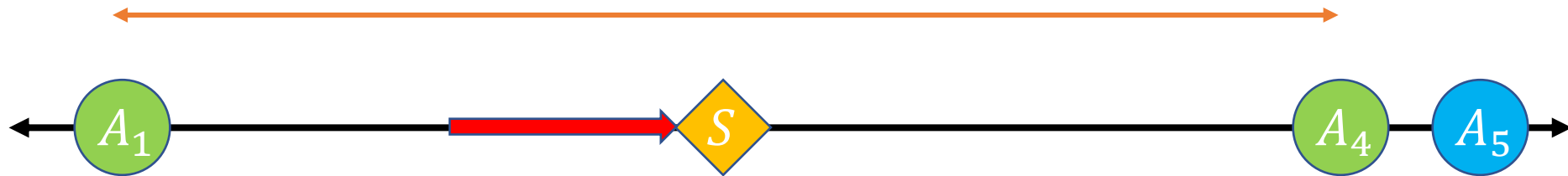
小課題 3 ($A_{i+1} - A_i \leq 100$)

- もう一度経過を見てみる
- A_l と A_r の距離は大きくなっていく



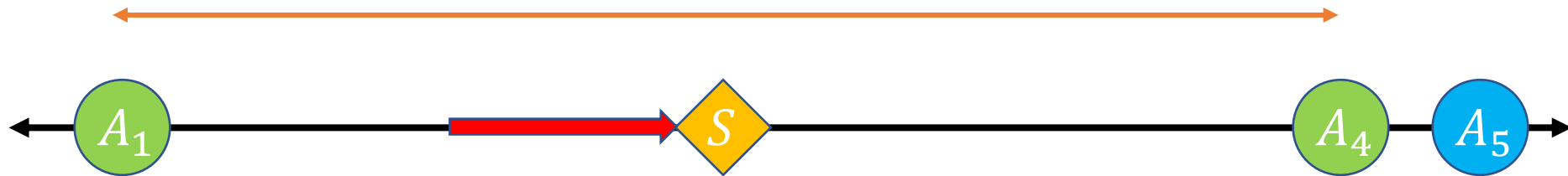
小課題 3 ($A_{i+1} - A_i \leq 100$)

- もう一度経過を見てみる
- A_l と A_r の距離は大きくなっていく



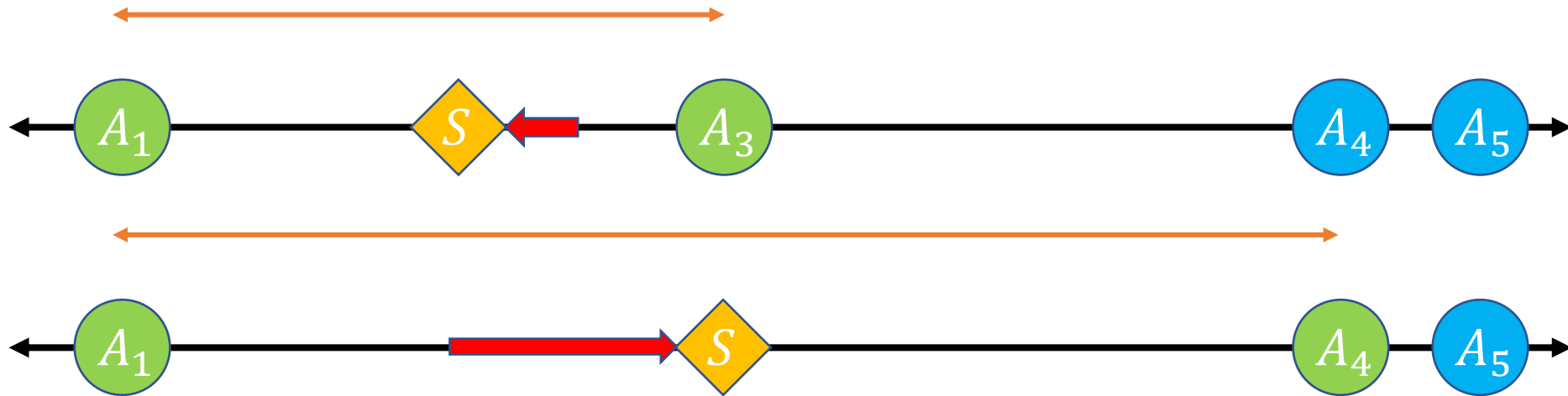
小課題 3 ($A_{i+1} - A_i \leq 100$)

- もう一度経過を見てみる
- A_l と A_r の距離は大きくなっていく
- どれくらい？



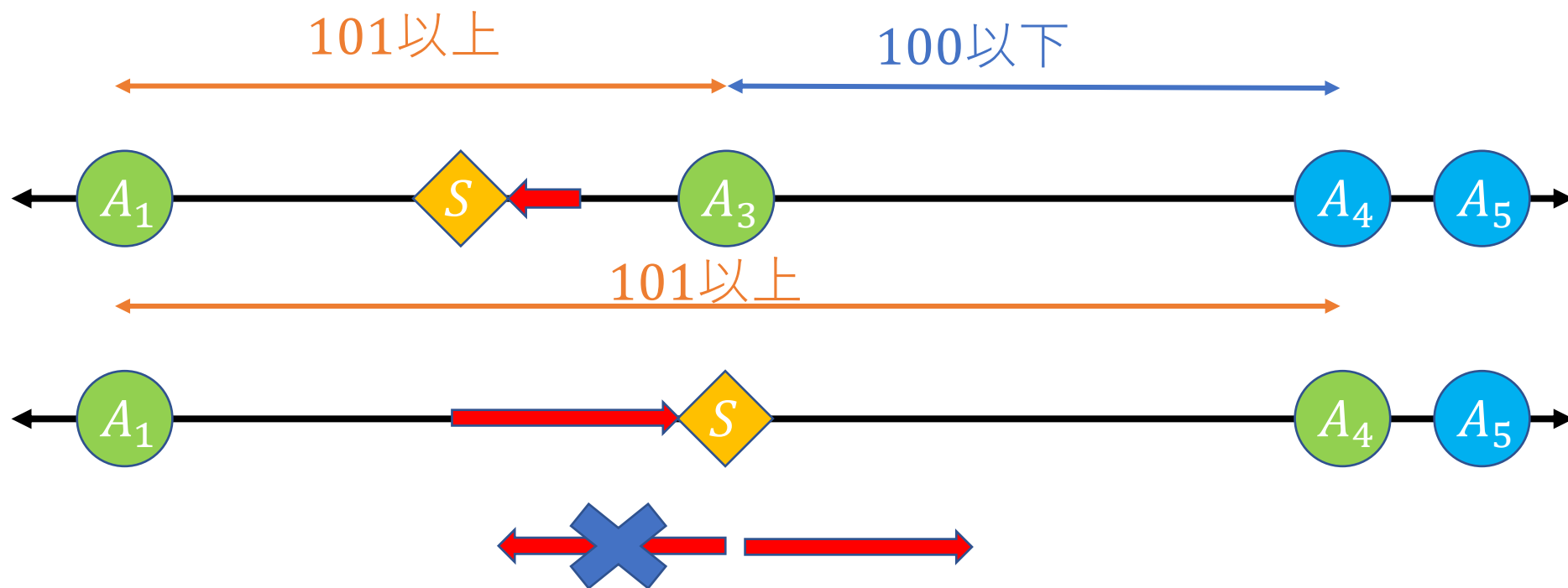
小課題 3 ($A_{i+1} - A_i \leq 100$)

- どれくらい A_l と A_r の距離が大きくなっていくか
- 一回で区間一つ分増える
- $A_i < A_{i+1}$ なので区間の長さは1以上→1以上増える



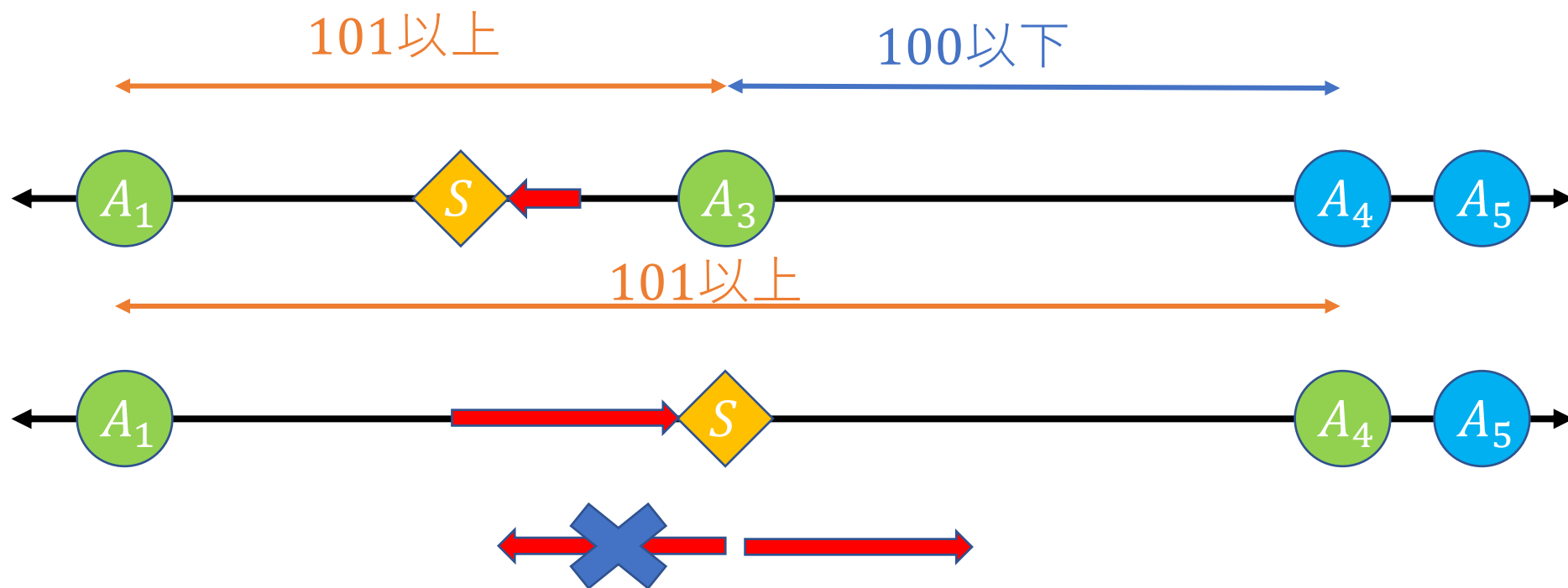
小課題 3 ($A_{i+1} - A_i \leq 100$)

- 100 回移動すると A_l と A_r の距離は101以上になる
- →それ以降は最後以外 A_l と A_r の間を飛び越えられない



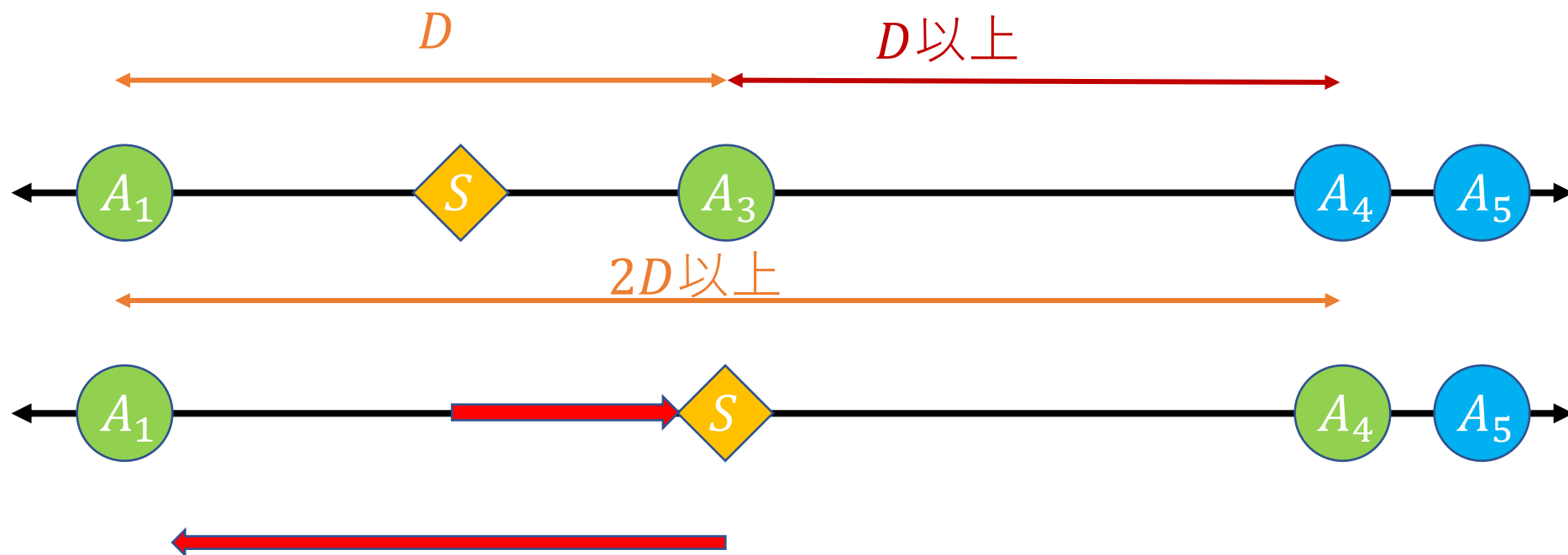
小課題 3 ($A_{i+1} - A_i \leq 100$)

- 100 回移動 $\rightarrow A_1$ か A_N まで同じ向きで移動 ($\rightarrow$ 逆の端まで移動)
- 計算回数は $100 \times Q$ くらい



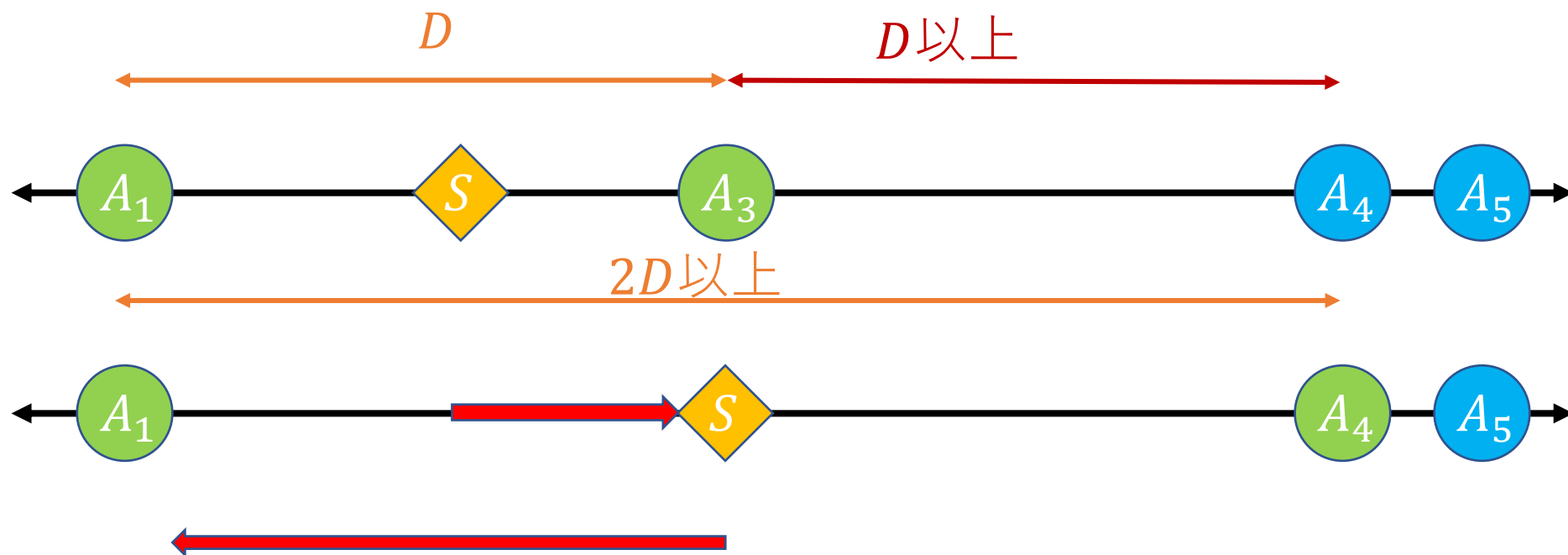
満点解法

- どれくらい A_l と A_r の距離が大きくなっていくか
- 進む向きが変わるときには距離 D が2倍以上になる



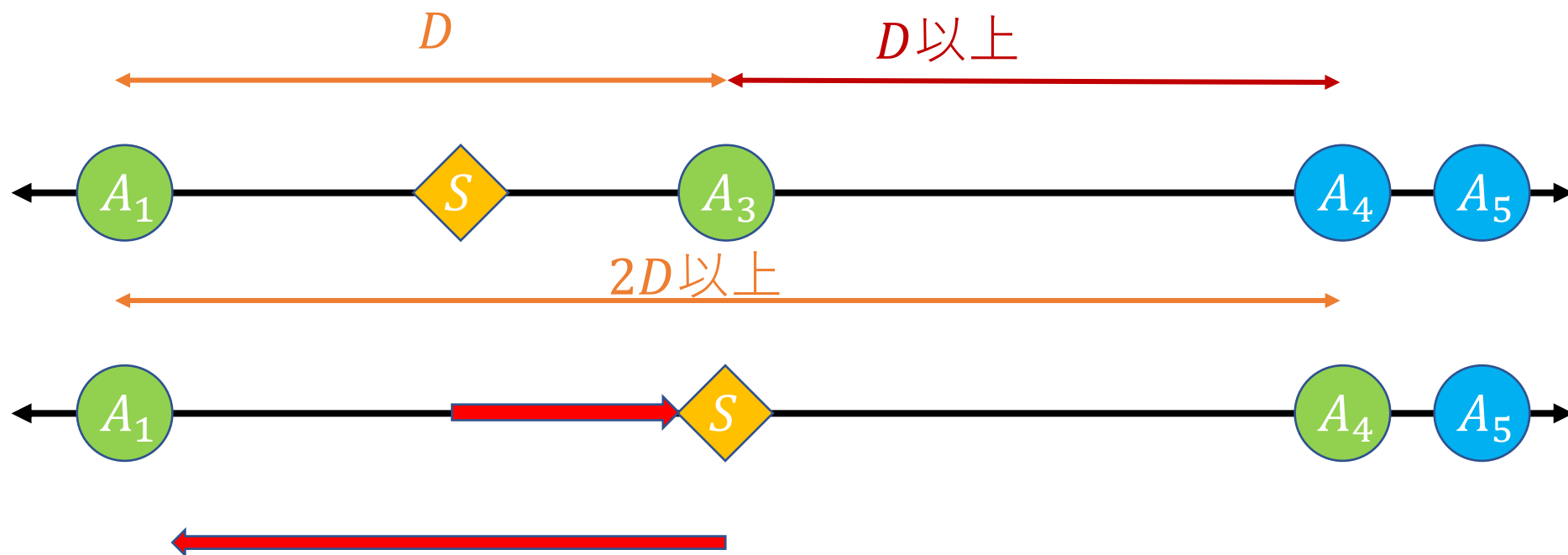
満点解法

- 距離 D が 2 倍以上にならないなら進む向きは変わらない
- 最初左右どちらに進むか調べ、その方向で距離 D が 2 倍以上になる初めての点を二分探索で求めて、そこまで同じ向きで進む



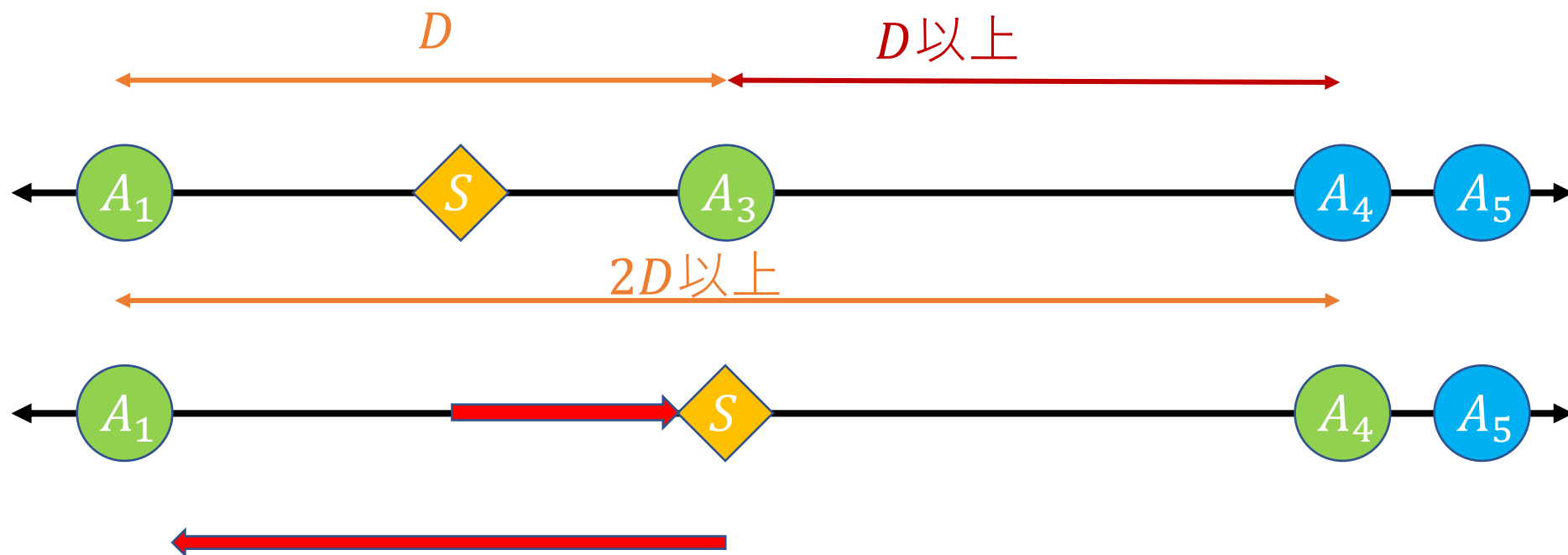
満点解法

- 距離 D は毎回 2 倍以上になるので、これを $O(\log \max A_i)$ 回繰り返せばよい
- 計算量は $O(N + Q \log N \log \max A_i)$

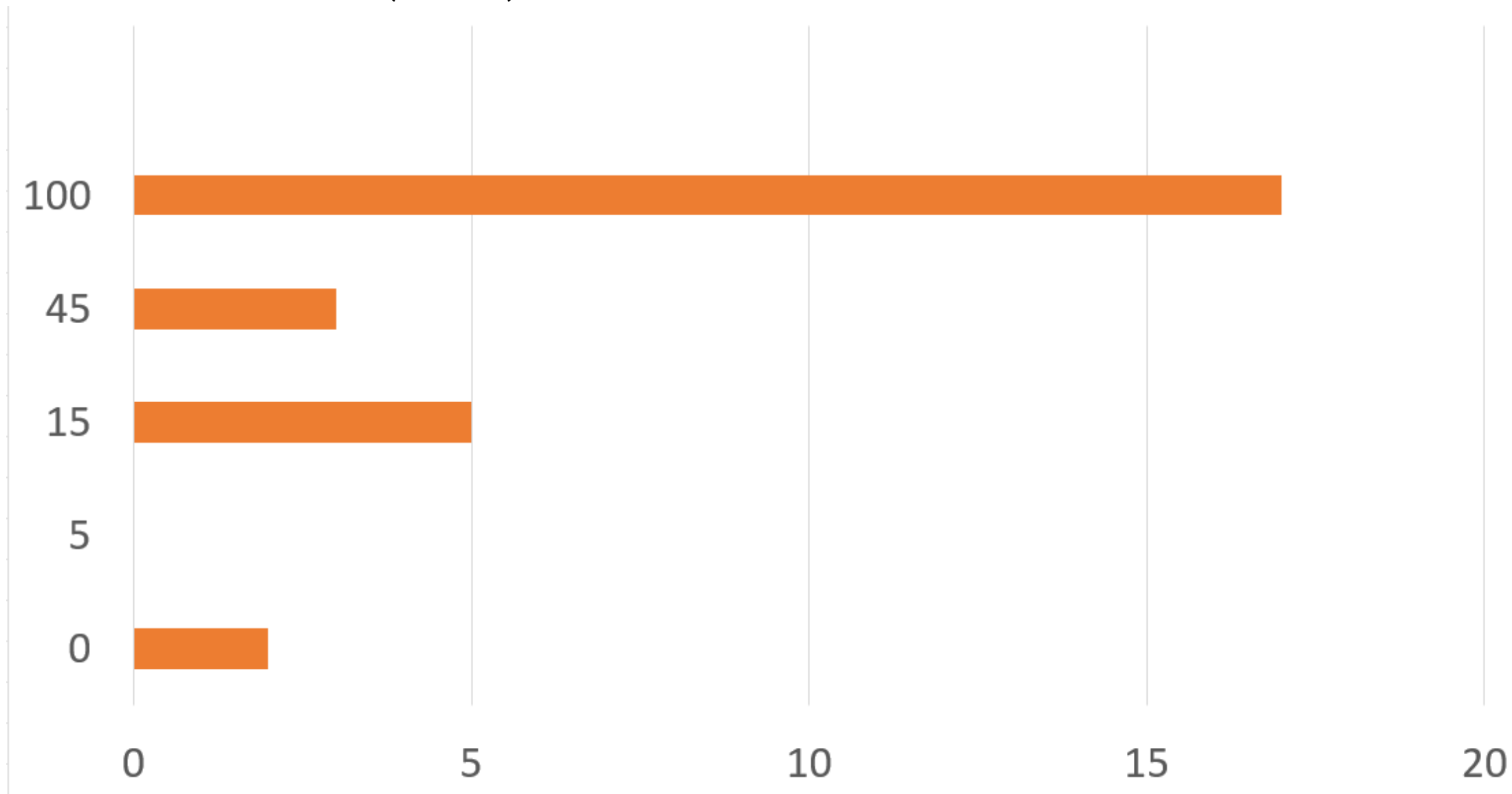


満点解法（別解）

- $A_{i+1} - A_i$ が距離 D 以上になる初めての場所をRMQ(Sparse Table)+二分探索で求めてもよい
- 計算量は $O(N + Q \log N \log \max A_i)$



得点分布 (JOI)



得点分布 (JOIG)

得点分布

